FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Circuitos Secuenciales Biestables

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar los efectos de la realimentación en circuitos lógicos.

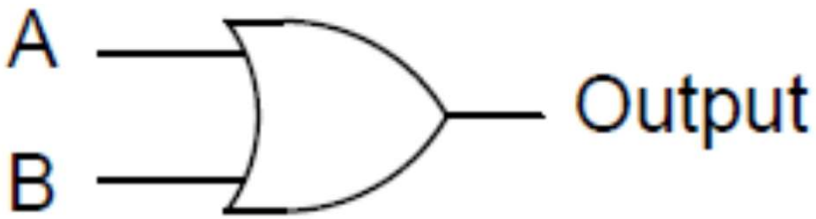
Estudiar las características de los circuitos secuenciales y su empleo como circuitos de memoria.

Conocer y aplicar los distintos tipos de circuitos biestables como así también sus características.

Aplicar técnicas para obtener un tipo de biestable a partir de otro.

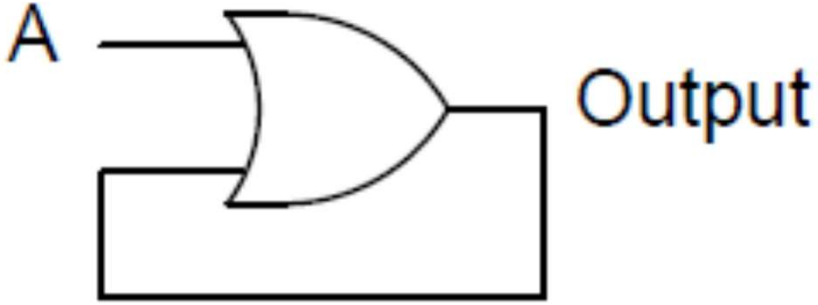
Realimentación en circuitos lógicos

Los circuitos vistos hasta ahora son circuitos **combinacionales**, ya que su salida responde a la **combinación** de sus **entradas**. Para cada una de las **2^n** combinaciones, la salida queda unívocamente establecida mediante su TV.
Como ejemplo una compuerta OR:



A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

Pero qué ocurre cuando realimentamos su salida:



A	Output
0	?
1	1

Circuitos secuenciales

La realimentación me permite obtener circuitos **secuenciales**, en los que la salida ya no responde solo a la combinación de sus **entradas**, sino que también depende del estado en que se encuentra su **salida**.

Combinacionales: $\mathbf{Z} = f(A, B, C)$

Secuenciales: $\mathbf{Z^{n+1}} = f(A^n, B^n, C^n, \mathbf{Z^n})$

El próximo estado depende también del estado actual.
El circuito recuerda en qué estado se encuentra, tiene memoria.
Un biestable es una celda básica de memoria RAM estática.

Circuitos secuenciales biestables

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Los circuitos secuenciales más simples son los biestables, poseen solo **2 estados estables** (0 o 1). Es una celda básica SRAM que me permite almacenar un bit. Podemos clasificarlos en principio:

Por su funcionamiento (TV):

SR
JK
D
T

Por su sincronismo:

Asincrónico	Latch / FF asincrónico?
Sincrónico de nivel (alto o bajo)	Latch c/habilit. / FF de nivel??
Sincrónico de flanco (asc. o desc.)	Flip Flop / FF de flanco??

Circuitos secuenciales biestables

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

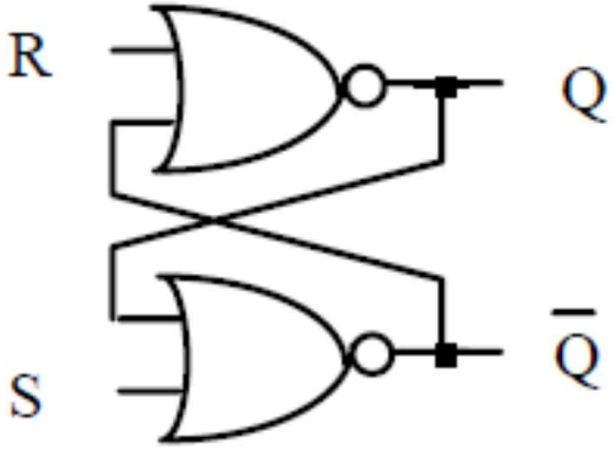
Para los biestables más comunes analizaremos sus **características**:

- TV reducida (o implícita)
- TV (ampliada)
- Ecuación característica
- Circuito lógico (interno)
- Símbolo gráfico
- Tabla de transiciones
- Diagrama de estados

Biestable S-R (Set-Reset)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Circuito lógico



TV reducida

S	R	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	x

$S \cdot R = 0$

TV ampliada

S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	x
1	1	1	x

Ecuación característica:

$Q^{n+1} = f(S, R, Q^n)$

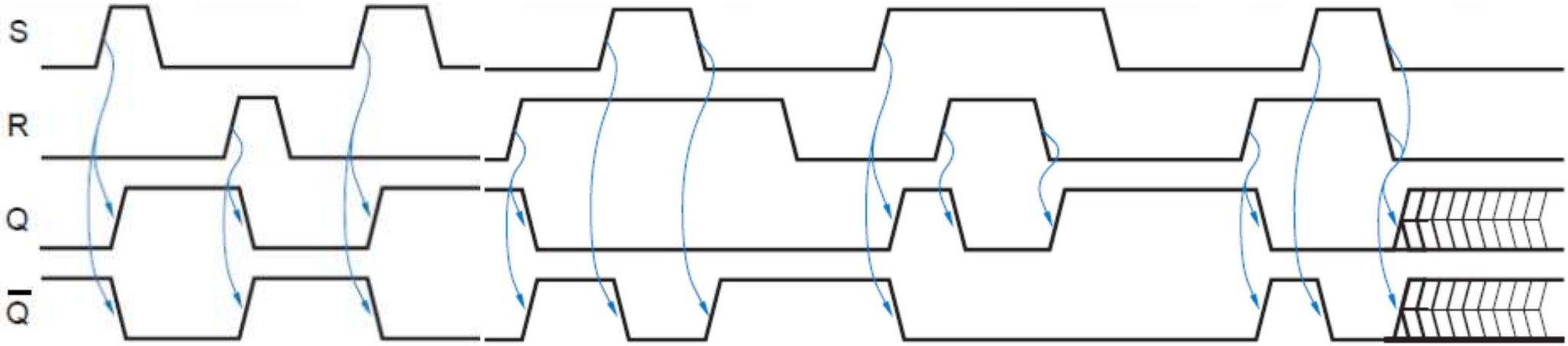
$$Q^{n+1} = S + \overline{R}\, Q^n$$

S \ R Qn	00	01	11	10
0	0	1	0	0
1	1	1	x	x

Biestable S-R (Set-Reset)

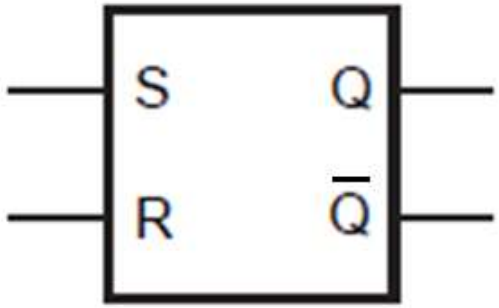
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

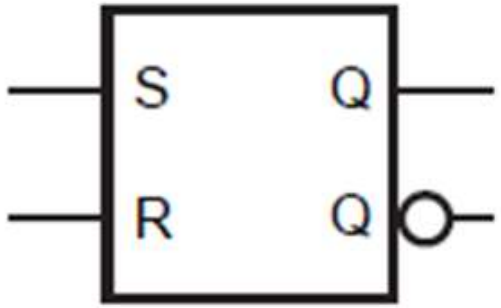
Diagrama temporal: funcionamiento normal y funcionamiento con R = S = 1



S	R	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	x

Símbolo gráfico





Biestable S-R (Set-Reset)

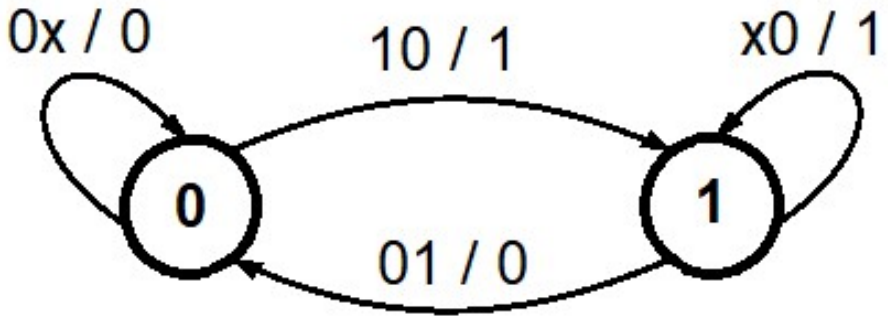
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

S	R	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	x
1	1	1	x

Tabla de transiciones

Qn	Qn+1	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

Diagrama de estados



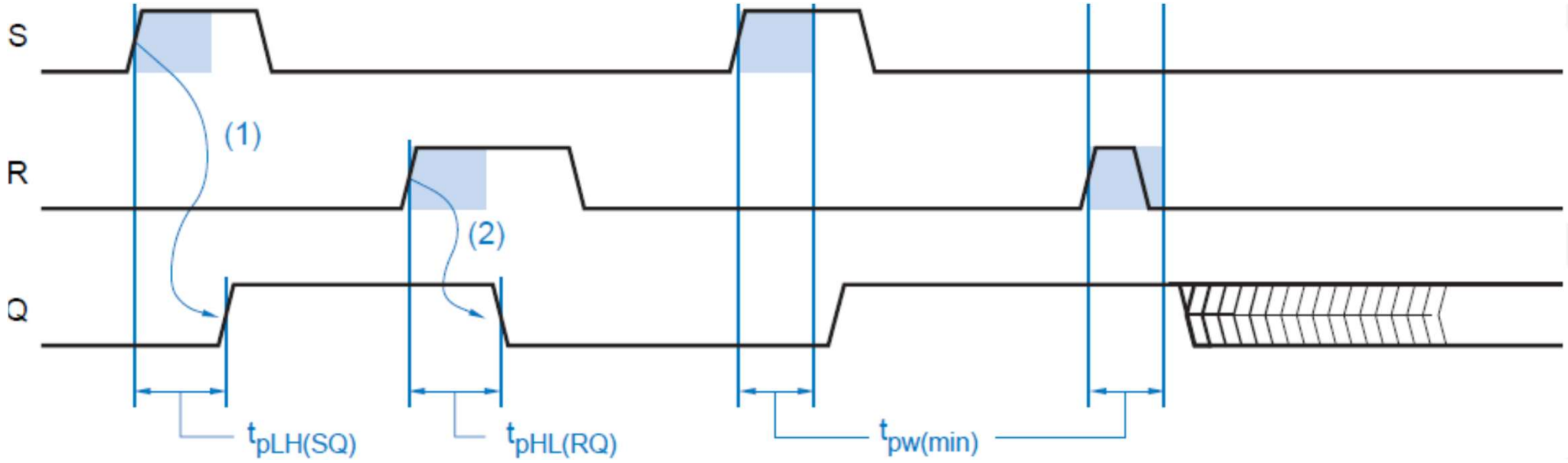
Biestable S-R (Set-Reset)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

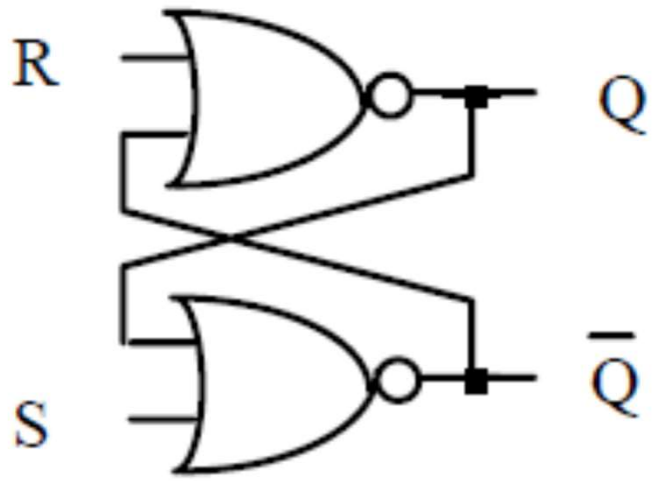
Tiempos:

Retardos de propagación: t_{pLH} y t_{pHL}

Ancho de pulso mínimo: $t_{pw(min)}$

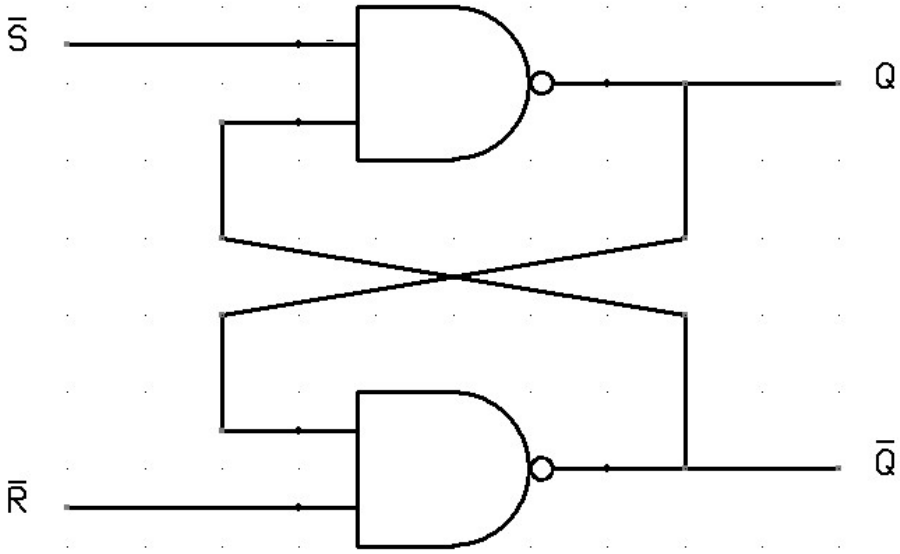


Biestable S-R con NAND



S	R	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	x

Implementación con NAND:



notS	notR	Qn+1
0	0	x
0	1	1
1	0	0
1	1	Qn

Biestables con habilitación por nivel (Ck)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

TV reducida

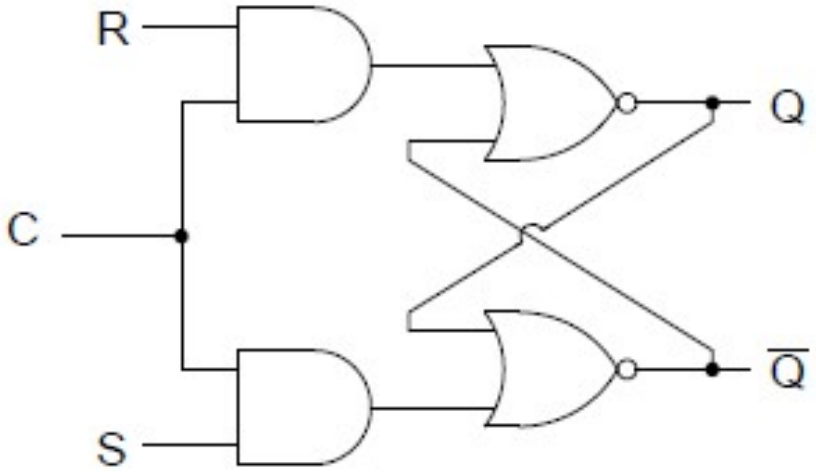
C	S	R	Qn+1
1	0	0	Qn
1	0	1	0
1	1	0	1
1	1	1	x
0	x	x	Qn

C es la temporización, reloj o clock (Ck)

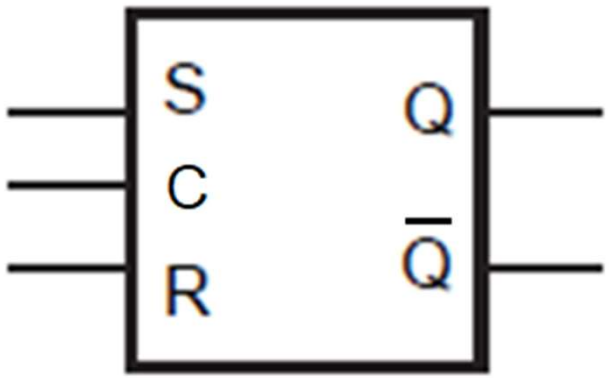
TV ampliada

C	S	R	Qn	Qn+1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	x
1	1	1	1	x
0	x	x	0	0
0	x	x	1	1

Circuito lógico

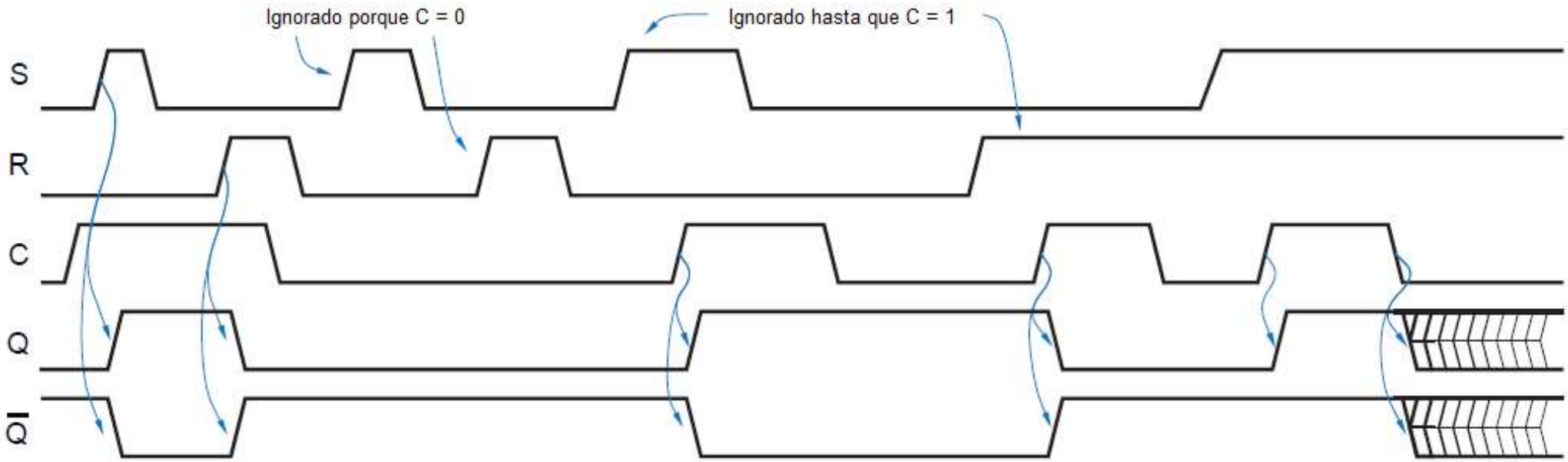


Símbolo gráfico

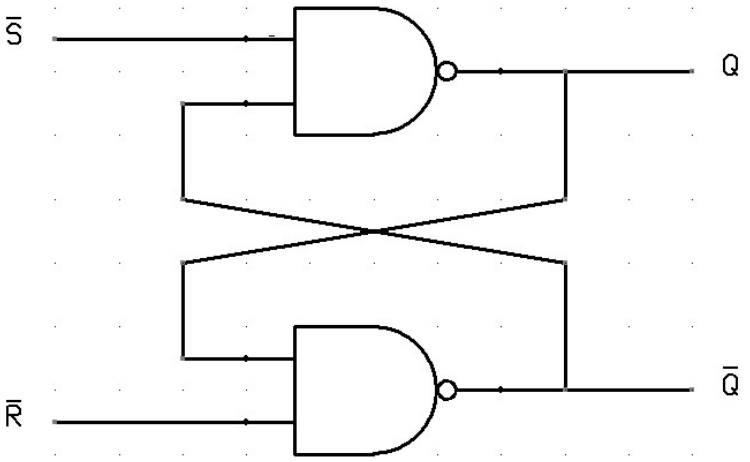


Biestables con habilitación por nivel (Ck)

Diagrama temporal: funcionamiento normal y funcionamiento con R = S = 1

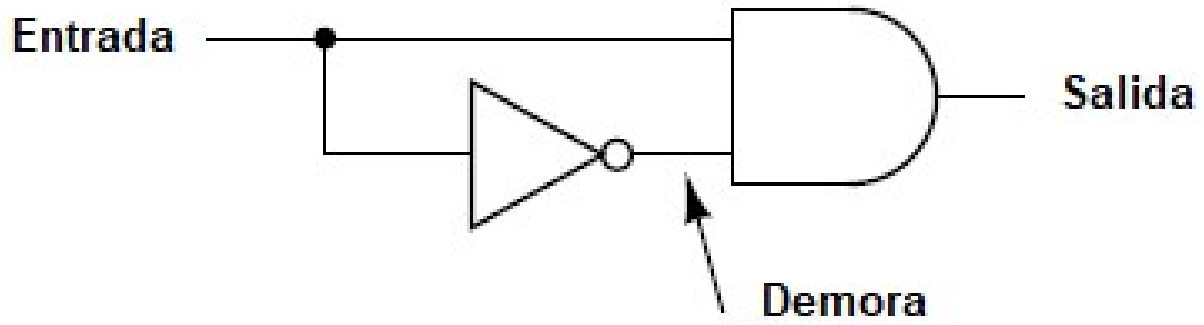


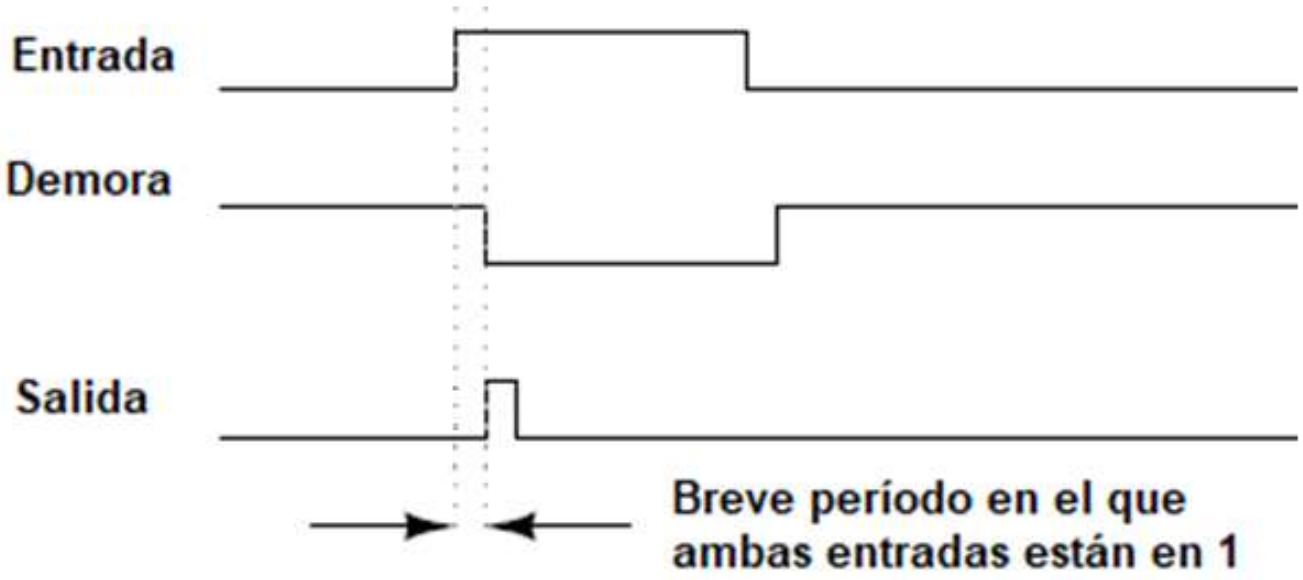
C	S	R	Qn+1
1	0	0	Qn
1	0	1	0
1	1	0	1
1	1	1	x
0	x	x	Qn



Biestables con habilitación por flanco (Ck)

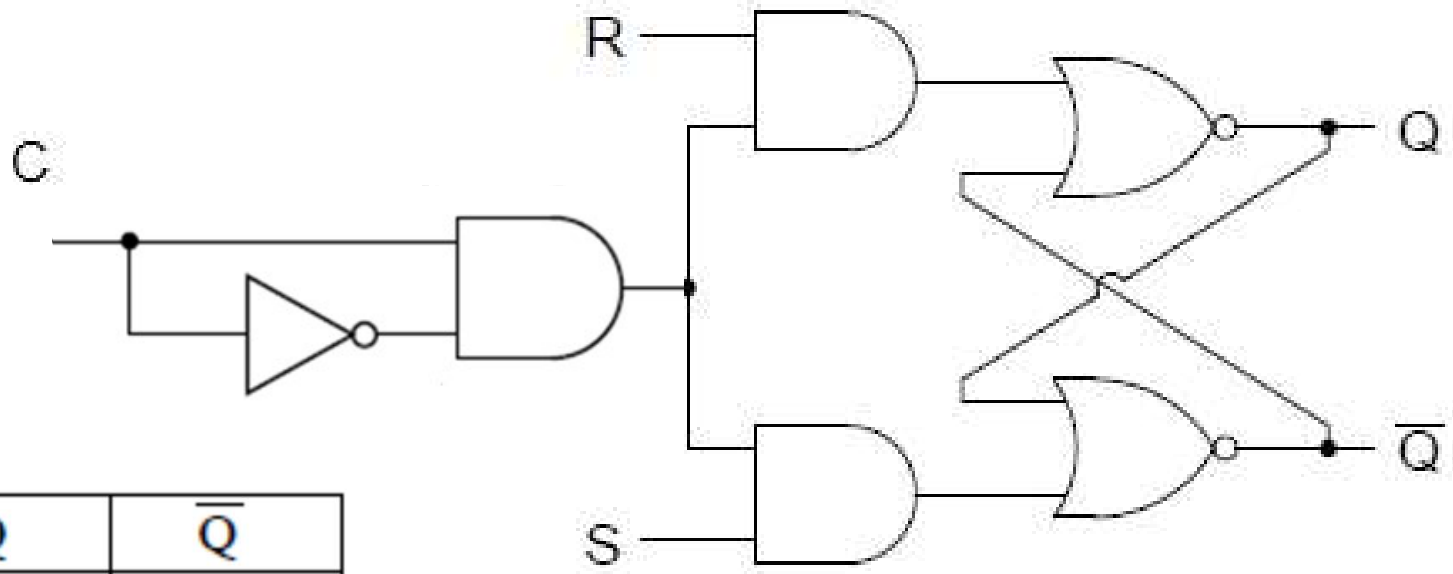
Para que se produzca **un solo cambio** por cada pulso de Ck.





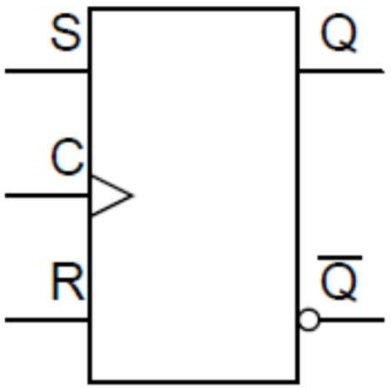
Biestables con habilitación por flanco (Ck)

Vemos el circuito y la TV de un SR de **flanco ascendente**.



C	S	R	Q	$\overline{Q}$
⌐	0	0	latch	latch
⌐	0	1	0	1
⌐	1	0	1	0
⌐	1	1	0	0
x	0	0	latch	latch
x	0	1	latch	latch
x	1	0	latch	latch
x	1	1	latch	latch

Símbolo gráfico

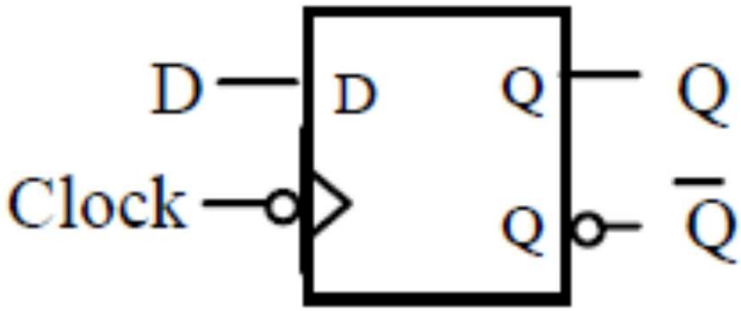


Biestables con habilitación por flanco (Ck)

También podemos obtener biestables de **flanco descendente**.

D	CLK_L	Q
0	↓	0
1	↓	1
x	0	last Q
x	1	last Q

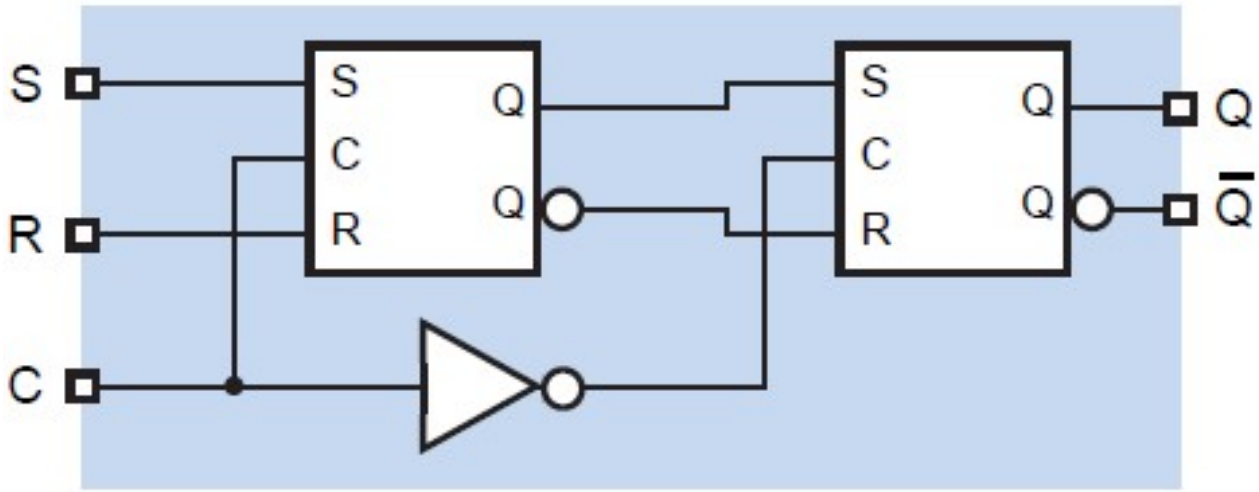
Símbolo gráfico



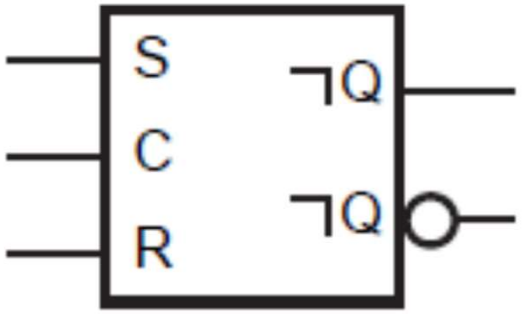
Biestables Maestro Esclavo

Necesitan que se produzcan 2 cambios en el nivel del Ck.
Con Ck = 1, el dato pasa a la salida del Master, y con Ck = 0, el dato pasa a la salida del Slave, quedando el Master deshabilitado.

S	R	C	Q
x	x	0	last Q
0	0	⊓	last Q
0	1	⊓	0
1	0	⊓	1
1	1	⊓	undef.



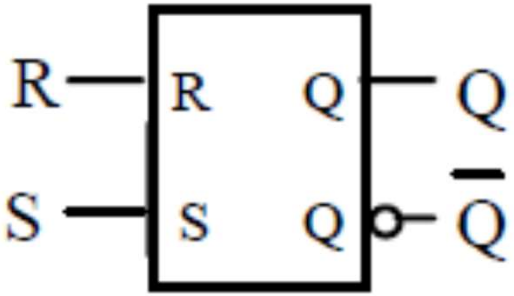
Símbolo gráfico



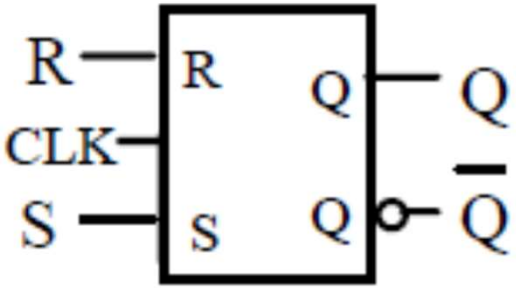
Resumen de sincronismos de biestables

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

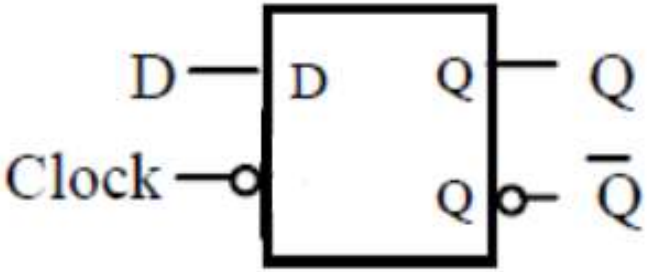
Asincrónico



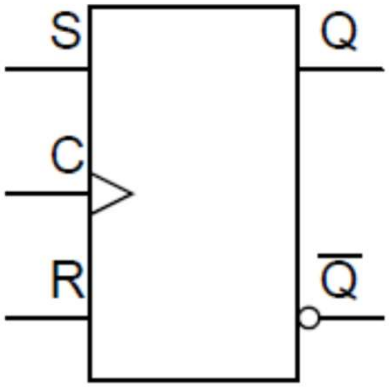
Nivel alto



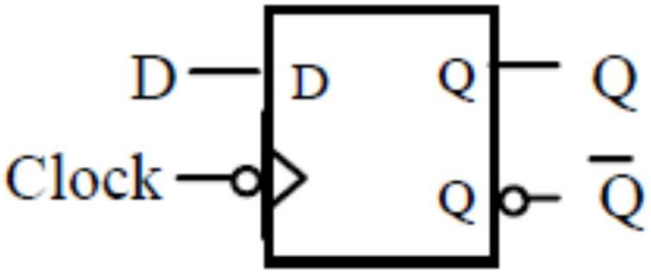
Nivel bajo



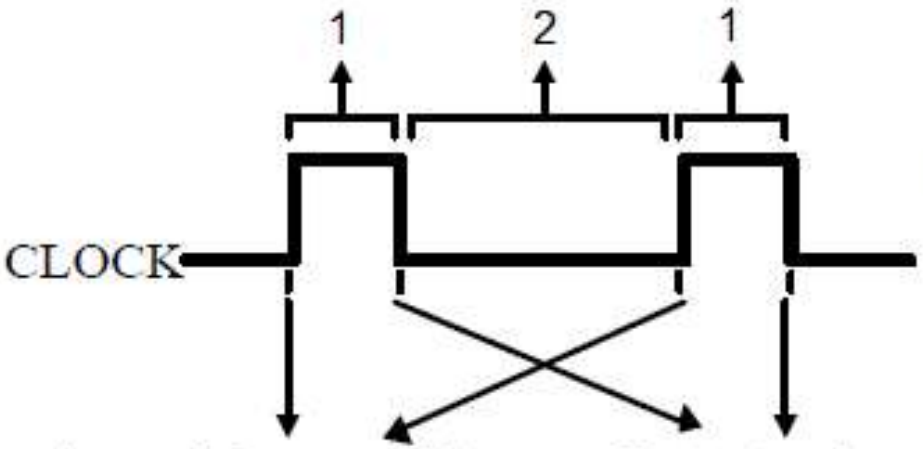
Flanco ascendente



Flanco descendente



Resumen de sincronismos de biestables



1 Instantes de cambio permitido con sincronización por nivel alto

2 Instantes de cambio permitido con sincronización por nivel bajo

Instantes de cambio permitido con sincronización por flanco ascendente

Instantes de cambio permitido con sincronización por flanco descendente

Nivel

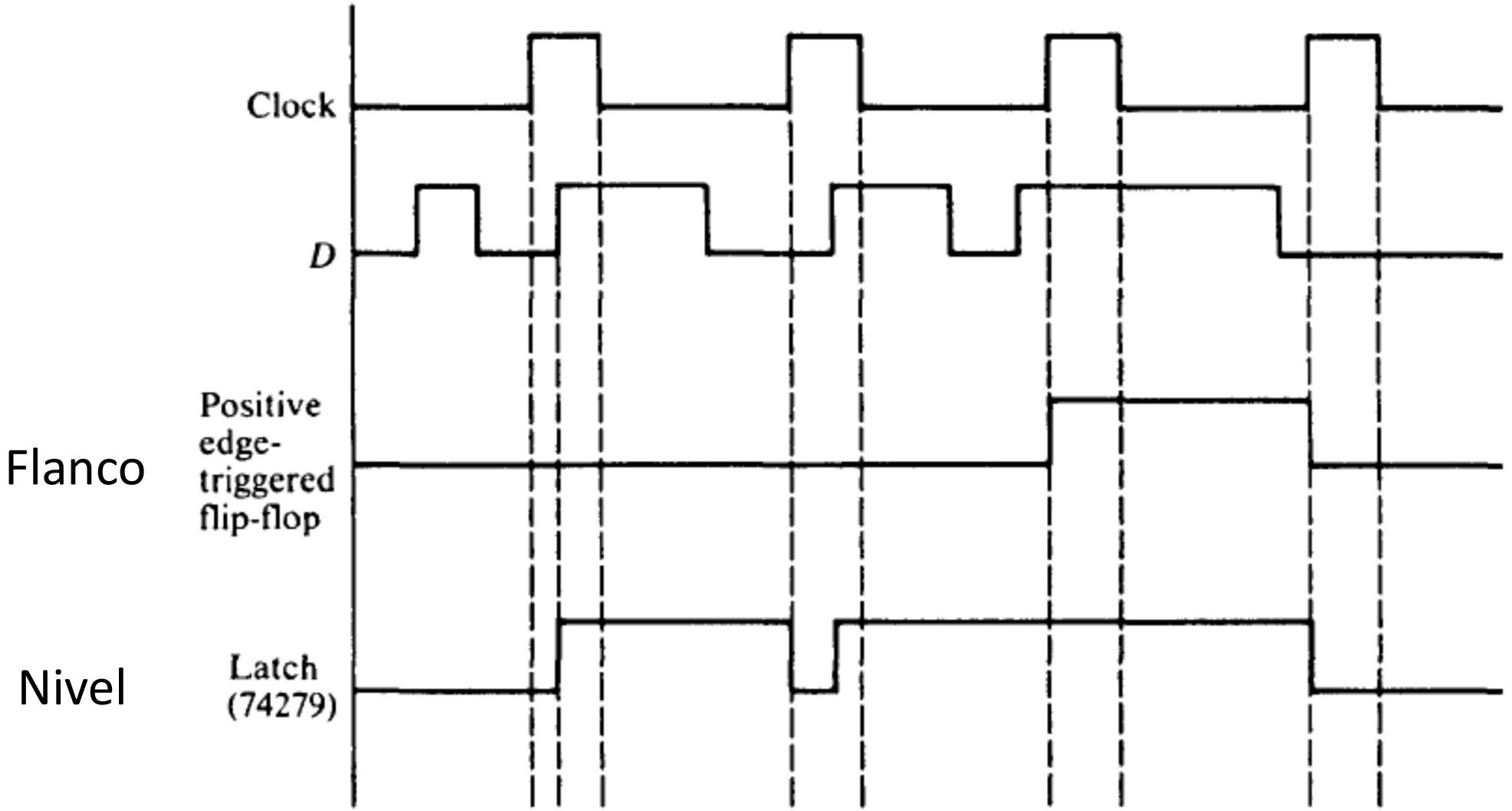
Flanco



Alto Bajo Ascendente Descendente

Resumen de sincronismos de biestables

Respuesta de un biestable de **flanco ascendente** y de uno de **nivel alto**.



Flanco

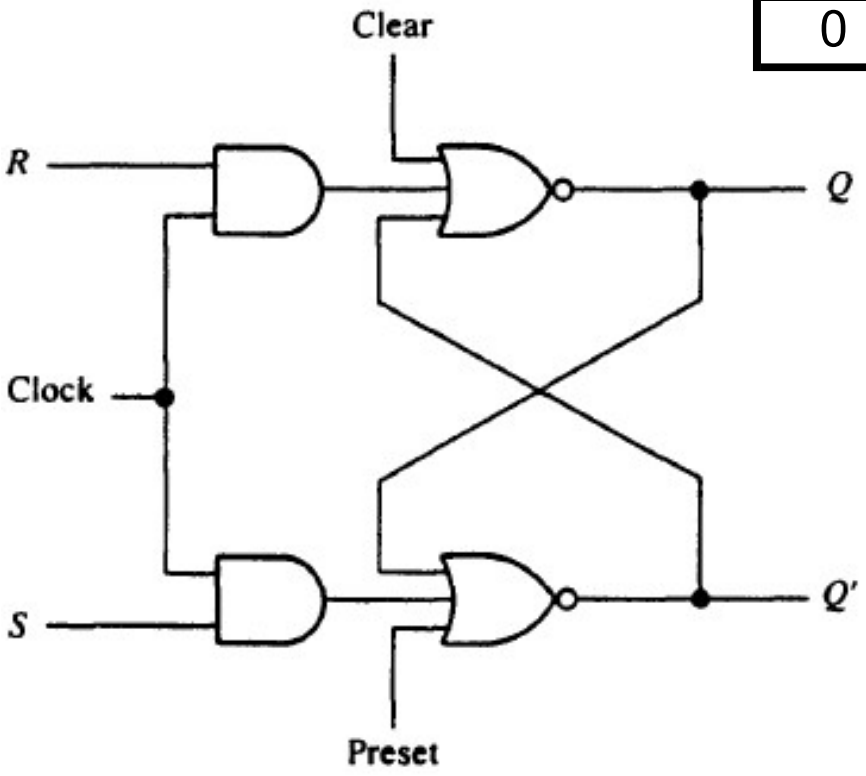
Nivel

Biestables con Entradas Asincrónicas

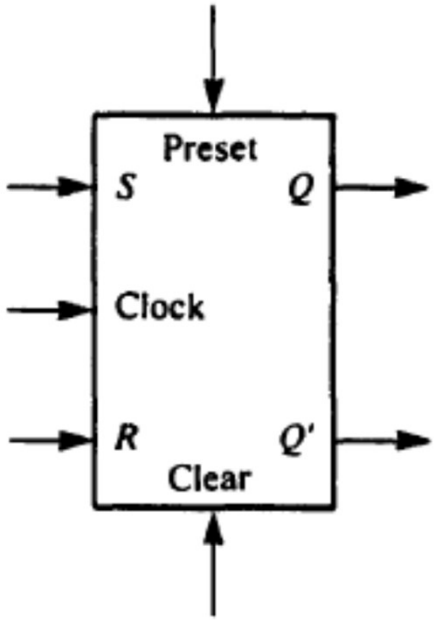
TV reducida:

Pr y Clr son **entradas asincrónicas**, no dependen de la temporización o clock (Ck).

Pr	Clr	Ck	S	R	Qn+1	not Qn+1
0	0	1	0	0	Qn	not Qn
0	0	1	0	1	0	1
0	0	1	1	0	1	0
0	0	1	1	1	0	0
0	1	x	x	x	0	1
1	0	x	x	x	1	0
1	1	x	x	x	0	0
0	0	0	x	x	Qn	not Qn



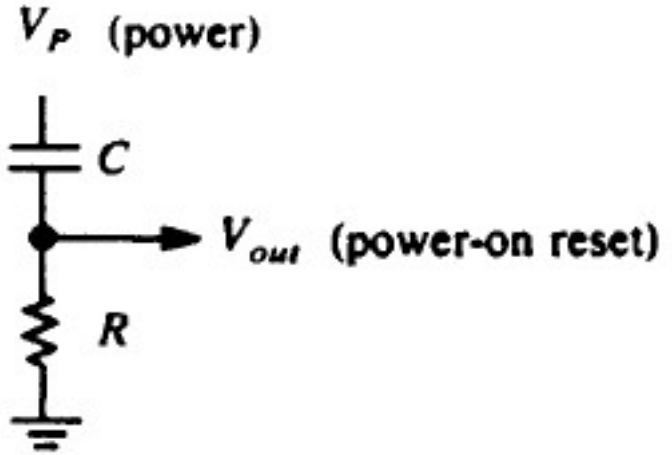
Circuito lógico

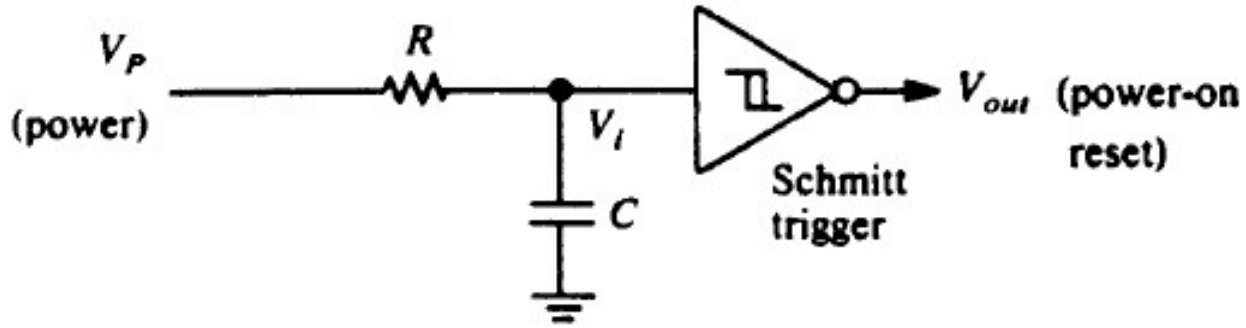


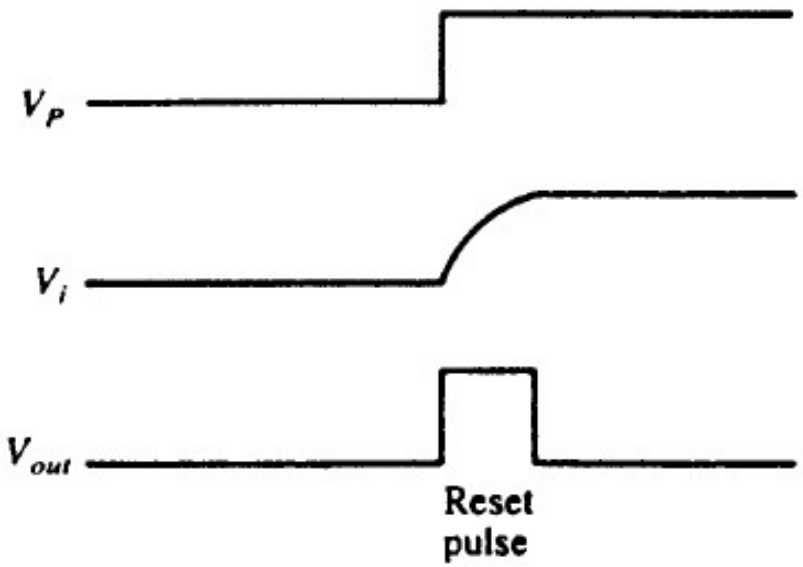
Símbolo gráfico

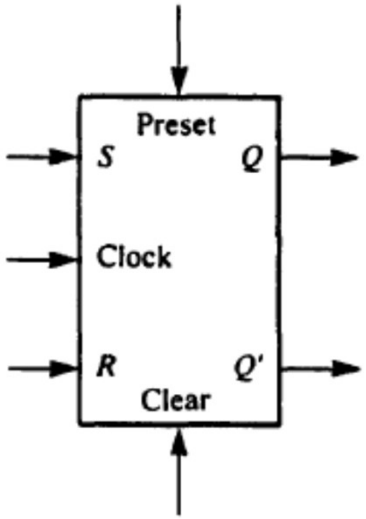
Power On Reset (POR)

Para forzar un biestable a un determinado estado inicial utilizo las entradas asincrónicas (Pr o Clr según sea 1 o 0 el estado inicial). El tiempo debe ser suficiente para que se establezca pero no demasiado largo.
Muchas veces se utilizan circuitos de **Power On Reset** (**POR**) para lograrlo.









Biestable J-K

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

TV reducida SR

S	R	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	x

TV reducida JK

J	K	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	not Qn

TV ampliada

J	K	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Equivalencias: J = S y K = R

Invierte estado con J = K = 1

Ecuación característica:

$$Q^{n+1} = f(J, K, Q^n)$$

$$Q^{n+1} = J\,\overline{Q^n} + \overline{K}\,Q^n$$

J \ K Qn	00	01	11	10
0	0	1	0	0
1	1	1	0	1

Biestable J-K

J	K	Qn	Qn+1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Tabla de transiciones

Qn	Qn+1	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Símbolo gráfico

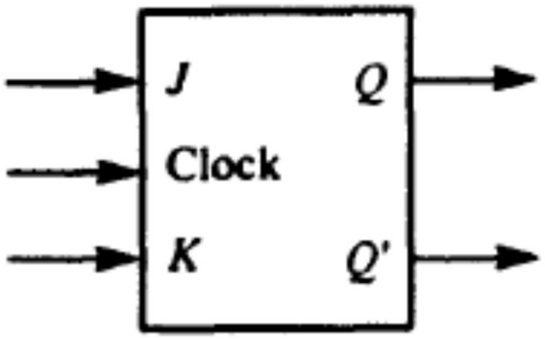
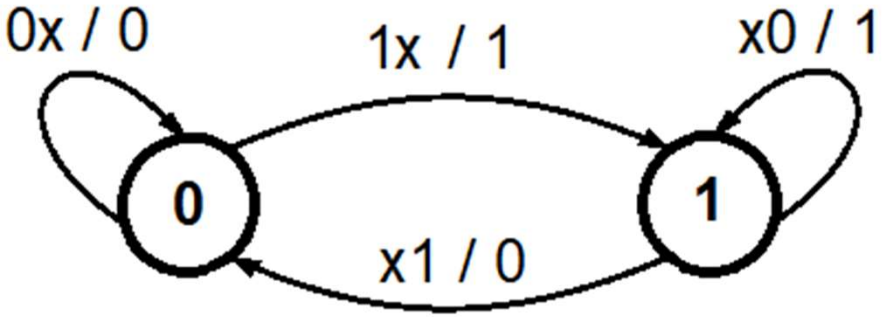


Diagrama de estados



Biestable D (Delay)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Posee una sola entrada.

Guarda el dato.

TV reducida

D	Qn+1
0	0
1	1

TV ampliada

D	Qn	Qn+1
0	0	0
0	1	0
1	0	1
1	1	1

Ecuación característica:

$$Q^{n+1} = f(D, Q^n)$$

$$Q^{n+1} = D$$

No depende de Q^n

D \ Qn	0	1
0	0	0
1	1	1

Biestable D (Delay)

D	Qn	Qn+1
0	0	0
0	1	0
1	0	1
1	1	1

Tabla de transiciones

Qn	Qn+1	D
0	0	0
0	1	1
1	0	0
1	1	1

Símbolo gráfico

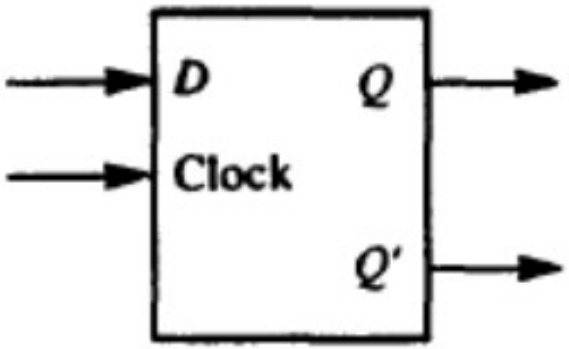
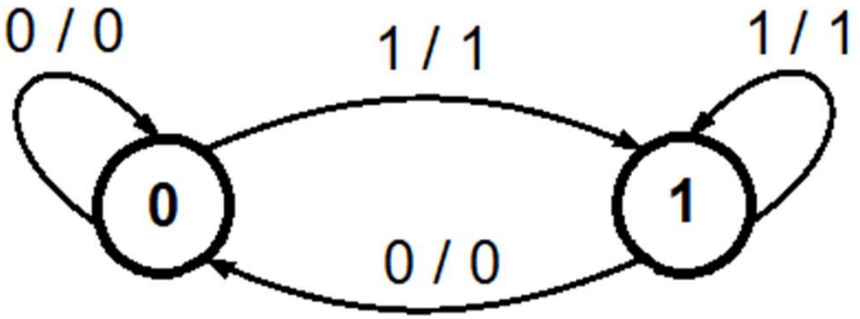


Diagrama de estados



Biestable T (Toggle)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Posee una sola entrada.

Con T = 1 invierte la salida.

TV reducida

T	Qn+1
0	Qn
1	not Qn

TV ampliada

T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

Ecuación característica:

$$Q^{n+1} = f(T, Q^n)$$

$$Q^{n+1} = T\,\overline{Q^n} + \overline{T}\,Q^n$$

T \ Qn	0	1
0	0	1
1	1	0

Biestable T (Toggle)

T	Qn	Qn+1
0	0	0
0	1	1
1	0	1
1	1	0

Tabla de transiciones

Qn	Qn+1	T
0	0	0
0	1	1
1	0	1
1	1	0

Símbolo gráfico

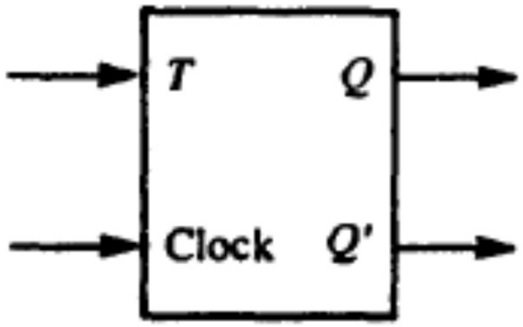
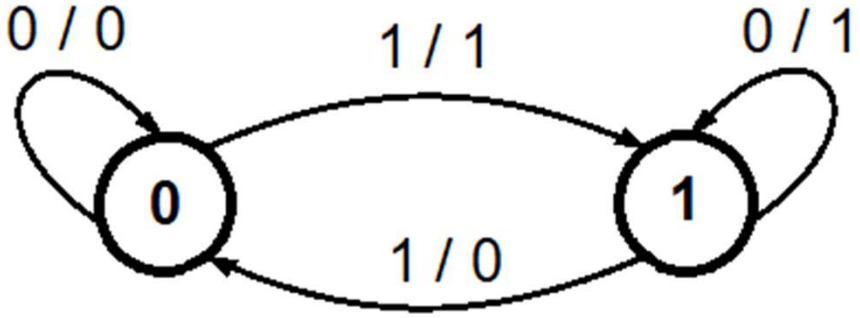


Diagrama de estados



Data sheets

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

SN74LS76A

Dual JK Flip-Flop with Set and Clear

MODE SELECT – TRUTH TABLE

OPERATING MODE	INPUTS				OUTPUTS	
	$\overline{S}_D$	$\overline{C}_D$	J	K	Q	$\overline{Q}$
Set	L	H	X	X	H	L
Reset (Clear)	H	L	X	X	L	H
*Undetermined	L	L	X	X	H	H
Toggle	H	H	h	h	$\overline{q}$	q
Load "0" (Reset)	H	H	l	h	L	H
Load "1" (Set)	H	H	h	l	H	L
Hold	H	H	l	l	q	$\overline{q}$

* Both outputs will be HIGH while both $\overline{S}_D$ and $\overline{C}_D$ are LOW, but the output states are unpredictable if $\overline{S}_D$ and $\overline{C}_D$ go HIGH simultaneously.

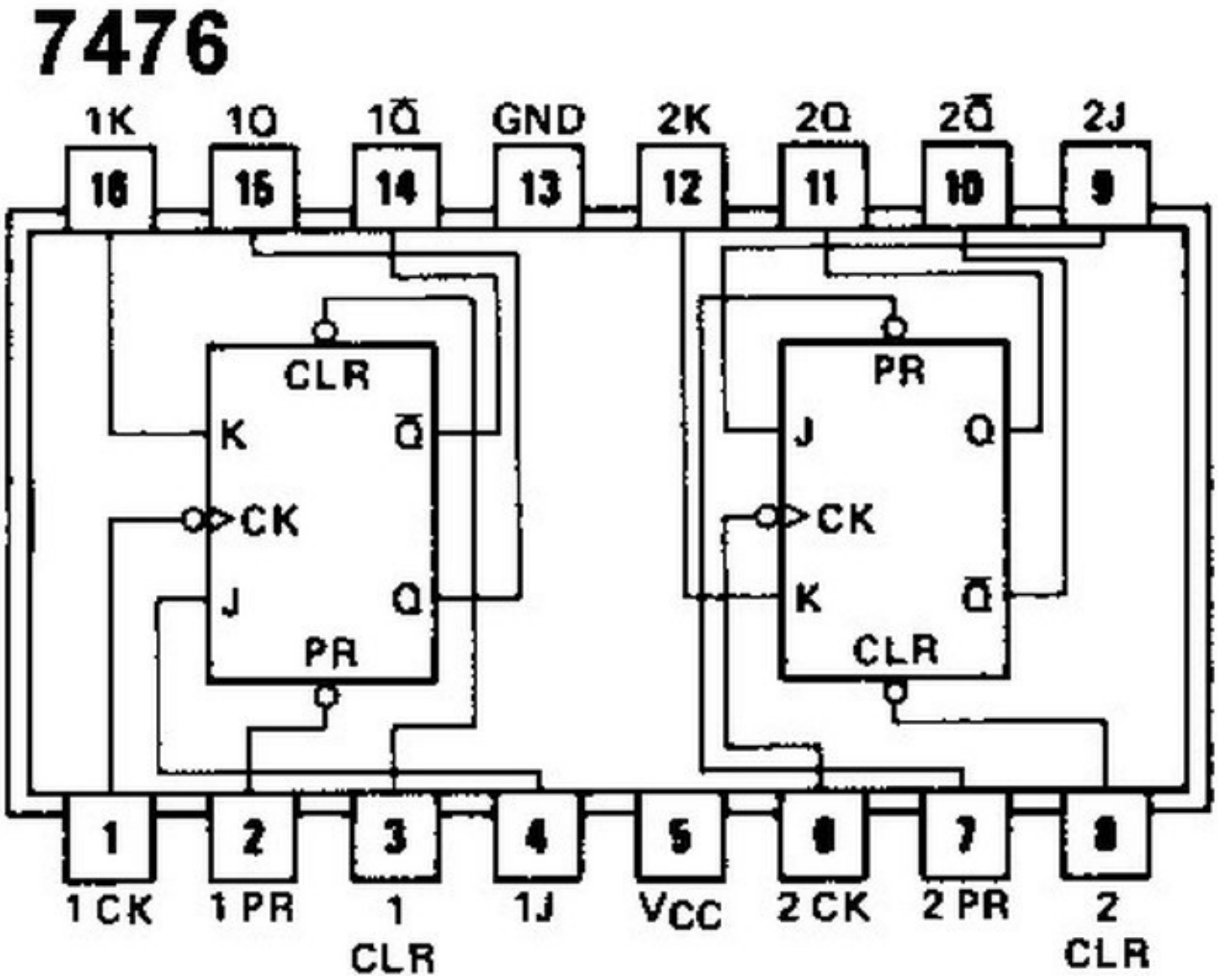
H, h = HIGH Voltage Level

L, l = LOW Voltage Level

X = Immaterial

l, h (q) = Lower case letters indicate the state of the referenced input (or output) one setup time prior to the HIGH–to–LOW clock transition

Data sheets


7476
1K
1Q
1Q̄
GND
2K
2Q
2Q̄
2J
16
15
14
13
12
11
10
9
CLR
K
Q̄
CK
J
Q
PR
PR
J
Q
CK
K
Q̄
CLR
1
2
3
4
5
6
7
8
1 CK
1 PR
1 CLR
1J
VCC
2 CK
2 PR
2 CLR

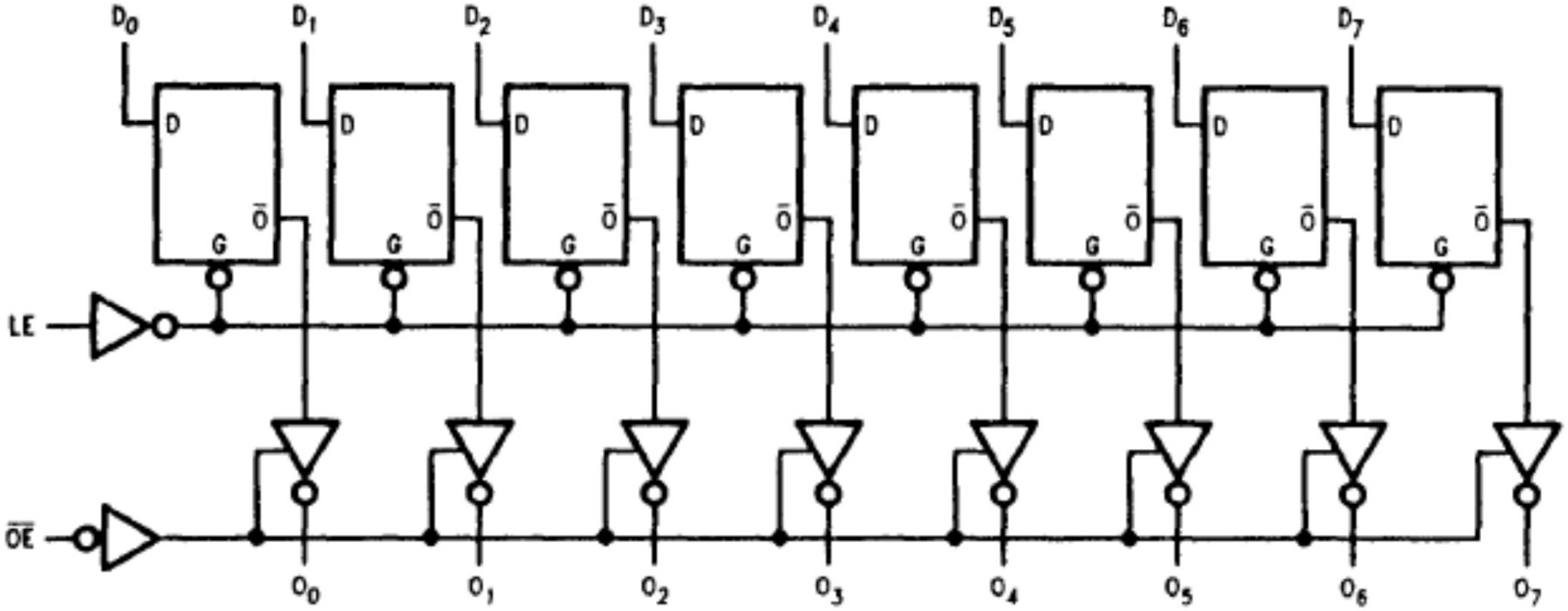
Data sheets

7475:

Truth Table

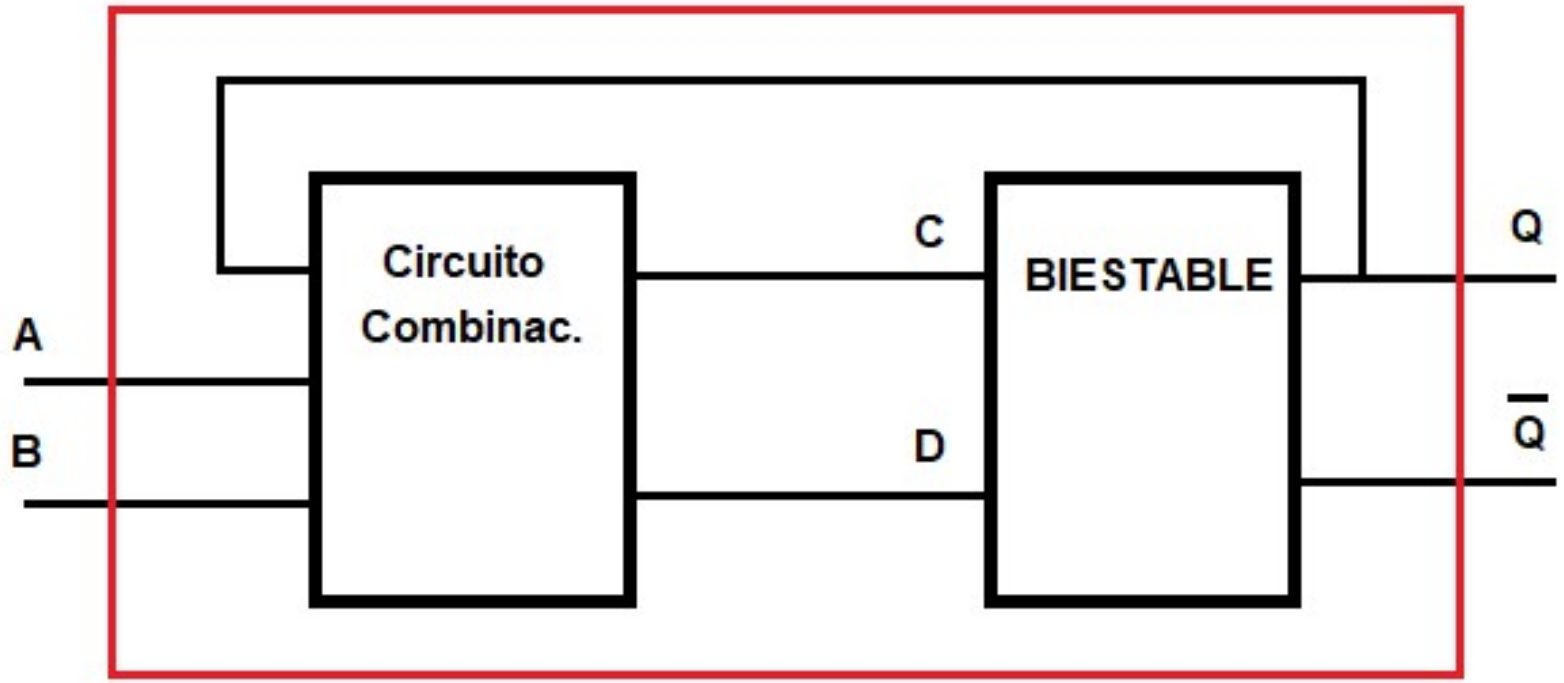
Inputs			Output
LE	$\overline{OE}$	D_n	O_n
H	L	H	H
H	L	L	L
L	L	X	O_n (no change)
X	H	X	Z

H = HIGH Voltage Level
L = LOW Voltage Level
X = Immaterial
Z = High Impedance State



Método de las Transiciones

El objetivo es obtener un FF (AB) a partir de otro (CD). (Sincrónico o asincrónico)



$$C = f_1(A, B, Q^n)$$

$$D = f_2(A, B, Q^n)$$

Vamos a estudiarlo mediante un ejemplo: Obtener el FF JK a partir del RS.

Obtendremos: **$S = f_1(J, K, Q^n)$** y **$R = f_2(J, K, Q^n)$**

Método de las Transiciones

Partimos de la TV ampliada del JK y analizamos sus transiciones con la TT del RS.

TT SR

Qn	Qn+1	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

TV JK

Obtenemos:

J	K	Qn	Qn+1	S	R
0	0	0	0	0	x
0	0	1	1	x	0
0	1	0	0	0	x
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	1	x	0
1	1	0	1	1	0
1	1	1	0	0	1

J \ K Qn	00	01	11	10
0	0	x	0	0
1	1	x	0	1

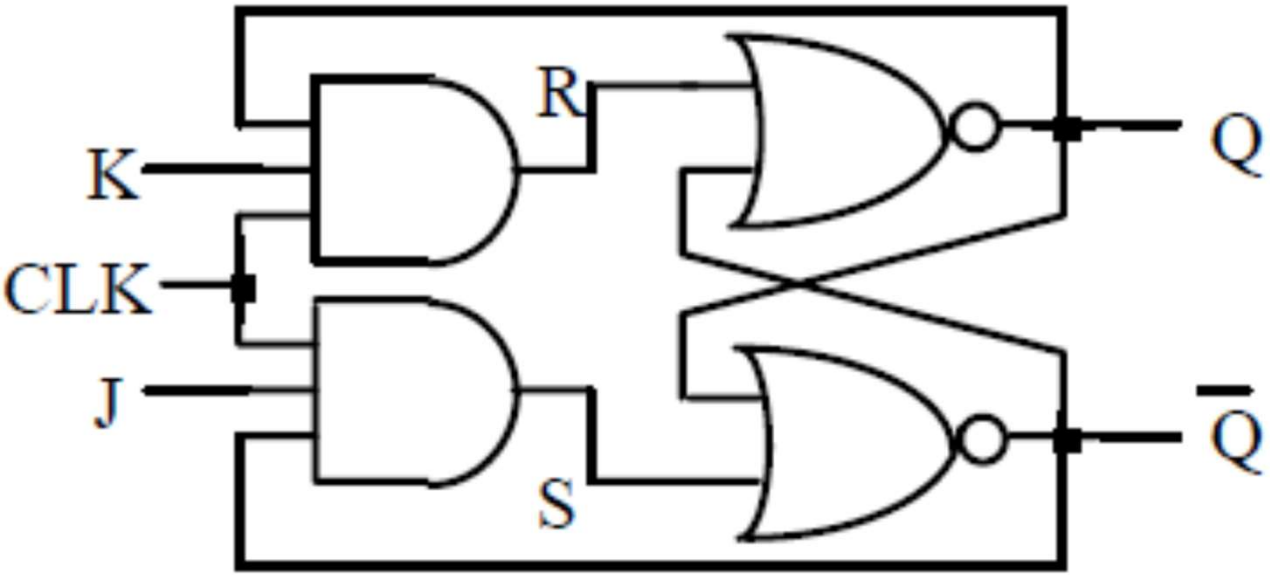
J \ K Qn	00	01	11	10
0	x	0	1	x
1	0	0	1	0

$$S = J\,\overline{Q^n}$$

$$R = K\,Q^n$$

Método de las Transiciones

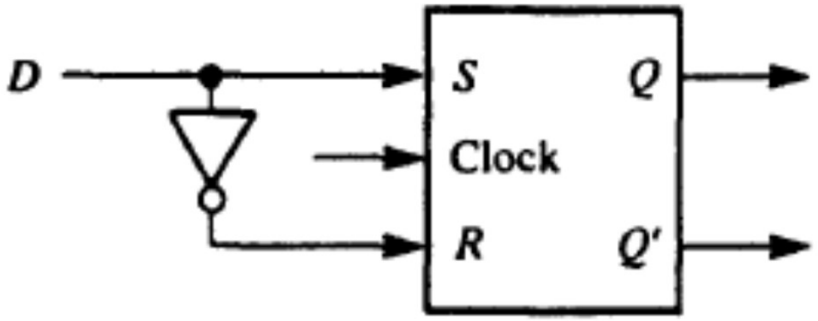
El circuito lógico de un JK sincrónico de nivel alto quedará:



$$S = J\,\overline{Q^n}\,Clk$$

$$R = K\,Q^n\,Clk$$

También puedo obtener mediante este método un D a partir de un SR, un D a partir de un JK, etc.



Método de las Ecuaciones Características

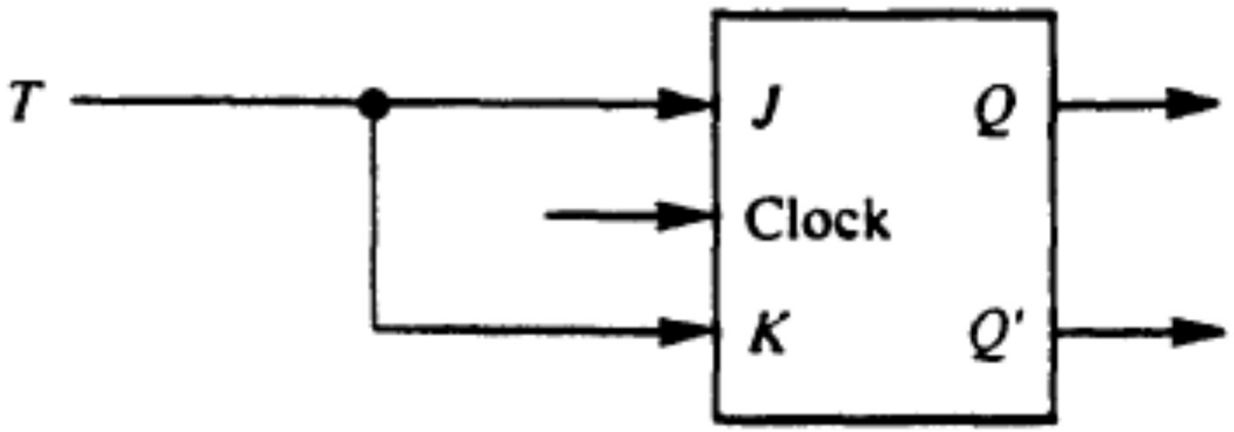
Consiste en igualar las EC de ambos FF. Ejemplo: Obtener un T a partir de un JK.

$$Q^{n+1} = J\,\overline{Q^n} + \overline{K}\,Q^n$$

$$Q^{n+1} = T\,\overline{Q^n} + \overline{T}\,Q^n$$

Comparando obtenemos:

$$J = T \qquad y \qquad K = T$$



Método de las Ecuaciones Características

Otro ejemplo: Obtener un D a partir de un JK.

$$Q^{n+1} = J\,\overline{Q^n} + \overline{K}\,Q^n$$

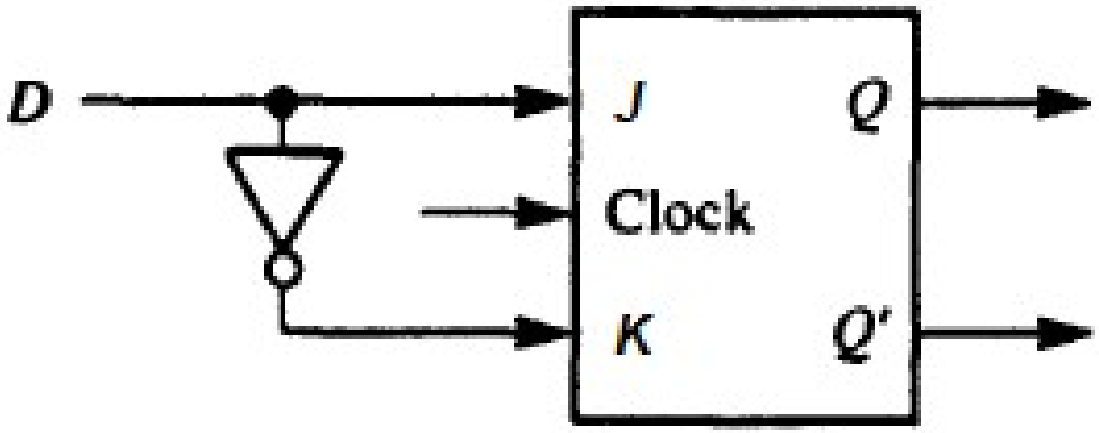
$$Q^{n+1} = D$$

$$Q^{n+1} = D\,\overline{Q^n} + D\,Q^n$$

Comparando obtenemos:

$$J = D \qquad y \qquad \overline{K} = D$$

Por lo tanto: $K = \overline{D}$

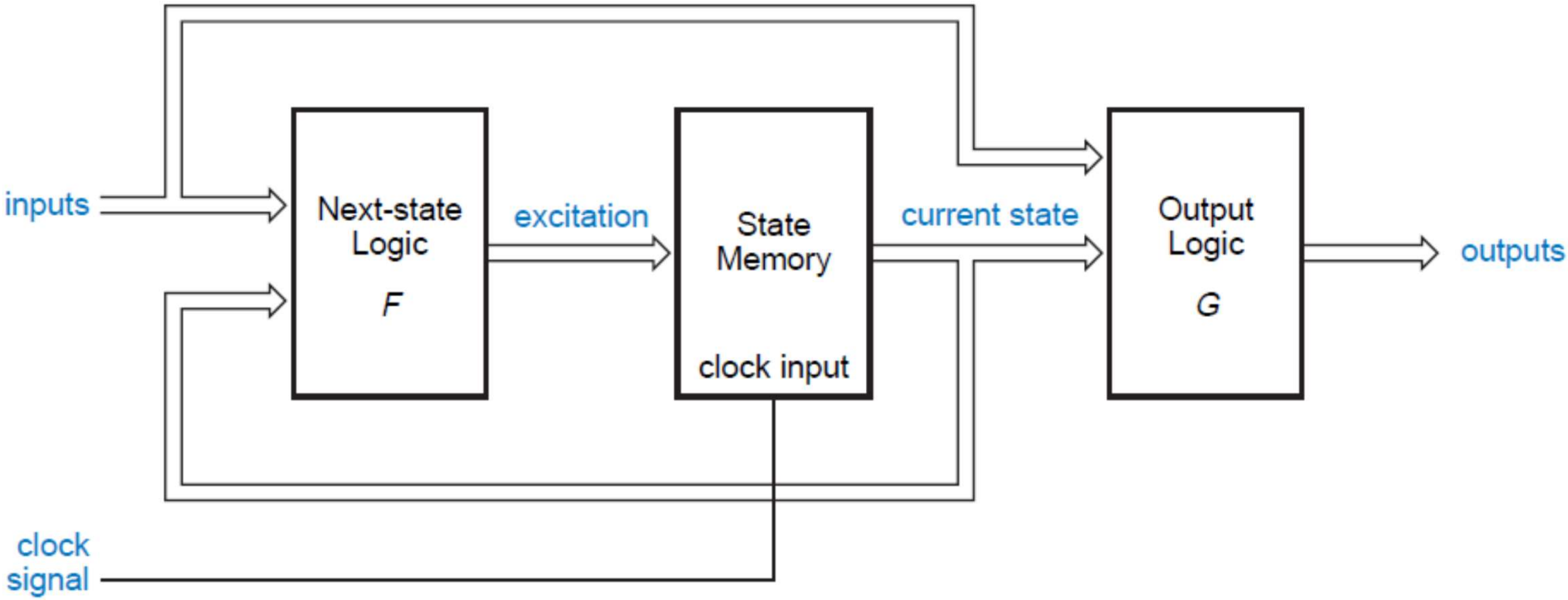


Máquina de Mealy

Memoria de estado: n biestables 2^n estados posibles reloj

Lógica de próximo estado: F = f(entradas, estado actual)

Lógica de salida: G = g(entradas, estado actual)

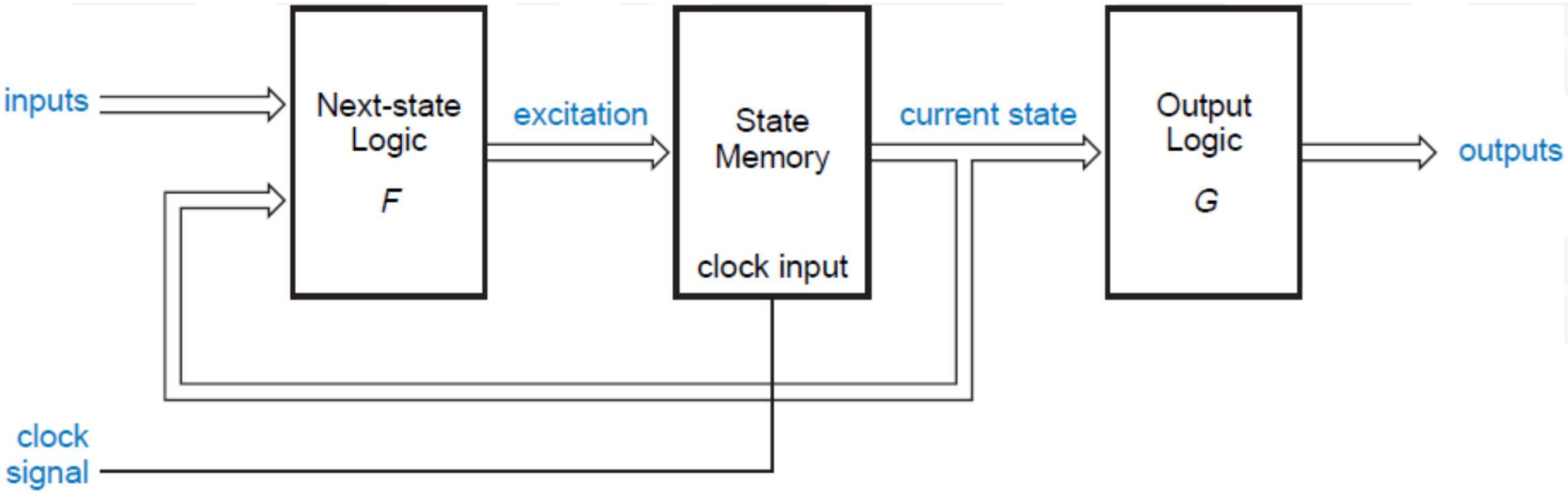


Máquina de Moore

Memoria de estado: n biestables 2^n estados posibles reloj

Lógica de próximo estado: F = f(entradas, estado actual)

Lógica de salida: G = g(estado actual)





FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Circuitos Secuenciales
Contadores

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar las características de los circuitos secuenciales en que los contadores pueden incluirse.

Estudiar un tipo de circuito secuencial capaz de almacenar y manejar información, así como generar secuencias para otros circuitos.

Redescubrir el concepto de estado de un circuito secuencial.

Conocer y aplicar los distintos tipos de contadores, sus códigos asociados, y el uso de manuales sobre el tema.

Obtener el manejo de los diagramas temporales y su realización.

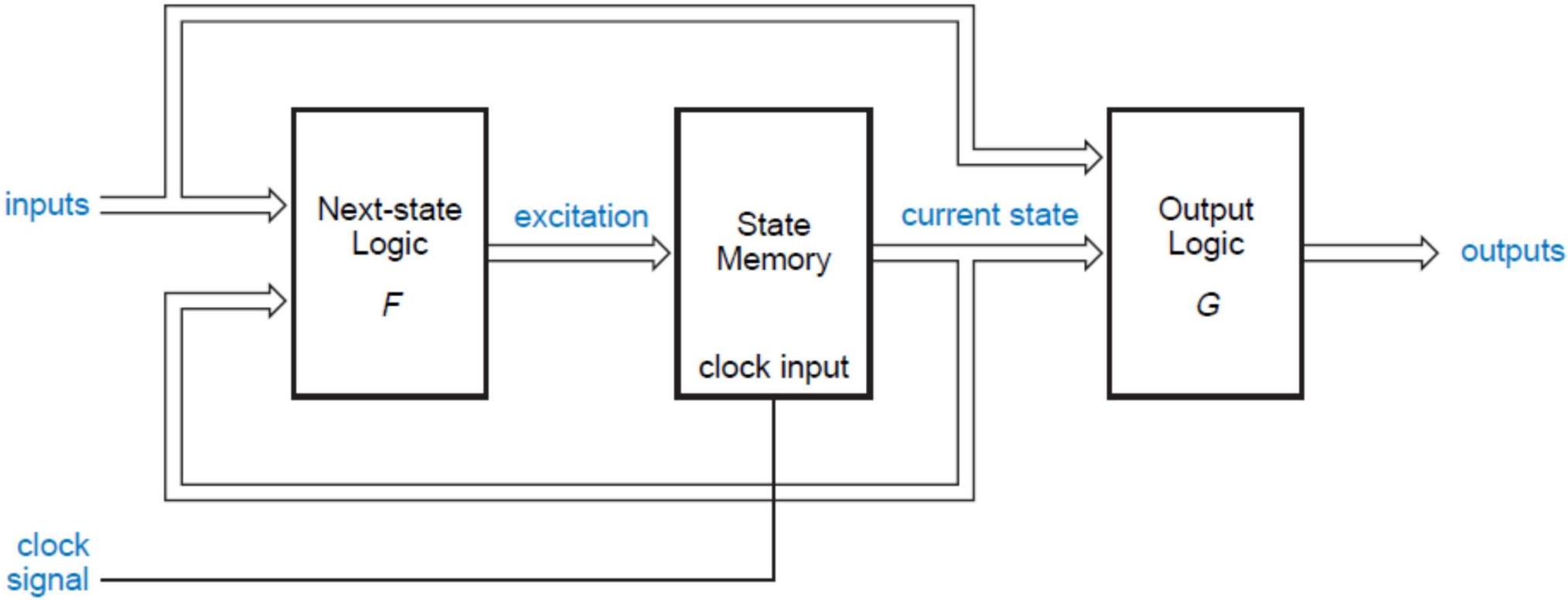
Analizar las formas de interconexión entre contadores.

Máquina de Mealy

Memoria de estado: n biestables 2^n estados posibles reloj

Lógica de próximo estado: F = f(entradas, estado actual)

Lógica de salida: G = g(entradas, estado actual)

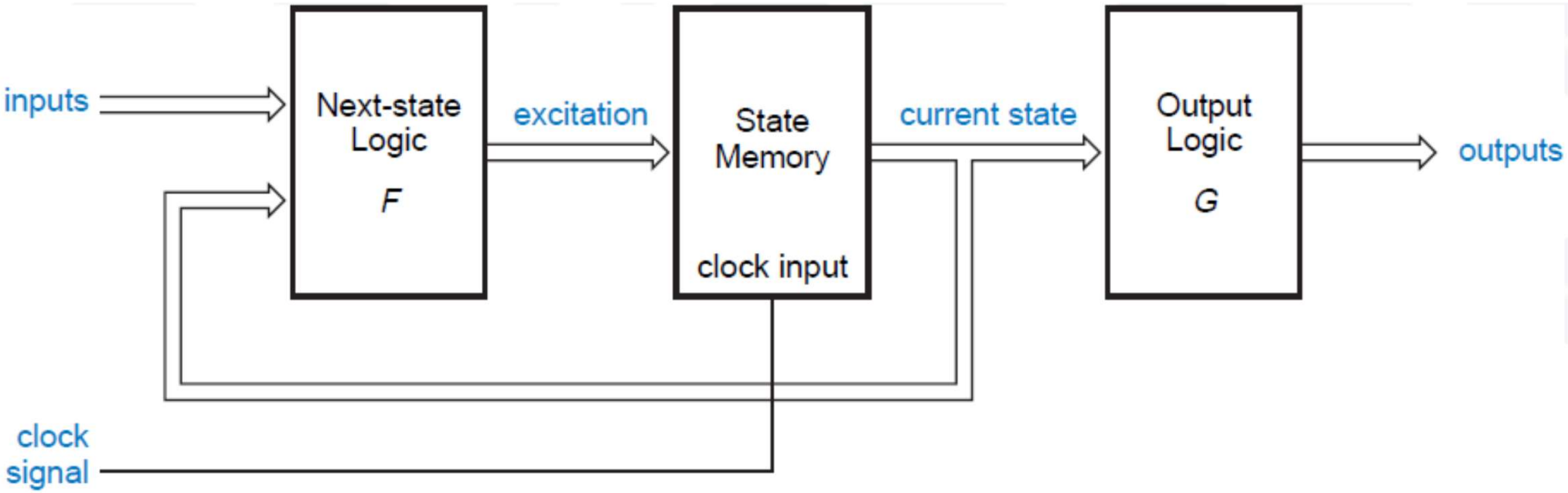


Máquina de Moore

Memoria de estado: n biestables 2^n estados posibles reloj

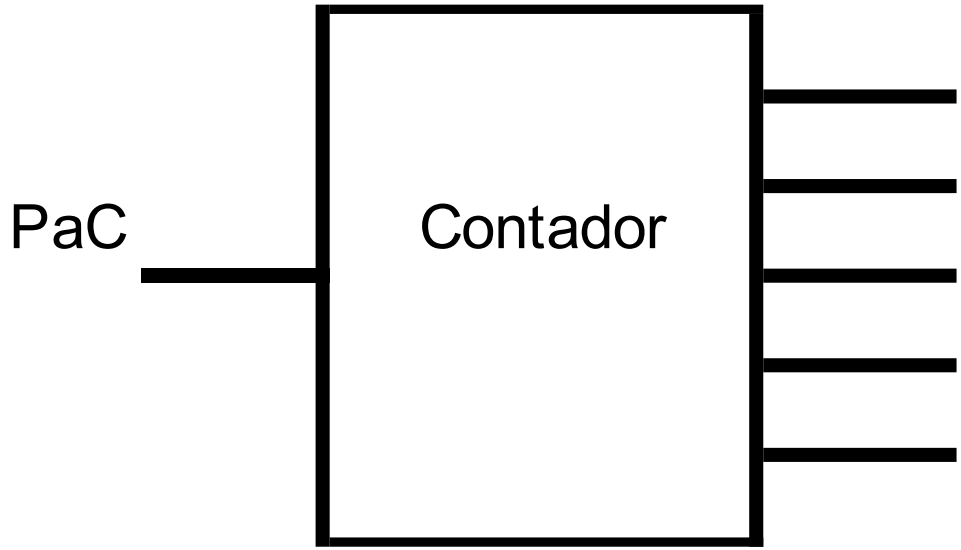
Lógica de próximo estado: F = f(entradas, estado actual)

Lógica de salida: G = g(estado actual)



Definiciones

Los **contadores** son **sistemas secuenciales** de memoria que **recuerdan** cuántos pulsos han sido aplicados a su única entrada.
La secuencia en que la información de cuenta aparece en la salida depende de la aplicación particular, acorde al diseño, y constituyen el **código de cuenta** del sistema.
En cada salida se generan secuencias repetitivas, son también llamados **secuenciadores**.

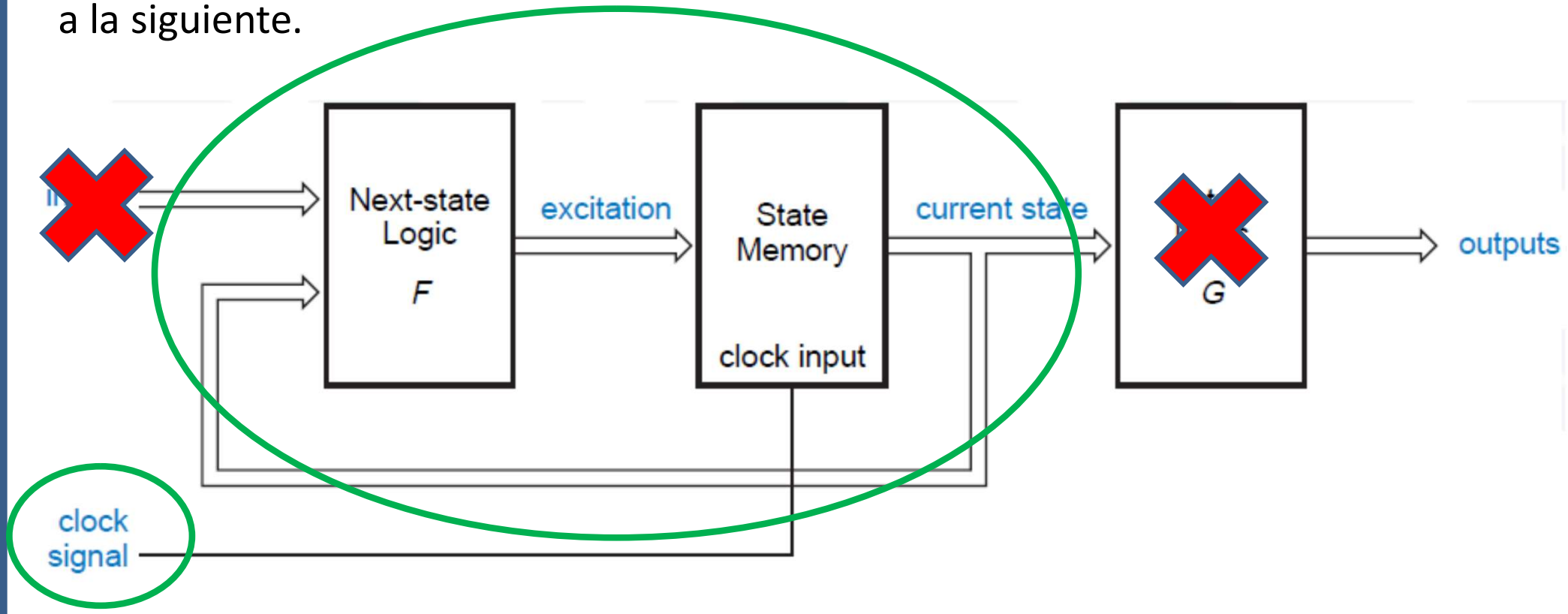


Definiciones

Un circuito **biestable** recuerda uno de dos estados posibles acorde con lo que ocurrió en sus entradas.

Un **contador** como **interconexión de n biestables** es un circuito secuencial de memoria capaz de recordar **hasta 2^n estados posibles**, es decir cada una de las 2^n combinaciones en sus salidas.

En el contador los pulsos a contar son aplicados a la entrada del circuito y por **cada pulso** en la entrada del mismo **hace avanzar** de una combinación de salida a la siguiente.

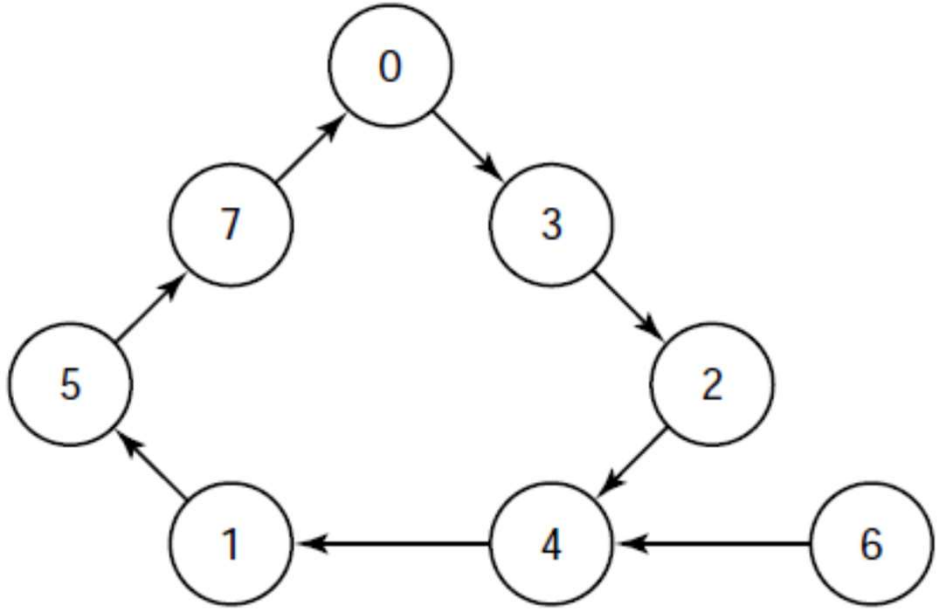


Definiciones

Estado es cada una de las **2^n combinaciones** que pueden formarse con n biestables y en la cual es factible que se encuentre el contador.

0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

3 biestables
8 estados posibles



M = 7

Módulo es la cantidad de pulsos que se deben aplicar a la entrada para que el contador vuelva al estado inicial luego de haber pasado por todos los estados para los cuales fue diseñado. Es decir que es la cantidad de estados que posee el código.

Definiciones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Código es el que se conforma con los valores obtenidos en la salida de los biestables. Depende de la forma de interconexión. Cada contador define finalmente un código de cuenta pero cada código podría ser implementado con distintas topologías circuitales.

10
01 **M = 3**
11
10

Biestables, se puede emplear cualquiera conocido. Sólo variará el tipo de lógica combinacional que se requerirá para cumplir con un código dado.

SR **JK** **T** **D**

Velocidad, depende del tipo de conexión realizada con los biestables.

Asincrónicos Sincrónicos

Definiciones

Sentido de la cuenta, se puede lograr que el conteo sea ascendente o descendente, normalmente mediante una entrada de control. También en ciertos circuitos se puede lograr desplazamientos a derecha o izquierda.

UP				**DOWN**
	00		00	
	01		11	
	10	**M = 4**	10	
	11		01	
	00		00	

Inicialización, es el estado que toma el sistema al ser energizado. Puede dejarse al azar o hacer que el contador vaya a un estado de salida particular desde el cual arranca su operación.

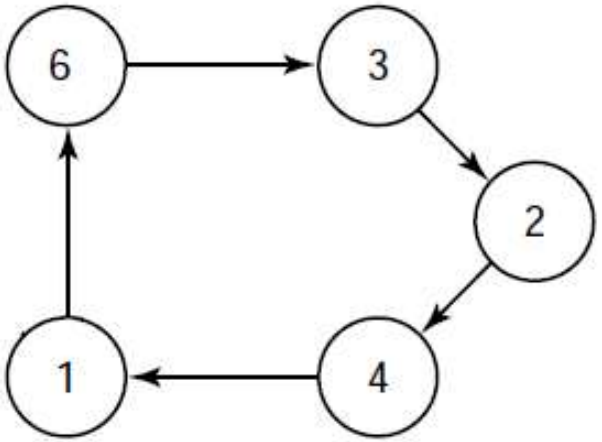
POR (Power On Reset)

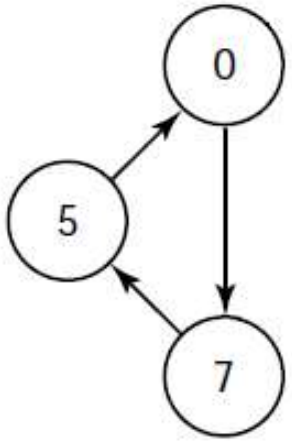
Definiciones

Seguridad, referido al hecho de evolucionar solo dentro del código prefijado o si por algún motivo se escapa del código regresará o no al código.

Si el código es menor que 2^n quedan estados no utilizados que podrían formar un **código secundario** o **secuencia prohibida**, y en caso de caer en ella el contador quedaría operando en la misma.

M = 5





M = 3
código
secundario

Decodificación de estados, utilizando circuitos para detectar determinados estados con fines específicos, por ejemplo regresar al estado inicial.

Clasificación

De acuerdo con la forma en que los **pulsos a contar (PaC)** ingresan al sistema se pueden clasificar en:

Sincrónicos: los pulsos a contar entran en los clocks de **todos** los biestables, todos ellos transicionan al "mismo tiempo".

Asincrónicos: **por lo menos uno** de los biestables no es controlado por los pulsos a contar, hay demoras en las transiciones.

También podemos distinguir según su **módulo**:

$M = 2^n$ todos los estados pertenecen al código

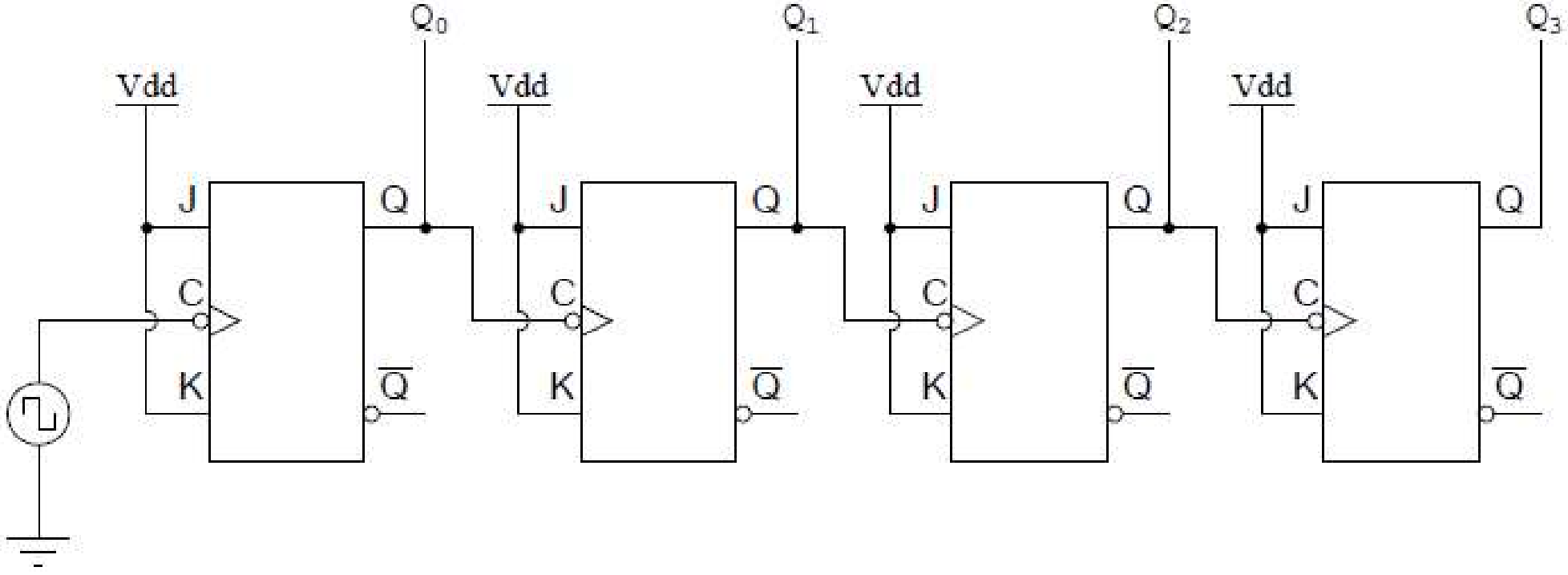
$M < 2^n$ estados sin usar (seguridad, decodificación, etc.)

donde n = cantidad de biestables

Contadores Asincrónicos de M = 2^n

Un biestable T, con T = 1, por cada pulso de Ck realizará un cambio en Q, por lo tanto la frecuencia de salida será **la mitad** de la frecuencia de entrada.
Usando T de flanco descendente tendremos un contador asincrónico ascendente (**UP counter**) de módulo **M = 2^n**. El código será **binario natural**.

T	Qn+1
0	Qn
1	not Qn



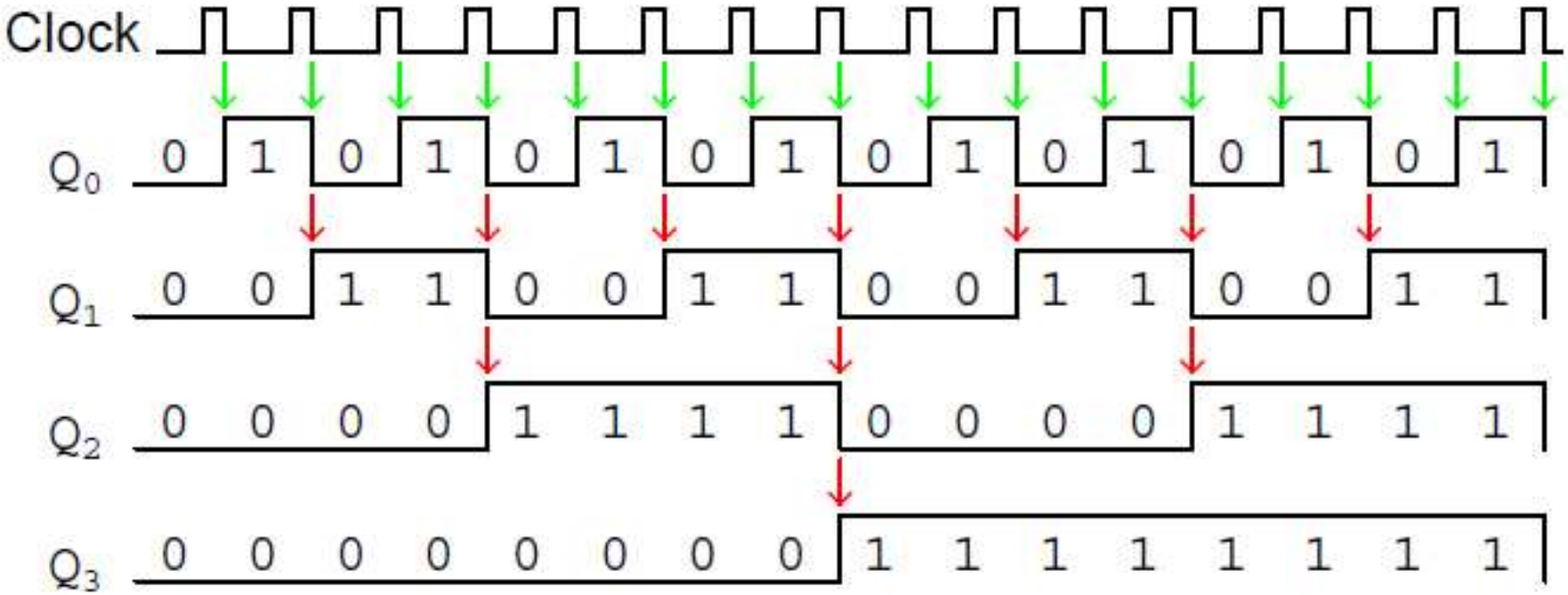
Contadores Asincrónicos de M = 2^n

Un contador asincrónico de Módulo M es un **divisor de frecuencia** por M, en este ejemplo M = 16. El código es binario natural.

T	Qn+1
0	Qn
1	not Qn

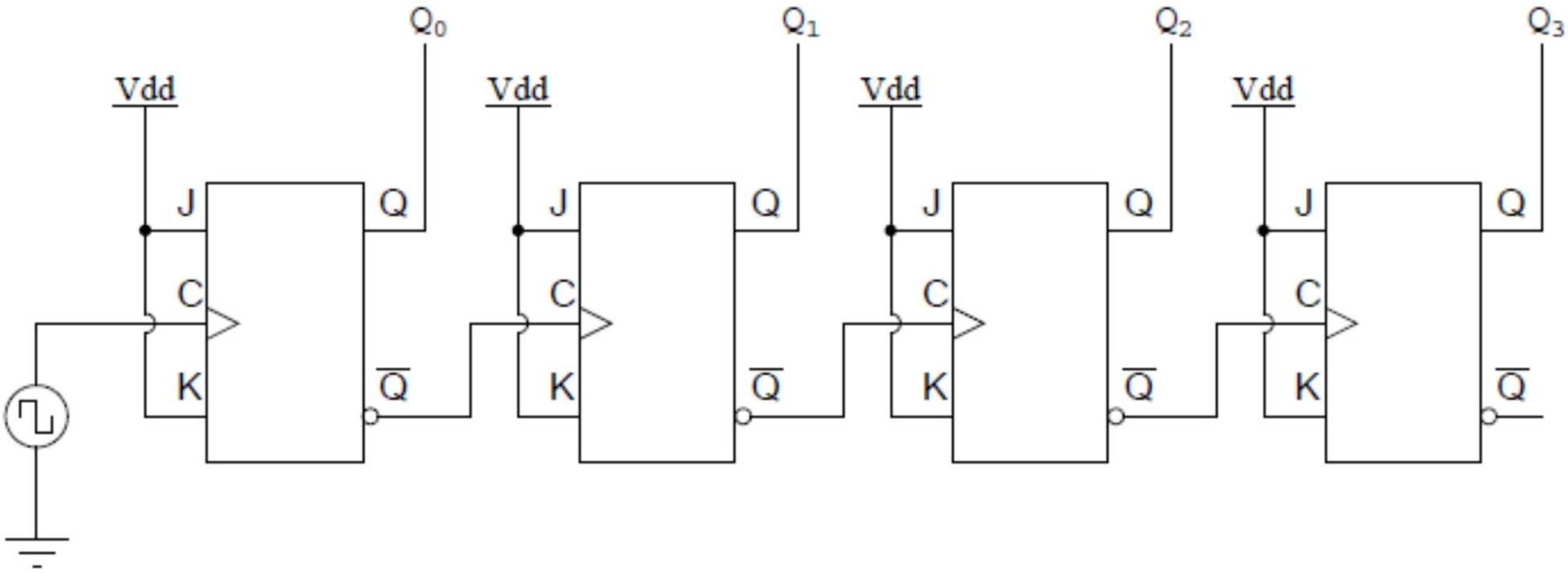
$Q_3Q_2Q_1Q_0$

0 0 0 0
0 0 0 1
0 0 1 0
0 0 1 1
0 1 0 0
0 1 0 1
0 1 1 0
0 1 1 1
1 0 0 0
1 0 0 1
1 0 1 0
1 0 1 1
1 1 0 0
1 1 0 1
1 1 1 0
1 1 1 1



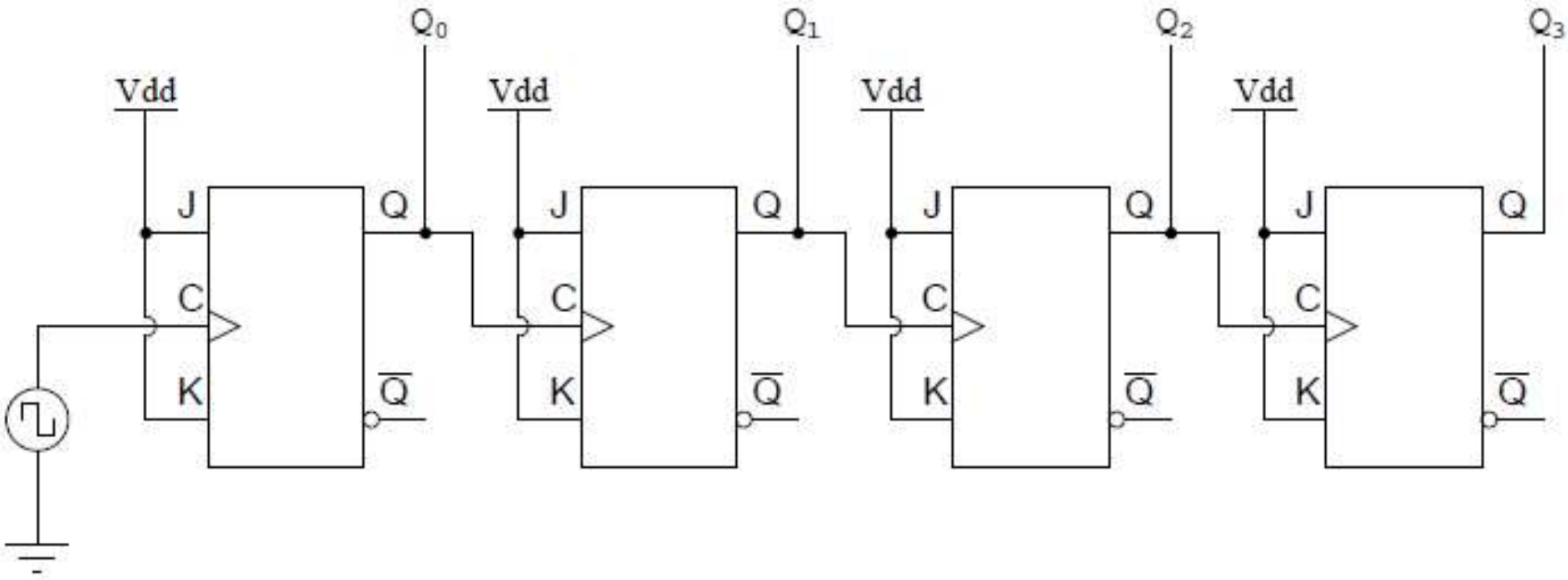
Contadores Asincrónicos de M = 2^n

Otra forma de obtener el mismo contador pero con biestables de **flanco ascendente**, en lugar de derivar desde Q lo hacemos desde not Q.



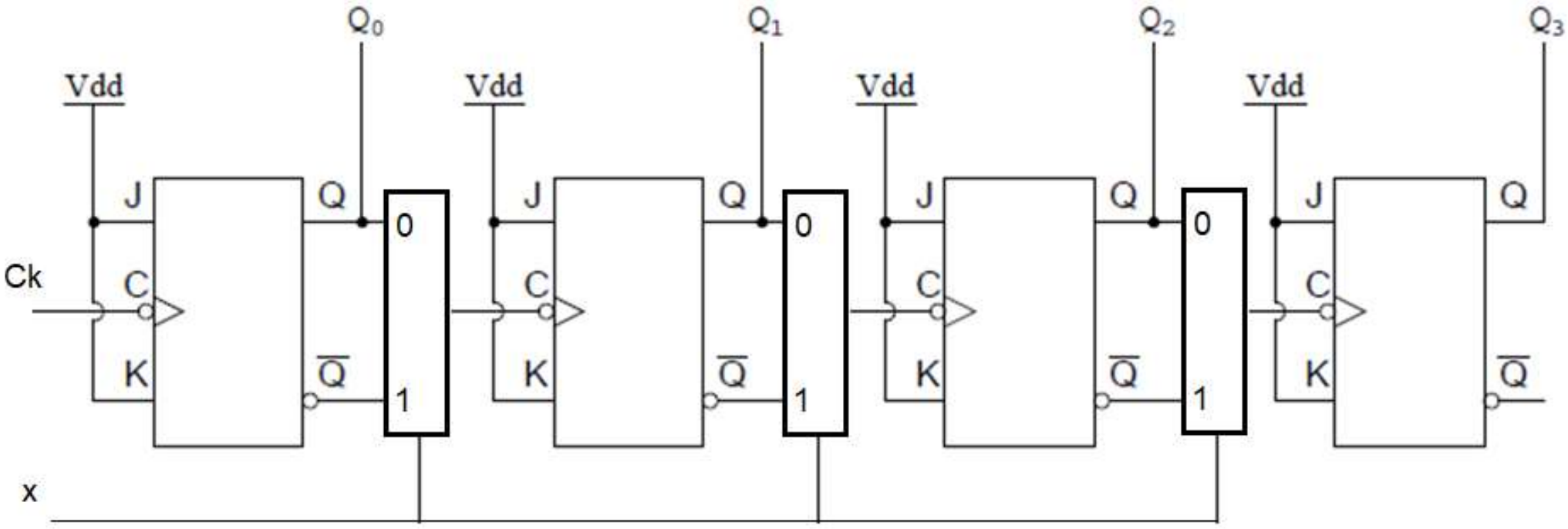
Contadores Asincrónicos de $M = 2^n$

Para obtener un contador asincrónico descendente (**DOWN counter**) de módulo **$M = 2^n$**, derivamos desde Q con biestables de flanco ascendente (o desde not Q con biestables de flanco descendente).



Contadores Asincrónicos Up/Down

Mediante una entrada adicional **x** y algunos **MUX** podemos obtener un contador ascendente / descendente (**UP / DOWN**) de módulo **M = 2^n**.

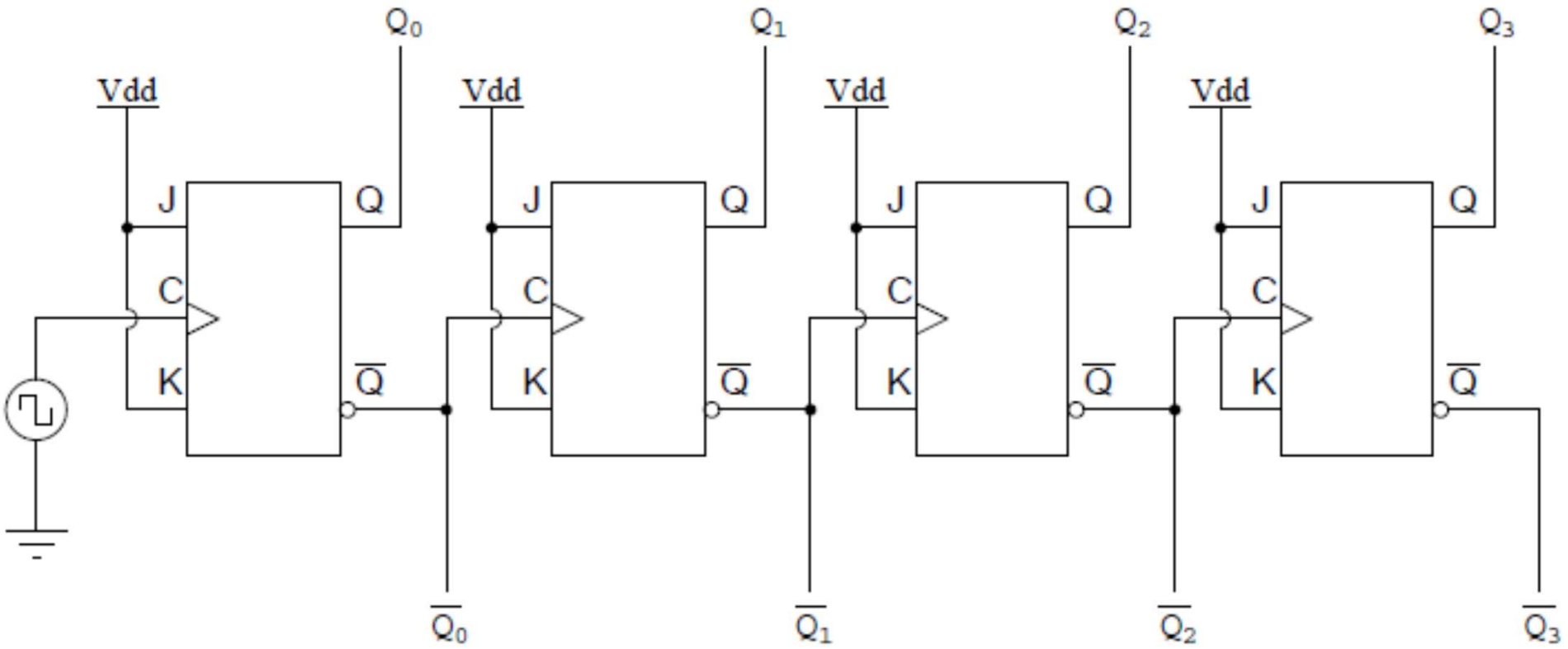


x = 0 **UP**

x = 1 **DOWN**

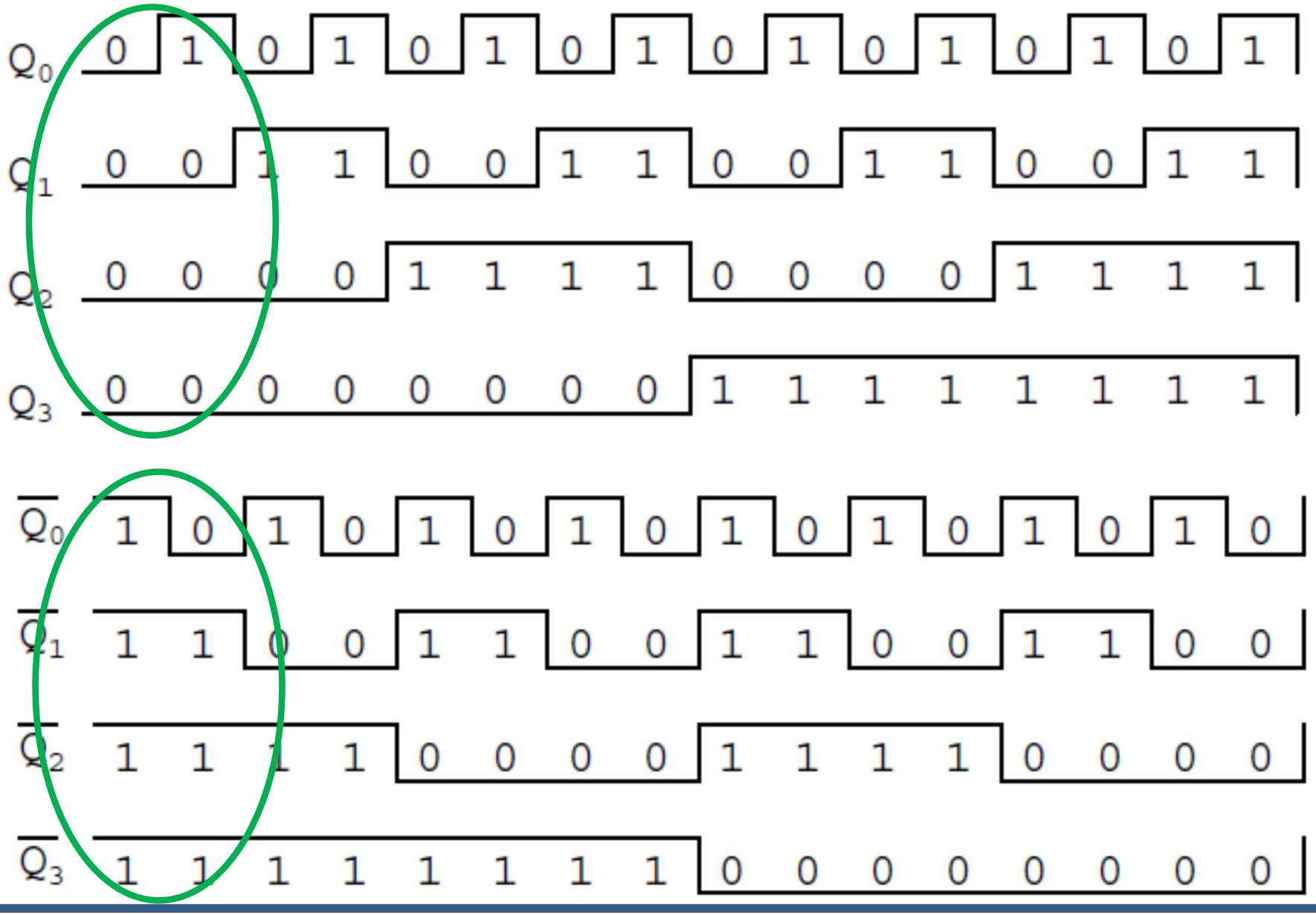
Contadores Asincrónicos Up/Down

También podemos obtener un contador **UP – DOWN** simultáneo. Tiene 2 salidas de n bits cada una.



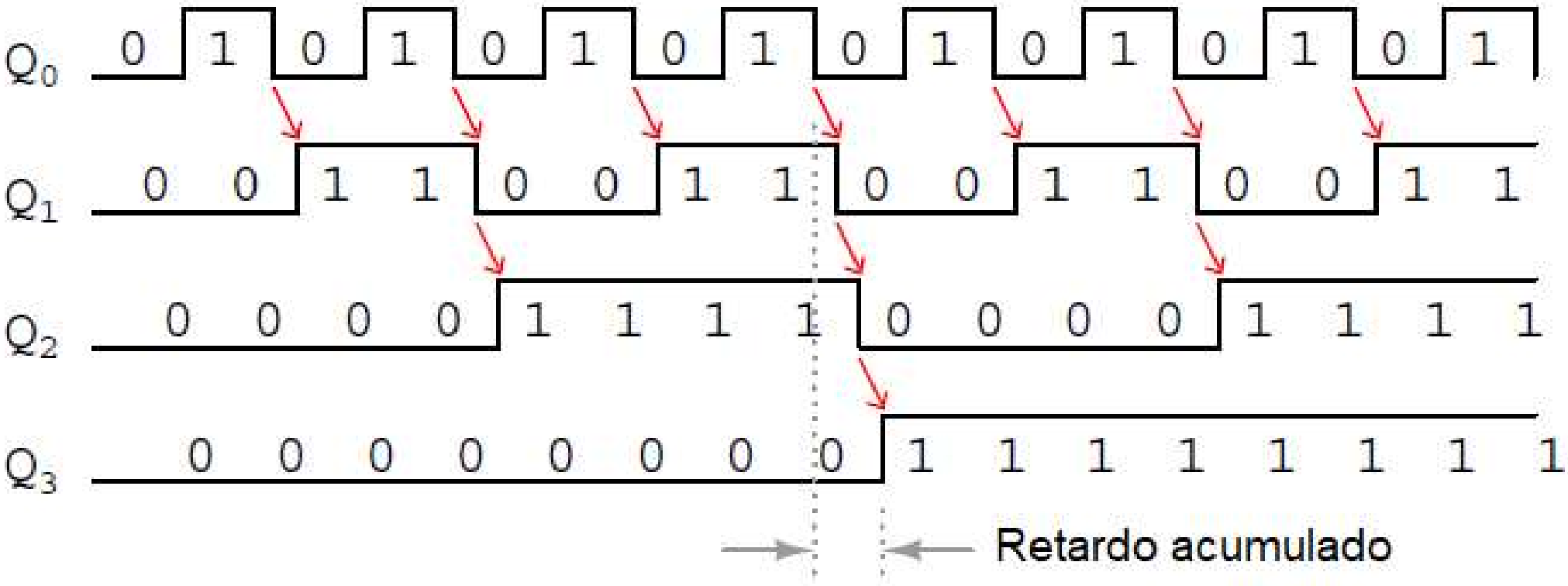
Contadores Asincrónicos Up/Down

El diagrama temporal será:



Contadores Asincrónicos – Retardos

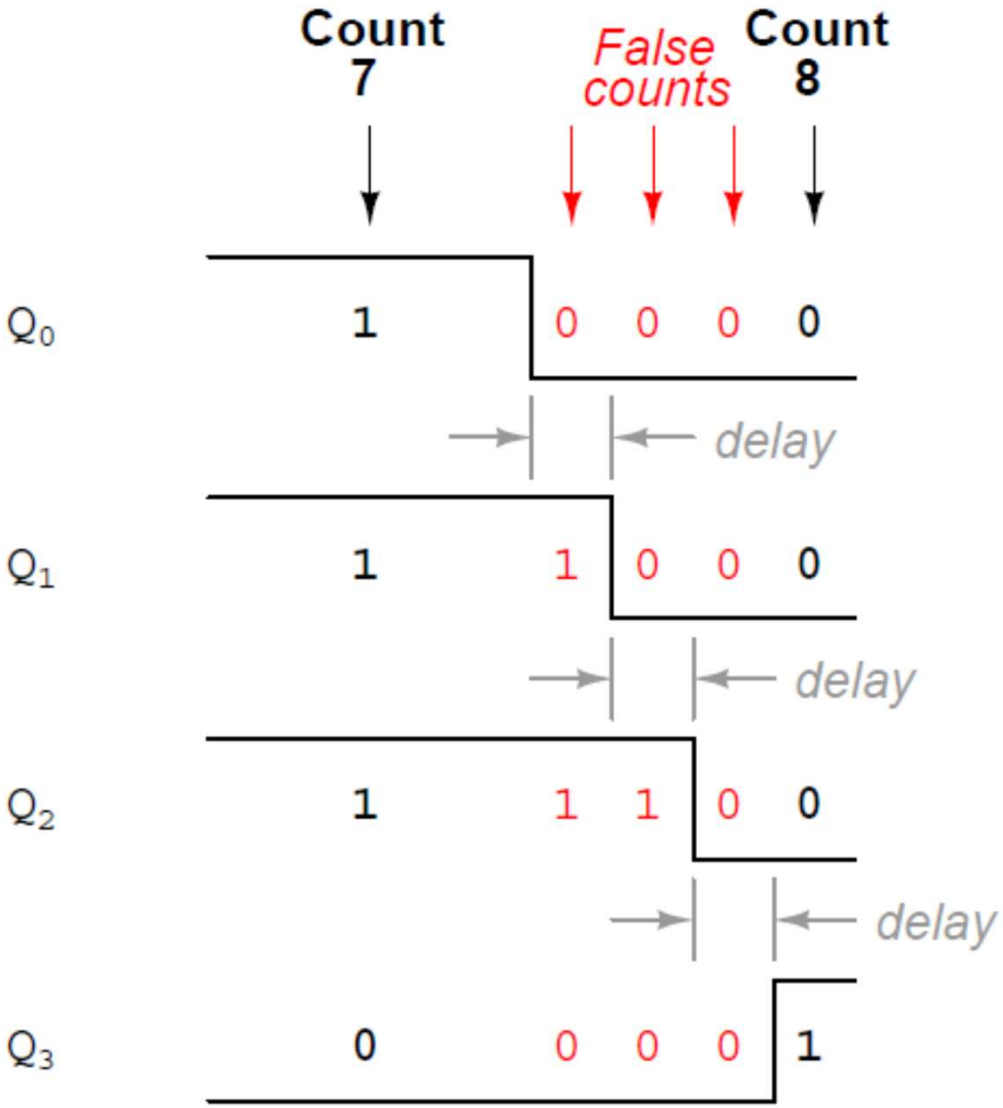
Todos los contadores mostrados hasta ahora comparten un **problema** común: el efecto **ripple o dominó**. Cuando varios bits cambian en una secuencia de conteo binario, no todos lo harán al mismo tiempo.



Si lo utilizo como **divisor** de frecuencia el límite lo dará **el 1er FF**, pero si necesito leer los estados (uso como **contador**), debo esperar a que cambien **los n FF**.

Contadores Asincrónicos – Retardos

Aparecerán estados **espurios** no deseados, que en caso de leer el estado en ese instante, la lectura será **errónea**.



Como **divisor** de frecuencia, el límite lo fija el 1er FF.

$$f_{max} = 1 / t_{pd}$$

Como **contador,** debo esperar a que cambien los n FF.

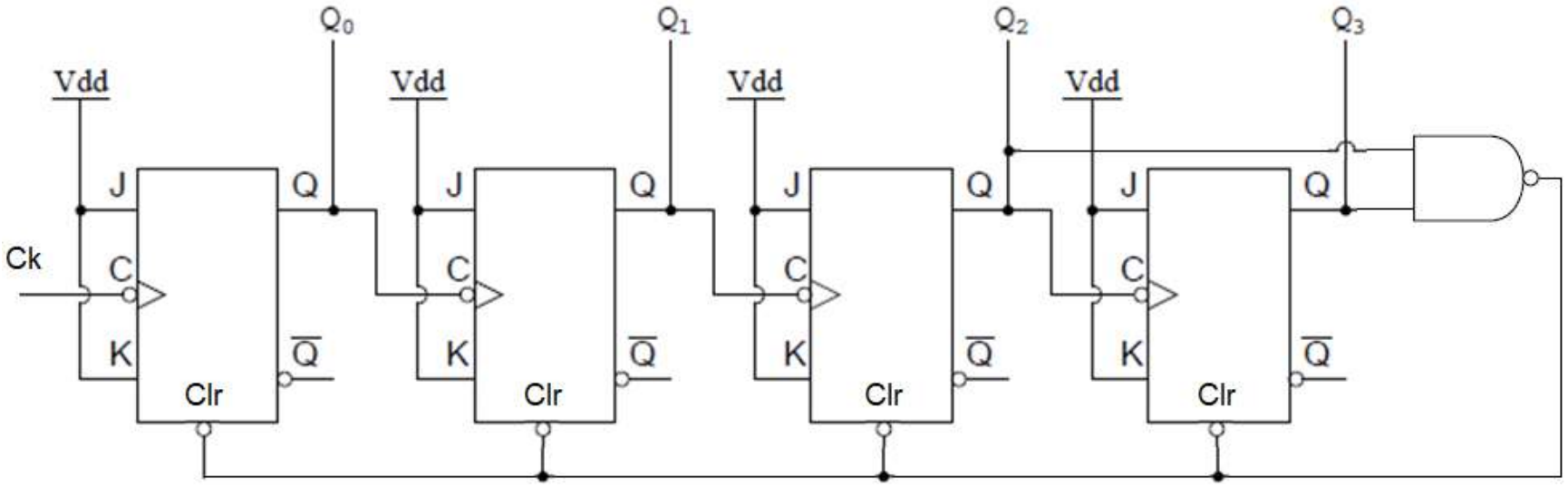
$$f_{max} = 1 / (n\, t_{pd} + t_L)$$

Contadores Asincrónicos de M < 2^n

Decodificando estados obtengo un contador asincrónico binario de **M < 2^n**.
Como ejemplo, decodificando el 12 binario tengo un contador de M = 12 (de 0 a 11 binario)

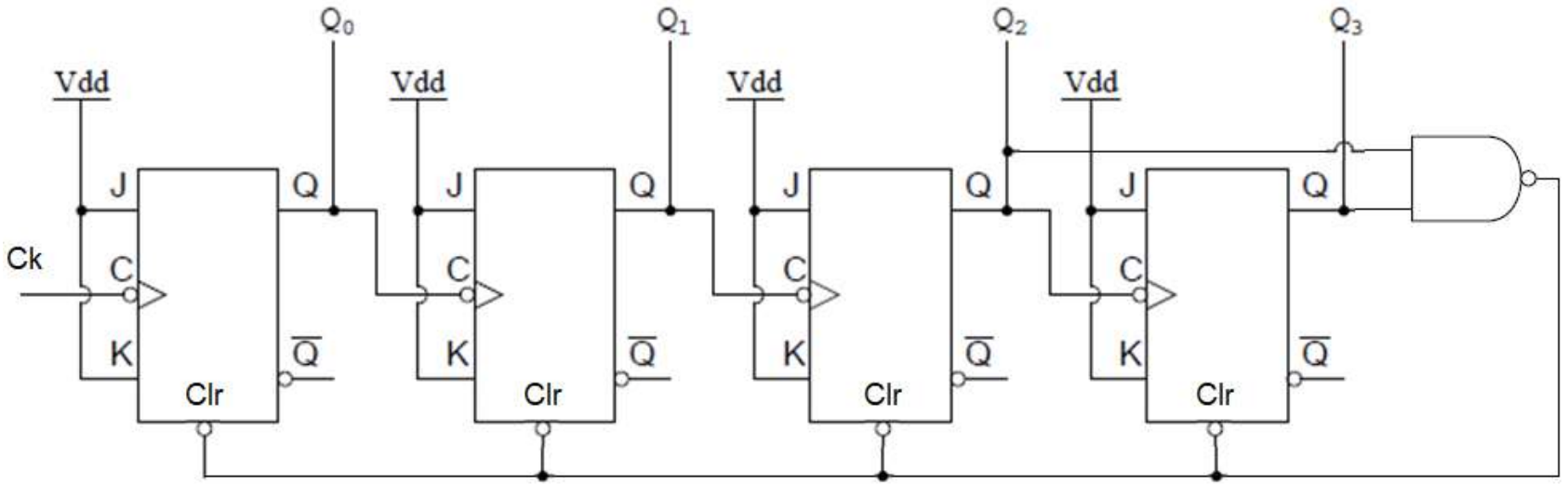
$Q_3Q_2Q_1Q_0$

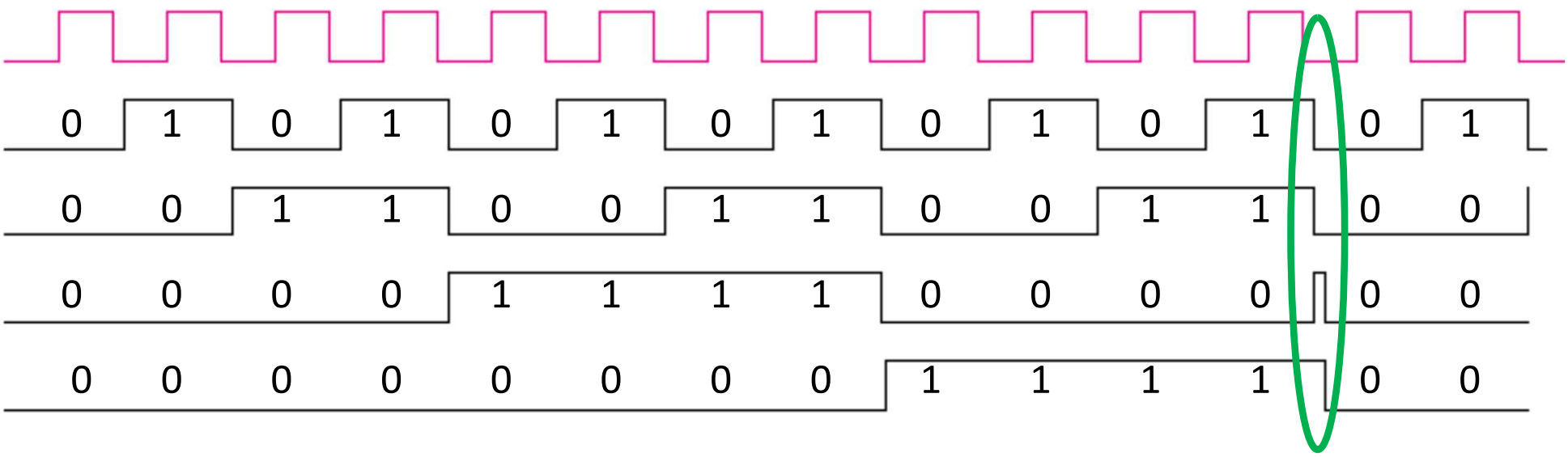
0 0 0 0
0 0 0 1
0 0 1 0
0 0 1 1
0 1 0 0
0 1 0 1
0 1 1 0
0 1 1 1
1 0 0 0
1 0 0 1
1 0 1 0
1 0 1 1
1 1 0 0
0 0 0 0



Contadores Asincrónicos de M < 2^n

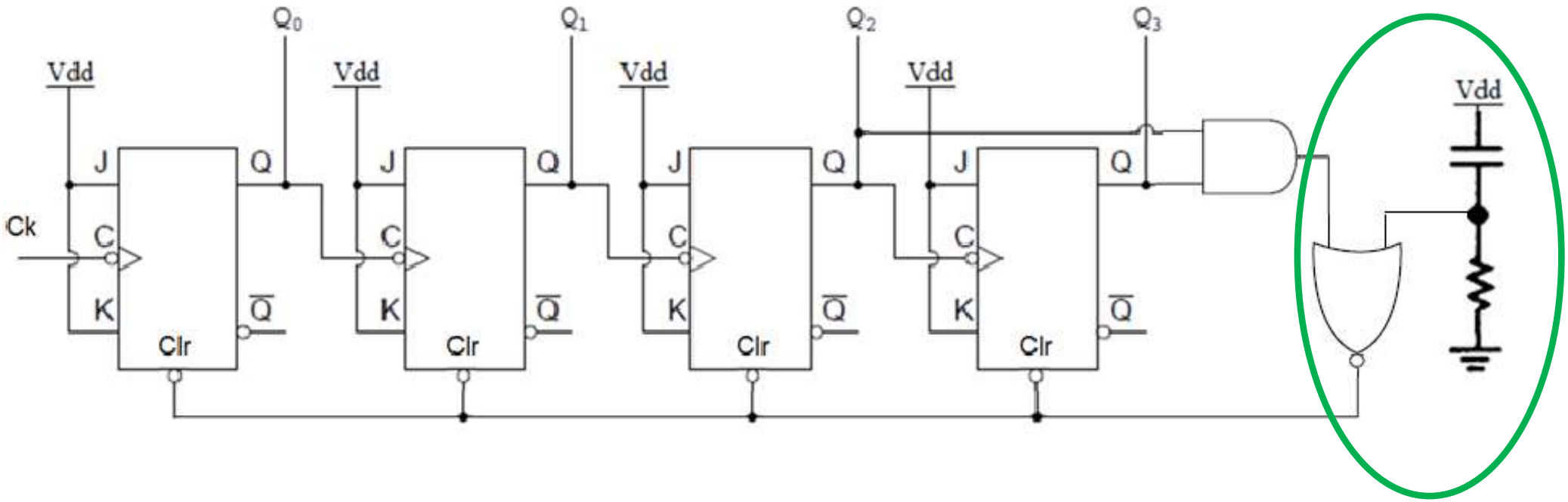
Diagrama de tiempos para el contador asincrónico de M = 12, vemos cómo actúa la NAND.





Contadores Asincrónicos – Inicialización

Aquí le hemos agregado un **POR** mediante un circuito RC y una compuerta OR (NOR ya que la NAND ahora es AND).
Se garantiza así que el contador **se inicialice** en el estado deseado (en este caso 0000).

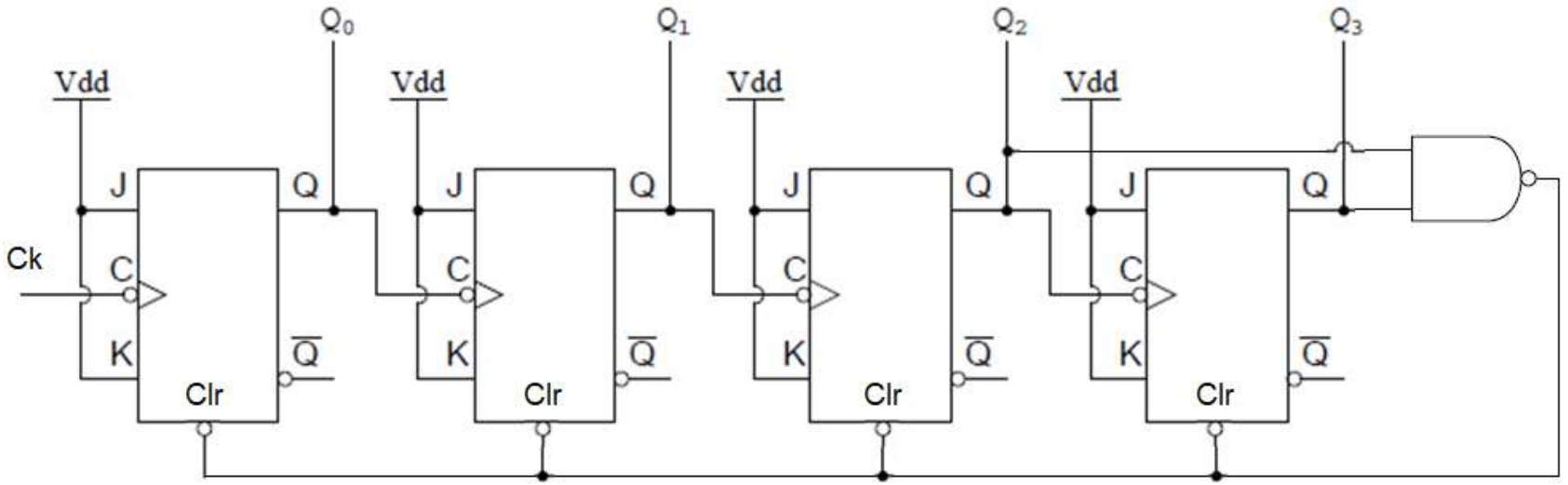


Análisis y Síntesis (Asincrónicos)

Estamos en condiciones de realizar ambas acciones:

Síntesis: Dado el módulo de un contador asincrónico de código binario natural (8421), de ser necesario decodifico el primer estado que no pertenece al código y actúo sobre los Clr de todos los biestables. Utilizo **n** biestables, con $\mathbf{2^{n-1} < M \leq 2^n}$.

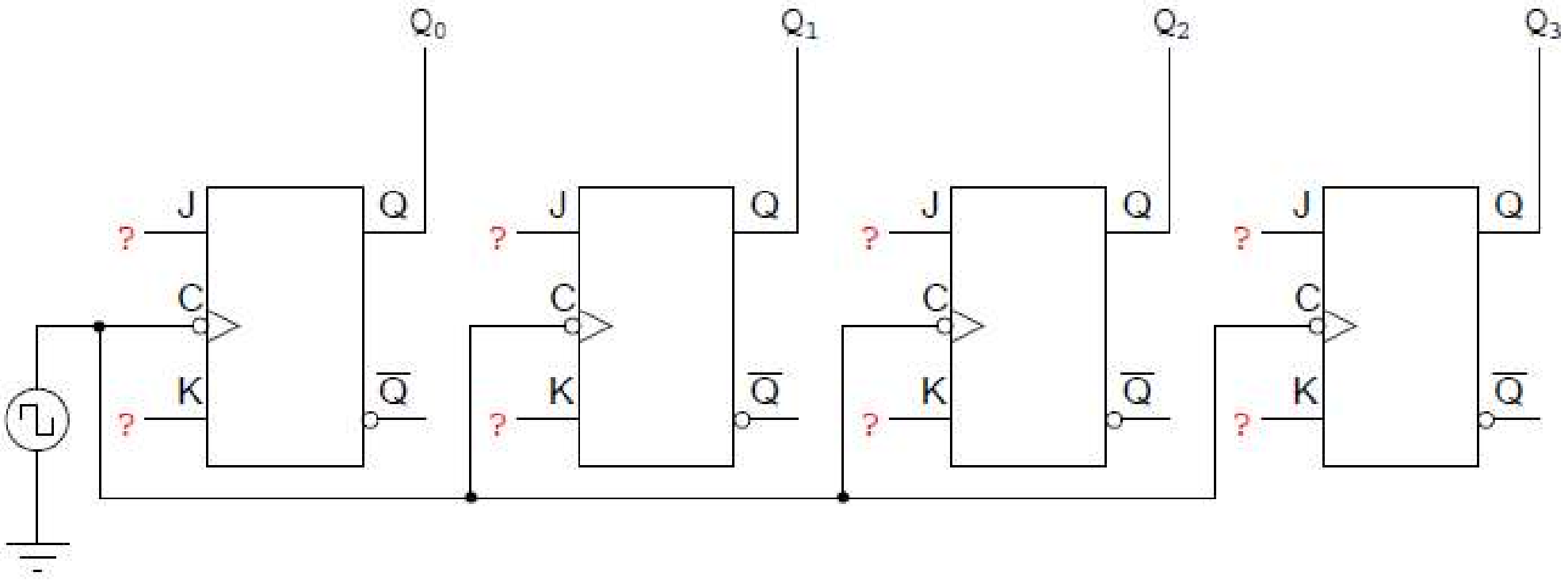
Análisis: Dado un circuito como el siguiente, obtener su módulo y su código (no necesariamente binario natural 8421). Plantear las **funciones de sus entradas** (J_i y K_i) y analizar sus transiciones. Obtener su **diagrama de estados**.



$Q_3Q_2Q_1Q_0$	
0 0 0 0	
0 0 0 1	
0 0 1 0	
0 0 1 1	
0 1 0 0	
0 1 0 1	
0 1 1 0	
0 1 1 1	
1 0 0 0	
1 0 0 1	
1 0 1 0	
1 0 1 1	1 1 0 0
0 0 0 0	

Contadores Sincrónicos – Síntesis

En los contadores sincrónicos **TODOS** los Ck de los biestables reciben los **PaC**, por lo tanto todos cambiarán **sincrónicamente**. Mayor **velocidad** de operación. Según el código deseado debemos condicionar a los biestables que deben cambiar.
El **diseño** consiste en obtener las funciones que condicionan las entradas de los FF determinando cuales deben cambiar. Podemos utilizar cualquier tipo de biestable. **POR** similar que en asincrónicos.



Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

<u>Método de las Transiciones</u>: Diseñaremos un contador binario natural de módulo M = 8 utilizando **JK**. Usaremos 3 biestables. A partir del código construimos la **TV**:

	$Q_3Q_2Q_1$
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111
0	000

	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	1	0	0	1	1
3	0	1	1	1	0	0
4	1	0	0	1	0	1
5	1	0	1	1	1	0
6	1	1	0	1	1	1
7	1	1	1	0	0	0

Contadores Sincrónicos – Síntesis – MT

Mediante la **TT** del JK analizaremos cada una de las transiciones Q_i^n a Q_i^{n+1}.

Qn	Qn+1	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

La **TV** quedará como sigue:

	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	0	1	0	x	0	x	1	x
1	0	0	1	0	1	0	0	x	1	x	x	1
2	0	1	0	0	1	1	0	x	x	0	1	x
3	0	1	1	1	0	0	1	x	x	1	x	1
4	1	0	0	1	0	1	x	0	0	x	1	x
5	1	0	1	1	1	0	x	0	1	x	x	1
6	1	1	0	1	1	1	x	0	x	0	1	x
7	1	1	1	0	0	0	x	1	x	1	x	1

Contadores Sincrónicos – Síntesis – MT

Planteamos los 6 mapas de los J_i y K_i, la solución será:

Q_2 \ Q_1Q_0	00	01	11	10
0	0	0	1	0
1	x	x	x	x

$$J_2 = Q_1\,Q_0$$

Q_2 \ Q_1Q_0	00	01	11	10
0	x	x	x	x
1	0	0	1	0

$$K_2 = Q_1\,Q_0$$

Q_2 \ Q_1Q_0	00	01	11	10
0	0	1	x	x
1	0	1	x	x

$$J_1 = Q_0$$

Q_2 \ Q_1Q_0	00	01	11	10
0	x	x	1	0
1	x	x	1	0

$$K_1 = Q_0$$

Q_2 \ Q_1Q_0	00	01	11	10
0	1	x	x	1
1	1	x	x	1

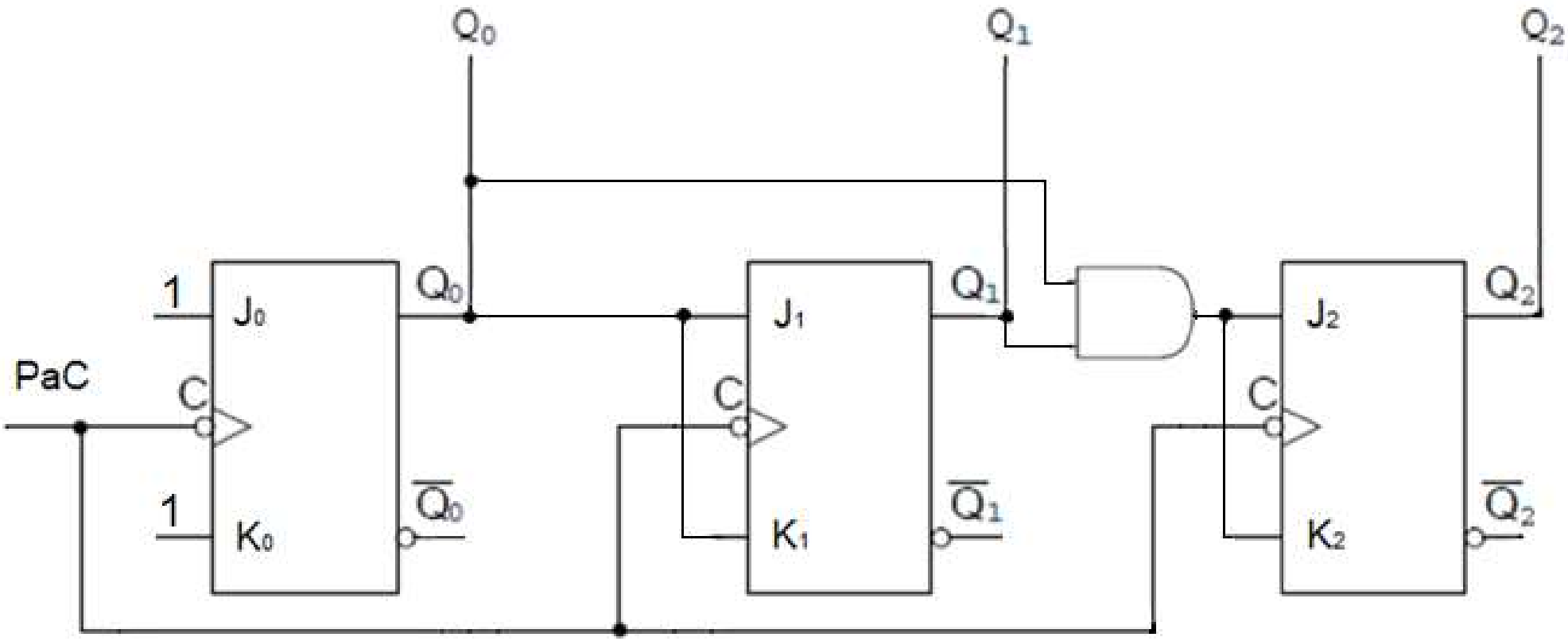
$$J_0 = 1$$

Q_2 \ Q_1Q_0	00	01	11	10
0	x	1	1	x
1	x	1	1	x

$$K_0 = 1$$

Contadores Sincrónicos – Síntesis – MT

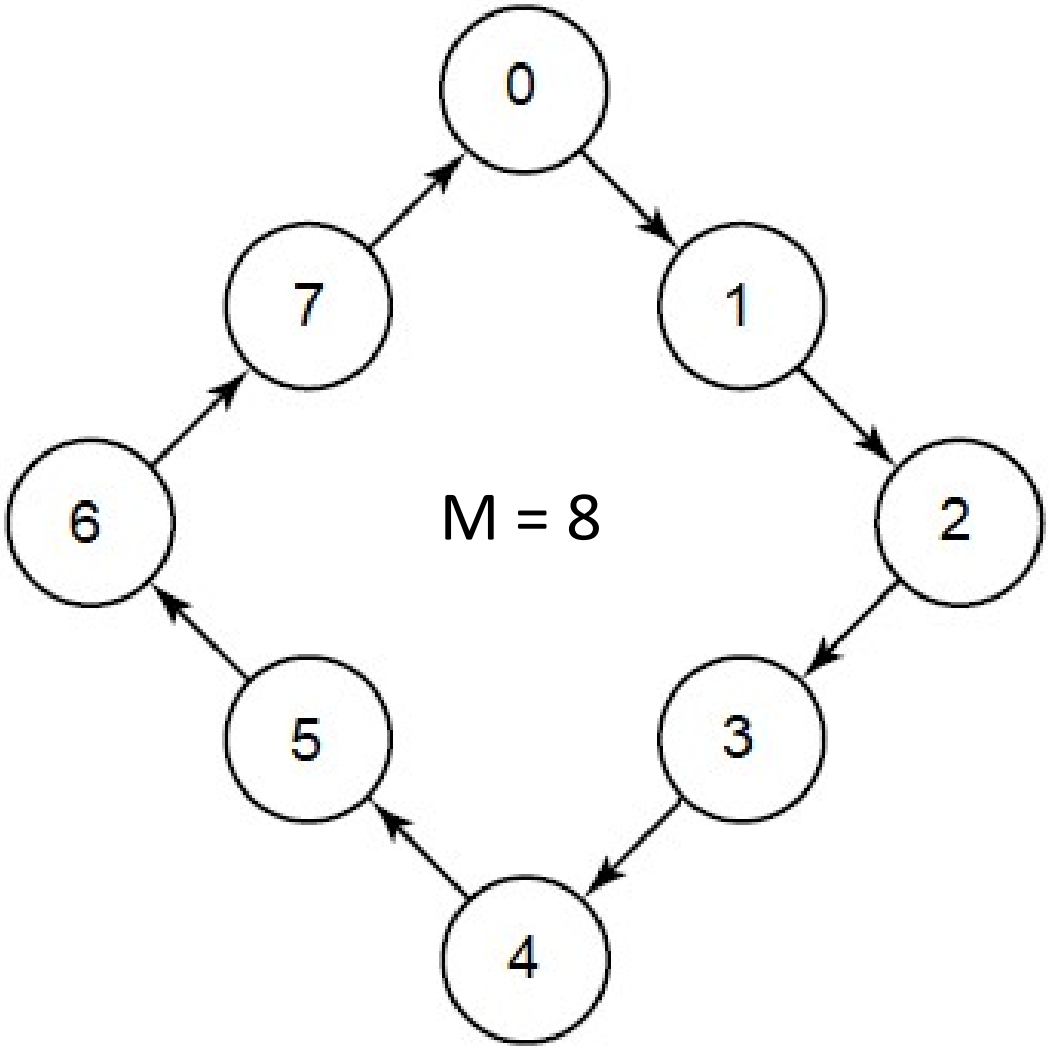
El circuito quedará:



Contadores Sincrónicos – Síntesis – MT

El diagrama de los 8 estados será:

	$Q_3Q_2Q_1$
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111
0	000



Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ahora nuevamente por el **Método de las Transiciones** pero con biestables **D**. Mediante la **TT** del D analizaremos las transiciones Q_i^n a Q_i^{n+1}. Teniendo en cuenta que $Q^{n+1} = D$, la **TV** quedará como sigue:

Qn	Qn+1	D
0	0	0
0	1	1
1	0	0
1	1	1

	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	D_2	D_1	D_0
0	0	0	0	0	0	1	0	0	1
1	0	0	1	0	1	0	0	1	0
2	0	1	0	0	1	1	0	1	1
3	0	1	1	1	0	0	1	0	0
4	1	0	0	1	0	1	1	0	1
5	1	0	1	1	1	0	1	1	0
6	1	1	0	1	1	1	1	1	1
7	1	1	1	0	0	0	0	0	0

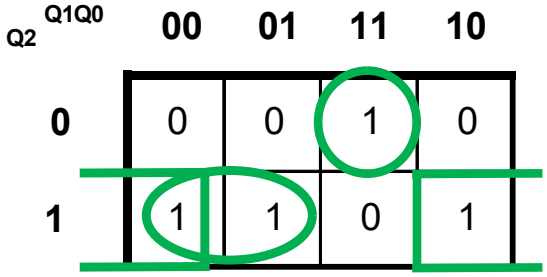
Debo resolver los 3 mapas K.

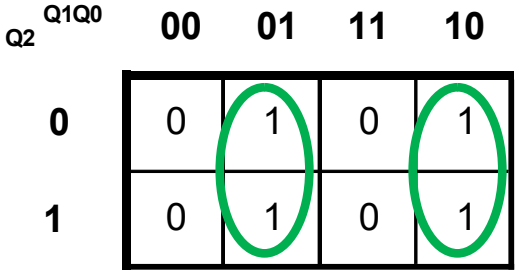
Ventaja: 3 mapas (D) vs 6 (JK).

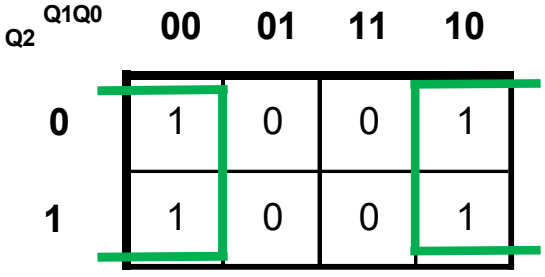
Desventaja: generalmente funciones más complejas (D).

Contadores Sincrónicos – Síntesis – MT

Planteamos 3 mapas K de los D_i y obtenemos las soluciones:







$$D_2 = Q_2\,\overline{Q}_1 + Q_2\,\overline{Q}_0 + \overline{Q}_2\,Q_1\,Q_0$$

$$D_1 = \overline{Q}_1\,Q_0 + Q_1\,\overline{Q}_0$$

$$D_0 = \overline{Q}_0$$

Como vemos, son funciones un poco más complejas que las obtenidas con JK (son 7 compuertas contra solo una).

Contadores Sincrónicos – Síntesis – MEC

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Método de la Ecuación Característica: Diseñaremos el mismo contador binario natural de M = 8 utilizando **JK**.

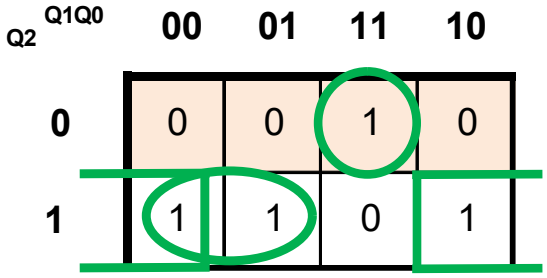
Tenemos la misma **TV** que antes:

	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	1
1	0	0	1	0	1	0
2	0	1	0	0	1	1
3	0	1	1	1	0	0
4	1	0	0	1	0	1
5	1	0	1	1	1	0
6	1	1	0	1	1	1
7	1	1	1	0	0	0

Contadores Sincrónicos – Síntesis – MEC

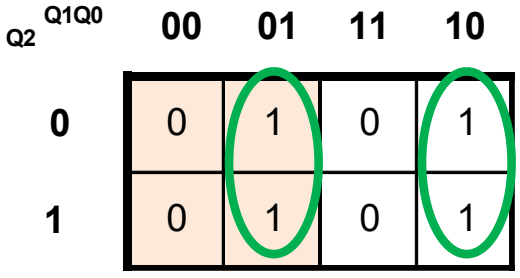
Planteamos 3 mapas K de los Q_i^{n+1} y los resolvemos en función de la ecuación característica del JK:

$$Q_i^{n+1} = J_i \, \overline{Q}_i + \overline{K}_i \, Q_i$$



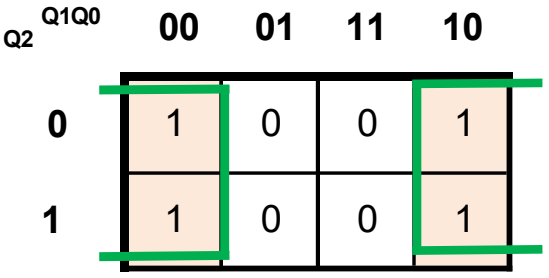
$$Q_2^{n+1} = Q_1 \, Q_0 \, \overline{Q}_2 + (\overline{Q}_1 + \overline{Q}_0) \, Q_2$$

$$J_2 = Q_1 \, Q_0 \qquad K_2 = Q_1 \, Q_0$$



$$Q_1^{n+1} = Q_0 \, \overline{Q}_1 + \overline{Q}_0 \, Q_1$$

$$J_1 = Q_0 \qquad K_1 = Q_0$$

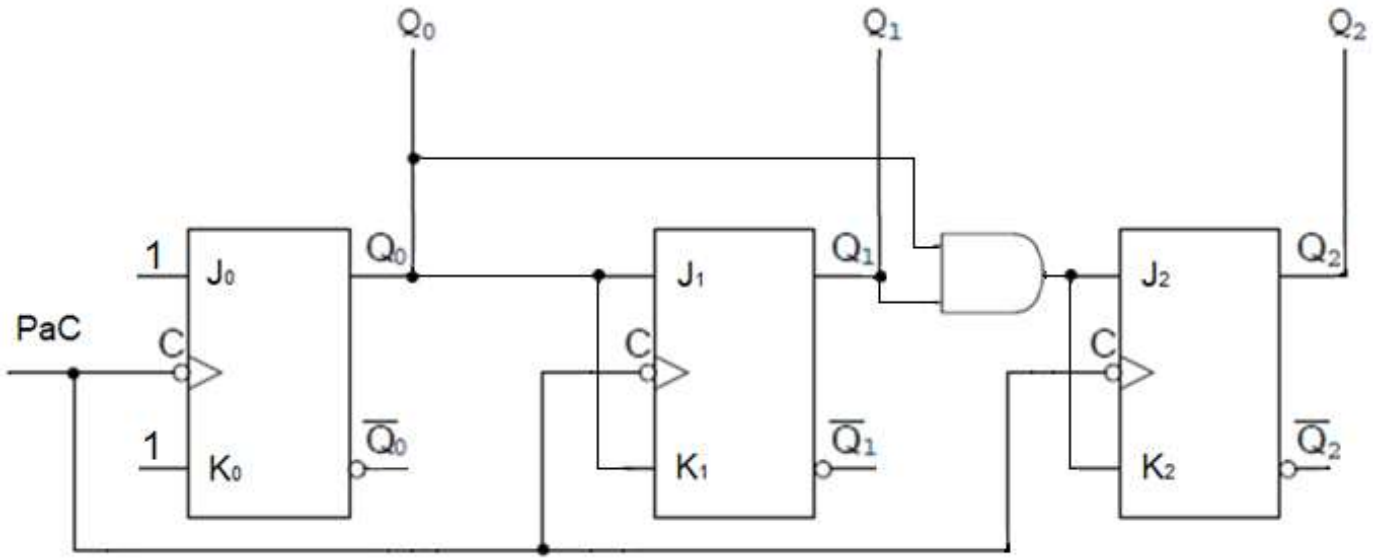


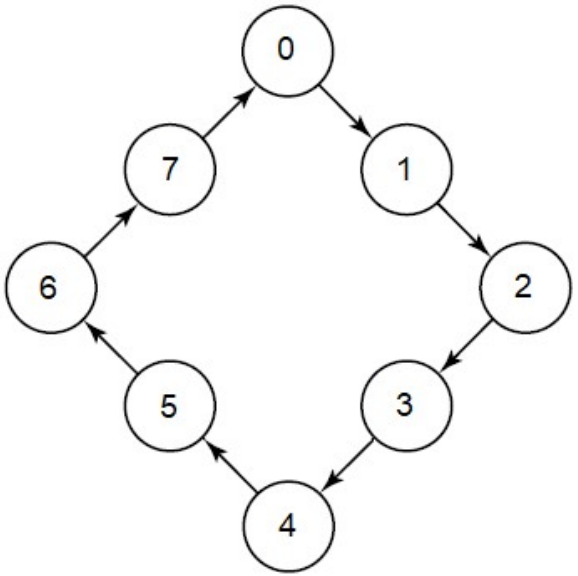
$$Q_0^{n+1} = 1 \, \overline{Q}_0 + 0 \, Q_0$$

$$J_0 = 1 \qquad K_0 = 1$$

Contadores Sincrónicos – Síntesis – MEC

El circuito y el diagrama de estados son los mismos que obtuvimos por el método de las Transiciones:





Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

<u>Método de las Transiciones</u>: Otro ejemplo más complejo. Diseñaremos un contador BCD Gray XS-3 utilizando **JK**. Usaremos 4 biestables. A partir del código Gray construimos la **TV**:

0	0 0 1 0
1	0 1 1 0
2	0 1 1 1
3	0 1 0 1
4	0 1 0 0
5	1 1 0 0
6	1 1 0 1
7	1 1 1 1
8	1 1 1 0
9	1 0 1 0

AB \ CD	00	01	11	10
00				0
01	4	3	2	1
11	5	6	7	8
10				9

	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	x	x	x	x
1	0	0	0	1	x	x	x	x
2	0	0	1	0	0	1	1	0
3	0	0	1	1	x	x	x	x
4	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0
6	0	1	1	0	0	1	1	1
7	0	1	1	1	0	1	0	1
8	1	0	0	0	x	x	x	x
9	1	0	0	1	x	x	x	x
10	1	0	1	0	0	0	1	0
11	1	0	1	1	x	x	x	x
12	1	1	0	0	1	1	0	1
13	1	1	0	1	1	1	1	1
14	1	1	1	0	1	0	1	0
15	1	1	1	1	1	1	1	0

Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Mediante la **TT** del JK analizaremos cada una de las transiciones Q_i^n a Q_i^{n+1}.
La **TV** quedará como sigue:

Qn	Qn+1	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	J_3	K_3	J_2	K_2	J_1	K_1	J_0	K_0
0	0	0	0	0	x	x	x	x	x	x	x	x	x	x	x	x
1	0	0	0	1	x	x	x	x	x	x	x	x	x	x	x	x
2	0	0	1	0	0	1	1	0	0	x	1	x	x	0	0	x
3	0	0	1	1	x	x	x	x	x	x	x	x	x	x	x	x
4	0	1	0	0	1	1	0	0	1	x	x	0	0	x	0	x
5	0	1	0	1	0	1	0	0	0	x	x	0	0	x	x	1
6	0	1	1	0	0	1	1	1	0	x	x	0	x	0	1	x
7	0	1	1	1	0	1	0	1	0	x	x	0	x	1	x	0
8	1	0	0	0	x	x	x	x	x	x	x	x	x	x	x	x
9	1	0	0	1	x	x	x	x	x	x	x	x	x	x	x	x
10	1	0	1	0	0	0	1	0	x	1	0	x	x	0	0	x
11	1	0	1	1	x	x	x	x	x	x	x	x	x	x	x	x
12	1	1	0	0	1	1	0	1	x	0	x	0	0	x	1	x
13	1	1	0	1	1	1	1	1	x	0	x	0	1	x	x	0
14	1	1	1	0	1	0	1	0	x	0	x	1	x	0	0	x
15	1	1	1	1	1	1	1	0	x	0	x	0	x	0	x	1

Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Planteamos los 8 mapas de los J_i y K_i, la solución será:

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	1	0	0	0
11	x	x	x	x
10	x	x	x	x

$$J_3 = \overline{Q}_1 \overline{Q}_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	x
01	x	x	x	x
11	0	0	0	0
10	x	x	x	1

$$K_3 = \overline{Q}_2$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	1
01	x	x	x	x
11	x	x	x	x
10	x	x	x	0

$$J_2 = \overline{Q}_3$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	x
01	0	0	0	0
11	0	0	0	1
10	x	x	x	x

$$K_2 = Q_3 Q_1 \overline{Q}_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	x
01	0	0	x	x
11	0	1	x	x
10	x	x	x	x

$$J_1 = Q_3 Q_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	x	x	1	0
11	x	x	0	0
10	x	x	x	0

$$K_1 = \overline{Q}_3 Q_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	0	x	x	1
11	1	x	x	0
10	x	x	x	0

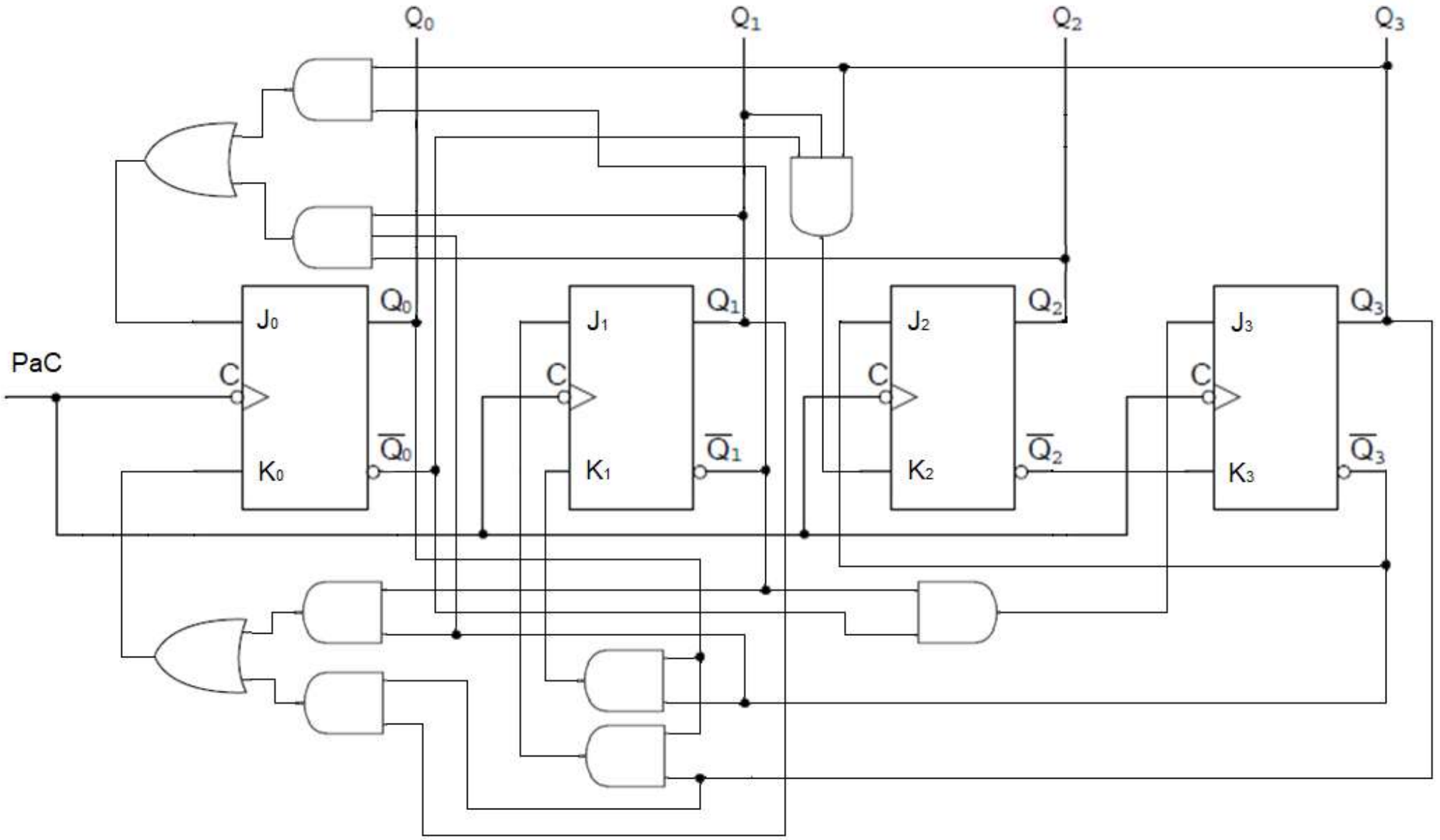
$$J_0 = \overline{Q}_3 Q_2 Q_1 + Q_3 \overline{Q}_1$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	x
01	x	1	0	x
11	x	0	1	x
10	x	x	x	x

$$K_0 = \overline{Q}_3 \overline{Q}_1 + Q_3 Q_1$$

Contadores Sincrónicos – Síntesis – MT

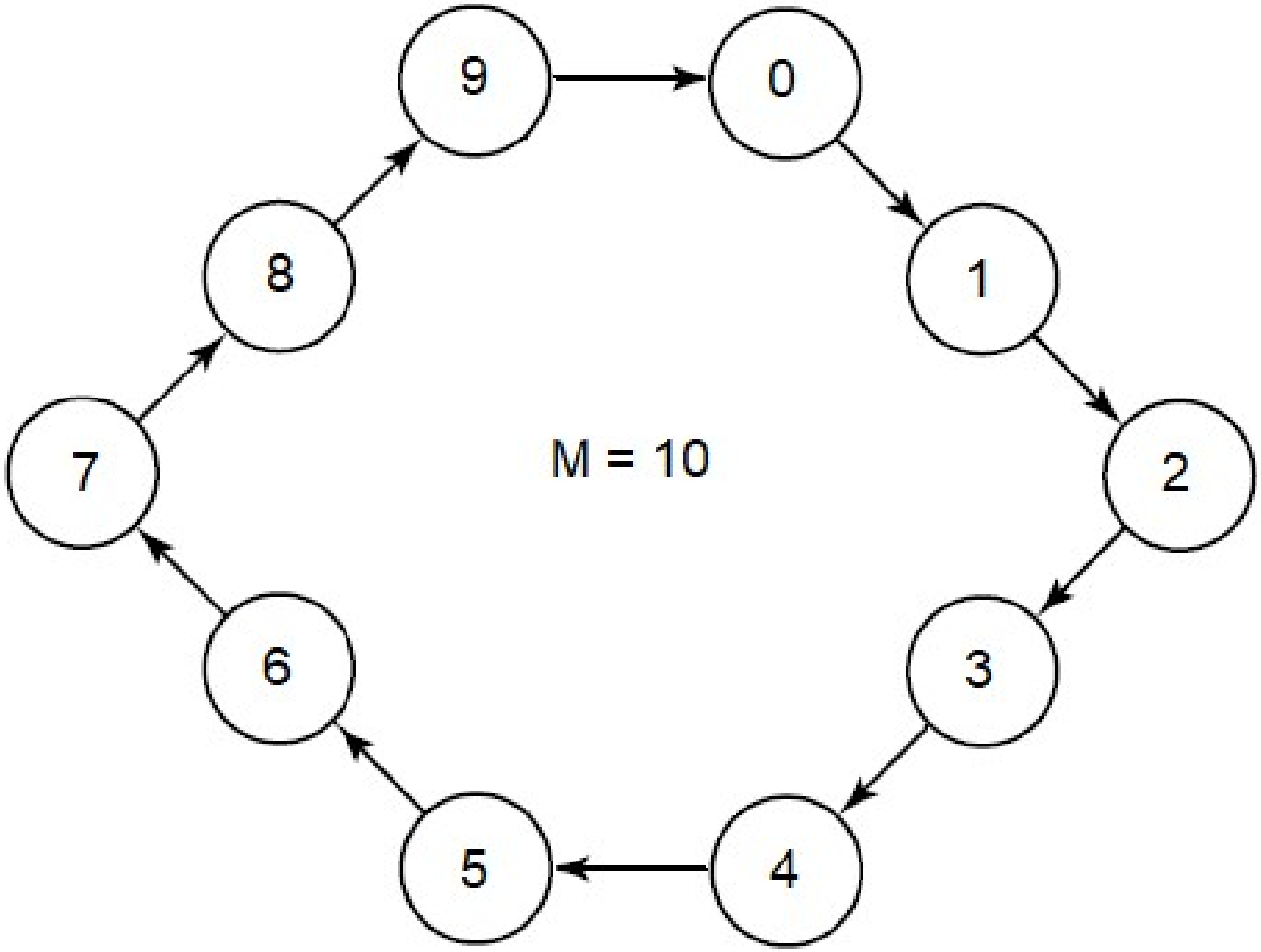
El circuito quedará:



Contadores Sincrónicos – Síntesis – MT

El diagrama de los 10 estados quedará:

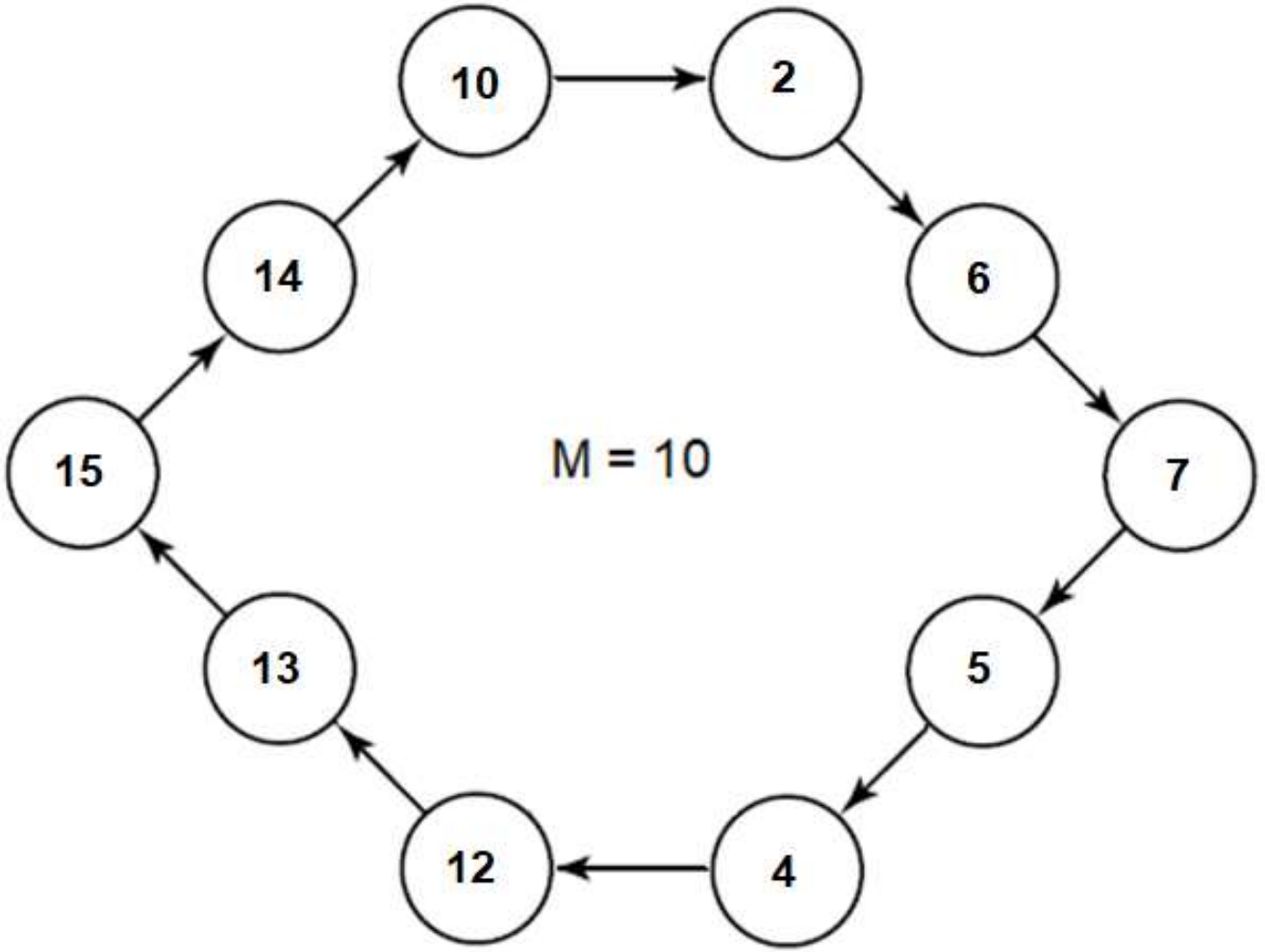
0	0	0	1	0
1	0	1	1	0
2	0	1	1	1
3	0	1	0	1
4	0	1	0	0
5	1	1	0	0
6	1	1	0	1
7	1	1	1	1
8	1	1	1	0
9	1	0	1	0



Contadores Sincrónicos – Síntesis – MT

Muchas veces se los designa con el estado pero en binario natural, en este caso quedaría:

0	0 0 1 0	2
1	0 1 1 0	6
2	0 1 1 1	7
3	0 1 0 1	5
4	0 1 0 0	4
5	1 1 0 0	12
6	1 1 0 1	13
7	1 1 1 1	15
8	1 1 1 0	14
9	1 0 1 0	10



En muchos casos es importante **analizar** qué ocurre con los **estados que no pertenecen al código**, lo veremos luego en Análisis de Contadores Sincrónicos.

Contadores Sincrónicos – Síntesis – MT

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ahora nuevamente por el **Método de las Transiciones** pero con biestables **D**. Mediante la **TT** del D analizaremos las transiciones Q_i^n a Q_i^{n+1}. Teniendo en cuenta que $Q^{n+1} = D$, la **TV** quedará como sigue:

Qn	Qn+1	D
0	0	0
0	1	1
1	0	0
1	1	1

	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}	D_3	D_2	D_1	D_0
0	0	0	0	0	x	x	x	x	x	x	x	x
1	0	0	0	1	x	x	x	x	x	x	x	x
2	0	0	1	0	0	1	1	0	0	1	1	0
3	0	0	1	1	x	x	x	x	x	x	x	x
4	0	1	0	0	1	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0	0	1	0	0
6	0	1	1	0	0	1	1	1	0	1	1	1
7	0	1	1	1	0	1	0	1	0	1	0	1
8	1	0	0	0	x	x	x	x	x	x	x	x
9	1	0	0	1	x	x	x	x	x	x	x	x
10	1	0	1	0	0	0	1	0	0	0	1	0
11	1	0	1	1	x	x	x	x	x	x	x	x
12	1	1	0	0	1	1	0	1	1	1	0	1
13	1	1	0	1	1	1	1	1	1	1	1	1
14	1	1	1	0	1	0	1	0	1	0	1	0
15	1	1	1	1	1	1	1	0	1	1	1	0

Debo resolver los 4 mapas K.

Ventaja: 4 mapas (D) vs 8 (JK).

Desventaja: generalmente funciones más complejas (D).

Contadores Sincrónicos – Síntesis – MT

Planteamos 4 mapas K de los D_i y obtenemos las soluciones:

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	1	0	0	0
11	1	1	1	1
10	x	x	x	0

$$D_3 = \overline{Q}_1 \overline{Q}_0 + Q_3 Q_2$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	1
01	1	1	1	1
11	1	1	1	0
10	x	x	x	0

$$D_2 = \overline{Q}_3 + \overline{Q}_1 + Q_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	1
01	0	0	0	1
11	0	1	1	1
10	x	x	x	1

$$D_1 = Q_3 Q_0 + Q_1 \overline{Q}_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	0	0	1	1
11	1	1	0	0
10	x	x	x	0

$$D_0 = \overline{Q}_3 Q_2 Q_1 + Q_3 \overline{Q}_1$$

Contadores Sincrónicos – Síntesis – MEC

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

<u>Método de la Ecuación Característica</u>: Diseñaremos el mismo contador BCD Gray XS-3 utilizando **JK**.

Tenemos la misma **TV** que antes:

	Q_3^n	Q_2^n	Q_1^n	Q_0^n	Q_3^{n+1}	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	x	x	x	x
1	0	0	0	1	x	x	x	x
2	0	0	1	0	0	1	1	0
3	0	0	1	1	x	x	x	x
4	0	1	0	0	1	1	0	0
5	0	1	0	1	0	1	0	0
6	0	1	1	0	0	1	1	1
7	0	1	1	1	0	1	0	1
8	1	0	0	0	x	x	x	x
9	1	0	0	1	x	x	x	x
10	1	0	1	0	0	0	1	0
11	1	0	1	1	x	x	x	x
12	1	1	0	0	1	1	0	1
13	1	1	0	1	1	1	1	1
14	1	1	1	0	1	0	1	0
15	1	1	1	1	1	1	1	0

Contadores Sincrónicos – Síntesis – MEC

Planteamos 4 mapas K de los Q_i^{n+1} y los resolvemos en función de la ecuación característica del JK:

$$Q_i^{n+1} = J_i \, \overline{Q}_i + \overline{K}_i \, Q_i$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	1	0	0	0
11	1	1	1	1
10	x	x	x	0

$$Q_3^{n+1} = (\overline{Q}_1 \, \overline{Q}_0) \, \overline{Q}_3 + Q_2 \, Q_3$$

$$J_3 = \overline{Q}_1 \, \overline{Q}_0 \qquad K_3 = \overline{Q}_2$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	1
01	1	1	1	1
11	1	1	1	0
10	x	x	x	0

$$Q_2^{n+1} = \overline{Q}_3 \; \overline{Q}_2 + (\overline{Q}_3 + \overline{Q}_1 + Q_0) \, Q_2$$

$$J_2 = \overline{Q}_3 \qquad K_2 = Q_3 \, Q_1 \, \overline{Q}_0$$

Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	1
01	0	0	0	1
11	0	1	1	1
10	x	x	x	1

$$Q_1^{n+1} = Q_3 \, Q0 \, \overline{Q}_1 + (Q_3 + \overline{Q}_0) \, Q_1$$

$$J_1 = Q_3 \, Q_0 \qquad K_1 = \overline{Q}_3 \, Q_0$$

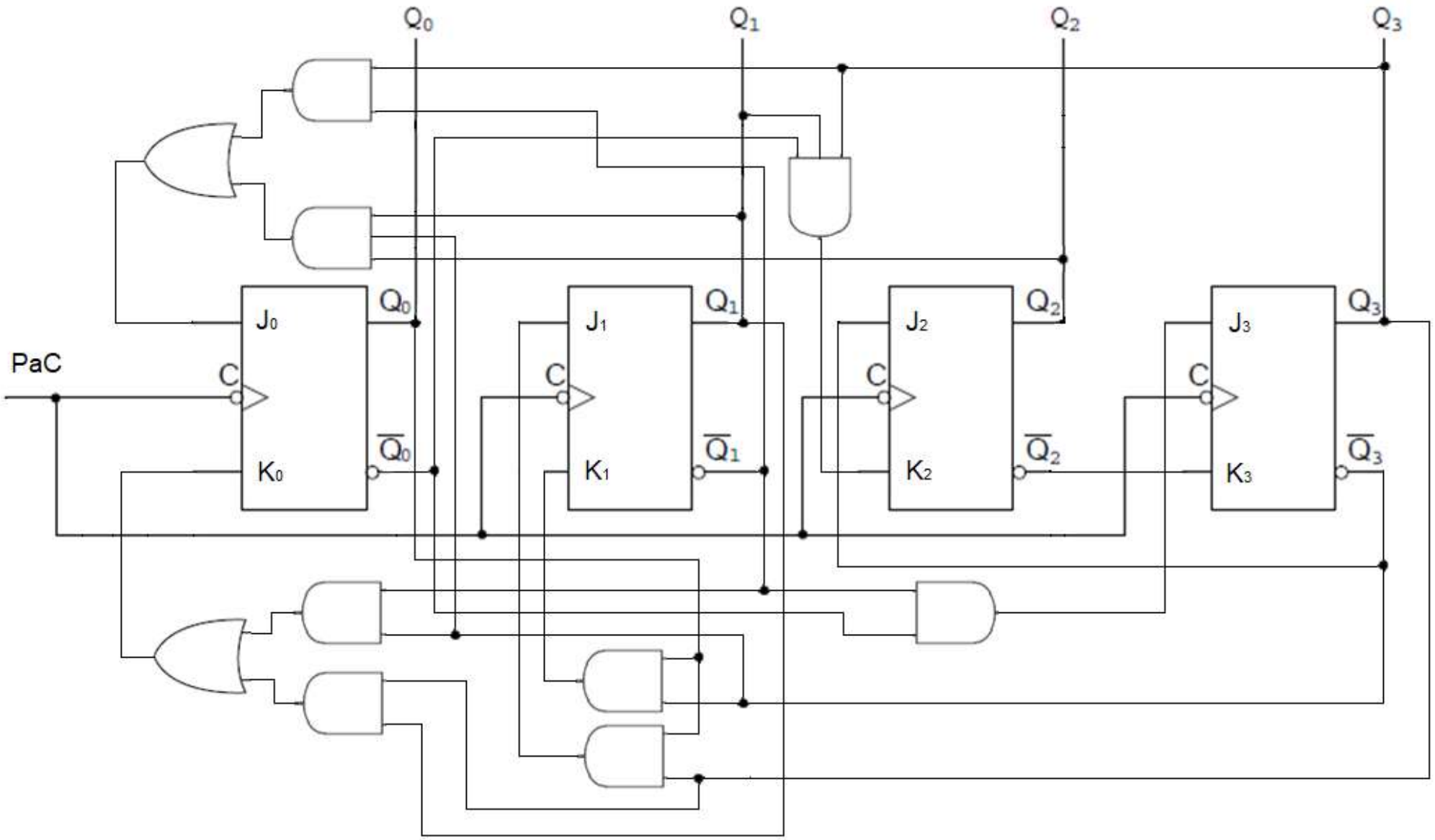
Q3Q2 \ Q1Q0	00	01	11	10
00	x	x	x	0
01	0	0	1	1
11	1	1	0	0
10	x	x	x	0

$$Q_0^{n+1} = (\overline{Q}_3 \, Q_2 \, Q_1 + Q_3 \, \overline{Q}_1) \, \overline{Q}_0 + (Q_3 \, \overline{Q}_1 + \overline{Q}_3 \, Q_1) \, Q_0$$

$$J_0 = \overline{Q}_3 \, Q_2 \, Q_1 + Q_3 \, \overline{Q}_1 \qquad K_0 = \overline{Q}_3 \, \overline{Q}_1 + Q_3 \, Q_1$$

Contadores Sincrónicos – Síntesis – MEC

El circuito es el mismo que obtuvimos por el método de las Transiciones:



Contadores Sincrónicos U/D – Síntesis

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Otro ejemplo: Diseñar un contador **binario natural** de módulo **M = 6**, que mediante una entrada x, pueda contar en forma ascendente o descendente. En caso que salga del código **asegurar que vuelva** al mismo.

La **TV** quedará como sigue:

	$Q_3Q_2Q_1$
0	000
1	001
2	010
3	011
4	100
5	101
0	000

Debo elegir **tipo** de FF y **método** de diseño: elijo JK y MEC.

	x	Q_2^n	Q_1^n	Q_0^n	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	0	0	1
1	0	0	0	1	0	1	0
2	0	0	1	0	0	1	1
3	0	0	1	1	1	0	0
4	0	1	0	0	1	0	1
5	0	1	0	1	0	0	0
6	0	1	1	0	0	0	0
7	0	1	1	1	0	0	0
0	1	0	0	0	1	0	1
1	1	0	0	1	0	0	0
2	1	0	1	0	0	0	1
3	1	0	1	1	0	1	0
4	1	1	0	0	0	1	1
5	1	1	0	1	1	0	0
6	1	1	1	0	0	0	0
7	1	1	1	1	0	0	0

Contadores Sincrónicos U/D – Síntesis

Ahora planteamos los 3 mapas K de los Q_i^{n+1} y los resolvemos en función de la ecuación característica del JK:

$$Q_i^{n+1} = J_i \, \overline{Q}_i + \overline{K}_i \, Q_i$$

xQ2 \ Q1Q0	00	01	11	10
00	0	0	1	0
01	1	0	0	0
11	0	1	0	0
10	1	0	0	0

xQ2 \ Q1Q0	00	01	11	10
00	0	1	0	1
01	0	0	0	0
11	1	0	0	0
10	0	0	1	0

xQ2 \ Q1Q0	00	01	11	10
00	1	0	0	1
01	1	0	0	0
11	1	0	0	0
10	1	0	0	1

$$Q_2^{n+1} = (\bar{x} \, Q_1 \, Q_0 + x \, \overline{Q}_1 \, \overline{Q}_0) \, \overline{Q}_2 + (\bar{x} \, \overline{Q}_1 \, \overline{Q}_0 + x \, \overline{Q}_1 \, Q_0) \, Q_2$$

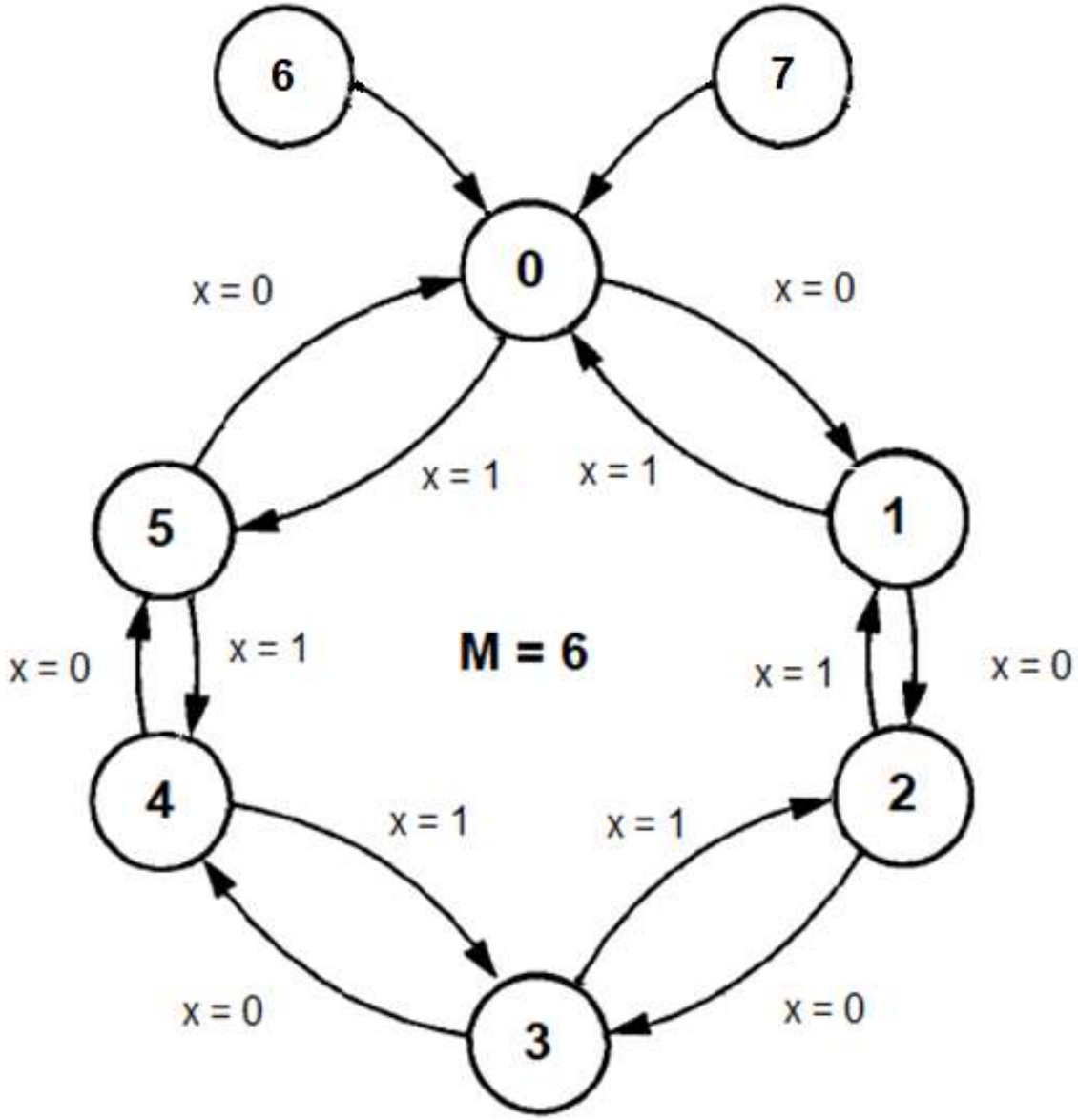
$$Q_1^{n+1} = (\bar{x} \, \overline{Q}_2 \, Q_0 + x \, Q_2 \, \overline{Q}_0) \, \overline{Q}_1 + (\bar{x} \, \overline{Q}_2 \, \overline{Q}_0 + x \, \overline{Q}_2 \, Q_0) \, Q_1$$

$$Q_0^{n+1} = (\overline{Q}_2 + \overline{Q}_1) \, \overline{Q}_0 + 0 \, Q_0$$

Contadores Sincrónicos U/D – Síntesis

El diagrama de estados quedará:

	$Q_3Q_2Q_1$
0	000
1	001
2	010
3	011
4	100
5	101
0	000

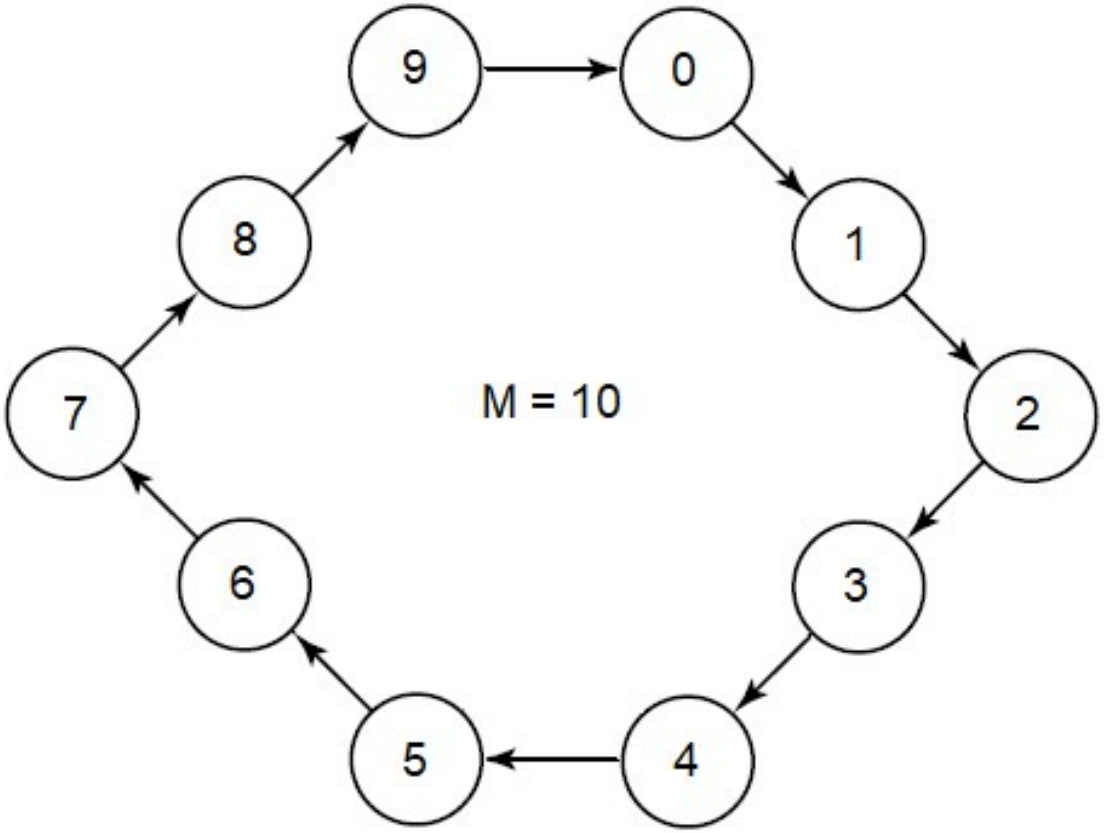


En la TV hemos forzado a los estados 6 y 7 (**para ambos valores de la entrada x)** a que evolucionen hacia el estado 0.

Contadores Sincrónicos – Análisis

Analizar implica que, **a partir del circuito** debemos obtener el **módulo**, **código** y **diagrama** de estados **completo.** En muchos casos es importante **analizar** qué ocurre con los **estados que no pertenecen al código**.

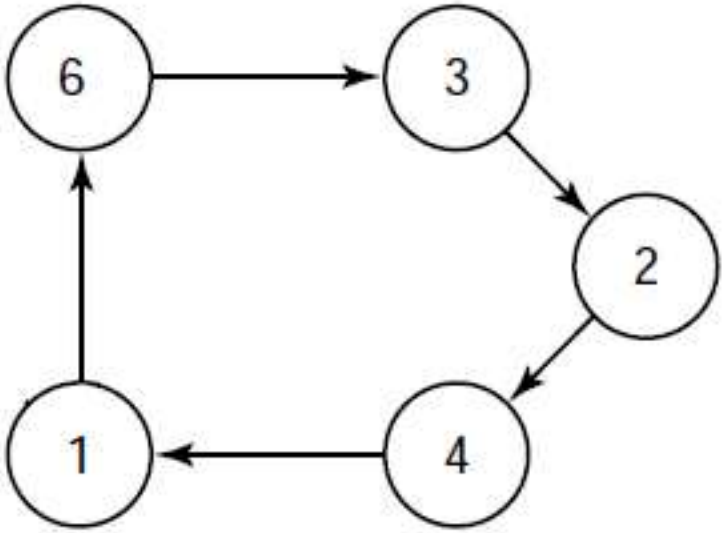
En este ejemplo con 4 biestables, hay 6 estados (del 10 al 15) que no pertenecen al código, podría haber **códigos secundarios** o transiciones hasta **regresar** al código.

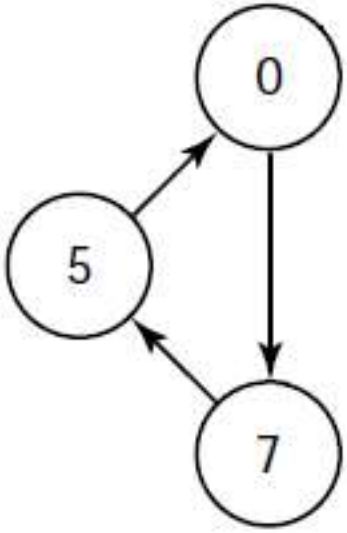


Contadores Sincrónicos – Análisis

En este contador vemos que hay un **código secundario**, si por alguna razón cae en alguno de los estados de ese código, continuará en ese código secundario en forma indefinida.

M = 5
código
principal

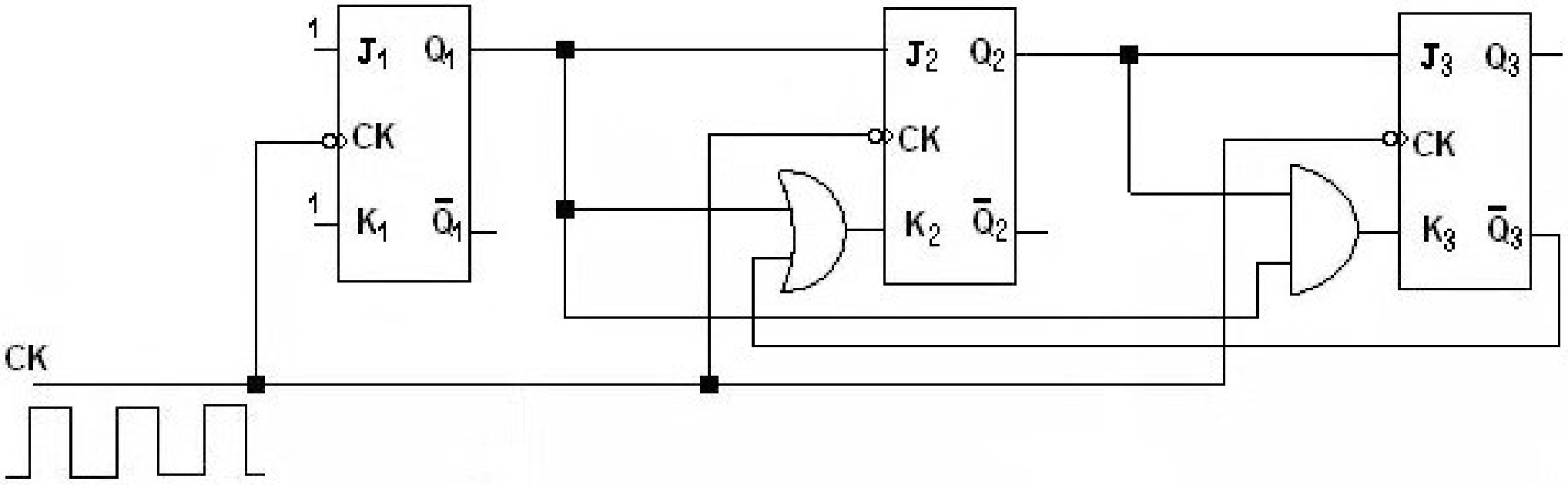




M = 3
código
secundario

Contadores Sincrónicos – Análisis

Ejemplo. A partir del circuito, debemos obtener el módulo, código y diagrama de estados **completo** del contador sincrónico dado.



Comienzo planteando las **ecuaciones** de las **entradas** de los **biestables**, en este caso:

$$J_3 = Q_2 \qquad J_2 = Q_1 \qquad J_1 = 1$$

$$K_3 = Q_1\,Q_2 \qquad K_2 = Q_1 + \bar{Q}_3 \qquad K_1 = 1$$

Contadores Sincrónicos – Análisis

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

A partir del **estado actual** obtengo las **entradas** actuales de los biestables y luego el **próximo estado**. Comienzo por ejemplo con el estado 0.

$J_3 = Q_2$ $\quad J_2 = Q_1$ $\quad J_1 = 1$

$K_3 = Q_1\,Q_2$ $\quad K_2 = Q_1 + \overline{Q}_3$ $\quad K_1 = 1$

J	K	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	not Qn

En rojo el estado en binario.

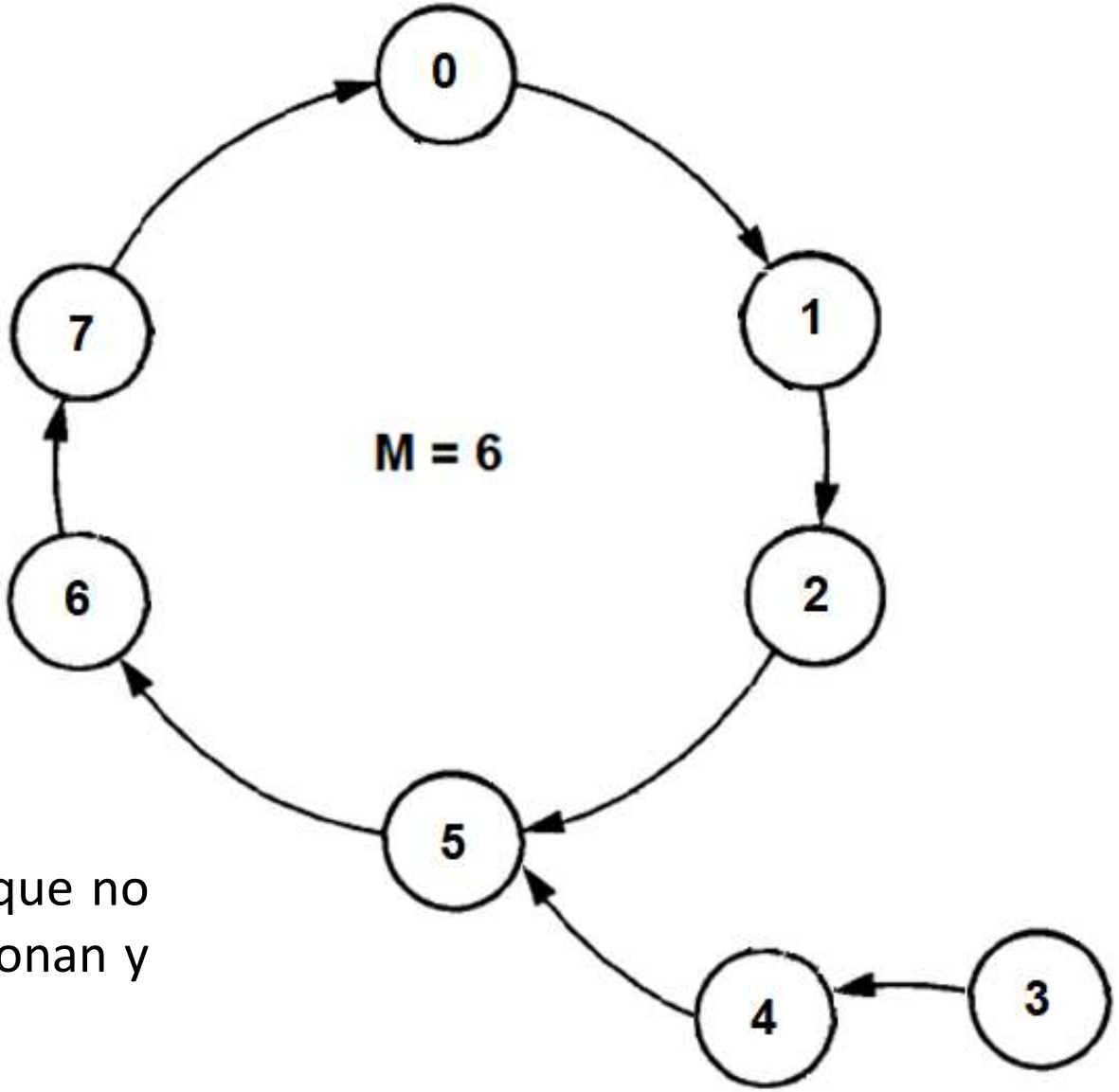
	$Q_3Q_2Q_1$	J_3K_3 J_2K_2 J_1K_1
0	0 0 0	0 0 0 1 1 1
1	0 0 1	0 0 1 1 1 1
2	0 1 0	1 0 0 1 1 1
5	1 0 1	0 0 1 1 1 1
6	1 1 0	1 0 0 0 1 1
7	1 1 1	1 1 1 1 1 1
0	0 0 0	**M = 6**

	$Q_3Q_2Q_1$	J_3K_3 J_2K_2 J_1K_1
3	0 1 1	1 1 1 1 1 1
4	1 0 0	0 0 0 0 1 1
5	1 0 1	regresa al código

Contadores Sincrónicos – Análisis

Nominando a los **estados** por su número en **binario**, el diagrama quedará:

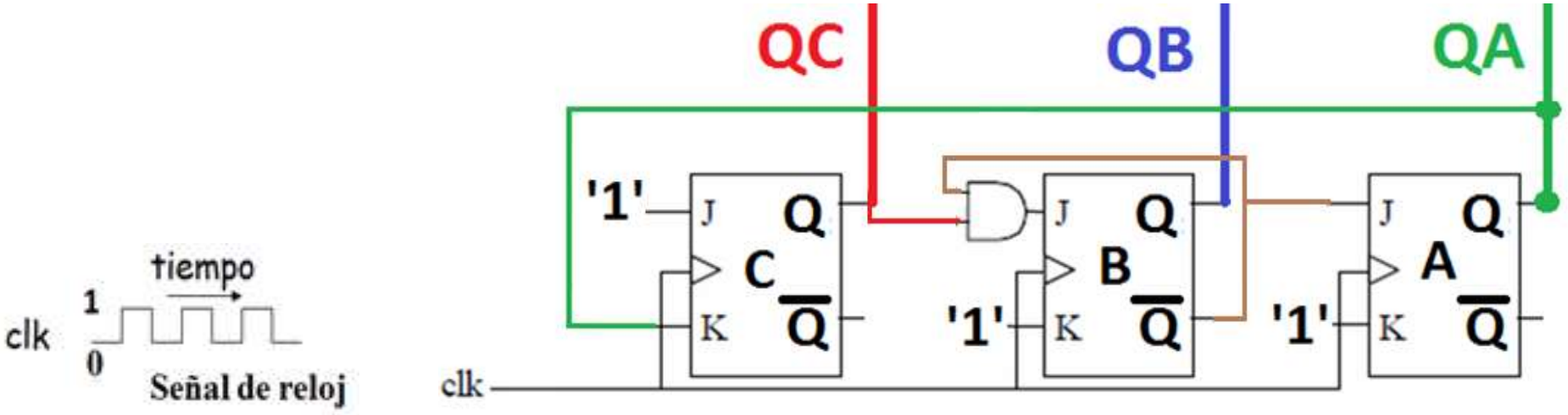
	$Q_3Q_2Q_1$
0	000
1	001
2	010
5	101
6	110
7	111
0	000



Vimos que los estados 3 y 4 que no pertenecen al código, evolucionan y **regresan** al mismo.

Contadores Sincrónicos – Análisis

Otro ejemplo pero con una metodología de trabajo distinta.



$J_C = 1$ $\quad J_B = Q_C \overline{Q}_B$ $\quad J_A = \overline{Q}_B$

$K_C = Q_A$ $\quad K_B = 1$ $\quad K_A = 1$

J	K	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	not Qn

Contadores Sincrónicos – Análisis

Construimos la TV a partir del **estado actual** obteniendo las **entradas** actuales de los biestables y a diferencia de antes, el **próximo estado** lo agrego a la TV.

$J_C = 1$ $\quad J_B = Q_C \overline{Q}_B$ $\quad J_A = \overline{Q}_B$

$K_C = Q_A$ $\quad K_B = 1$ $\quad K_A = 1$

J	K	Qn+1
0	0	Qn
0	1	0
1	0	1
1	1	not Qn

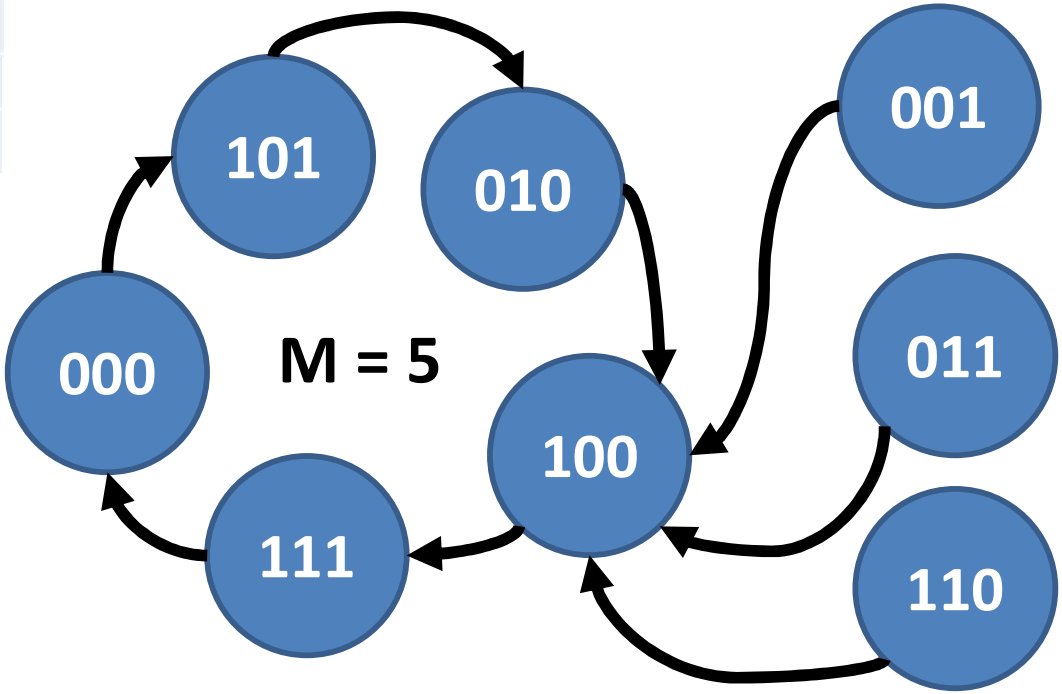
Qn C	Qn B	Qn A	JC	KC	JB	KB	JA	KA	Qn+1 C	Qn+1 B	Qn+1 A
0	0	0	1	0	0	1	1	1	1	0	1
0	0	1	1	1	0	1	1	1	1	0	0
0	1	0	1	0	0	1	0	1	1	0	0
0	1	1	1	1	0	1	0	1	1	0	0
1	0	0	1	0	1	1	1	1	1	1	1
1	0	1	1	1	1	1	1	1	0	1	0
1	1	0	1	0	0	1	0	1	1	0	0
1	1	1	1	1	0	1	0	1	0	0	0

Contadores Sincrónicos – Análisis

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ahora construimos el **diagrama de estados**.

Qn C	Qn B	Qn A	Qn+1 C	Qn+1 B	Qn+1 A
0	0	0	1	0	1
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	0	0
1	0	0	1	1	1
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	0	0	0



Contadores Sincrónicos – Análisis

Analizaremos con la primer metodología qué ocurre con los **estados no utilizados** del contador **BCD Gray XS-3** diseñado anteriormente.

$J_3 = \overline{Q}_1\,\overline{Q}_0$ $\quad J_2 = \overline{Q}_3$ $\quad J_1 = Q_3\,Q_0$ $\quad J_0 = \overline{Q}_3\,Q_2\,Q_1 + Q_3\,\overline{Q}_1$

$K_3 = \overline{Q}_2$ $\quad K_2 = Q_3\,Q_1\,\overline{Q}_0$ $\quad K_1 = \overline{Q}_3\,Q_0$ $\quad K_0 = \overline{Q}_3\,\overline{Q}_1 + Q_3\,Q_1$

0	0 0 1 0	2
1	0 1 1 0	6
2	0 1 1 1	7
3	0 1 0 1	5
4	0 1 0 0	4
5	1 1 0 0	12
6	1 1 0 1	13
7	1 1 1 1	15
8	1 1 1 0	14
9	1 0 1 0	10

	$Q_3Q_2Q_1Q_0$	J_3K_3 J_2K_2 J_1K_1 J_0K_0
0	0 0 0 0	1 1 1 0 0 0 0 1
12	1 1 0 0	regresa al código

8	1 0 0 0	1 1 0 0 0 0 1 0
1	0 0 0 1	0 1 1 0 0 1 0 0
5	0 1 0 1	regresa al código

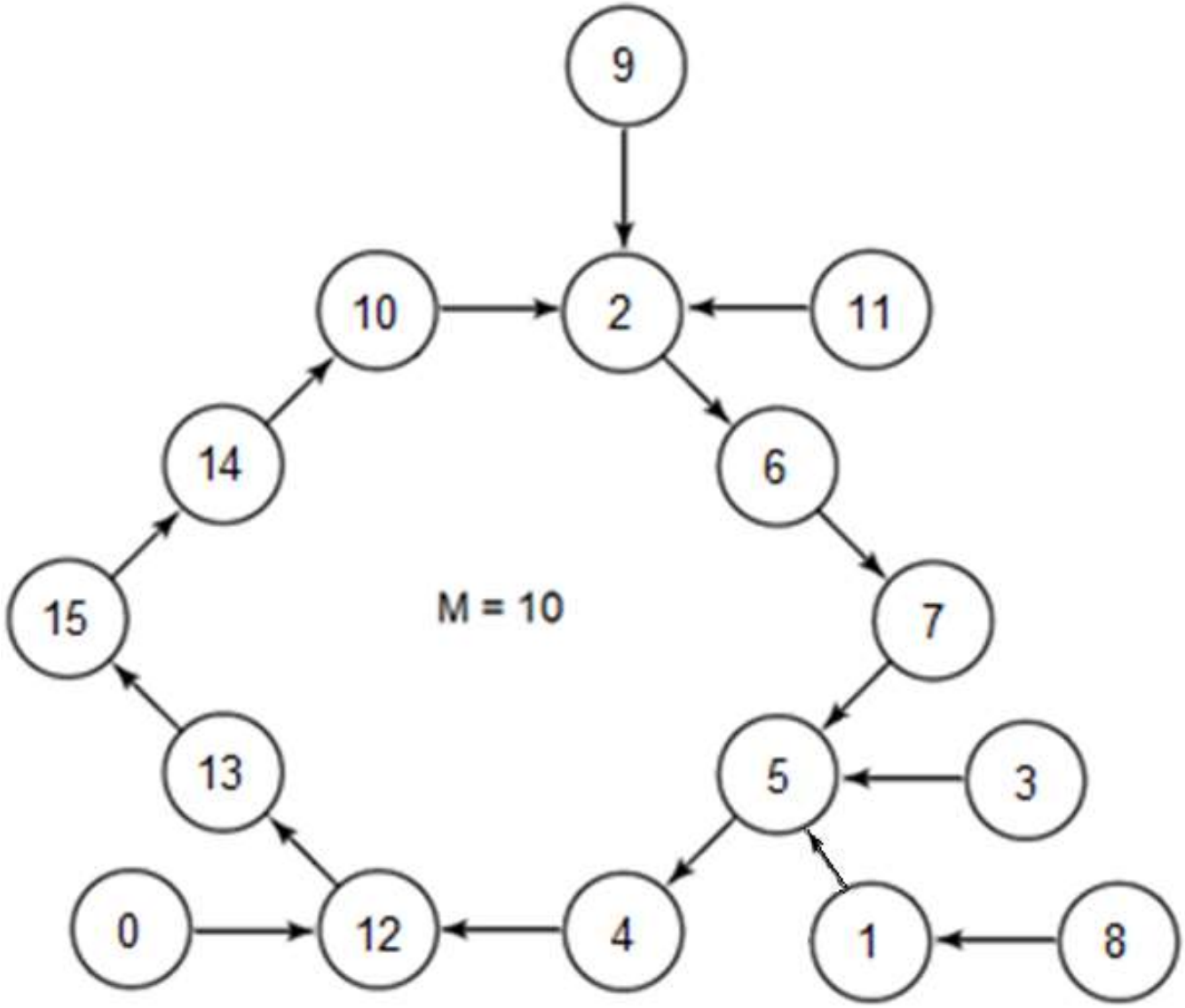
3	0 0 1 1	0 1 1 0 0 1 0 0
5	0 1 0 1	regresa al código

	$Q_3Q_2Q_1Q_0$	J_3K_3 J_2K_2 J_1K_1 J_0K_0
9	1 0 0 1	0 1 0 0 1 0 0 1
2	0 0 1 0	regresa al código

11	1 0 1 1	0 1 0 0 1 0 0 1
2	0 0 1 0	regresa al código

Contadores Sincrónicos – Análisis

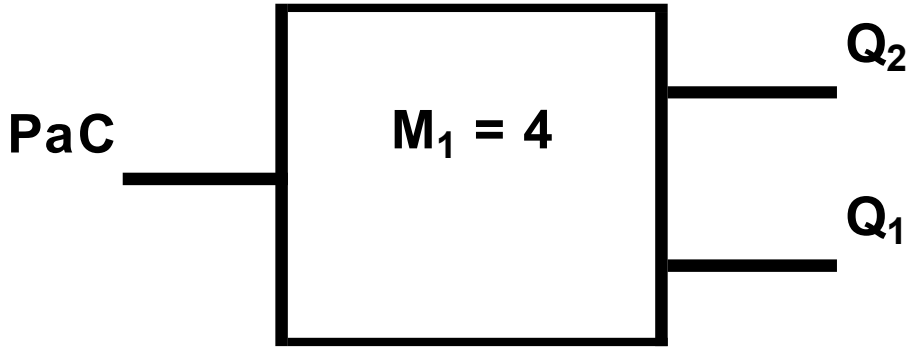
El diagrama de estados **completo** (2^n estados) quedará (expresados en binario):

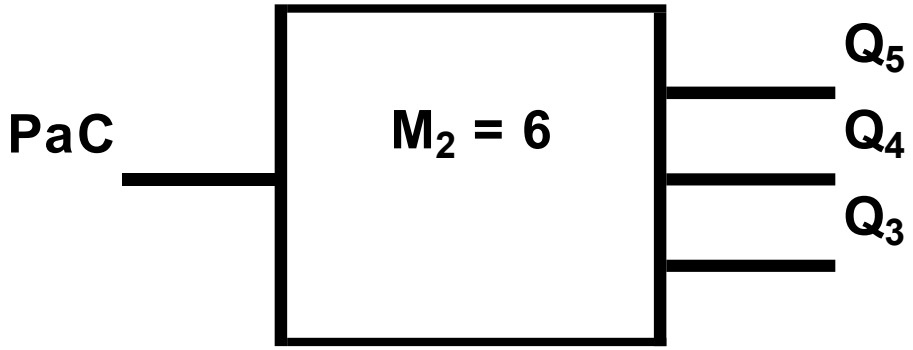


Interconexión de Contadores

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

¿Qué ocurre si **interconectamos contadores**?
Ejemplo con 2 contadores binarios $M_1 = 4$ y $M_2 = 6$. Supondremos contadores de flanco descendente.



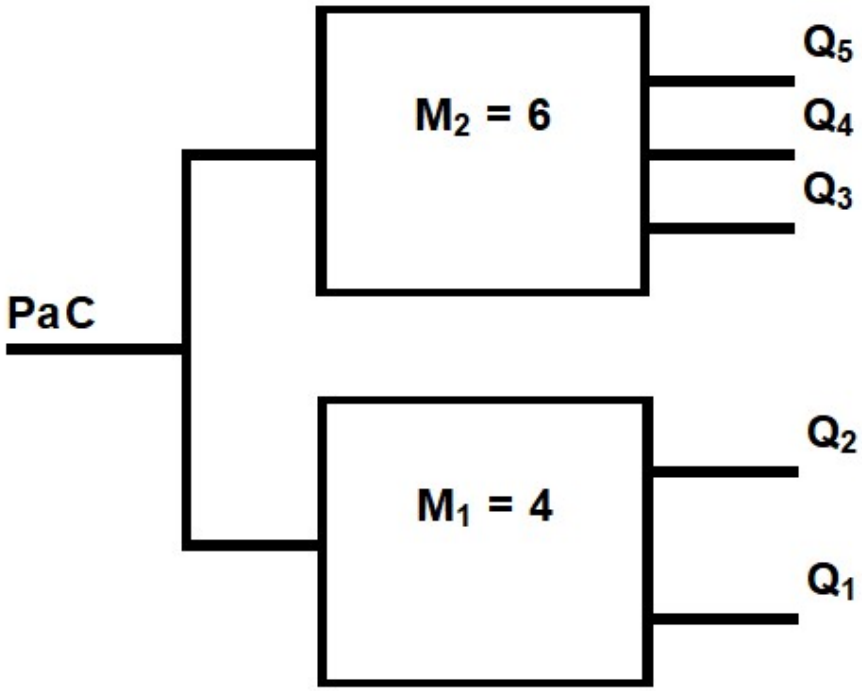


Q_2	Q_1
0	0
0	1
1	0
1	1
0	0

Q_5	Q_4	Q_3
0	0	0
0	0	1
0	1	0
0	1	1
1	0	0
1	0	1
0	0	0

Interconexión de Contadores

Acoplando los contadores en forma **sincrónica** tendremos un módulo resultante M = 12, esto se debe a que ambos contadores evolucionan en cada pulso.



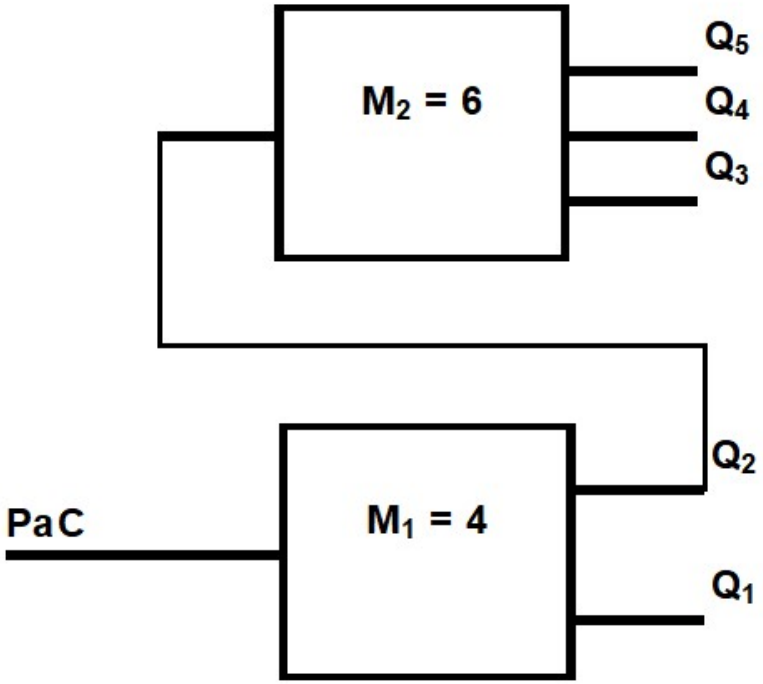
Q_5 Q_4 Q_3	Q_2 Q_1
0 0 0	0 0
0 0 1	0 1
0 1 0	1 0
0 1 1	1 1
1 0 0	0 0
1 0 1	0 1
0 0 0	1 0
0 0 1	1 1
0 1 0	0 0
0 1 1	0 1
1 0 0	1 0
1 0 1	1 1
0 0 0	0 0

Vemos que el módulo resultante es el **mínimo común múltiplo** de los módulos.

M = m.c.m. (M_1, M_2)

Interconexión de Contadores

En cambio acoplando en forma **asincrónica**, por ejemplo tomando Q_2 como Ck del contador 2, tendremos M = 24.



Vemos que el módulo resultante es el **producto** de los módulos.

$$M = M_1 \cdot M_2$$

El objetivo de la interconexión de contadores es obtener un **módulo mayor**, por lo que la interconexión sincrónica no tiene mucho sentido frente a la asincrónica.

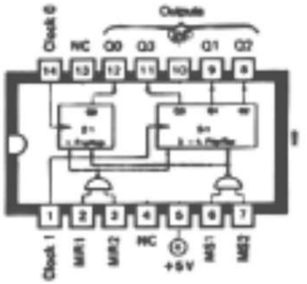
$Q_5Q_4Q_3$	Q_2Q_1
0 0 0	0 0
0 0 0	0 1
0 0 0	1 0
0 0 0	1 1
0 0 1	0 0
0 0 1	0 1
0 0 1	1 0
0 0 1	1 1
0 1 0	0 0
0 1 0	0 1
0 1 0	1 0
0 1 0	1 1
0 1 1	0 0
0 1 1	0 1
0 1 1	1 0
0 1 1	1 1
1 0 0	0 0
1 0 0	0 1
1 0 0	1 0
1 0 0	1 1
1 0 1	0 0
1 0 1	0 1
1 0 1	1 0
1 0 1	1 1
0 0 0	0 0

Interconexión de Contadores

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo CI **7490**.

Se trata de un contador **BCD binario** obtenido como la **interconexión asincrónica** de 2 contadores de módulos 2 y 5.



Description:
This package contains a divide-by-two and a divide-by-five counter.

Mode of operation:
The device consists of 4 flip-flops which are connected internally to create one divide-by-2 and one divide-by-5 counters.
All flip-flops have a common reset line controlled by two inputs MR1 and MR2. When both MR1 and MR2 are high the counters are cleared.
Flip-flop 1 is not internally connected to the other stages, thus providing a variety of counter sequences:

a) Counting to 10 (BCD): Output Q0 is connected to the Clock 1 input. The input pulses are applied to the Clock 0 input and the divided signal is extracted at Q3. The device counts in binary code up to 9, with the outputs returning to zero on the 10th clock pulse. Pins 2, 3 and 6, 7 must be grounded.

b) Divide by 2 and divide by 5: Flip-flop 1 is used as a 2:1 divider and flip-flops 2,3 and 4 are used as a 5:1 divider.

c) Symmetrical biquinary divider 10:1: Q3 is connected to Clock 0. Clock 1 is used as the clock input. A symmetrical square-wave with a frequency 1/10 that of the input signal is obtained at output Q0.

The flip-flops are triggered on the falling edge of the clock pulse.
When inputs MS1 and MS2 are both high the counter is preset to 9.

Reset/set inputs				Outputs			
MR1	MR2	MS1	MS2	Q0	Q1	Q2	Q3
H	H	L	X	L	L	L	L
H	H	X	L	L	L	L	L
X	X	H	H	H	L	L	H
L	X	L	X	Counting			
X	L	X	L	Counting			
L	X	X	L	Counting			
X	L	L	X	Counting			

Application:
Counters and dividers 2:1, 5:1 and 10:1

Data:									
Min. guaranteed clock frequency flip-flop 1	MHz	32					3	32	
Min. guaranteed clock frequency flip-flops 2-4	MHz	16					3	16	
Supply Current	mA	32					3.2	9	

Families:	Std	ALS	AS	F	H	L	LS	S
	●					●	●	

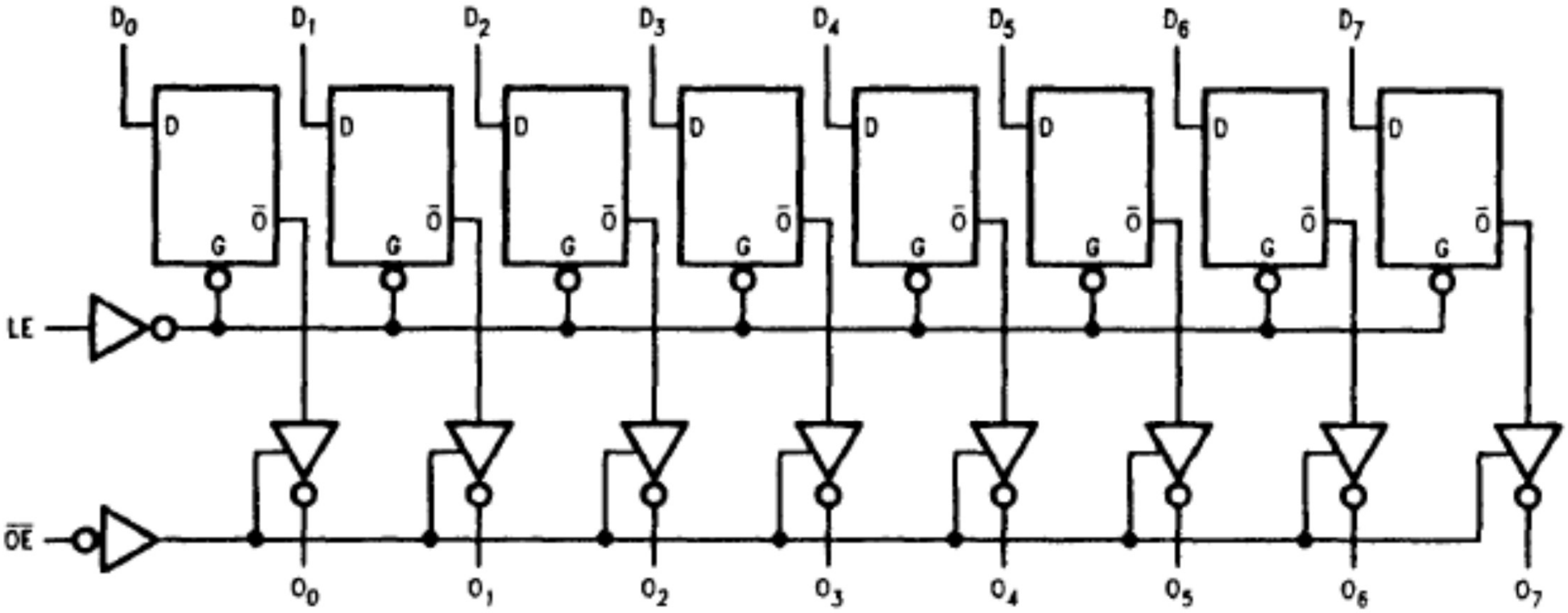
DECADE COUNTER

7490

Registros de almacenamiento

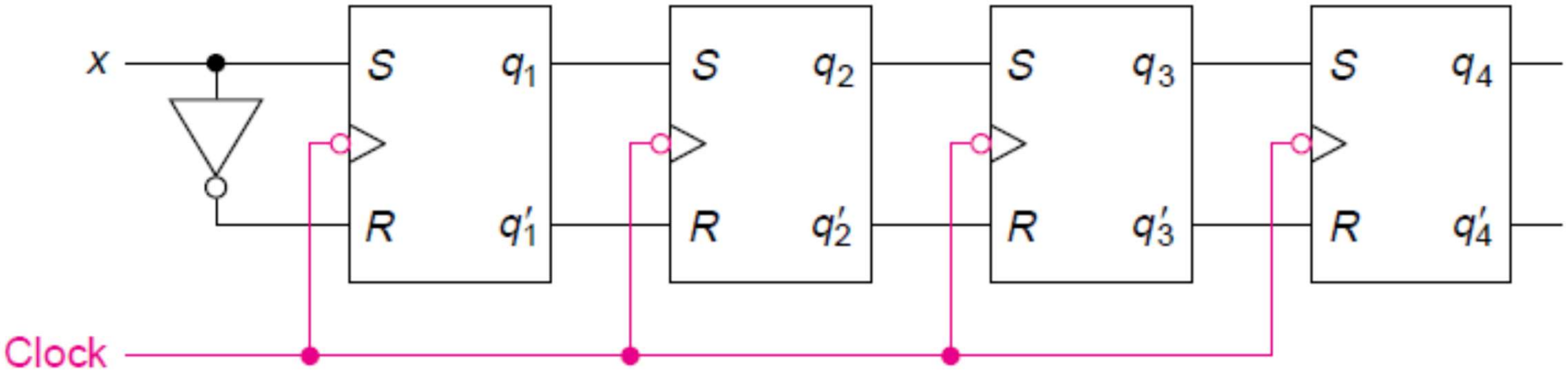
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un registro de **almacenamiento** es un conjunto de celdas básicas de memoria (biestables) que permite **almacenar** la información.
La entrada y la salida de los datos es en **paralelo**. CI 7475.



Registros de desplazamiento (SR)

Un registro de **desplazamiento** (**Shift Register**) es un conjunto de celdas básicas de memoria (biestables) que permite almacenar y **desplazar** (shift) la información ingresada. La **entrada** y la **salida** de los datos es en **serie**.

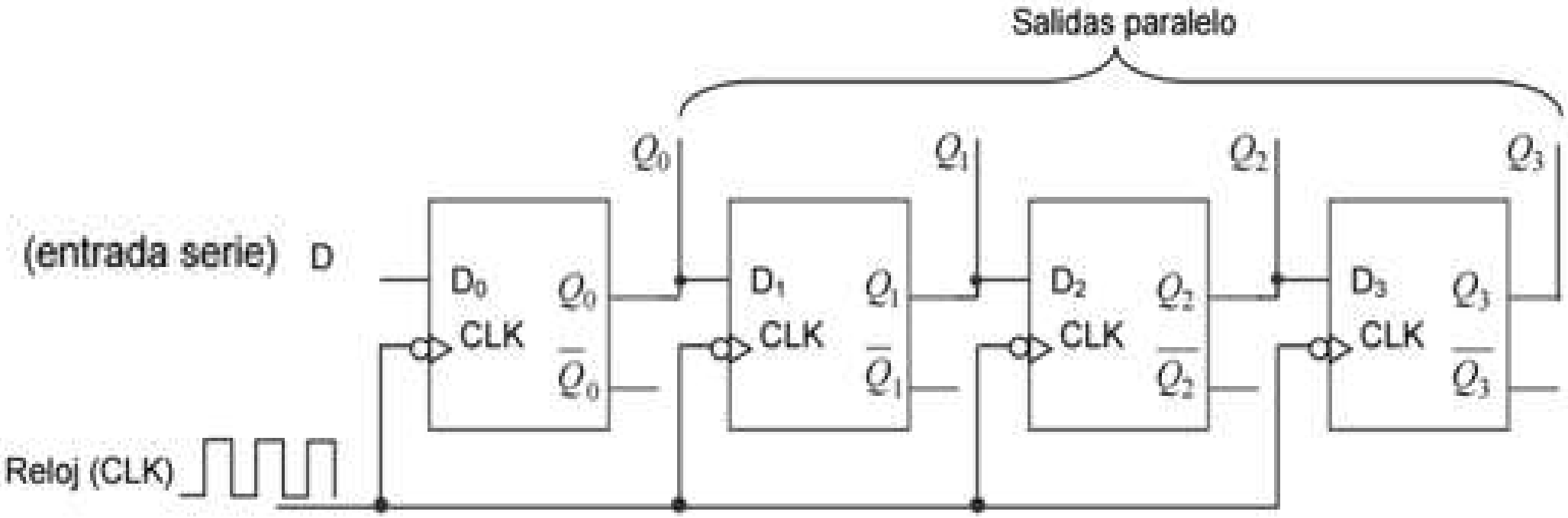


Asumiendo que se **inicializan** los FF en **0**, si se ingresan **datos** a través de **x**, el diagrama de tiempo será:

x	1	0	1	1	1	0	1	1	1	1	0	0	0				
q_1	0	1	0	1	1	1	0	1	1	1	1	0	0	0			
q_2	0	0	1	0	1	1	1	0	1	1	1	1	0	0	0		
q_3	0	0	0	1	0	1	1	1	0	1	1	1	1	0	0	0	
q_4	0	0	0	0	1	0	1	1	1	0	1	1	1	1	0	0	0

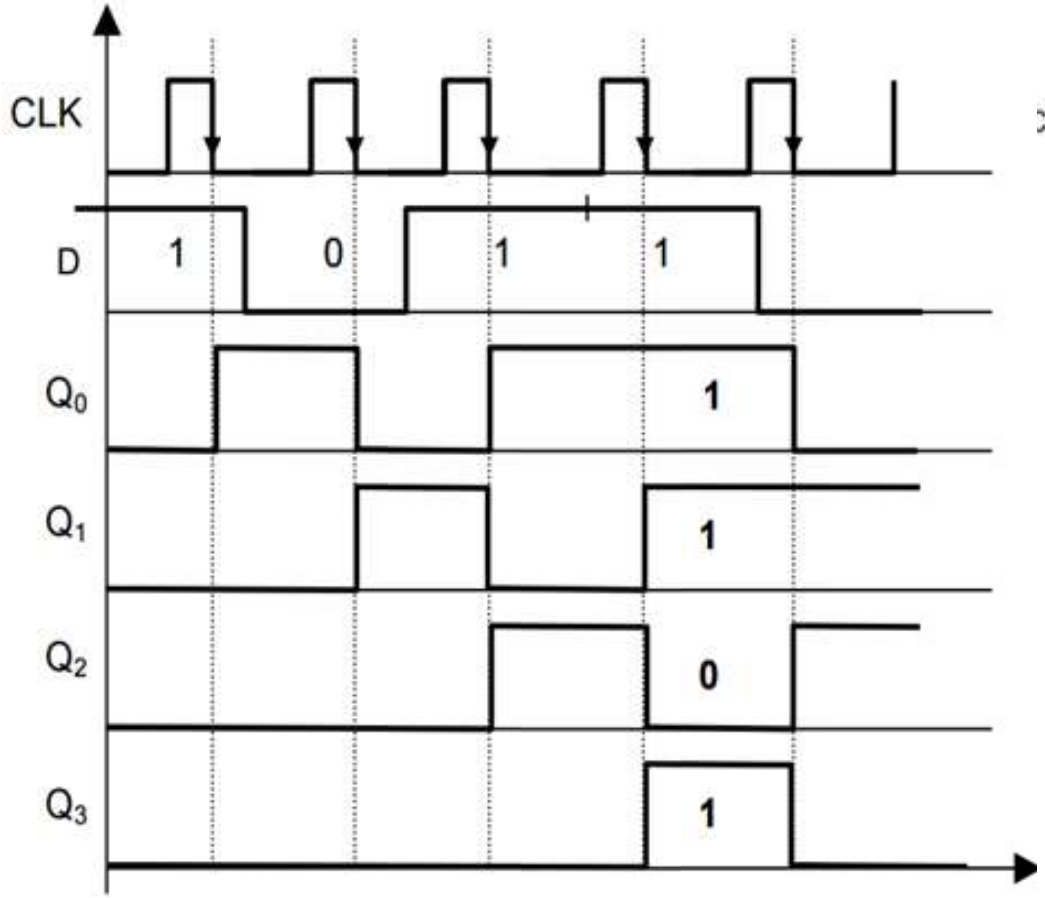
SR Serie Paralelo

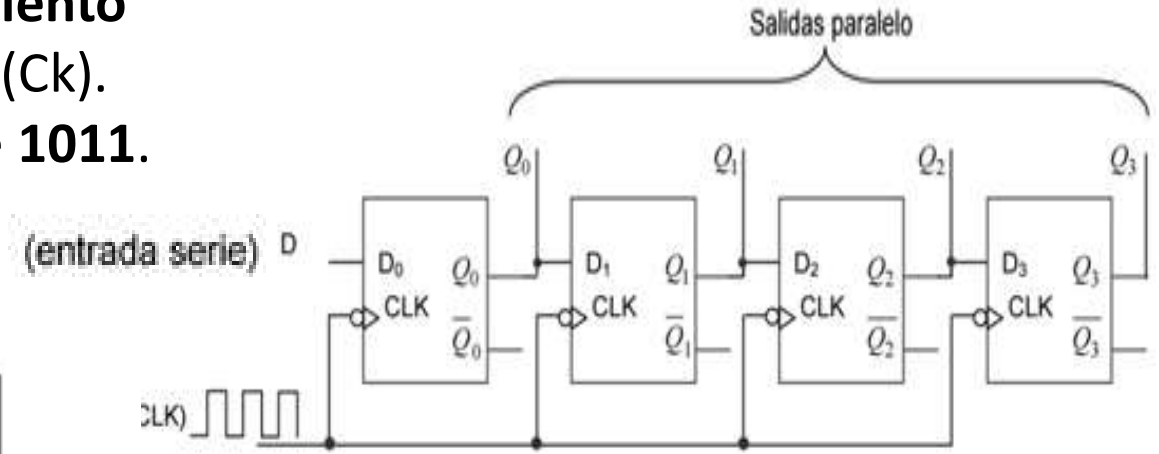
Los **SR** (**registro de desplazamiento**) son muy útiles para **almacenar** y **desplazar** datos, pero también para **transformar** palabras de **información en paralelo** a una sucesión de **bits en serie**, o también transformar una sucesión de **datos en serie** en una palabra de **datos en paralelo**.
Para implementar un **SR** puedo utilizar **biestables SR**, **JK** o **D**.
Aquí vemos un **SR** con **entrada serie y salida paralelo** en base a biestables **D**.



SR Serie Paralelo

Aquí vemos el **ingreso** y **desplazamiento** de los datos en función del tiempo (Ck). Como ejemplo cargaremos en serie **1011**.

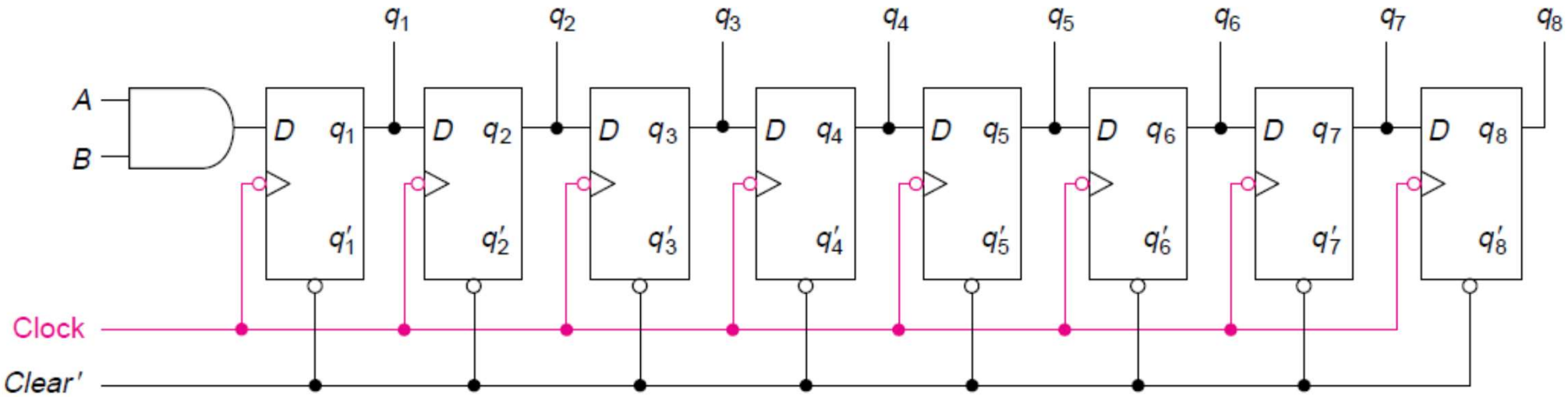




D	CLK	Q_3	Q_2	Q_1	Q_0
1	↓	0	0	0	1
0	↓	0	0	1	0
1	↓	0	1	0	1
1	↓	1	0	1	1

SR Serie Paralelo

Ejemplo. CI 74164. SR con entrada serie y salida paralelo de 8 bits.



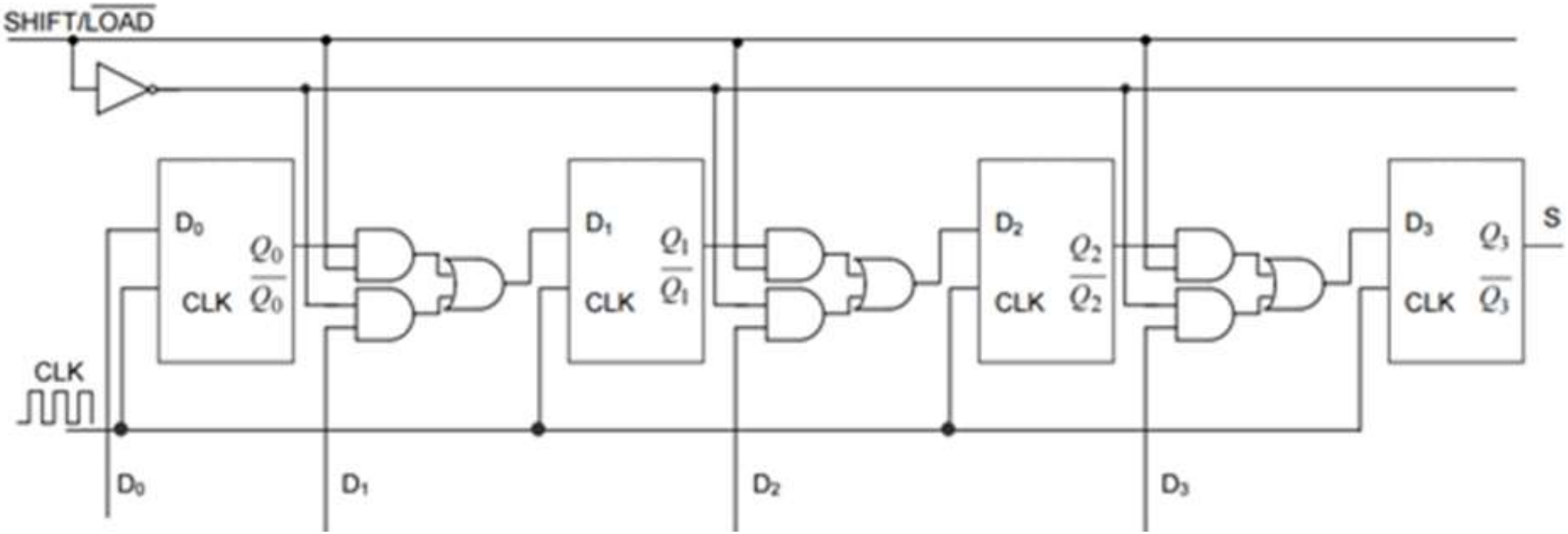
SR Paralelo Serie

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ahora vemos un **SR** con **entrada paralelo y salida serie** en base a biestables **D**.

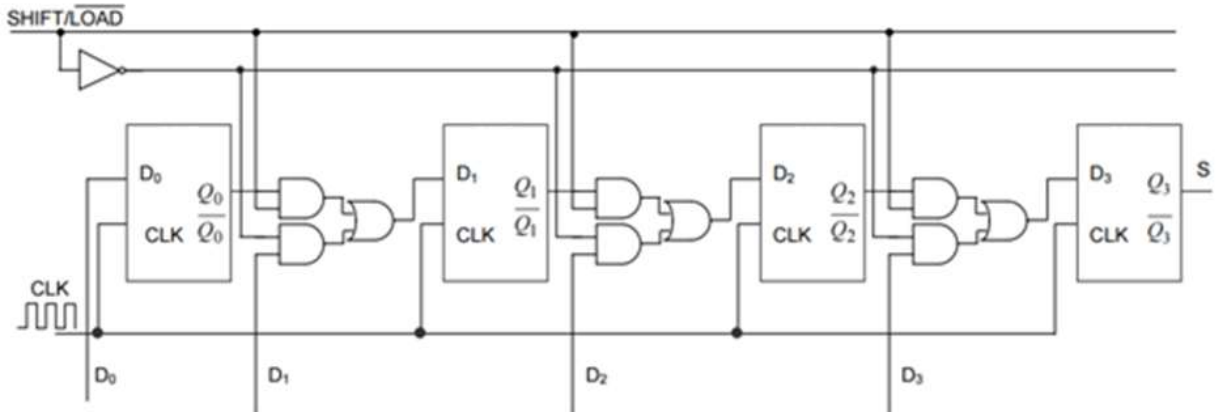
Mediante una entrada se selecciona **desplazamiento** o **carga** de datos.

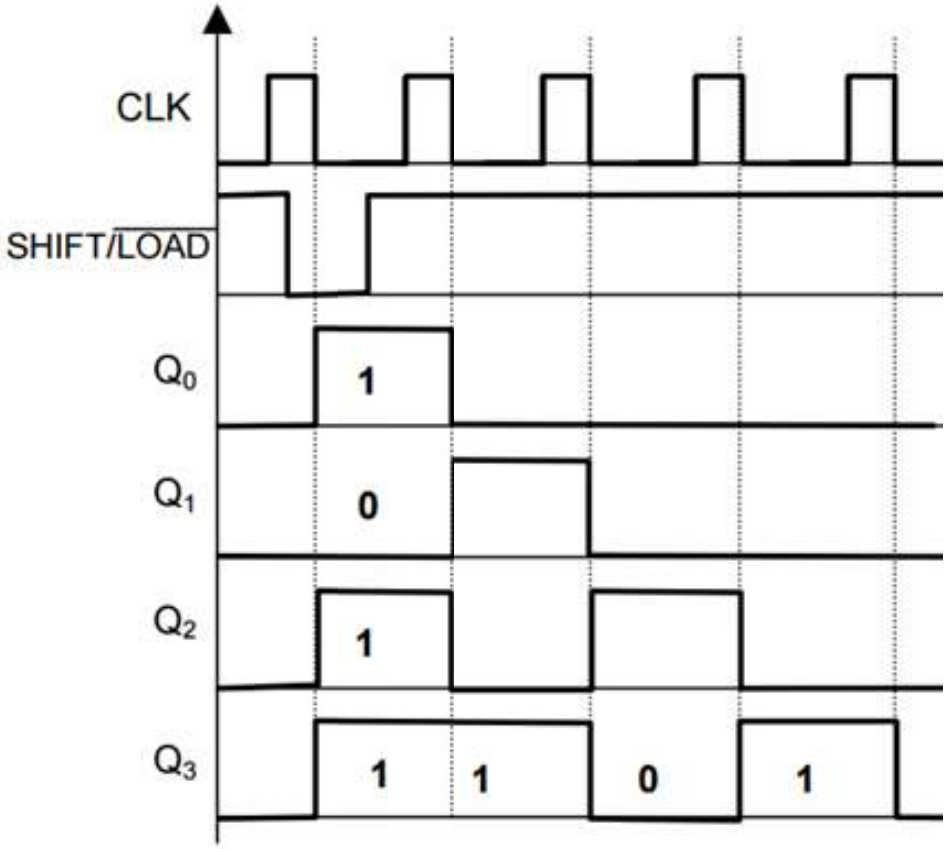
Como ejemplo cargaremos en paralelo **1101 ($D_3D_2D_1D_0$)**.



SR Paralelo Serie

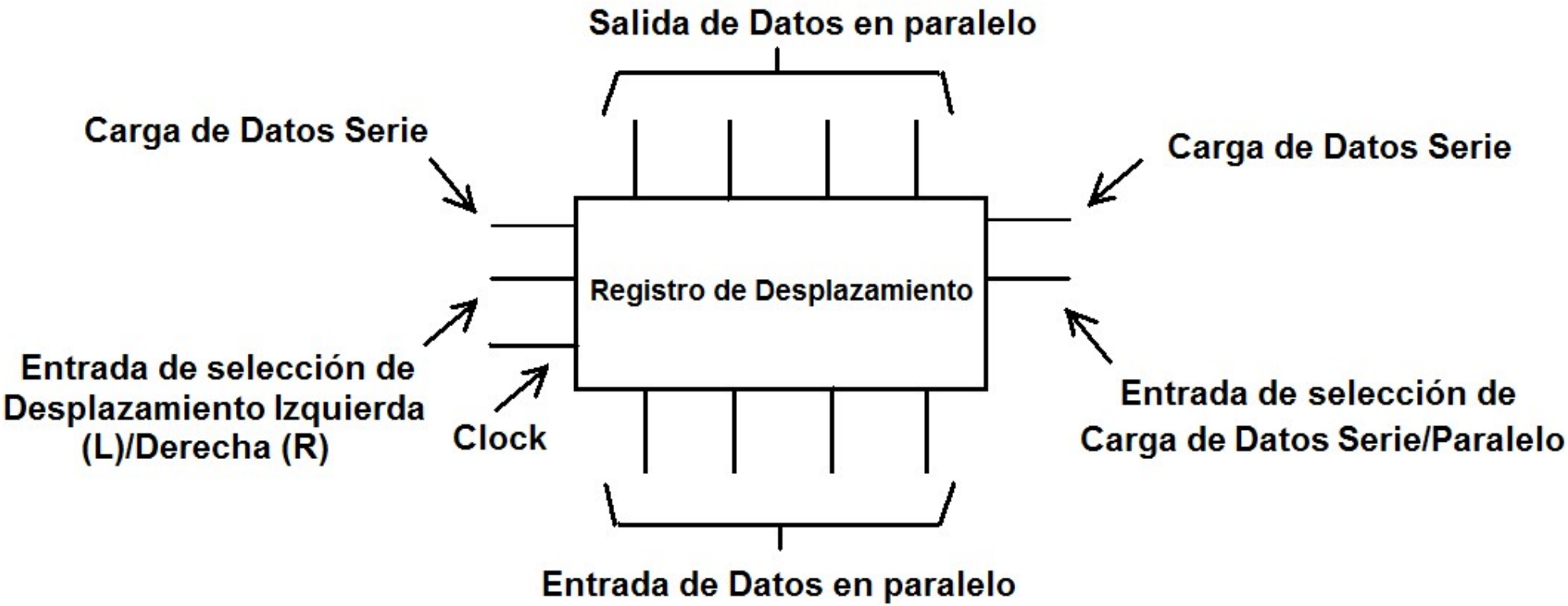
Aquí vemos el **ingreso** y **desplazamiento** de los datos del ejemplo en función del tiempo (Ck).





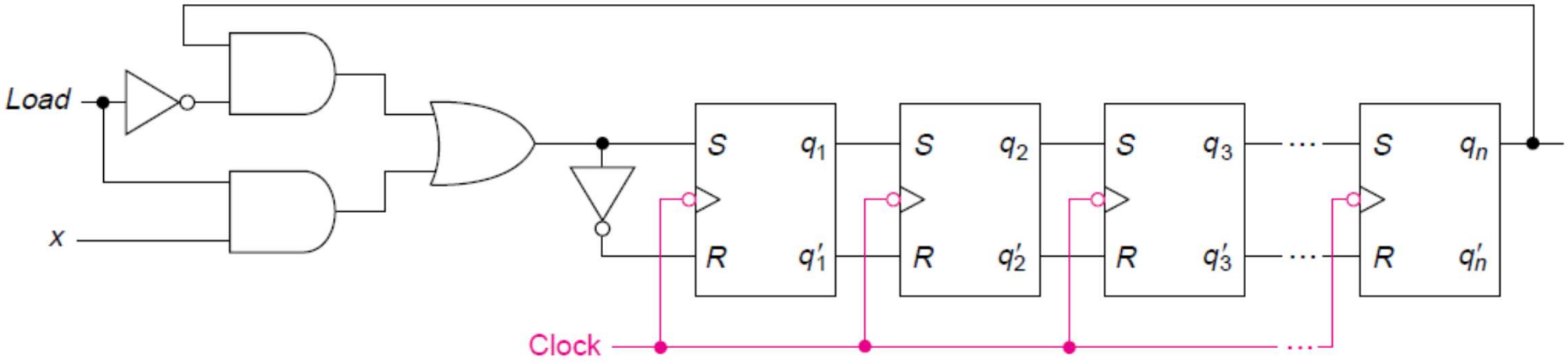
SR Universal

Un **SR universal** permite ingresar y extraer la información tanto en **serie** como en **paralelo**, inclusive en muchos casos es posible seleccionar el **desplazamiento** hacia la **izquierda** o hacia la **derecha**.



SR con realimentación

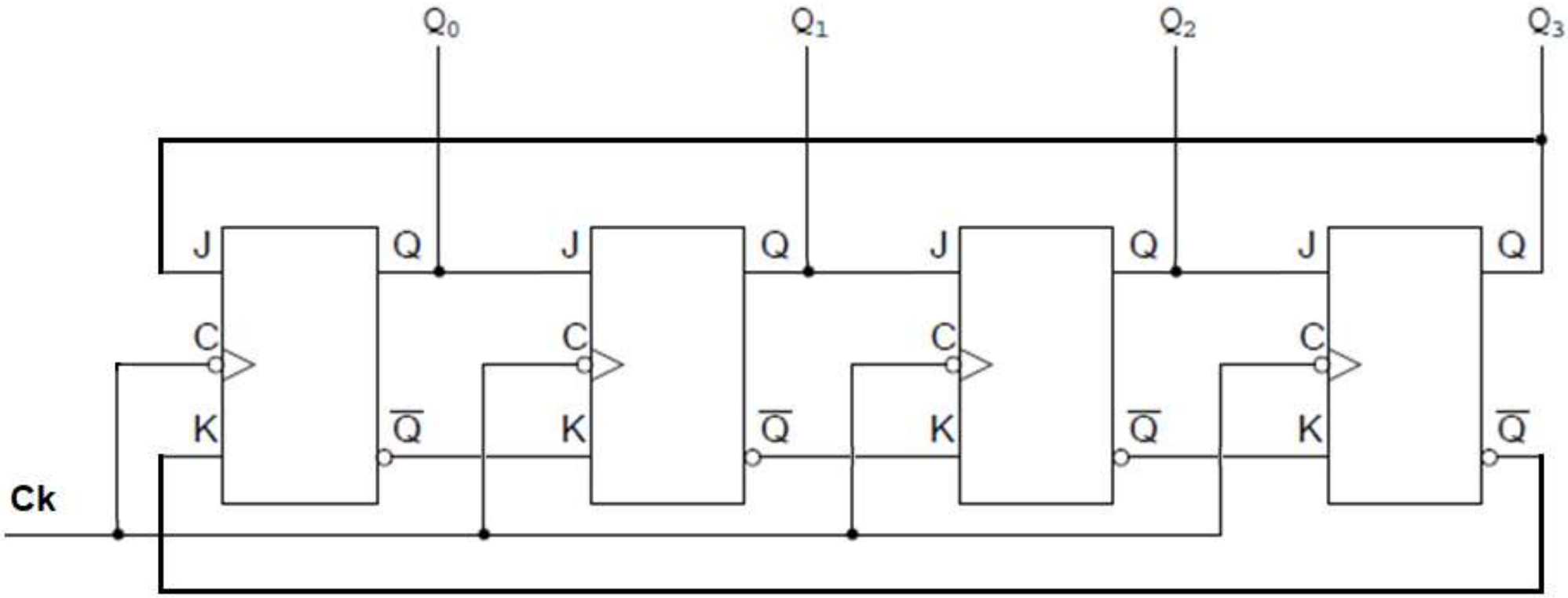
Un SR con **realimentación** permite reingresar el dato presente en la salida del mismo o ingresar datos nuevos según el estado de la entrada **Load**.



Este concepto me permite realizar **contadores** a partir de **registros de desplazamiento**.

Contador en anillo

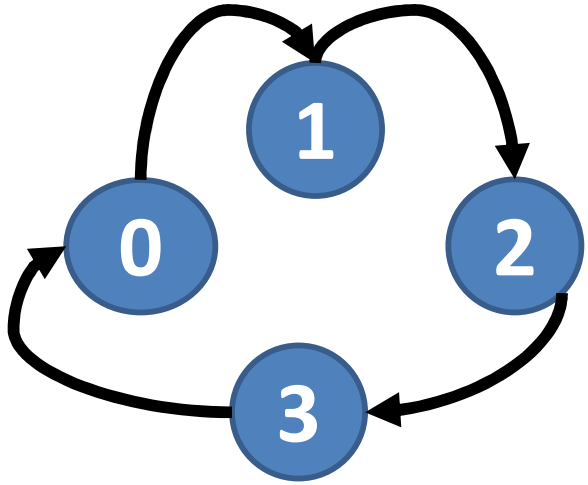
Realimentando un **SR** implementado con JK, y **precargando 0001** ($Q_3Q_2Q_1Q_0$) obtenemos un contador en anillo de módulo **M = 4**.
No se grafica el circuito de precarga (POR).



Contador en anillo

La secuencia de avance de los datos y el diagrama de **estados** serán:

Pulso	Q_3	Q_2	Q_1	Q_0
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	1	0	0	0



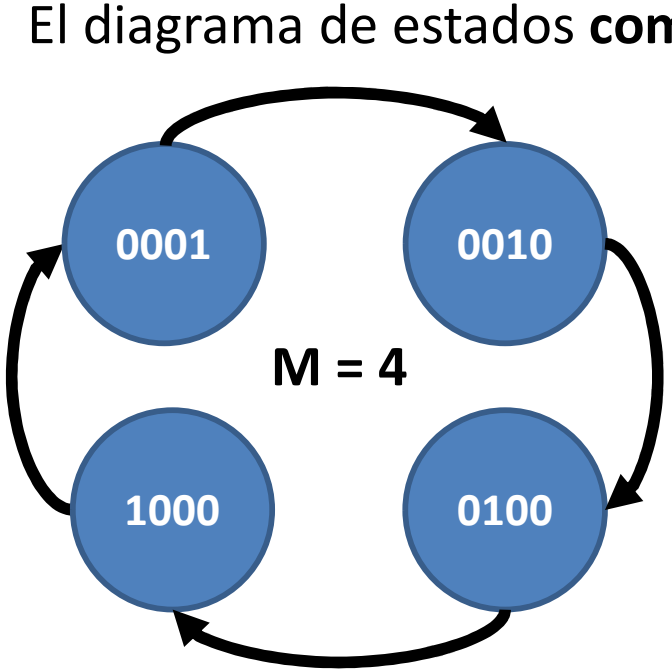
Si precargamos dos 1 consecutivos obtendremos otro código de M = 4, lo mismo para tres 1 consecutivos. ¿Cuál es el principal?
Generalizando para un contador de anillo, el módulo mayor (principal) es igual a la cantidad de biestables.

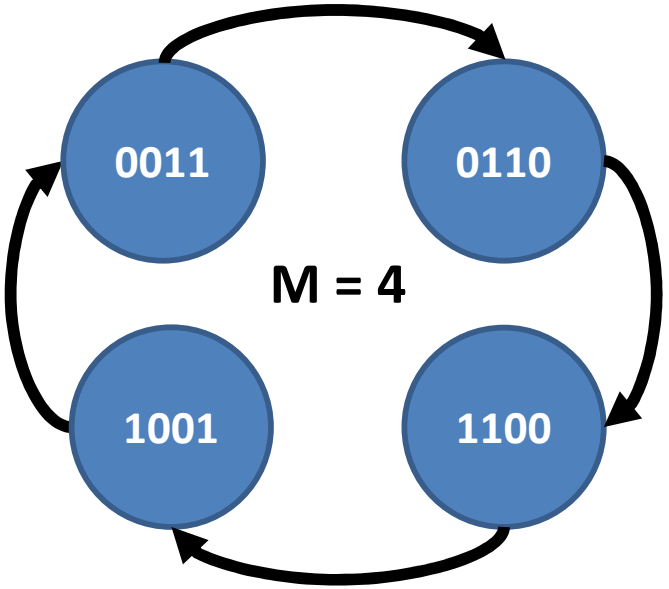
M = n **$< 2^n$** >>> siempre hay estados sin usar

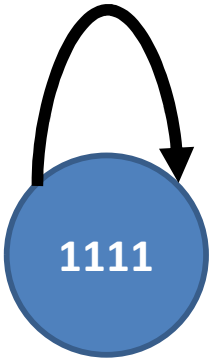
Si precargamos o cae en 0101 se formará un código secundario de M = 2.
Si cae en 0000 o 1111 quedará en ese estado en forma indefinida.

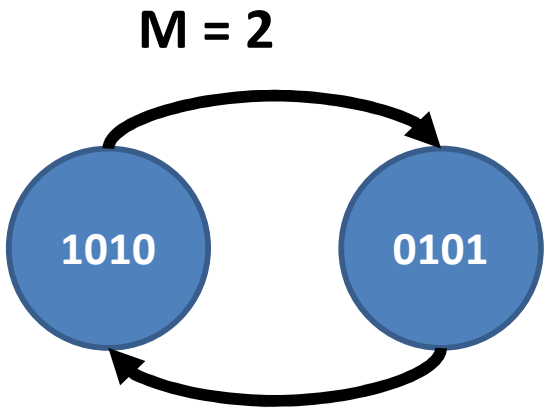
Contador en anillo

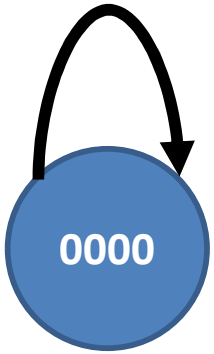
El diagrama de estados **completo** ($Q_3Q_2Q_1Q_0$).

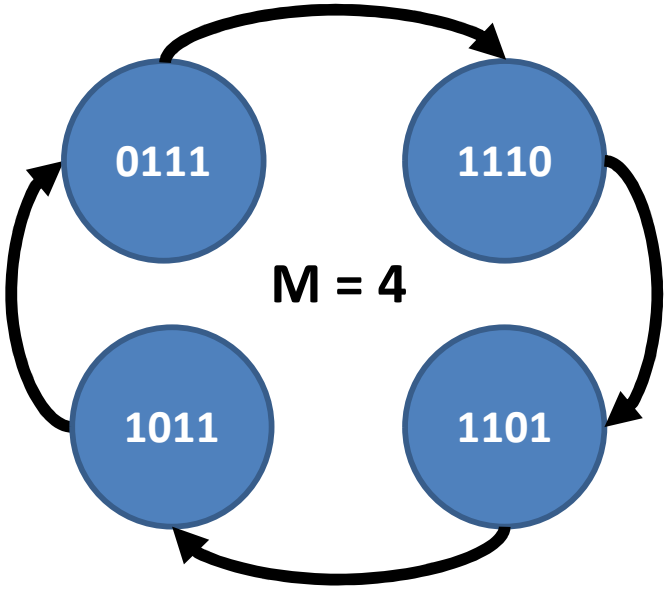






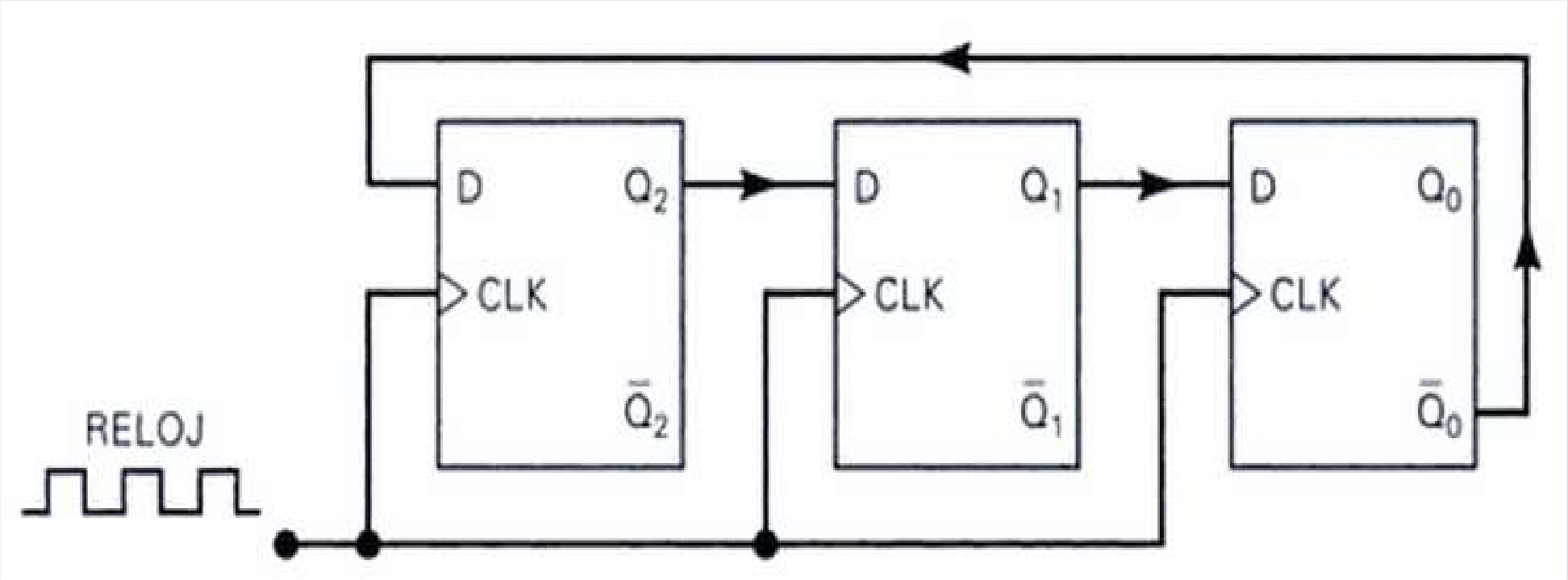






Contador de Johnson

Si **invertimos** la **realimentación**, se obtiene un contador de **Johnson** o **de anillo trenzado**.

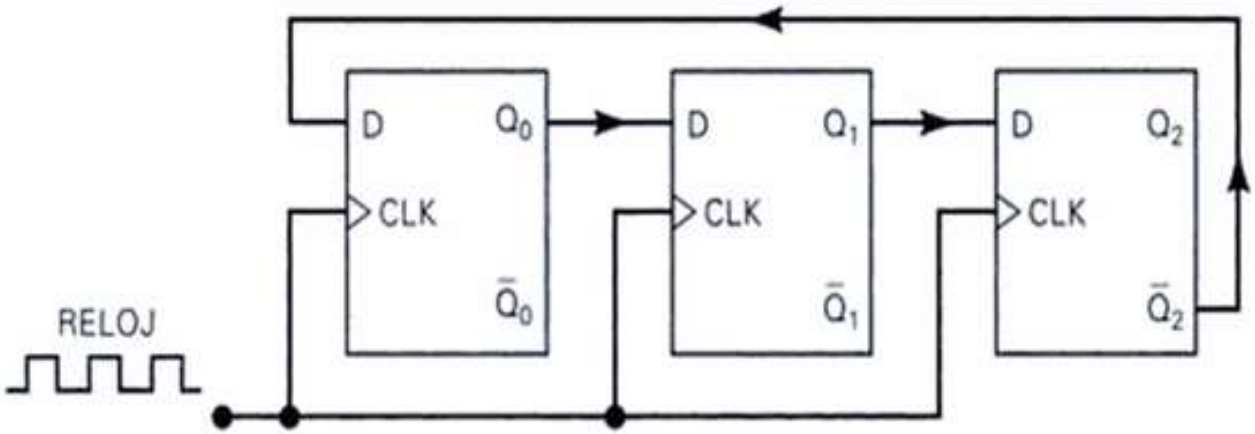


Si en este SR **precargamos 000** obtendremos un código de M = 6.
Generalizando para un contador de Johnson, el módulo principal es igual al doble de la cantidad de biestables.

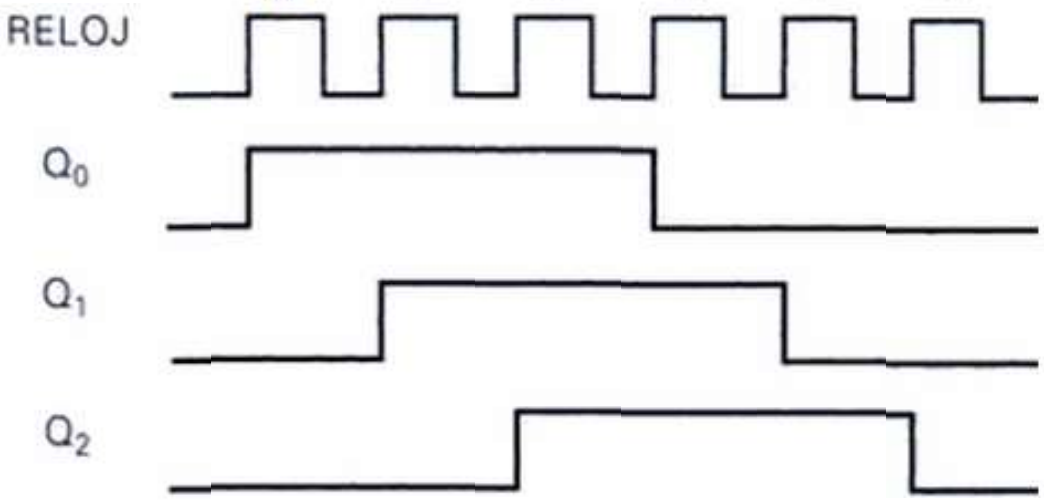
M = 2 . n **< 2^n** >>> siempre hay estados sin usar

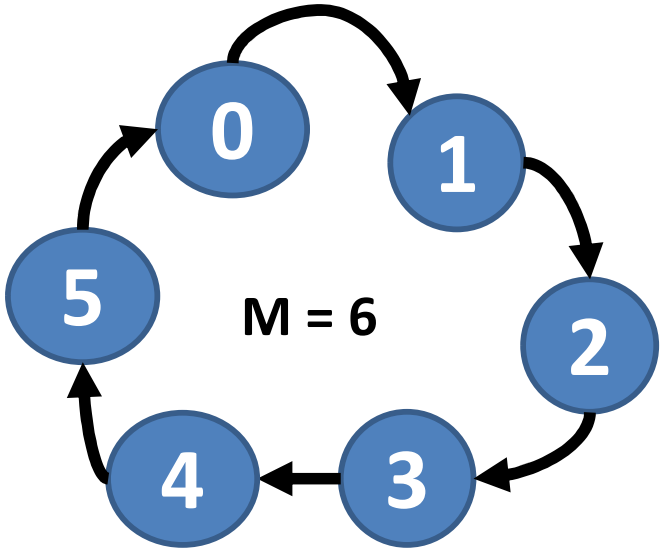
Contador de Johnson

La secuencia de avance de los datos y el diagrama de **estados** serán:



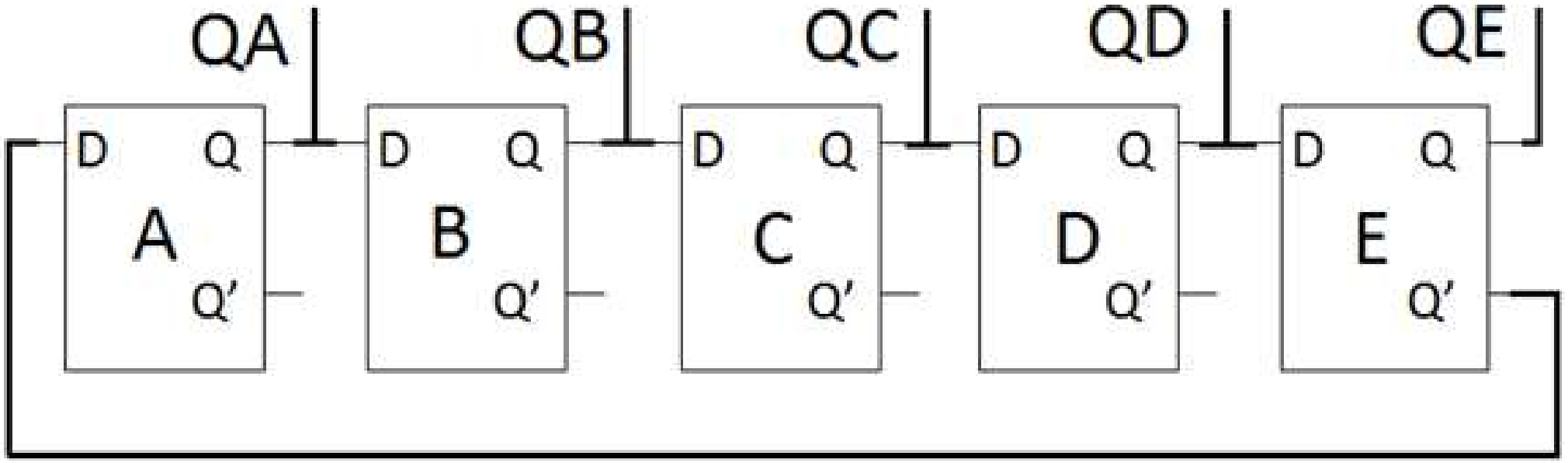
Estado	Q_2	Q_1	Q_0
0	0	0	0
1	0	0	1
2	0	1	1
3	1	1	1
4	1	1	0
5	1	0	0





Contador de Johnson

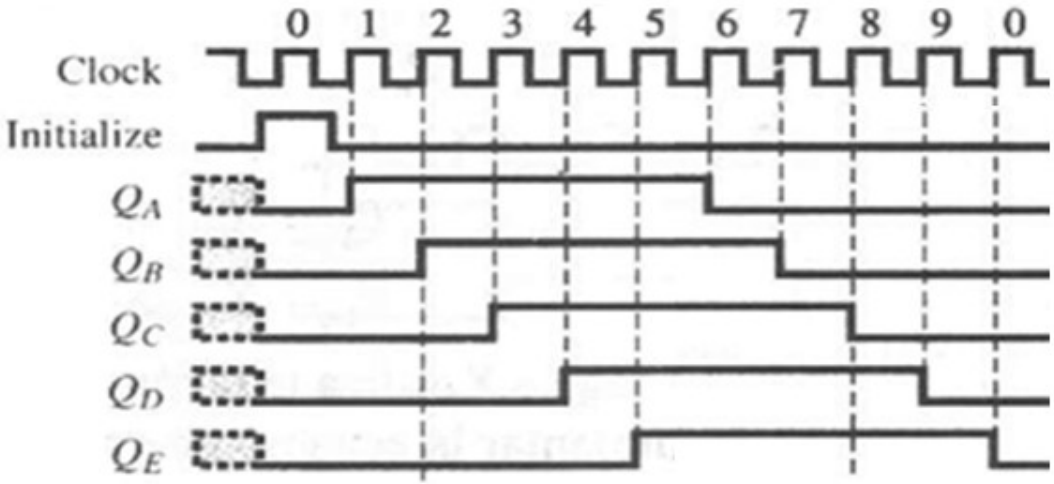
Con 5 biestables **precargando 00000**, podemos obtener un contador de código **BCD** de **Johnson**. Por simplicidad no se grafica el POR y el Ck.

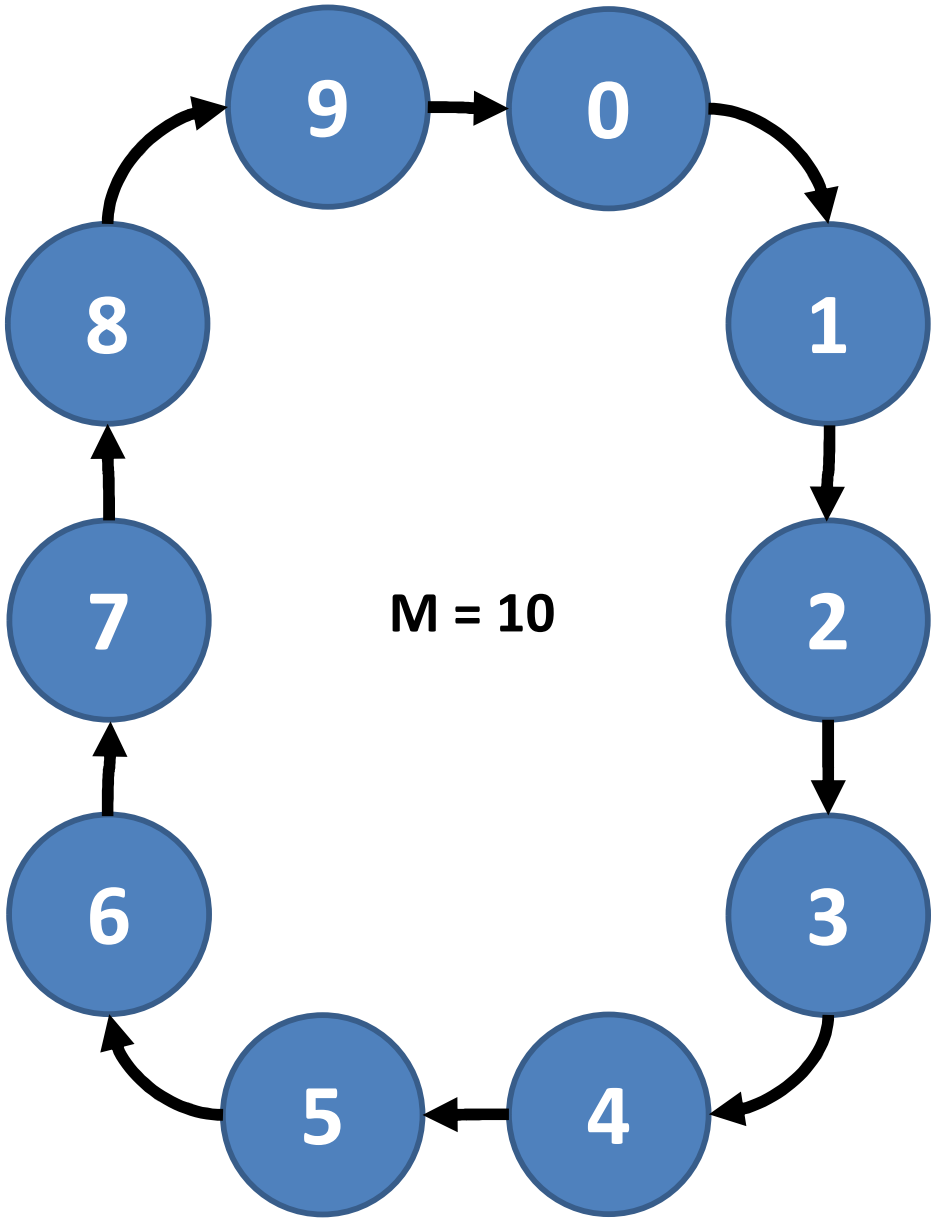


Contador de Johnson

La secuencia de avance de los datos y el diagrama de **estados** serán:

	Q_E	Q_D	Q_C	Q_B	Q_A
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	1
3	0	0	1	1	1
4	0	1	1	1	1
5	1	1	1	1	1
6	1	1	1	1	0
7	1	1	1	0	0
8	1	1	0	0	0
9	1	0	0	0	0







FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Información en los Sistemas Digitales

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Diferenciar entre magnitudes analógicas, digitales y binarias.

Introducir la lógica binaria y su aplicación al diseño de circuitos digitales.

Analizar el concepto de conversión de información analógica a formato digital y viceversa.

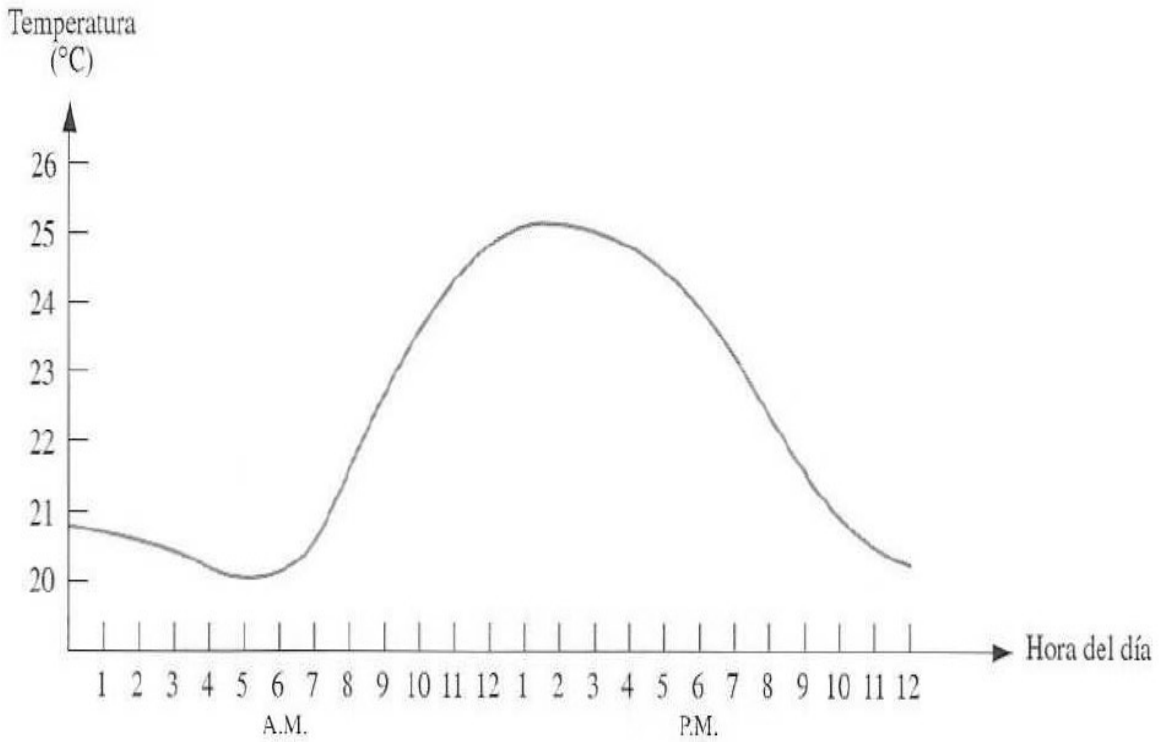
Justificar la necesidad de utilización de conversores analógico/digital y digital/analógico.

Magnitud Analógica

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Las magnitudes físicas normalmente son de tipo **analógico**, es decir que dentro de un cierto rango pueden tomar una cantidad infinita de valores.

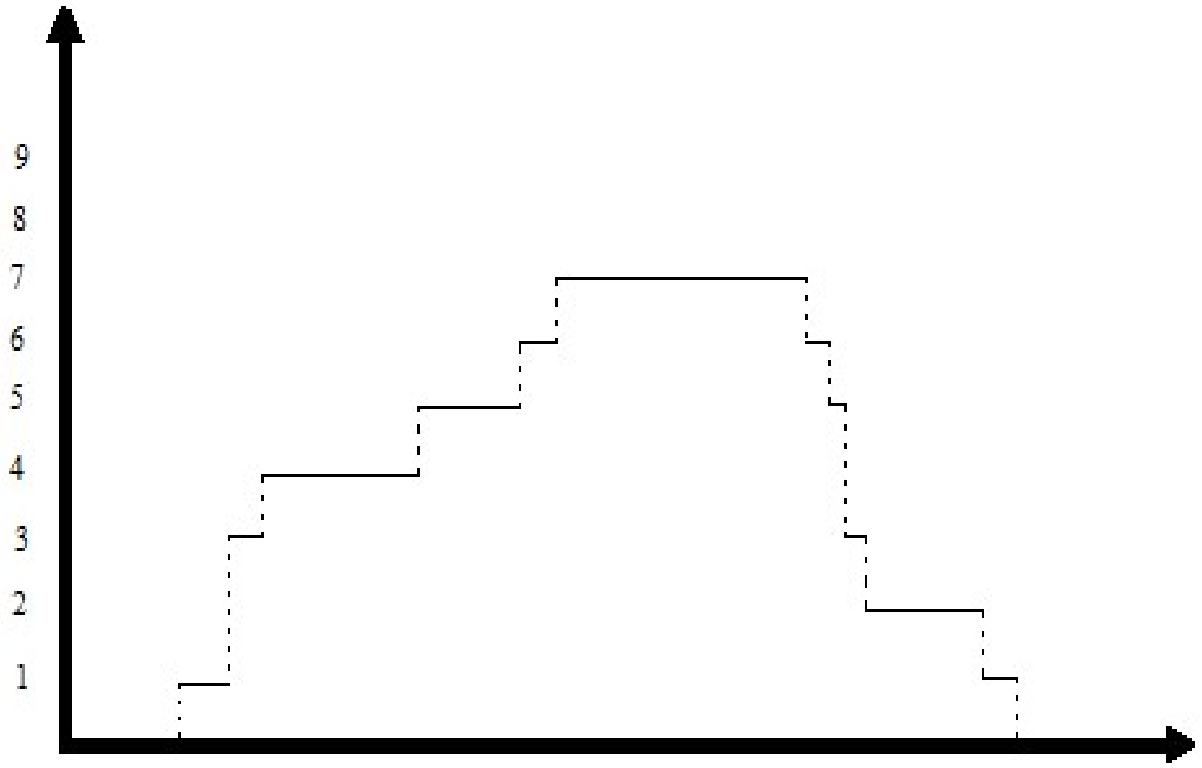
Por ejemplo, la temperatura a lo largo de un período de tiempo podrá variar entre un máximo y un mínimo determinados, pero dentro de ese rango podrá tomar infinitos valores como 23,4709241065…… grados centígrados, es una magnitud analógica, está representada por una **función continua**.



Magnitud Digital

Por el contrario, una función **digital** solo puede tomar una cantidad finita de valores.

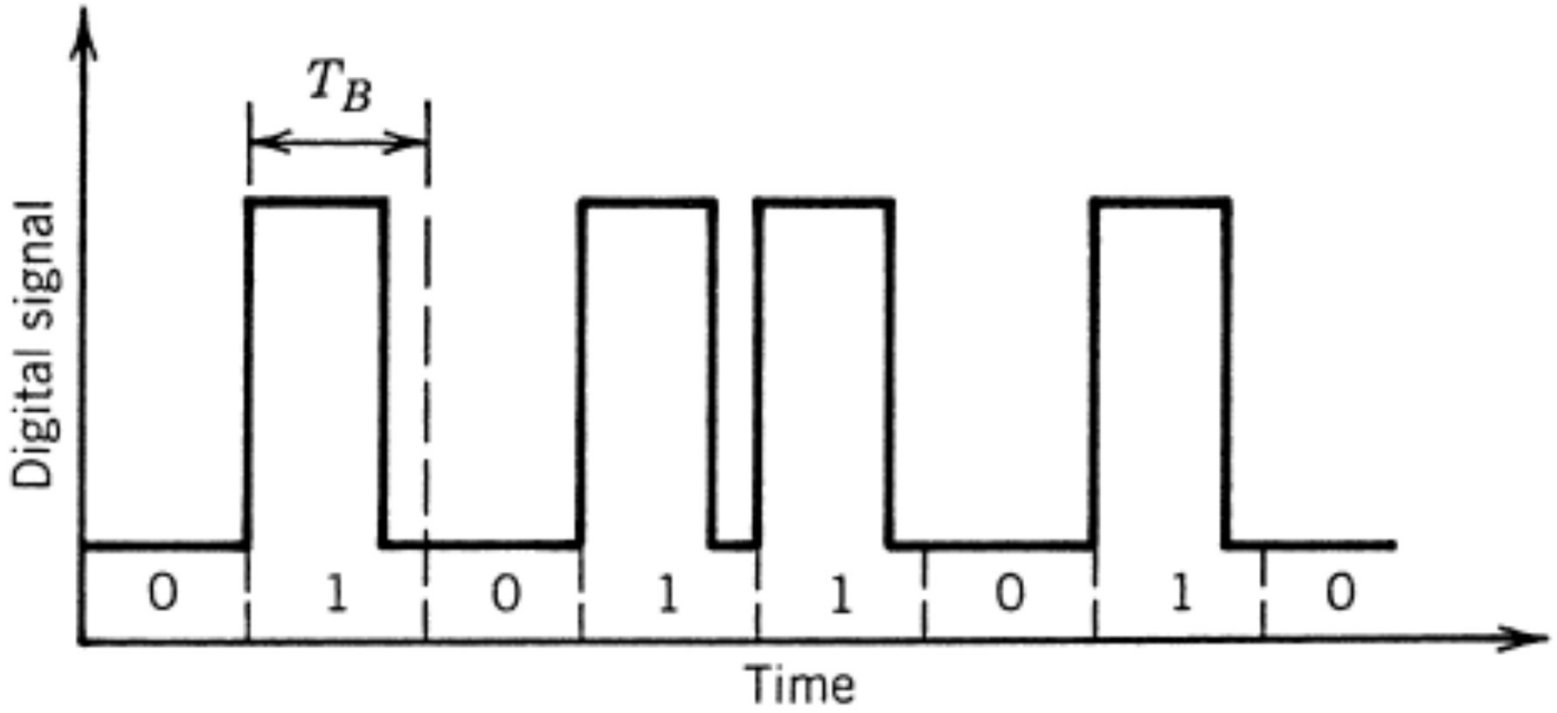
Por ejemplo, la cantidad de personas en una sala en un determinado período de tiempo podrá variar entre un máximo y un mínimo determinados, pero dentro de ese rango solo podrá tomar algunos valores, en este caso enteros, es una magnitud discreta, digital, y está representada por una **función discontinua.**



Magnitud Binaria

Si bien una función **digital** puede tomar una cantidad finita de valores, una función **binaria** solo puede tomar 2 valores.

Es decir que una señal binaria es un caso particular de señal digital. Una señal binaria es digital, pero no necesariamente una señal digital es binaria.



Ejemplos de funciones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Lámpara con interruptor común: función **digital binaria,** solo puede tomar 2 valores, encendida o apagada.

Lámpara con dimmer: función **“analógica”,** puede tomar “infinitos” valores de intensidad.

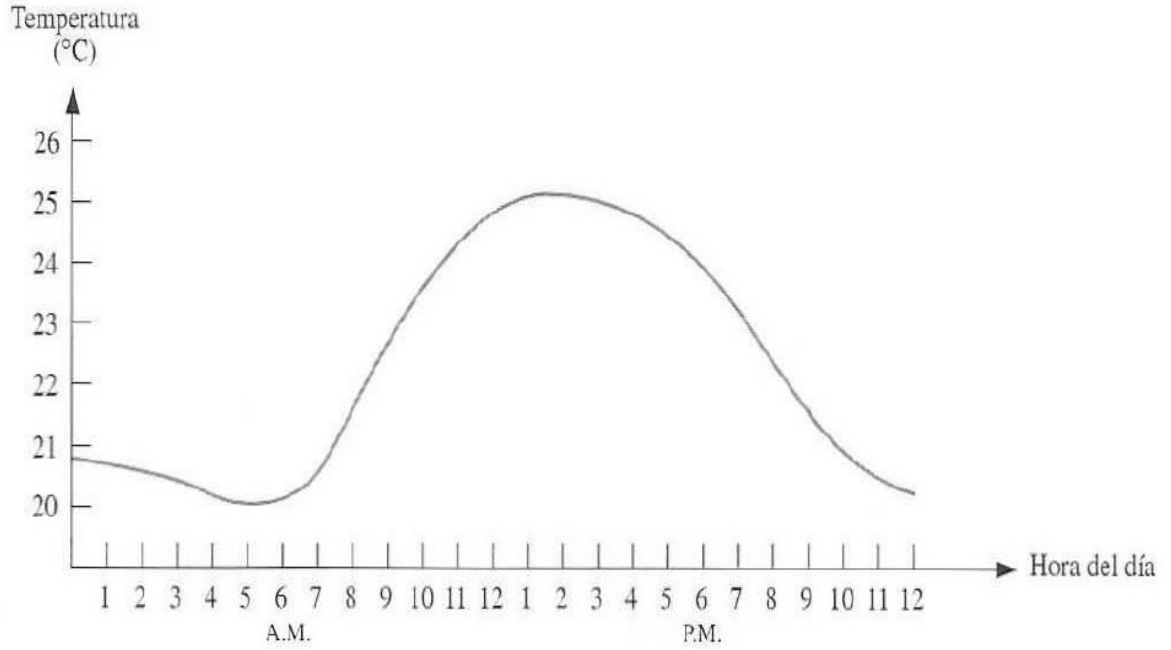
Lámpara con llave de 10 posiciones: función **digital decimal,** solo puede tomar 10 valores de intensidad.

Por qué los **sistemas digitales** son **binarios**?

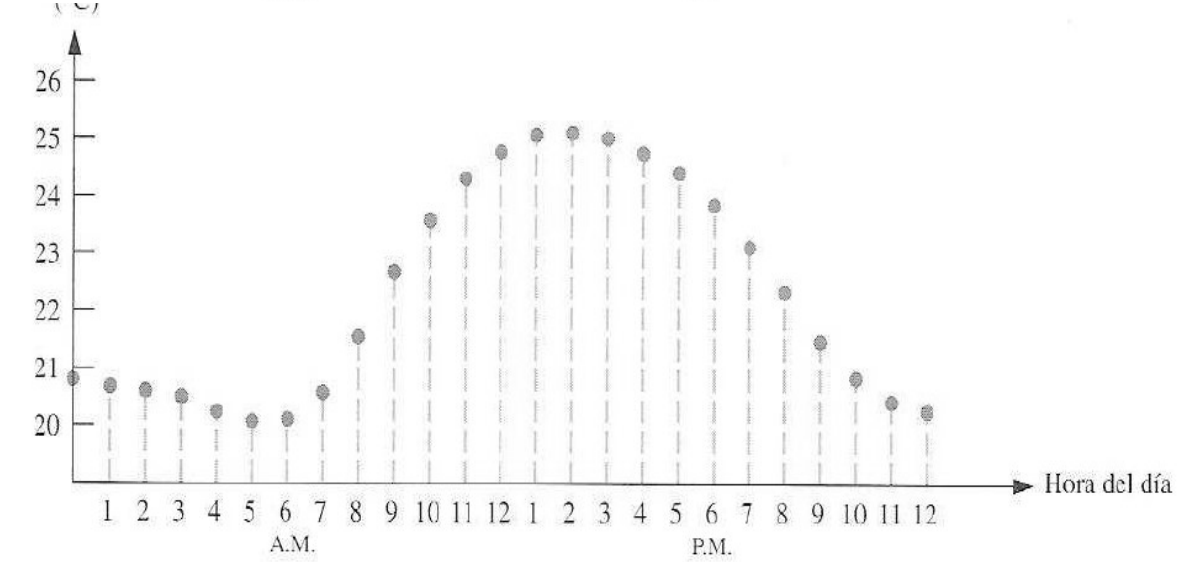
Conversión Analógico Digital

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Una señal analógica se puede convertir a formato digital muestreando a intervalos regulares de tiempo, para esto se utiliza un **conversor analógico/digital** o **A/D Converter.**

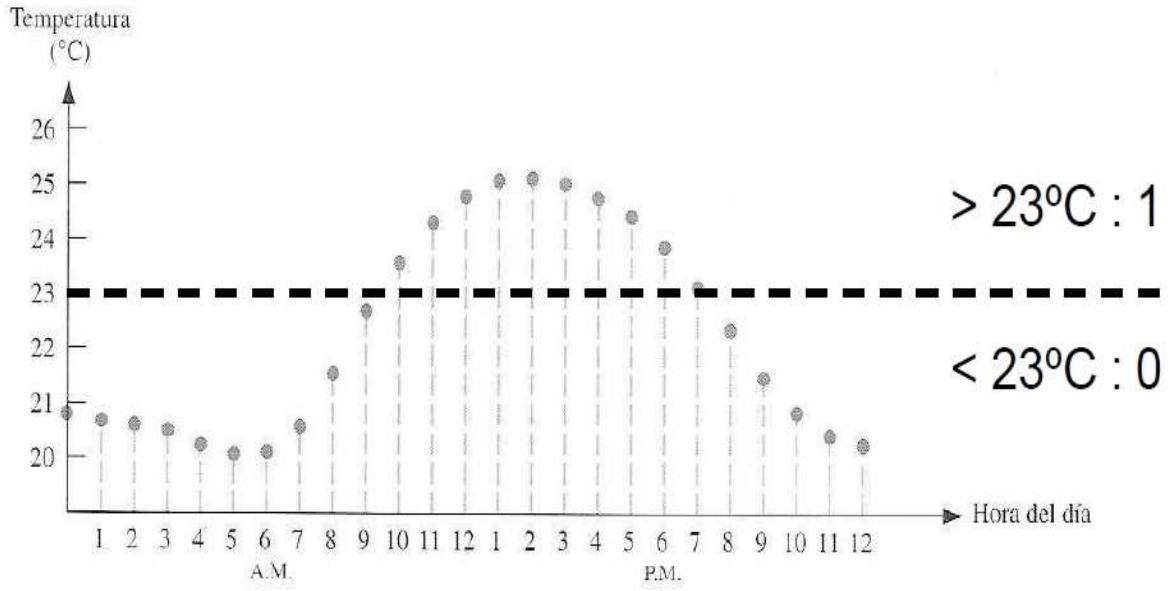


Por ejemplo, para la función temperatura vista anteriormente tendríamos esta sucesión de muestras.

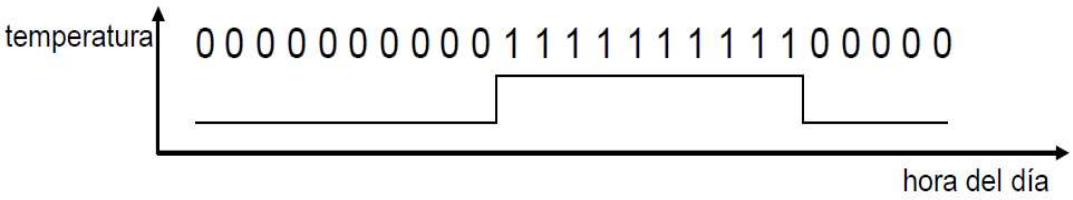


Conversión Analógico Digital

Si creamos 2 niveles posibles (binario) de temperaturas, > 23°C y < 23°C, las muestras caerán en ambos grupos o niveles.



La correspondiente digitalización en 2 niveles resulta en una función digital y en este caso binaria, la que ha perdido calidad.

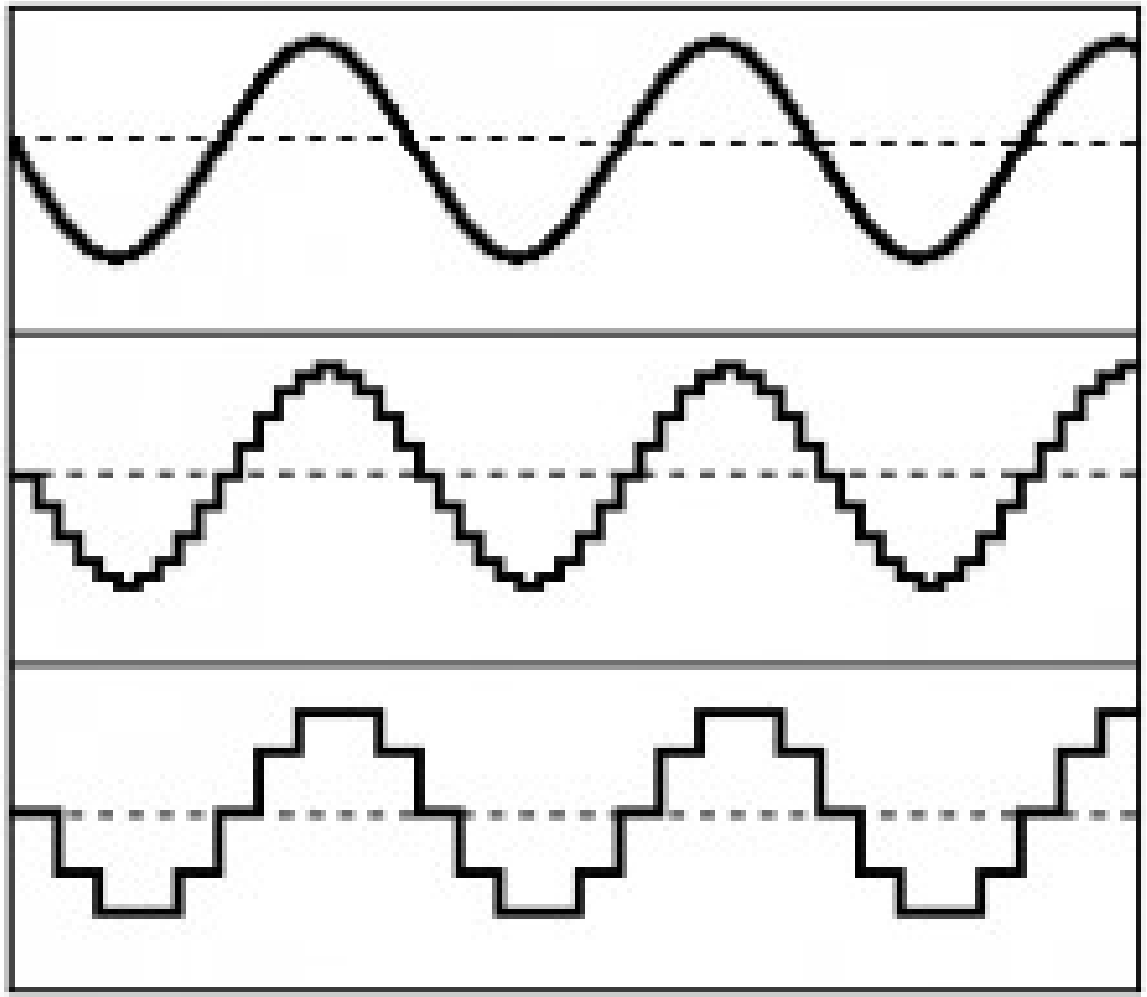


Conversión Analógico Digital

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Aumentando la cantidad de niveles posibles vamos mejorando la reproducción de la señal original.
Para poder identificar más niveles necesitamos una mayor cantidad de bits.

Esto tiene un costo, ya que por cada muestra debo enviar más bits (mayor ancho de banda) o almacenar más bits (mayor tamaño del archivo).

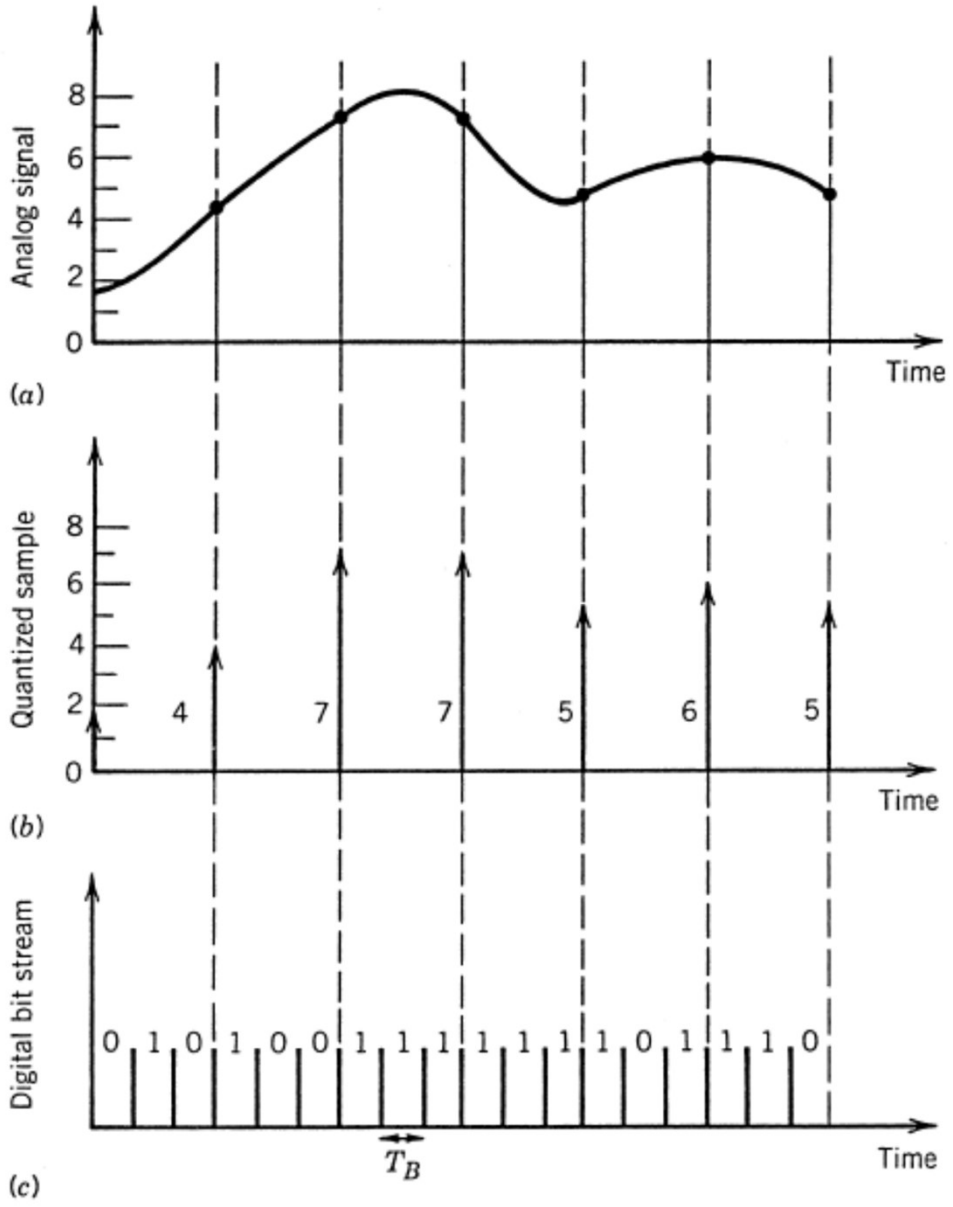


Conversión Analógico Digital

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

La digitalización conduce a un ruido o error adicional, conocido como ruido o **error** de **cuantificación** o **cuantización.**

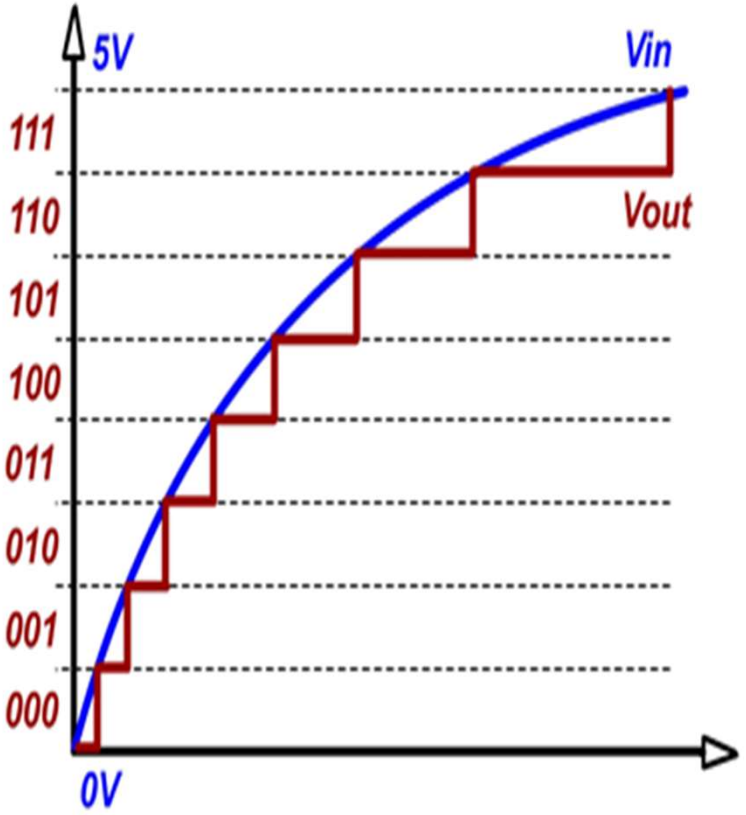
Para disminuirlo debo aumentar la cantidad de niveles posibles, lo que implicará aumentar también la cantidad de bits por muestra, por ejemplo: con **3 bits** por muestra tendremos $2^3 = 8$ intervalos.



Conversión Analógico Digital

Puedo aumentar la cantidad de bits hasta tener tantos intervalos que aseguren que el error de cuantificación sea **aceptable**, pero el **costo** de esta mejora es que por cada muestra necesito una cantidad mayor de bits.

Esto implica mayor ancho de banda y mayor tamaño de archivos.



Ventajas de la Información Digital

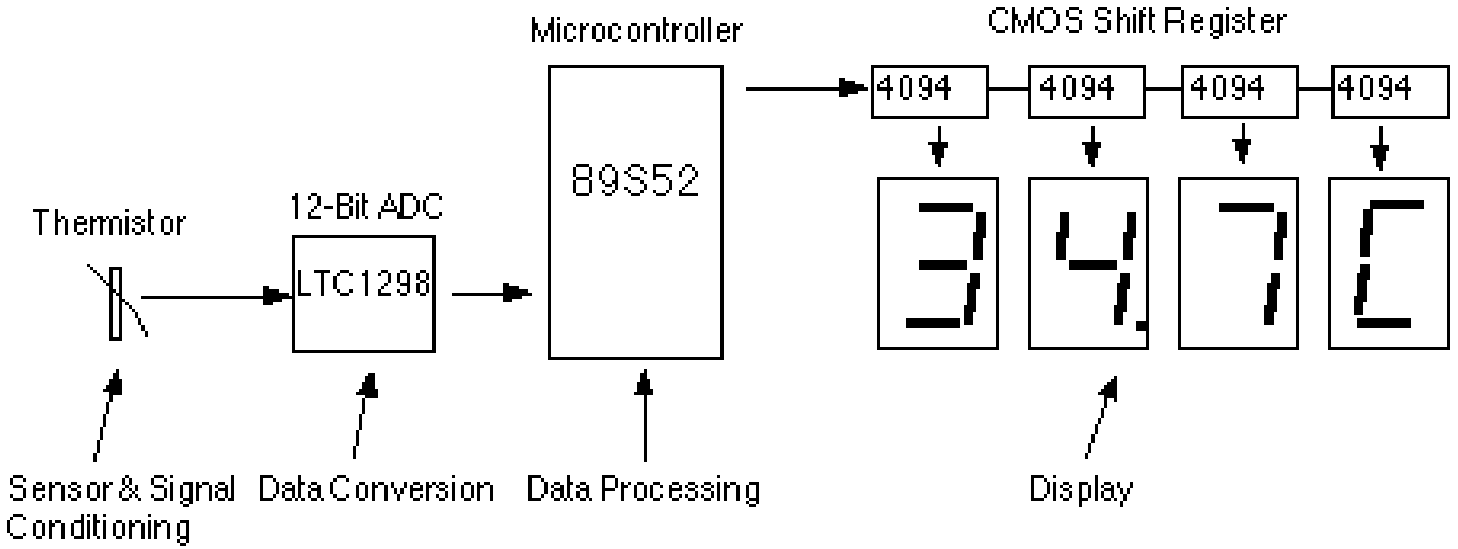
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

El almacenamiento es más compacto.

Procesamiento y Transmisión de Información más eficiente.

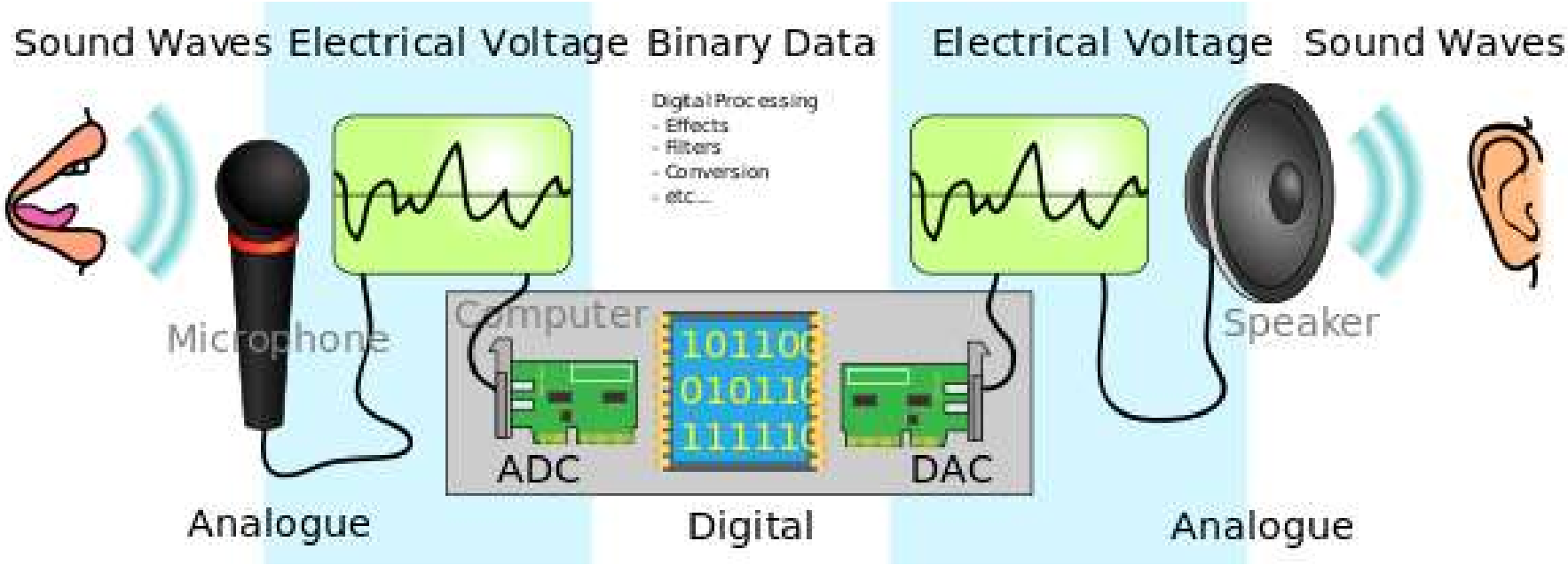
Mejor tolerancia al ruido eléctrico.

Mayor velocidad de procesamiento.



Conversión Digital Analógica

Para restaurar la información analógica digitalizada previamente, por ejemplo sonido, debo utilizar un **conversor digital/analógico** o **D/A Converter.**



FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Sistemas Numéricos

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Comprender la representación de los números en distintos sistemas de numeración y las conversiones entre los mismos.

Introducir los elementos que definen un sistema numérico posicional.

Realizar operaciones aritméticas básicas en binario.

Plantear las diferentes formas de representación de los números negativos. Emplear las mismas para realizar operaciones aritméticas.

Interpretar las funciones de los indicadores (flags) de las unidades aritméticas: transporte, rebalse, signo, paridad, cero, etc.

Definiciones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Definiremos un **sistema de numeración** como un conjunto de símbolos y reglas de generación que permitan construir todos los números válidos del sistema. Un sistema de numeración puede representarse como:

N = S + R

- **N** es el sistema de numeración considerado.
- **S** son los símbolos permitidos del sistema.
- **R** son las reglas de generación que nos indican que números son válidos y cuales no en el sistema.

Dentro de los sistemas numéricos podemos distinguir:

No posicionales

Posicionales, pesados o ponderados

Sistemas numéricos No Posicionales

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

No Posicionales o No Ponderados son aquellos en los cual el valor de los símbolos que componen el sistema es fijo, o sea no depende de la posición que ocupe este dentro del número, ejemplo de este tipo es el sistema romano.



Sistemas numéricos Posicionales

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Posicionales, Pesados o Ponderados son aquellos que el valor de los símbolos depende del valor que se les ha asignado y también de la posición que ocupa el símbolo dentro del número.

Podemos decir que cada símbolo tiene dos valores, uno **absoluto** que depende del valor del signo y otro **relativo** que depende de la posición que ocupa dentro del mismo.

Dígito es cada uno de los símbolos diferentes que constituyen el sistema.

Un dígito binario se denomina **bit**, que proviene del inglés "**Bi**nary Dig**it**".

Base es la cantidad de dígitos que lo forman. Las cantidades mayores a la base se obtienen combinando en forma adecuada los diferentes dígitos del sistema.

$$3435{,}62_{|10} = 3000 + 400 + 30 + 5 + 0{,}6 + 0{,}02$$

$$3435{,}62_{|10} = 3 \cdot 10^3 + 4 \cdot 10^2 + 3 \cdot 10^1 + 5 \cdot 10^0 + 6 \cdot 10^{-1} + 2 \cdot 10^{-2}$$

Sistemas numéricos Posicionales

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Generalizando, un número **N** de **n** dígitos enteros y **k** dígitos fraccionarios, expresado en la base **B**, será:

$$N_B = a_{-k} \cdot B^{-k} + a_{-k+1} \cdot B^{-k+1} + \ldots\ldots\ldots + a_0 \cdot B^0 + \ldots\ldots\ldots + a_{n-2} \cdot B^{n-2} + a_{n-1} \cdot B^{n-1}$$

$$N_B = \sum_{i=-k}^{n-1} a_i \cdot B^i$$

Representación en otras bases

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En toda base **B** tengo **B símbolos**, los números se van formando tal como en decimal.

La base **siempre** se escribe 10.

En **hexadecimal** (16) necesito 6 símbolos más, uso las **6 primeras letras**.

A mayor base, menor cantidad de dígitos.

Bases						
10	**2**	**3**	**4**	**7**	**8**	**16**
0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	**10**	2	2	2	2	2
3	11	**10**	3	3	3	3
4	100	11	**10**	4	4	4
5	101	12	11	5	5	5
6	110	20	12	6	6	6
7	111	21	13	**10**	7	7
8	1000	22	20	11	**10**	8
9	1001	100	21	12	11	9
10	1010	101	22	13	12	A
11	1011	102	23	14	13	B
12	1100	110	30	15	14	C
13	1101	111	31	16	15	D
14	1110	112	32	20	16	E
15	1111	120	33	21	17	F
16	10000	121	100	22	20	**10**
17	10001	122	101	23	21	11
18	10010	200	102	24	22	12
19	10011	201	103	25	23	13

Dígitos binarios y conjuntos de bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Los circuitos lógicos binarios trabajan con señales eléctricas que solo pueden tener dos estados posibles. En **lógica positiva** los estados se representan con dos niveles de tensión eléctrica: **Bajo** (0) y **Alto** (1).

Un dígito binario se denomina **bit**, que proviene del inglés "**Bi**nary Dig**it**".

Los conjuntos de 4, 8, 16, 32 bits reciben denominaciones particulares según veremos a continuación o también "**palabra** de n bits".

Los conjuntos de bits son utilizados para **representar** números, letras, símbolos, etc.

Dígitos binarios y conjuntos de bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Tamaño	Valor mínimo y máximo	Denominación
1 bit	0 – 1	Bit
4 bits	0000 – 1111	Nibble
8 bits	0000 0000 – 1111 1111	Byte
16 bits	0000 0000 0000 0000 – 1111 1111 1111 1111	Word
32 bits	0000 0000 0000 0000 0000 0000 0000 0000 – 1111 1111 1111 1111 1111 1111 1111 1111	Double Word

Cambio de otra base a decimal

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Utilizando esta **representación polinómica** podemos convertir un número expresado en una base cualquiera al sistema de numeración decimal.

Por ejemplo para el número binario $1010,01_{|2}$, tenemos k=2, n=4 y B=2:

$$1010,01_{|2} = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} =$$
$$= (8) + (0) + (2) + (0) + (0/2) + (1/4) = 10,25$$

Para el número expresado en base 3 $201,02_{|3}$, tenemos k=2, n=3 y B=3:

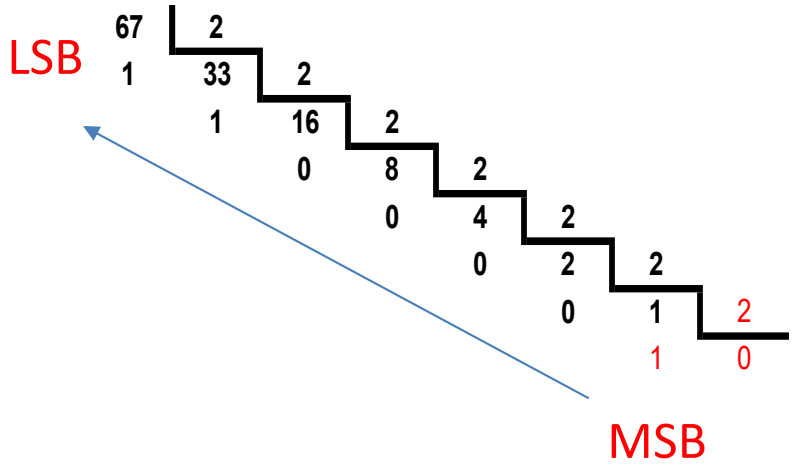
$$201,02_{|3} = 2 \times 3^2 + 0 \times 3^1 + 1 \times 3^0 + 0 \times 3^{-1} + 2 \times 3^{-2} =$$
$$= (18) + (0) + (1) + (0/3) + (2/9) = 19,22222222$$

Definición de MSB y LSB, MSD y LSD.

Cambio de decimal a otra base

Podemos convertir cualquier número decimal a otra base mediante el siguiente método, lo veremos con un ejemplo $67{,}74_{|10}$ a base 2.

Tomamos la **parte entera** y calculamos los residuos de sucesivas divisiones enteras por la base de llegada:



Teniendo en cuenta el sentido (der. a izq.) tenemos: $67 = 1000011_{|2}$

Podemos verificarlo:

$$1000011_{|2} = 1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 67$$

Cambio de decimal a otra base

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Para la **parte fraccionaria** multiplicamos por la base de llegada y la parte entera del producto la agregamos al número, luego seguimos multiplicando la parte fraccionaria:

$0{,}74 \times 2 = 1{,}48$ >>>>> 1

$0{,}48 \times 2 = 0{,}96$ >>>>> 0

$0{,}96 \times 2 = 1{,}92$ >>>>> 1

$0{,}92 \times 2 = 1{,}84$ >>>>> 1

Entonces obtenemos:

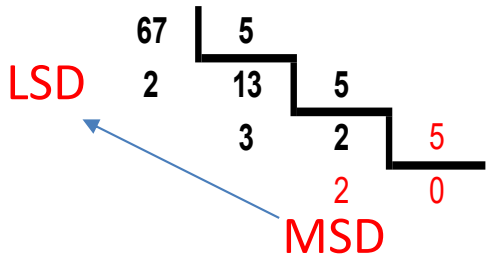
$$67{,}74 = 1000011{,}1011_{|2}$$

Pero hasta qué cifra seguimos si los restos no se hacen cero? Depende del error de truncamiento que queramos, será menor que el peso del último dígito a la derecha: $\epsilon < B^{-k}$

En este caso tenemos B = 2 y k = 4, por lo tanto $\epsilon < 2^{-4}$

Cambio de decimal a otra base

Ahora pasemos el mismo número $67,74_{|10}$ a base 5, pero como requisito adicional **$\epsilon < 1/100$**:



Tenemos: $67 = 232_{|5}$

Podemos verificarlo:

$$232_{|5} = 2 \times 5^2 + 3 \times 5^1 + 2 \times 5^0 = 67$$

Para la parte fraccionaria:

0,74 x 5 = 3,7	>>>>>	3
0,7 x 5 = 3,5	>>>>>	3
0,5 x 5 = 2,5	>>>>>	2

Resultando: $67,74 = 232,332_{|5}$ con $\epsilon < 5^{-3} = 1/125 < 1/100$

Base potencia de otra

Para pasar por ejemplo de base 7 a base 5, puedo hacerlo de forma directa pero operando en la base de llegada, por lo tanto lo más recomendable es pasar de base 7 a base 10 y luego de base 10 a base 5.

En el caso particular de una base que es potencia de la otra, $\mathbf{B_2 = B_1^k}$, puedo usar un método más sencillo: utilizando **k** bits para representar en binario un dígito del sistema de numeración de base $\mathbf{2^k}$, siendo **k** un número entero, por lo que para base $\mathbf{8 = 2^3}$ se requieren tres bits por dígito, en tanto que para base $\mathbf{16 = 2^4}$ se requieren cuatro bits por dígito.

Ejemplo: pasar $346,D_{|H}$ a binario y a octal.

3 4 6 , $D_{|H}$

$0\,0\,1\,1\,0\,1\,0\,0\,0\,1\,1\,0\,,1\,1\,0\,1_{|2}$

$0\,0\,1\,1\,0\,1\,0\,0\,0\,1\,1\,0\,,1\,1\,0\,1\,0\,0_{|2}$

1 5 0 6 , 6 $4_{|8}$

Aritmética en otras bases

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

+	0	1	2	3	4	5	6	7	8	9
0	0	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9	10
2	2	3	4	5	6	7	8	9	10	11
3	3	4	5	6	7	8	9	10	11	12
4	4	5	6	7	8	9	10	11	12	13
5	5	6	7	8	9	10	11	12	13	14
6	6	7	8	9	10	11	12	13	14	15
7	7	8	9	10	11	12	13	14	15	16
8	8	9	10	11	12	13	14	15	16	17
9	9	10	11	12	13	14	15	16	17	18

x	0	1	2	3	4	5	6	7	8	9
0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9
2	0	2	4	6	8	10	12	14	16	18
3	0	3	6	9	12	15	18	21	24	27
4	0	4	8	12	16	20	24	28	32	36
5	0	5	10	15	20	25	30	35	40	45
6	0	6	12	18	24	30	36	42	48	54
7	0	7	14	21	28	35	42	49	56	63
8	0	8	16	24	32	40	48	56	64	72
9	0	9	18	27	36	45	54	63	72	81

+	0	1
0	0	1
1	1	10

x	0	1
0	0	0
1	0	1

+	0	1	2
0	0	1	2
1	1	2	10
2	2	10	11

x	0	1	2
0	0	0	0
1	0	1	2
2	0	2	11

Tablas de suma y producto en bases 10, 2 y 3

Aritmética binaria

Este ejemplo que veremos es muy simple, por ejemplo queremos **sumar** y **multiplicar** dos números binarios, $6 = 110_{|2}$ y $5 = 101_{|2}$.

$$\begin{array}{cccc} & 1 & 1 & 0 \\ + & 1 & 0 & 1 \\ \hline 1 & 0 & 1 & 1 \end{array}$$

$$\begin{array}{cccc} & & 1 & 1 & 0 \\ \times & & 1 & 0 & 1 \\ \hline & & 1 & 1 & 0 \\ + & 1 & 1 & 0 & 0 \\ \hline & 1 & 1 & 1 & 1 & 0 \end{array}$$

Vemos que obviamente el resultado de la suma y la multiplicación dan 11 y 30 respectivamente.

Los sistemas digitales normalmente utilizan un algoritmo para realizar el producto, **no necesitan la tabla del producto binario**, solo la de suma.

Suma y resta binaria

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Suma: ya vimos la tabla de suma binaria, aparece el “me llevo” a la columna inmediata a la izquierda (carry).

+	0	1
0	0	1
1	1	10

A	B	Suma	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Resta: de la misma forma en la resta aparece el “pido prestado” de la columna inmediata a la izquierda (borrow).

A	B	Resta	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Complementos

<u>Complemento a la base (o al módulo)</u>: (o complemento a **10**) es lo que le falta a un número para alcanzar a una potencia de la base.

$$C(A) = B^n - A$$

Ejemplo: $C_{10}(345) = 10^3 - 345 = 655$

$$\begin{array}{rr} 862 & 862 \\ - \ \underline{345} & +\underline{655} \\ 517 & 1517 \end{array}$$

<u>Complemento a la base (o al módulo) menos 1</u>: (o complemento a **9**) es lo que le falta a un número para alcanzar a una potencia de la base menos 1.

$$C(A) = (B^n - 1) - A$$

$C_9(345) = (10^3 - 1) - 345 = 999 - 345 = 654$

$$\begin{array}{rr} 862 & 862 \\ - \ \underline{345} & +\underline{654} \\ 517 & 1516 \\ & \underline{+1} \\ & 517 \end{array}$$

Reglas: Complemento dígito a dígito.

Si hay acarreo lo debo sumar.

Complementos en binario

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Complemento a la base (o al módulo): (o complemento a **2**) es lo que le falta a un número para alcanzar a una potencia de la base.

$$C(A) = B^n - A$$

Ejemplo: $C_2(0101) = 2^4 - 0101_{|2} = 10000_{|2} - 0101_{|2} = 1011_{|2}$

Regla: recorro de derecha a izquierda, el primer uno que encuentro queda como está y después invierto todos los bits.

Complemento a la base (o al módulo) menos 1: (o complemento a **1**) es lo que le falta a un número para alcanzar a una potencia de la base menos 1.

$$C(A) = (B^n - 1) - A$$

Ejemplo: $C_1(0101) = (2^4 - 1) - 0101_{|2} = 1111_{|2} - 0101_{|2} = 1010_{|2}$

Regla: complemento bit a bit.

Otra forma: $\mathbf{C_{B-1} = C_B - 1}$ >>>>> $\mathbf{C_B = C_{B-1} + 1}$

$$C_2(0101) = C_1(0101) + 1 = 1010_{|2} + 1 = 1011_{|2}$$

Representación de números negativos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Convención: para representar números negativos los sistemas digitales utilizan el bit más significativo para indicar el signo, si toma el valor 0 interpreta que es un número positivo, y si es 1 que corresponde a uno negativo.

Convenciones de representación: complementos permiten realizar restas mediante sumas.

Ejemplo: 1011 representa 11, - 3, - 5, o - 4 según cada uno de los casos.

Número binario	Convención			
	Valor Abs.	BS y VA	C_2	C_1
0000	0	+0	0	+0
0001	1	1	1	1
0010	2	2	2	2
0011	3	3	3	3
0100	4	4	4	4
0101	5	5	5	5
0110	6	6	6	6
0111	7	7	7	7
1000	8	-0	-8	-7
1001	9	-1	-7	-6
1010	10	-2	-6	-5
1011	11	-3	-5	-4
1100	12	-4	-4	-3
1101	13	-5	-3	-2
1110	14	-6	-2	-1
1111	15	-7	-1	-0
Rango de representación	$\{0 ; +2^n - 1\}$	$\{-2^{n-1} - 1 ; +2^{n-1} - 1\}$ ∃ +0 ^ -0	$\{-2^{n-1} ; +2^{n-1} - 1\}$	$\{-2^{n-1} - 1 ; +2^{n-1} - 1\}$ ∃ +0 ^ -0

Flags o banderas de la UAL

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Indican características del resultado de una operación. Los circuitos directamente realizan la operación, la interpretación del resultado la hace el usuario o el programador del sistema digital. Dentro de un sistema microprocesado, estos indicadores generalmente forman parte de un registro llamado de estado o de códigos de condición.

C: Carry (acarreo): Cuando sumo dos números de **n bits** puede ocurrir que necesite **n + 1 bits,** este bit adicional es el **Carry.** Es importante tener el concepto que es **independiente** de que los operandos tengan signo o no. O sea que se puede producir sumando números con signo o sin signo.

V: Overflow (desbordamiento o rebalse): Este fenómeno ocurre solo cuando trabajo con números **con signo.** Sucede solo cuando sumo dos positivos o dos negativos y el resultado tiene el signo contrario, es decir que se cambia el **bit de signo,** lo cual es una incoherencia. Nunca existirá Overflow en la suma de un número negativo y un número positivo.
Si bien la UAL siempre lo informa, cuando sumo números sin signo puedo ignorarlo.

Flags o banderas de la UAL

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Z: Zero (cero): Esta bandera indica que el resultado es cero.

N: Negative (negativo): También llamado S (sign), indica el signo del resultado.

P: Parity (paridad): es una convención, por ejemplo usando paridad **par** este bit se pone en 1 si la cantidad de unos del resultado es impar**,** y se pone en 0, si la cantidad de unos del resultado es par**.** Esto es así para que toda la palabra completa (resultado más bit de paridad) **tenga una cantidad par de unos.** Lo contrario ocurre usando paridad **impar.**

H: Half-Carry (acarreo de primer nibble): En el caso de realizar operaciones de suma con números binarios de 8 bits, esta bandera indica que se produjo un acarreo en la suma de los 4 bits menos significativos.

Operaciones con complementos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

2	0010	$C = 0$
$+\ \underline{3}$	$+\ \underline{0011}$	$V = 0$
5	0101	$N = 0$
		$Z = 0$

5	0101	$C = 0$
$+\ \underline{6}$	$+\ \underline{0110}$	$V = 1$
11	1011	$N = 1$
		$Z = 0$

Sin signo **ignoro** el Overflow!!!

C_1:

4	0100	$C = 1$
$-\ \underline{1}$	$+\ \underline{1110}$	$V = 0$
3	10010	$N = 0$
	$\underline{+\ 1}$	$Z = 0$
	0011	

En C_1 debo **sumar** el Carry!!!
(desventaja pero más fácil complementar)

C_2:

4	0100	$C = 1$
$-\ \underline{1}$	$+\ \underline{1111}$	$V = 0$
3	10011	$N = 0$
		$Z = 0$

En C_2 **no sumo** el Carry!!!
(ventaja pero más difícil complementar)

Operaciones con complementos

C_1:

3	0011	C = 0
- 5	+1010	V = 0
- 2	1101	N = 1
		Z = 0

C_2:

3	0011	C = 0
- 5	+1011	V = 0
- 2	1110	N = 1
		Z = 0

C_1:

- 4	1011	C = 1
- 2	+ 1101	V = 0
- 6	11000	N = 1
	+ 1	Z = 0
	1001	

C_2:

- 4	1100	C = 1
- 2	+ 1110	V = 0
- 6	11010	N = 1
		Z = 0

En C_1 debo **sumar** el Carry!!!

En C_2 **no sumo** el Carry!!!

Debo **complementar** siempre los resultados **negativos**!!!

$C_1(1001) = 0110$

$C_2(1010) = 0110$

En ambos casos el resultado es: – 6

Operaciones con complementos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

C_1:

3	0011	$C = 0$
-3	$+\ 1100$	$V = 0$
0	1111	$N = 1$
		$Z = 0$

C_2:

3	0011	$C = 1$
-3	$+\ 1101$	$V = 0$
0	10000	$N = 0$
		$Z = 1$

Vemos que la **misma cuenta** en complementos distintos arroja valores de flags distintos.

La UAL no interpreta resultados más allá de los Flags, para un mismo par de sumandos, los **resultados** deben interpretarse según la **convención** que se esté utilizando. Por ejemplo:

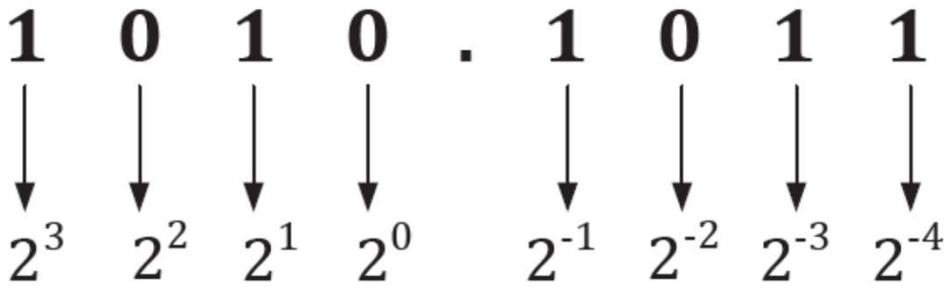
0101	$C = 1$
$+\ 1110$	$V = 0$
10011	$N = 0$
	$Z = 0$

Si es una suma de números **sin signo** (5+14), considerando el Carry el resultado es 19, si se trata de **C_1** (5-1), debo sumar el Carry obteniendo como resultado 4, y si se trata de **C_2** (5-2), el resultado es 3.

Punto fijo

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: utilizo 1 byte para representar un número positivo fraccionario



$$8+2 = 10 \; + \; 0{,}5+0{,}125+0{,}0625 = 0{,}6875$$

$$10{,}6875$$

Puedo representar los siguientes rangos:

$$\{0\} \; ; \; \{ + 0{,}0001_{|2} \; ; \; + 1111{,}1111_{|2}\}$$

Punto flotante

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En muchos cálculos el intervalo de números que se usan es muy grande.

La masa del electrón, 9×10^{-28} gramos

0000000000000000000000000000.000000000000000000000009

La masa del Sol, 2×10^{33} gramos

2000000000000000000000000000000.000000000000000000000000

$$\mathbf{n = f \times 10^{e}}$$

$$\mathbf{\pm 1,Mantisa \times 2^{E}}$$

Punto flotante

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Norma IEEE 754:

Simple precisión: 32 bits (1 + 8 + 23)

signo	exponente	mantisa
1 bit	8 bits	23 bits

Doble precisión: 64 bits (1 + 11 + 52)

signo	exponente	mantisa
1 bit	11 bits	52 bits

Sumas y más sumas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Cuáles de las siguientes operaciones son correctas?

1 + 1 = 2	Suma aritmética B > 2
1 + 1 = 10	Suma aritmética B = 2
1 + 1 = 1	Suma lógica

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Álgebra de Boole

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar el concepto de sistema digital y su forma de estudio.

Estudiar la base matemática que permita la descripción, el análisis y la síntesis de un sistema digital.

Desarrollar los conceptos de funciones lógicas como herramientas de descripción del comportamiento de los circuitos elementales (compuertas) utilizados como bloques unitarios constructivos de otros más complejos.

Definiciones

Un álgebra puede definirse con un conjunto de elementos, un conjunto de operadores y un número de axiomas no probados o **postulados**.

En 1854 George **Boole** presentó un tratamiento sistemático de la lógica y desarrolló para este propósito un sistema algebraico que ahora se conoce como álgebra de Boole o álgebra booleana.

En 1938 Claude E. **Shannon** introdujo un álgebra booleana de dos valores denominada álgebra de interruptores, en la cual demostró que las propiedades de los circuitos eléctricos biestables con interruptores pueden representarse con este álgebra.

Para la definición formal del álgebra booleana, se emplean los postulados formulados por Edward V. **Huntington** en 1904. Estos postulados o axiomas no son únicos para definir el álgebra booleana, también se han usado otros conjuntos de postulados distintos a los que plantearemos aquí.

Definiciones

El álgebra booleana es una estructura algebraica definida en un conjunto de elementos M junto con dos operadores binarios + y • siempre y cuando se cumpla con los postulados de Huntington. Al operador binario + se lo denomina unión, disyunción o suma lógica (**OR**), y al operador binario • se lo denomina intersección, conjunción o producto lógico (**AND**). Además, tenemos una tercera operación que es llamada complemento, negación o inversión (**NOT**). Las operaciones OR, AND y NOT son las fundamentales de este álgebra.

El álgebra booleana rige la lógica binaria, la teoría de conjuntos, la lógica proposicional, la lógica de contactos, interruptores o pulsadores, etc.

Principio de Dualidad

Establece que a toda relación o ley lógica le corresponderá su ley dual, formada mediante el intercambio de los operadores de suma con los de producto, y de los 1s con los 0s, y viceversa.

Postulados de Huntington

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

P1) Conmutatividad

P1a) $A + B = B + A$

P1b) $A \bullet B = B \bullet A$

P2) Distributividad

P2a) $A \bullet (B + C) = (A \bullet B) + (A \bullet C)$

P2b) $A + (B \bullet C) = (A + B) \bullet (A + C)$

P3) Identidad o Invariancia

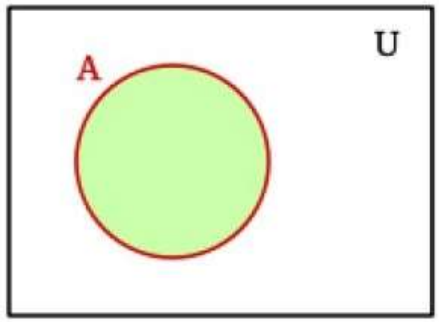
P3a) $A + 0 = A$

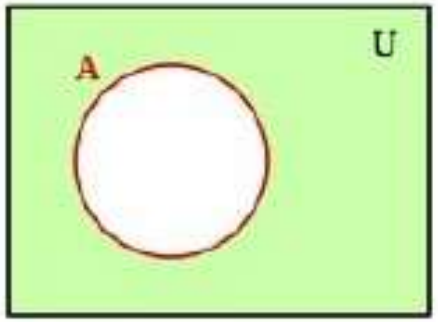
P3b) $A \bullet 1 = A$

P4) Complemento

P4a) $A + \bar{A} = 1$

P4b) $A \bullet \bar{A} = 0$

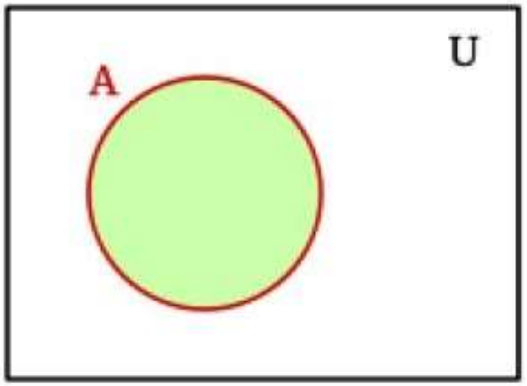




Teoremas o Propiedades

Los **teoremas** se demuestran mediante los **postulados de Huntington**

T1) <u>Idempotencia</u>



T1a) **A + A = A**

Demostración:

A + A	**= (A + A) • 1**	por P3b)
	= (A + A) • (A + Ā)	P4a)
	= A + (A • Ā)	P2b)
	= A + 0	P4b)
	= A	P3a) l.q.q.d.

Teoremas o Propiedades

T1b) **A • A = A**

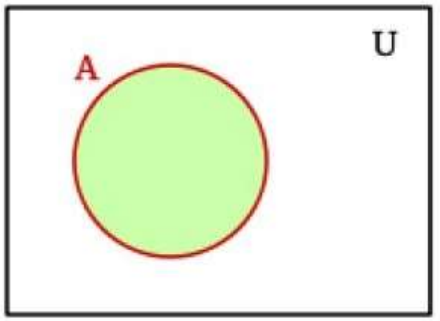
Demostración:

A • A	**= (A • A) + 0**	por P3a)	
	= (A • A) + (A • Ā)	P4b)	
	= A • (A + Ā)	P2a)	
	= A • 1	P4a)	
	= A	P3b)	l.q.q.d.

T2) <u>Existencia de elementos nulos</u>

T2a) **A + 1 = 1**

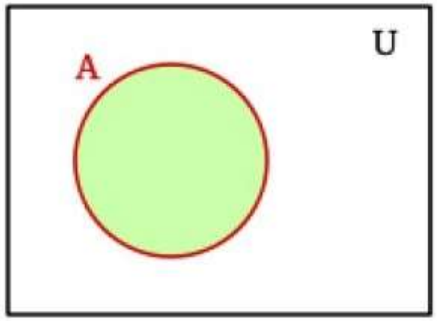
T2b) **A • 0 = 0**



Teoremas o Propiedades

T3) <u>Involución</u> (único)

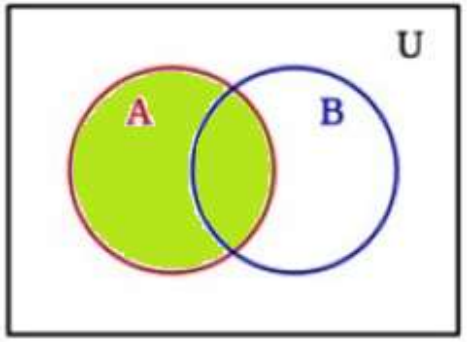
$$\overline{\overline{A}} = A$$



T4) <u>Absorción</u>

T4a) **A + A • B = A**

T4b) **A • (A + B) = A**



T5) <u>Asociatividad</u>

T5a) **A + (B + C) = (A + B) + C**

T5b) **A • (B • C) = (A • B) • C**

Teoremas o Propiedades

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

T6) <u>Leyes de De Morgan</u>

T6a) $\overline{A + B} = \overline{A} \bullet \overline{B}$

T6b) $\overline{A \bullet B} = \overline{A} + \overline{B}$

Corolarios:

$$\overline{\overline{A + B}} = \overline{A} \bullet \overline{B}$$

$$\overline{\overline{A \bullet B}} = \overline{A} + \overline{B}$$

Las leyes de De Morgan y sus corolarios son muy útiles y se aplican permanentemente ya que permiten transformar sumas en productos y viceversa.

Función lógica y Tabla de verdad

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Una función lógica booleana es una expresión formada por variables lógicas binarias y operaciones lógicas (OR, AND, NOT).

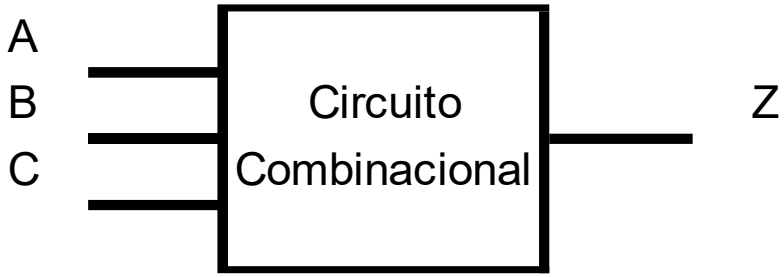
Para conocer el valor de Z para diferentes valores de las variables A, B y C se genera la **tabla de verdad** de la función. Para **n** variables tiene $\mathbf{2^n}$ filas.

Un ejemplo de función booleana Z = f(A,B,C) sería: Z = A + B • C.

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Ejemplo:

Z = Lámpara encendida
A = Llave encendida
B = Puerta cerrada
C = Ventana cerrada



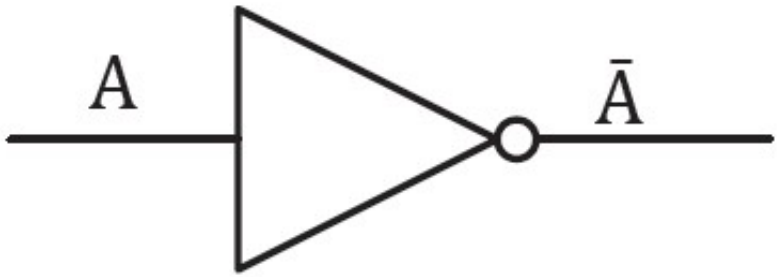
Operaciones y Compuertas lógicas

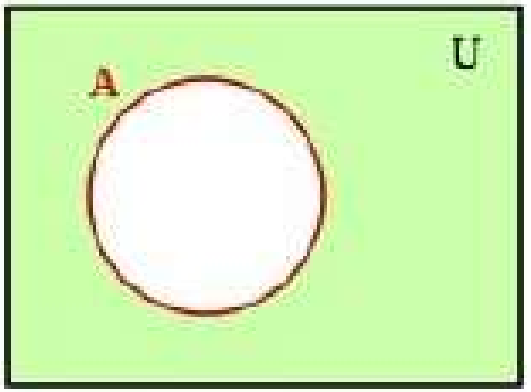
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

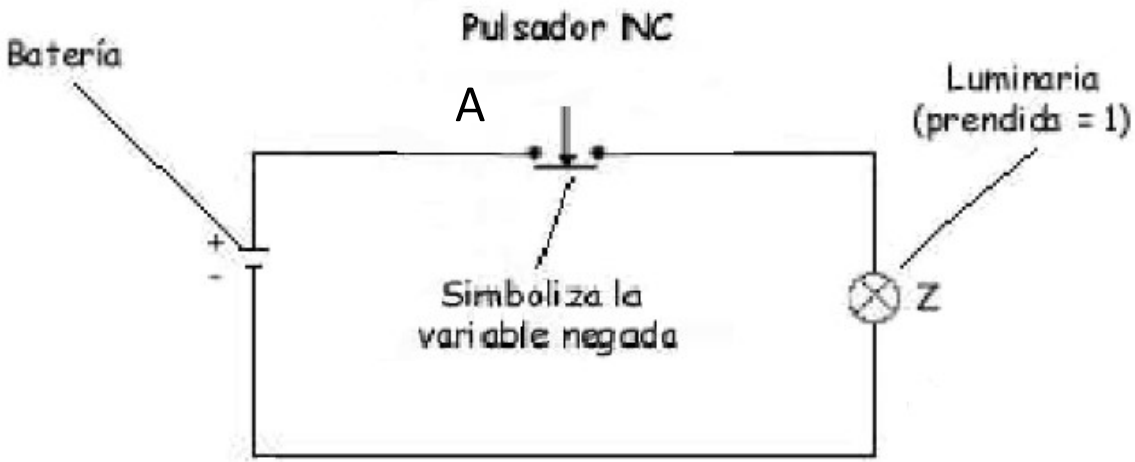
NOT: complemento, negación o inversión.

$$Z = \bar{A}$$

A	$\bar{A}$
0	1
1	0





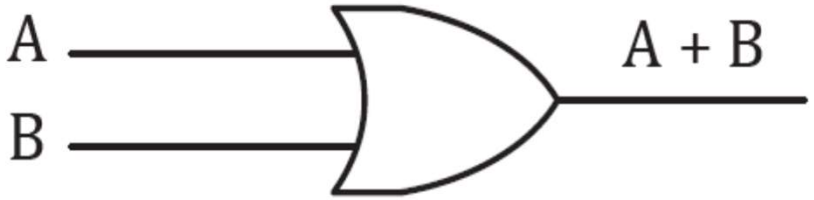


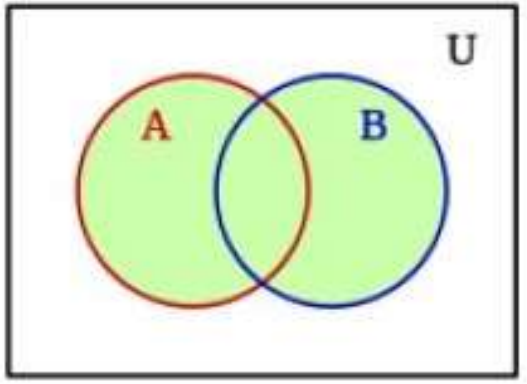
Operaciones y Compuertas lógicas

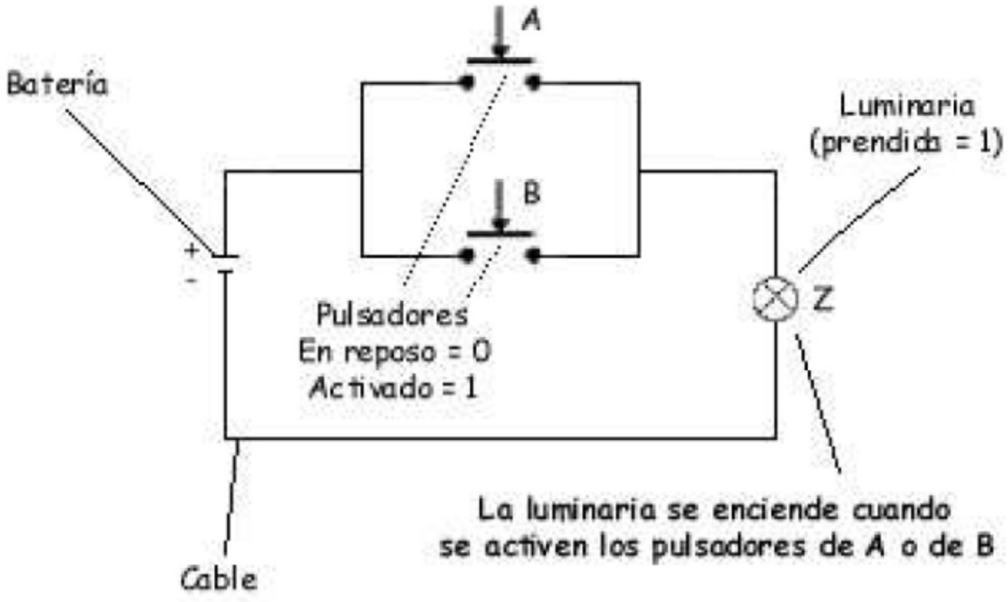
OR: operación "O", suma lógica, disyunción o unión

Z = A + B

A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	1







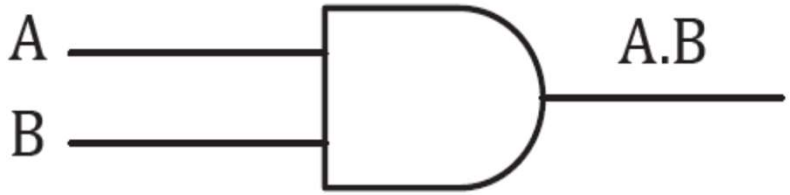
Operaciones y Compuertas lógicas

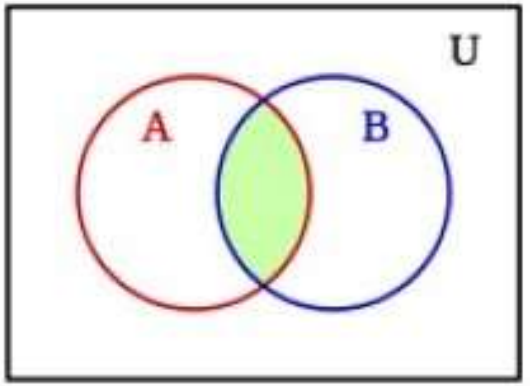
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

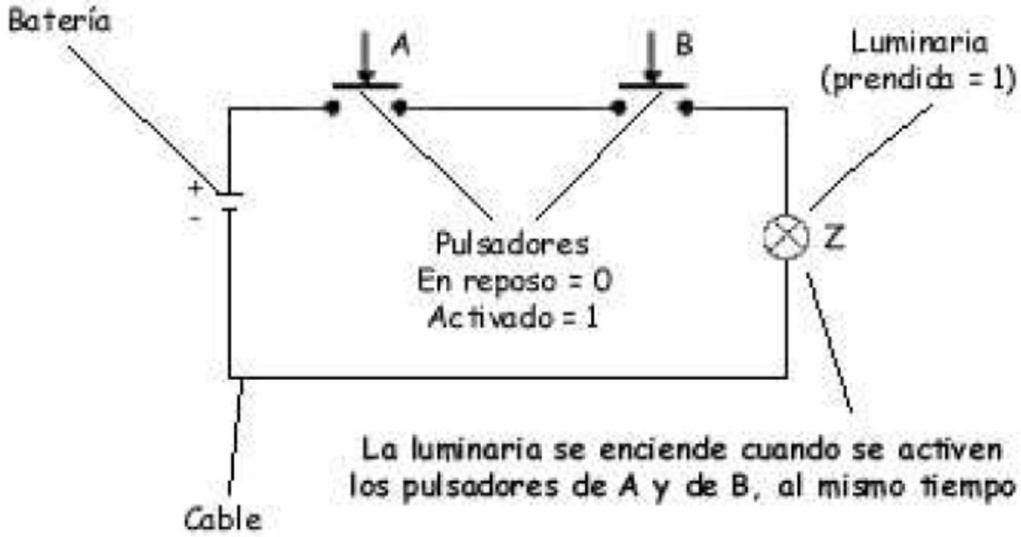
AND: operación "Y", producto lógico, conjunción o intersección

$$Z = A \bullet B$$

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1





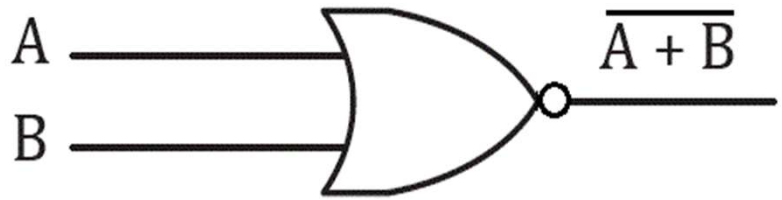


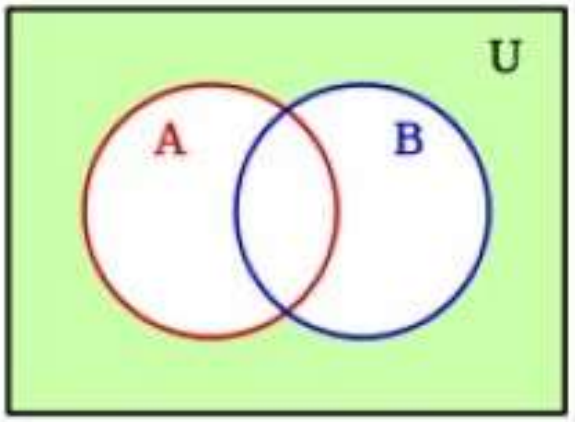
Operaciones y Compuertas lógicas

NOR: operación “No O”, suma lógica negada o disyunción opuesta

$$Z = \overline{A + B}$$

A	B	$\overline{A + B}$
0	0	1
0	1	0
1	0	0
1	1	0



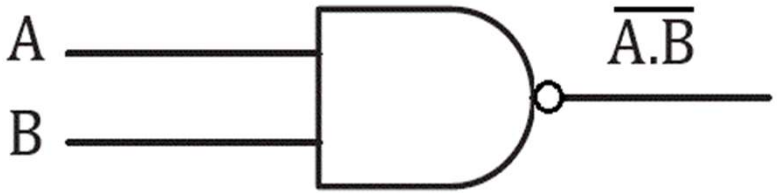


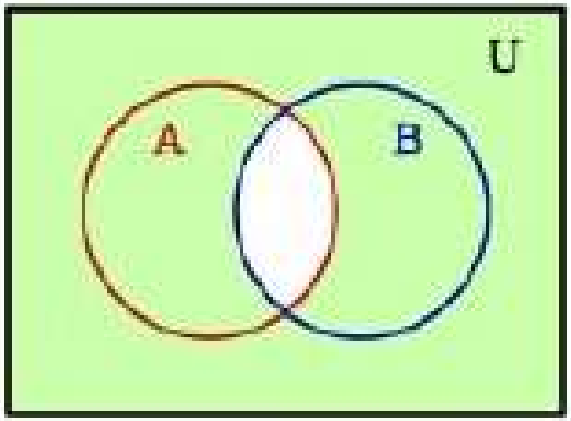
Operaciones y Compuertas lógicas

NAND: operación “No Y”, producto lógico negado o conjunción opuesta

$$Z = \overline{A \bullet B}$$

A	B	$\overline{A.B}$
0	0	1
0	1	1
1	0	1
1	1	0





Operaciones y Compuertas lógicas

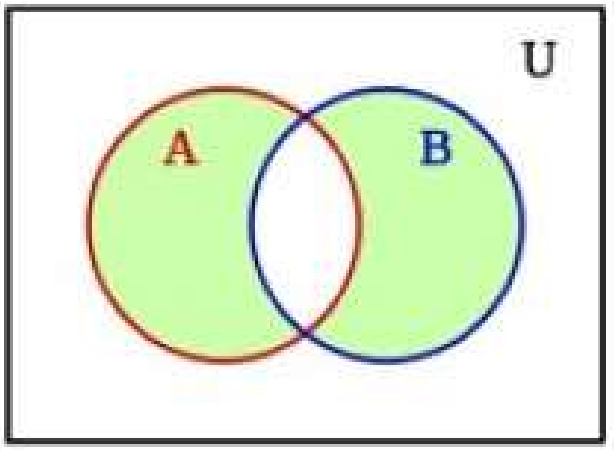
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

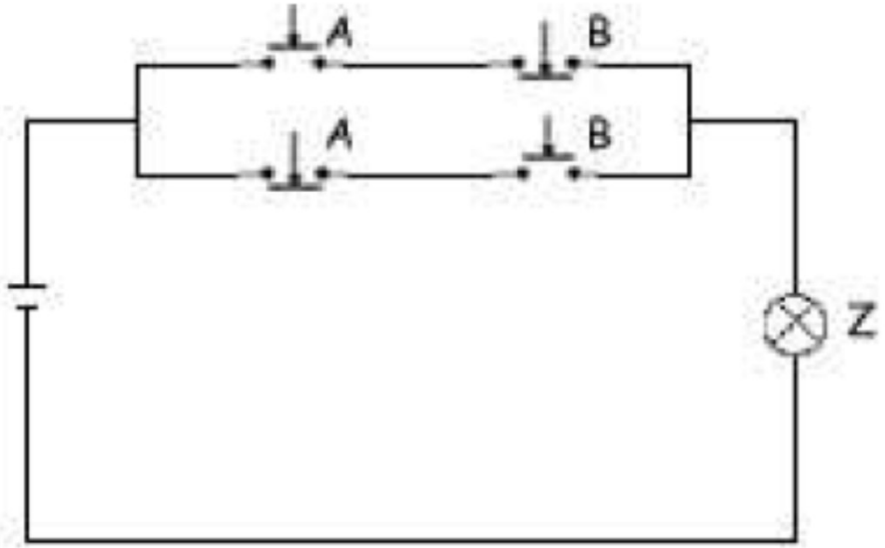
X-OR: operación “O Exclusiva”, suma lógica exclusiva o disyunción exclusiva

$$Z = A \oplus B$$

A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0





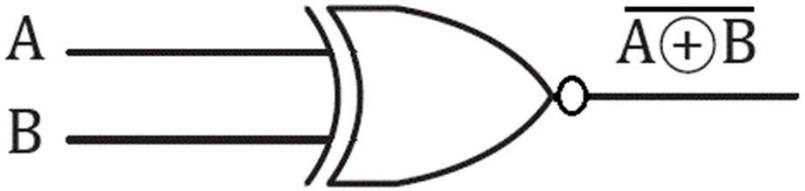


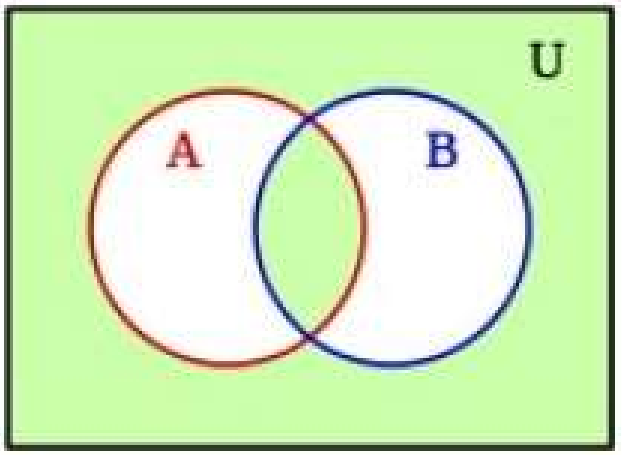
Operaciones y Compuertas lógicas

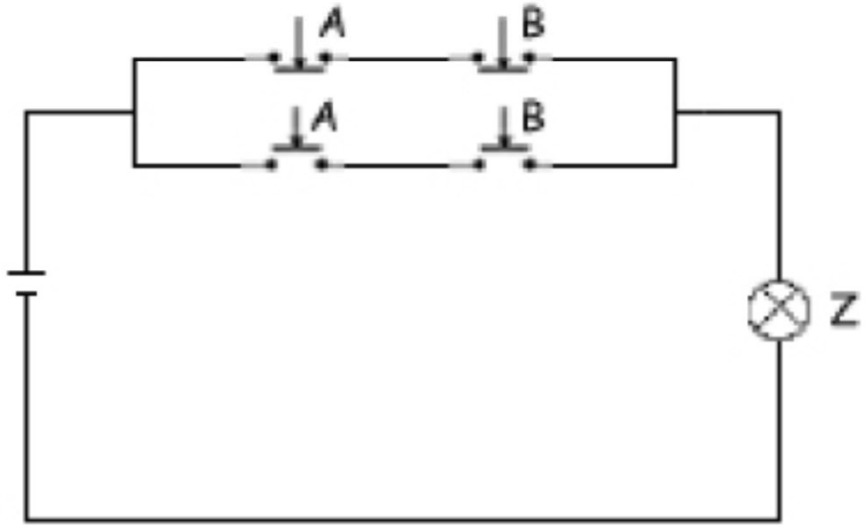
X-NOR: operación “No O Exclusiva” o suma lógica exclusiva negada

$$Z = \overline{A \oplus B}$$

A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1





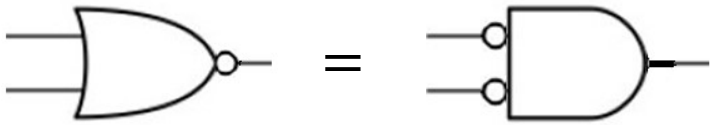


Equivalencia de compuertas

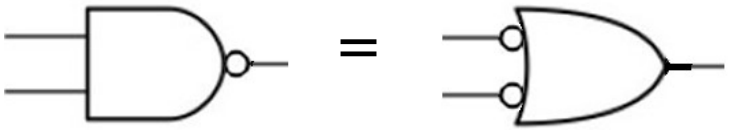
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Leyes de De Morgan

$$\overline{A + B} = \overline{A} \bullet \overline{B}$$



$$\overline{A \bullet B} = \overline{A} + \overline{B}$$

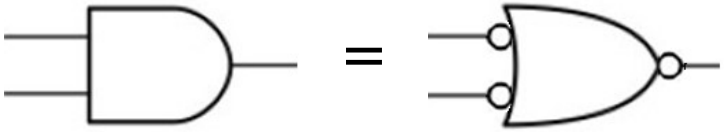


Corolarios:

$$\overline{A + B} = \overline{\overline{A} \bullet \overline{B}}$$



$$\overline{A \bullet B} = \overline{\overline{A} + \overline{B}}$$



Las leyes de De Morgan permiten transformar sumas en productos y viceversa.

Operaciones lógicas con registros

Las operaciones lógicas se pueden aplicar no solo a variables lógicas (1 bit) sino también a registros (conjunto de bits).

Por ejemplo, tenemos 2 registros de 8 bits, A y B:

A = 10100110 B = 00111010

El resultado de la operación A **AND** B resulta:

$Z = A \bullet B = (10100110) \bullet (00111010) = 00100010$

El resultado de la operación A **OR** B resulta:

$Z = A + B = (10100110) + (00111010) = 10111110$

Operaciones **NOT**:

$Z = NOT(A) = 01011001$

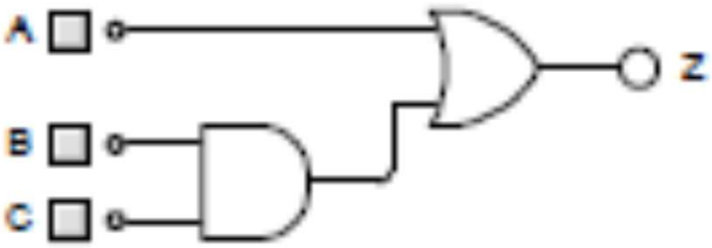
$Z = NOT(A + B) = 01000001$

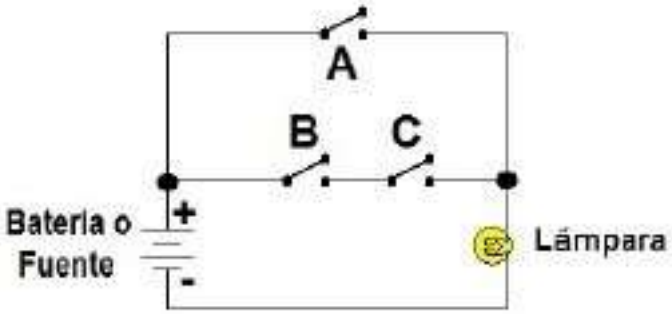
Proposiciones y Funciones lógicas

Supongamos la siguiente **proposición**: la lámpara (Z) deberá encenderse cuando el interruptor (A) se encuentre encendido **o** cuando la puerta (sensor B) **y** la ventana (sensor C) estén ambas cerradas. En este caso **simple** podemos obtener la **función**: $Z = A + B \bullet C$

Z = Lámpara encendida
A = Interruptor encendido
B = Puerta cerrada
C = Ventana cerrada

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



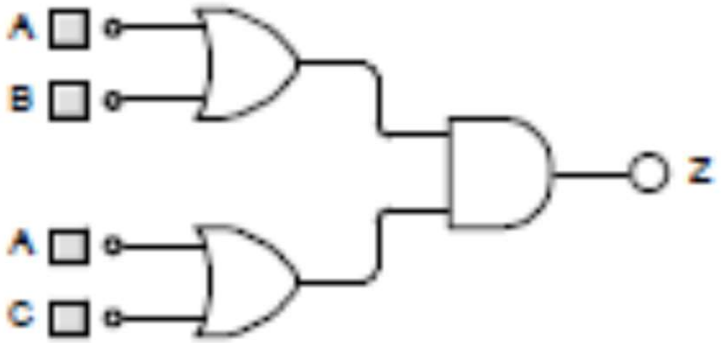


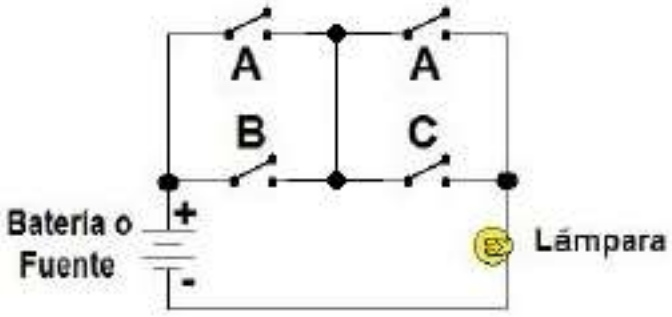
Proposiciones y Funciones lógicas

De la función anterior: $Z = A + B \bullet C$

Operando obtenemos: $Z = (A + B) \bullet (A + C)$

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1





Son **funciones equivalentes**, responden a la misma tabla de verdad, pero tienen distinto **costo lógico**.

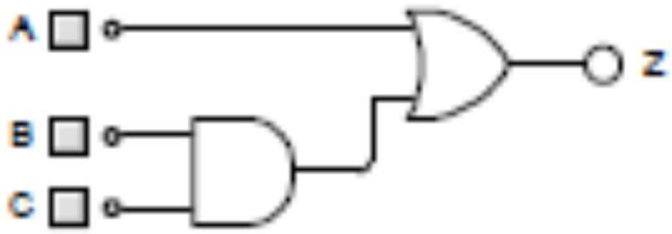
Función mínima - Costo lógico

Uno de los objetivos del estudio del Álgebra de Boole es encontrar la expresión mínima de una función o **función mínima**.

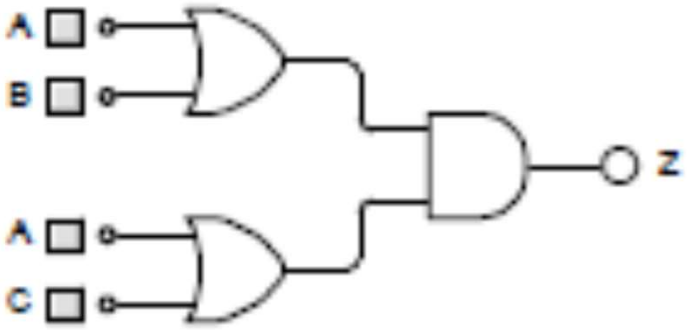
Para una determinada TV, la función mínima es la que tiene el menor **costo lógico**. Para comparar funciones equivalentes verificamos los siguientes criterios:

1) Menor cantidad de niveles (menos retardo)
2) Menor cantidad de compuertas
3) Menor cantidad de entradas

$Z = A + B \bullet C$



$Z = (A + B) \bullet (A + C)$



Formas normales o canónicas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Minitérmino (m): Es un producto lógico (AND) en el cual figuran **todas** las variables lógicas de la función. Estas variables pueden estar o no negadas.

Maxitérmino (M): Es una suma lógica (OR) en la cual figuran **todas** las variables lógicas de la función. Estas variables pueden estar o no negadas.

Forma Normal Disyuntiva:

Está dada por la sumatoria de los **minitérminos** para los cuales la función vale 1. Es decir, es una suma de productos (**SPm**).

Forma Normal Conjuntiva

Está dada por el producto de los **maxitérminos** para los cuales la función vale 0. Es decir es un producto de sumas (**PSM**).

Toda función lógica se puede representar por sus **FN**. (¿pero es la **mínima**?)

Formas normales o canónicas

Hallaremos las formas normales de una función a partir de su **TV**:

FND (SPm): En las filas donde la función vale 1 se forma el producto de todas las variables, remplazando los 0's por su respectiva variable negada, y los 1's por su correspondiente variable sin negar. Luego se realiza la suma lógica de los minitérminos así determinados. El número de minitérminos será igual al número de 1's en la TV de la función.

A	B	C	Z	
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\bar{A}BC$
1	0	0	1	$A\bar{B}\bar{C}$
1	0	1	1	$A\bar{B}C$
1	1	0	1	$AB\bar{C}$
1	1	1	1	ABC

$$Z = f(A,B,C) = \bar{A}BC + A\bar{B}\bar{C} + A\bar{B}C + AB\bar{C} + ABC$$

Formas normales o canónicas

De la misma forma y teniendo en cuenta el principio de Dualidad:

FNC (PSM): En las filas donde la función vale 0 se forma la suma de todas las variables, reemplazando los 1's por su respectiva variable negada, y los 0's por su correspondiente variable sin negar. Luego se realiza el producto lógico de los maxiterminos así constituidos. El número de maxitérminos será igual al número de 0's en la TV de la función.

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$\boldsymbol{A + B + C}$$
$$\boldsymbol{A + B + \overline{C}}$$
$$\boldsymbol{A + \overline{B} + C}$$

$$\boldsymbol{Z = f(A, B, C) = (A + B + C)\ \ (A + B + \overline{C})\ \ (A + \overline{B} + C)}$$

Minimización mediante Álgebra de Boole

Podemos **minimizar** cualquier función aplicando los postulados y propiedades del **Álgebra de Boole**.

Como ejemplo, a partir de una FND:

$$Z = ABC + AB\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + \overline{A}BC$$

$$Z = AB\,(C + \overline{C}) + A\overline{B}\,(C + \overline{C}) + \overline{A}BC + \mathbf{ABC}$$

$$Z = AB\ + A\overline{B}\ + (\overline{A} + A)\,BC$$

$$Z = A\,(B\ + \overline{B}\,) + 1\,BC$$

$$Z = A + BC$$

Obtención de FNs mediante Álg. Boole

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

A partir de cualquier función podemos obtener ambas **FND** y **FNC** aplicando postulados y propiedades del **Álgebra de Boole**.

Como ejemplo, la función ya analizada:

A	B	C	Z
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

$$Z = A + BC$$

$$Z = A \,.\, 1 \,.\, 1 + 1 \,.\, BC$$

$$Z = A\,(B + \overline{B})\,(C + \overline{C}) + (A + \overline{A})\,BC$$

$$Z = ABC + AB\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + \mathbf{ABC} + \overline{A}BC$$

$$Z = ABC + AB\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + \overline{A}BC$$

Conclusiones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

1) Estamos en condiciones de realizar **análisis** y **síntesis** de circuitos lógicos.
2) Una función de 3 niveles, puede ser mínima?
3) Simplificar, minimizar, sintetizar, aquí son sinónimos, pero para qué?

Menor cantidad de niveles

Menor cantidad de compuertas

Mayor velocidad

Menor consumo de energía

Menor tamaño

Menor costo económico

Métodos algorítmicos de minimización:

Mapa de Veitch – Karnaugh
Método de Quine – McCluskey
Método de Consenso, etc.



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Mapa de Veitch – Karnaugh

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Reforzar el concepto de equivalencia entre funciones y el concepto de función mínima.

Realizar la optimización de un circuito lógico que posibilite obtener la performance adecuada en velocidad, complejidad, costo y consumo de energía.

Aplicar métodos sistematizados para la obtención de funciones mínimas.

Implementar funciones lógicas mediante un solo tipo de compuertas aplicando las leyes de De Morgan.

Analizar los riesgos de un circuito lógico combinacional.

Función mínima - Costo lógico

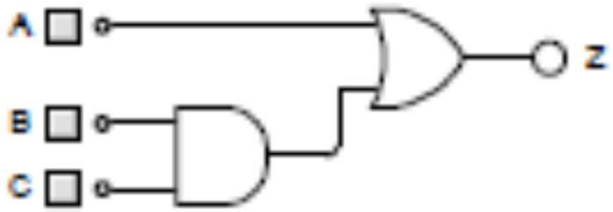
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Para una cierta TV, la **función mínima** es la que tiene el menor **costo lógico**.
Para comparar funciones equivalentes verificamos los siguientes criterios:

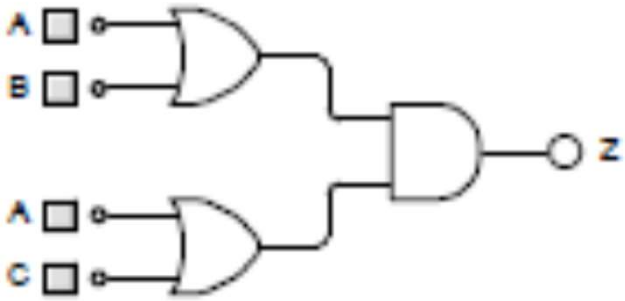
1) Menor cantidad de niveles
2) Menor cantidad de compuertas
3) Menor cantidad de entradas

Resultando: mayor velocidad, menor consumo, menor tamaño, menor costo.

$Z = A + B \bullet C$



$Z = (A + B) \bullet (A + C)$



Síntesis de funciones lógicas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Estamos en condiciones de realizar la **síntesis** de circuitos lógicos aplicando los postulados y propiedades del **Álgebra de Boole**.

$$Z = ABC + AB\overline{C} + A\overline{B}C + A\overline{B}\overline{C} + \overline{A}BC$$

$$Z = AB\,(C + \overline{C}) + A\overline{B}\,(C + \overline{C}) + \overline{A}BC + ABC$$

$$Z = AB\ + A\overline{B}\ + (\overline{A} + A)\,BC$$

$$Z = A\,(B\ + \overline{B}\,) + 1\,BC$$

$$Z = A + BC$$

Estos mismos **agrupamientos** de **minitérminos** podemos realizarlos en forma gráfica si los disponemos en celdas **adyacentes** cuando solo difieran en una variable. Lo mismo podríamos hacer operando a partir de los **maxitérminos** de la función.

Métodos de minimización

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Veremos en profundidad el método de **Veitch – Karnaugh,** uno de los métodos sistemáticos para la minimización, simplificación o síntesis de funciones lógicas. La ubicación de las celdas (**minitérminos** o **maxitérminos**) para 2, 3 y 4 variables quedará por ejemplo:

A \ B	0	1
0	0	1
1	2	3

A \ BC	00	01	11	10
0	0	1	3	2
1	4	5	7	6

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

	A	B	Z
0	0	0	?
1	0	1	?
2	1	0	?
3	1	1	?

Otros métodos algorítmicos de minimización utilizados:

Método de Consenso (mayormente 2 o 3 variables)

Método de Quine – McCluskey (mayormente más de 5 variables)

Mapa de Karnaugh

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Si decimos que dos **celdas** son **adyacentes** cuando comparten un lado pero no cuando comparten un vértice, vemos que dos celdas adyacentes solo difieren en una variable por lo tanto puedo agruparlas eliminando la variable diferente. Notar que la última columna es adyacente a la primera.

Para la TV de la función de 3 variables ya estudiada con anterioridad tendremos que disponer los 1s y 0s de la siguiente forma, es decir que solo hemos transcripto la TV al mapa K.



	A	B	C	Z
0	0	0	0	0
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	1
5	1	0	1	1
6	1	1	0	1
7	1	1	1	1

A \ BC	00	01	11	10
0	0	0	1	0
1	1	1	1	1

Mapa de Karnaugh

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Si resuelvo agrupando 1s voy a obtener una (o más) **SP** (no necesariamente minitérminos) que será la/s **SP mínima/s**, mientras que si resuelvo agrupando 0s voy a obtener un (o más) **PS** (no maxitérminos) que será el/los **PS mínimo/s**.

La **función mínima** será la/s menor/es de todas ellas, es decir que para obtenerla debo resolver por 1s y por 0s.

Para resolver por **SP** (por 1s) es conveniente colocar sólo los 1s en el diagrama y para resolver por **PS** (por 0s) es conveniente solo colocar los 0s.

A \ BC	00	01	11	10
0			1	
1	1	1	1	1

A \ BC	00	01	11	10
0	0	0		0
1				

Definición de Implicantes

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Implicante (I) de **orden n** es todo agrupamiento de **2^n celdas** (0s o 1s) que me permite eliminar **n variables**.

Implicante primo (IP) es todo implicante que **no puede ser contenido** por otro de mayor orden.

Implicante primo esencial (IPE) es todo implicante primo que tiene al menos una celda que **solo él** contiene.

IP de orden 0 >>> minitérmino (o maxitérmino)

Debo encontrar todos los **IP** e **IPE** y **nominarlos**, todos los IPE formarán parte de la solución, si quedan celdas sin cubrir se deberá tomar la menor cantidad de IP para cubrirlas.

Resolviendo por **1s**, los IP resultan productos de las variables que no cambian, si la variable es 1 se debe tomar sin negar, si vale 0 se debe tomar negada. La solución será una suma de los IPE (+IP), resultando una **SP**.

Resolviendo por **0s**, los IP resultan sumas de las variables que no cambian, si la variable es 0 se debe tomar sin negar, si vale 1 se debe tomar negada. La solución será un producto de los IPE (+IP), resultando un **PS**.

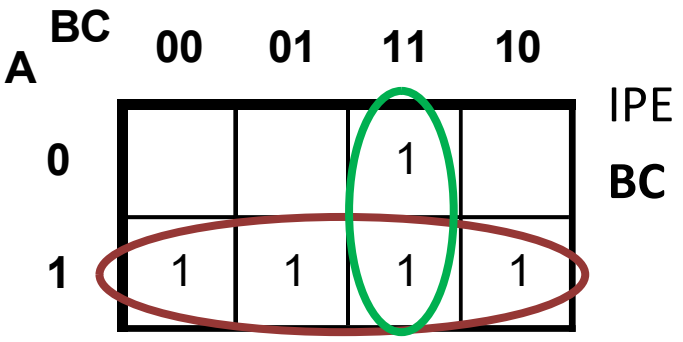
Determinación de los IP e IPE

Resolviendo por 1s encontramos 2 IPE:

* el vertical, de orden 1, cambia A, por lo tanto se elimina, y como B y C ambas valen 1 en las 2 celdas que lo componen, van sin negar: **BC**

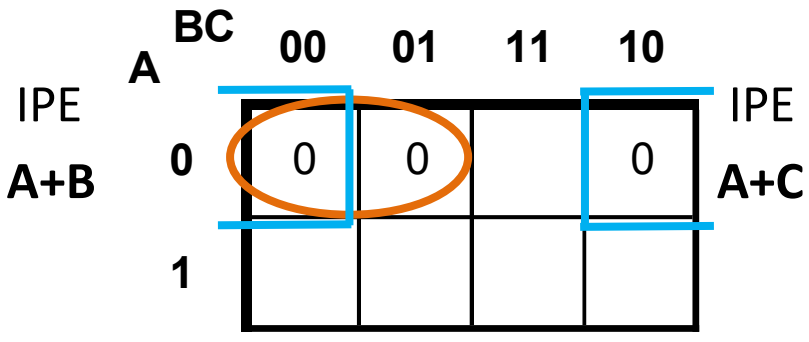
* el horizontal, de orden 2, cambian B y C, por lo tanto se eliminan, y como A vale 1 en las 4 celdas, va sin negar: **A**

Z_1 será la suma de los 2 IPEs, y de esta forma tengo cubiertos todos los 1s.



IPE **A**

Z_1 = A + BC

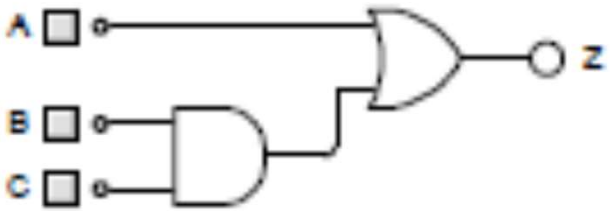


Z_0 = (A+B) (A+C)

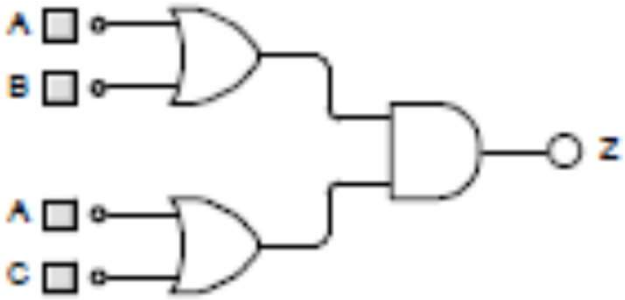
Determinación de la Función mínima

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

$Z_1 = A + B\,C$



$Z_0 = (A + B)\,(A + C)$



Observamos que resolviendo por 1s tenemos una sola función mínima de tipo **SP** (2 niveles).

Resolviendo por 0s también tenemos una sola función mínima pero de tipo **PS** (2 niveles).

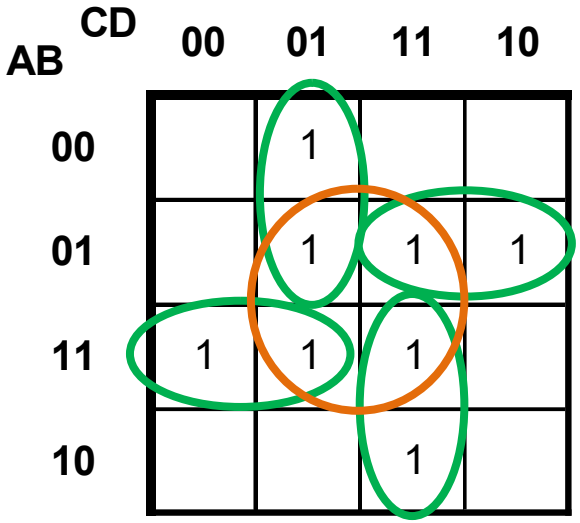
Para este ejemplo vemos que de ambas, la solución por 1s es la **mínima**, ya que tiene menor cantidad de compuertas.

Asumimos la disponibilidad de **todas las variables y sus complementos**.

IP de mayor orden no significa IPE

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En este ejemplo el IP de mayor orden no es IPE.



El IP **BD** (orden 2) no es IPE, y como con los 4 IPE (orden 1) cubro todos los 1s, colocar a BD en la solución haría que ya no sea mínima.

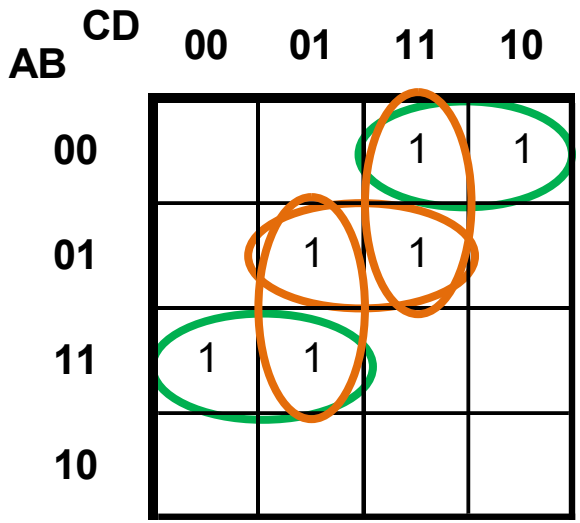
La función mínima por 1s resulta entonces:

$$Z = \overline{A}\overline{C}D + \overline{A}BC + AB\overline{C} + ACD$$

Caso con IPEs e IPs

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En este ejemplo tenemos 2 IPE, el resto son solo IPs.



Con los 2 IPE cubro cuatro 1s, me quedan dos 1s sin cubrir, el menor costo lo obtengo usando un solo IP para cubrirlos. La función mínima por 1s resulta:

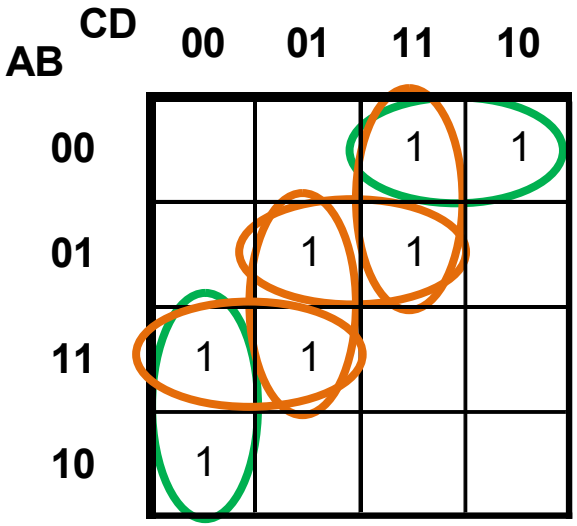
IPE IPE IP

$$Z = \overline{A}\overline{B}C + AB\overline{C} + \overline{A}BD$$

Caso con soluciones múltiples

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Aquí también tenemos 2 IPE, el resto son solo IPs.



Con los 2 IPE cubro cuatro 1s, me quedan tres 1s sin cubrir, el menor costo lo obtengo tomando 2 IPs para cubrirlos. Las funciones mínimas por 1s resultan:

$$Z_1 = \overline{A}\overline{B}C + A\overline{C}\overline{D} + \overline{A}BD + AB\overline{C}$$

$$Z_2 = \overline{A}\overline{B}C + A\overline{C}\overline{D} + \overline{A}CD + B\overline{C}D$$

$$Z_3 = \overline{A}\overline{B}C + A\overline{C}\overline{D} + \overline{A}BD + B\overline{C}D$$

Mapa K de 5 variables

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

A = 0

BC \ DE	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

A = 1

BC \ DE	00	01	11	10
00	16	17	19	18
01	20	21	23	22
11	28	29	31	30
10	24	25	27	26

Mapa K de 6 variables

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

B = 0

A = 0

CD \ EF	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11	12	13	15	14
10	8	9	11	10

A = 1

CD \ EF	00	01	11	10
00	32	33	35	34
01	36	37	39	38
11	44	45	47	46
10	40	41	43	42

B = 1

A = 0

CD \ EF	00	01	11	10
00	16	17	19	18
01	20	21	23	22
11	28	29	31	30
10	24	25	27	26

A = 1

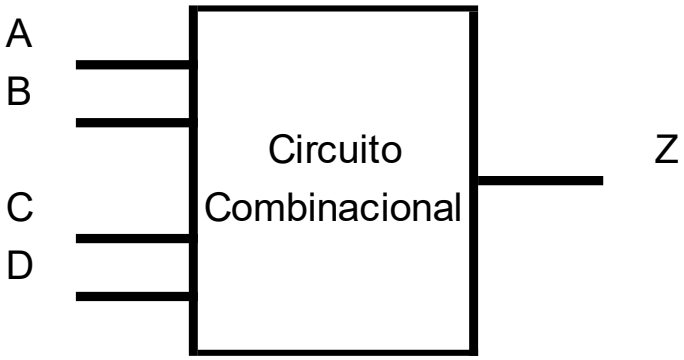
CD \ EF	00	01	11	10
00	48	49	51	50
01	52	53	55	54
11	60	61	63	62
10	56	57	59	58

Funciones no totalmente definidas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Veamos el caso de una función que tiene algunas combinaciones de sus entradas no especificadas, por ejemplo la siguiente **proposición**:

Sintetizar un circuito que recibe un dígito decimal (0 a 9) codificado en binario (BCD) y que su salida presente un 1 si el número recibido es **primo**.



	A	B	C	D	Z
0	0	0	0	0	0
1	0	0	0	1	0
2	0	0	1	0	1
3	0	0	1	1	1
4	0	1	0	0	0
5	0	1	0	1	1
6	0	1	1	0	0
7	0	1	1	1	1
8	1	0	0	0	0
9	1	0	0	1	0
10	1	0	1	0	X
11	1	0	1	1	X
12	1	1	0	0	X
13	1	1	0	1	X
14	1	1	1	0	X
15	1	1	1	1	X

Funciones no totalmente definidas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Tengo **6 combinaciones** que **nunca** se van a dar (a menos que haya errores o ruidos). No me importa que valor tomaría la salida en esos hipotéticos casos (don't care o redundancias).

Otra forma de expresar las **canónicas**:

$$\boldsymbol{Z(A,B,C,D) = \Sigma_m(2,3,5,7) + d(10,11,12,13,14,15)}$$

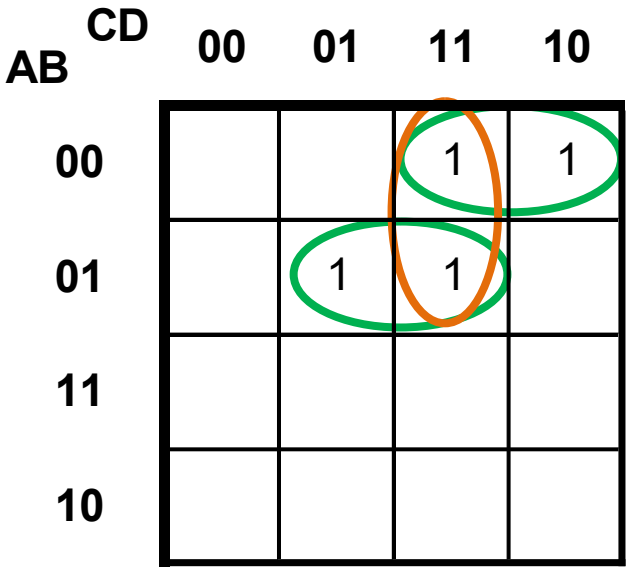
$$\boldsymbol{Z(A,B,C,D) = \Pi_M(0,1,4,6,8,9) + d(10,11,12,13,14,15)}$$

Es decir que podemos tener la misma información de la TV expresada como la **sumatoria** de los **minitérminos** o como la **productoria** de los **maxitérminos**, más en ambos casos los términos indiferentes si existen.

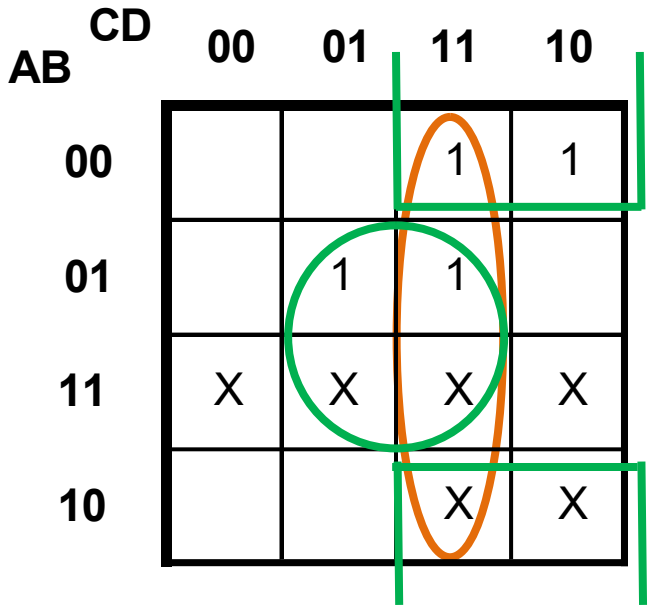
Funciones no totalmente definidas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Tomando las X como 0 o 1 según más me convenga para minimizar obtengo funciones de **menor costo lógico**.



$$Z = \overline{A}\overline{B}C + \overline{A}BD$$



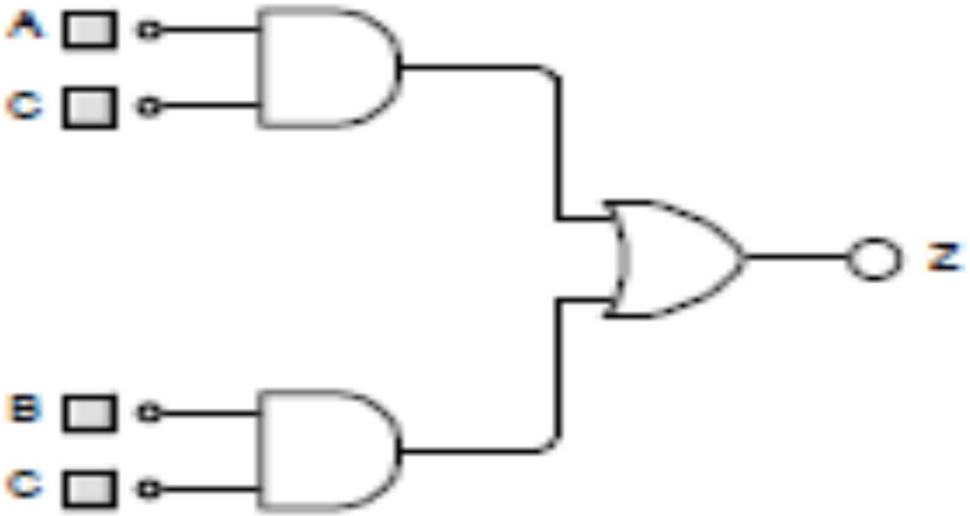
$$Z = \overline{B}C + BD$$

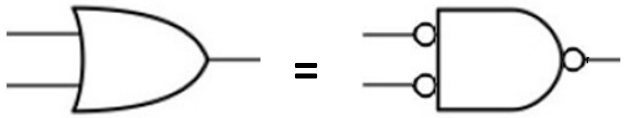
Ambas funciones son **equivalentes** (responden a la misma TV) pero **solo** en el rango donde la **función está definida**, no necesariamente en el resto.

Implementación monocompuerta

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En muchos casos resulta conveniente implementar las funciones obtenidas como **SP (AND – OR)** o **PS (OR – AND)** con un único tipo de compuertas, solo **NAND** o solo **NOR** respectivamente.





Aplicando De Morgan:

$$Z = AC + BC$$

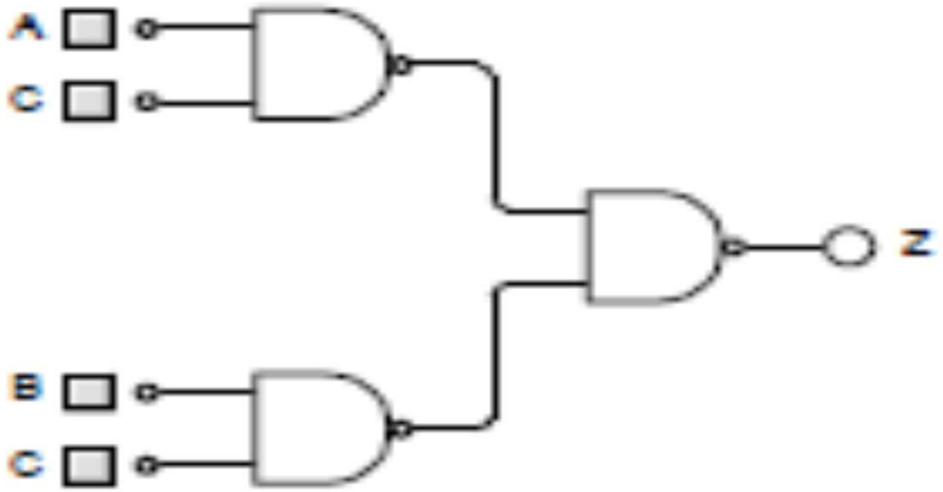
$$Z = \overline{\overline{AC + BC}}$$

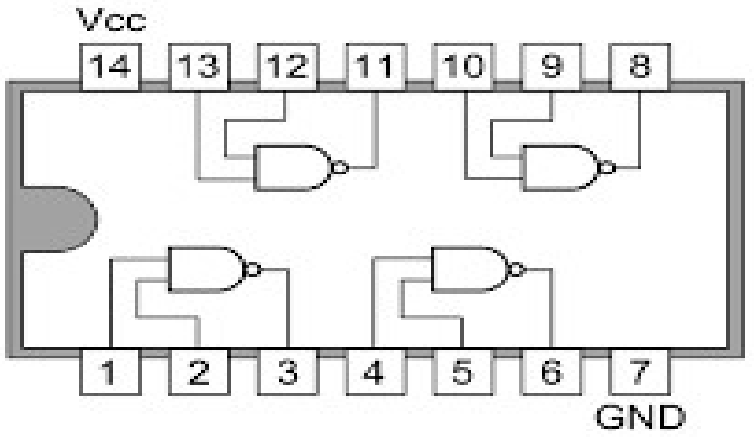
$$Z = \overline{\overline{AC} \cdot \overline{BC}}$$

Implementación monocompuerta

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

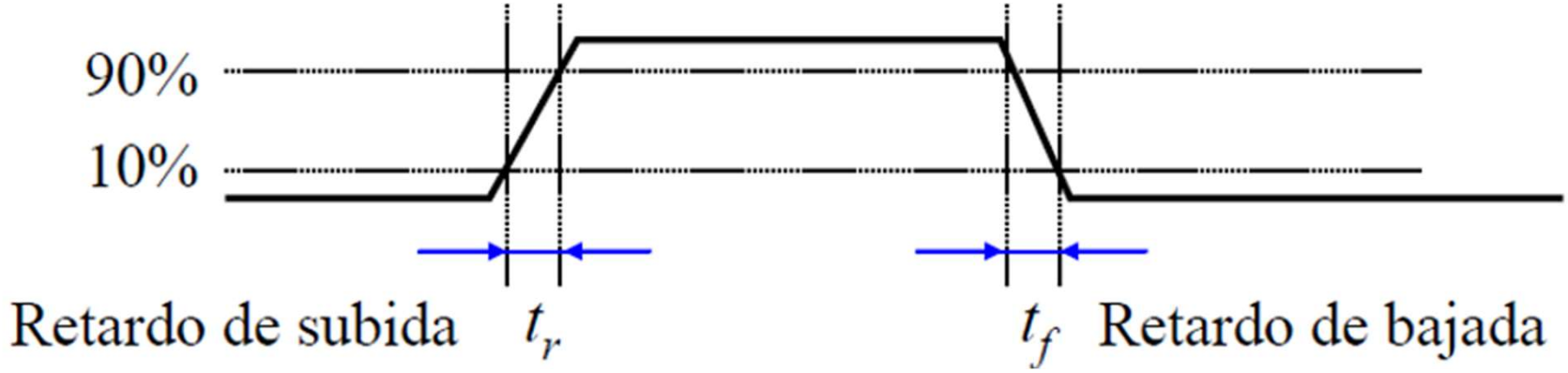
Implementación con **solo NAND** (un solo CI 74ALS00)



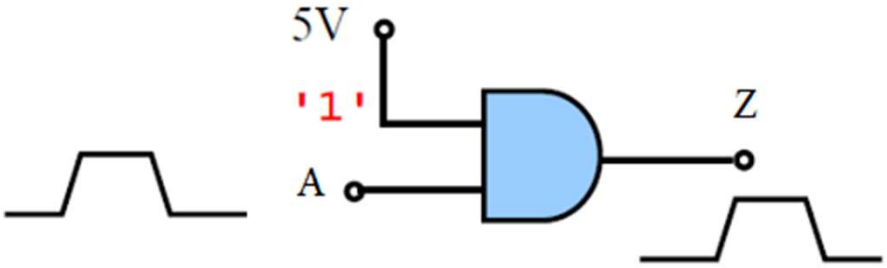


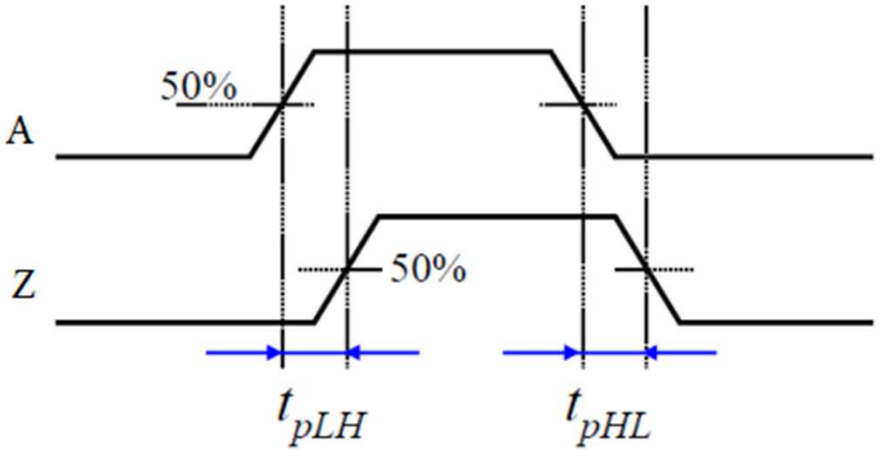
Retardos

La conmutación de 0 a 1 o de 1 a 0 no son instantáneas, existe un **retardo de conmutación**.



Asimismo, cuando se produce un cambio en una entrada, si corresponde un cambio en la salida, no es instantáneo, existe un **retardo de propagación**.

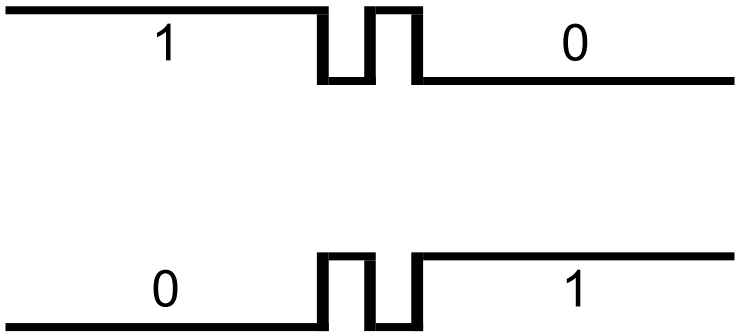




Riesgos dinámicos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un **riesgo dinámico** es la posibilidad de que en un circuito lógico, luego de un cambio en una de sus entradas que debería producir un único cambio en la salida, se produzcan 3 cambios en la misma. Esto es debido a los retardos de los diferentes caminos de la señal.



Teorema:

Una función implementada en **2 niveles** (**SP** o **PS**) está **libre de riesgos dinámicos**.

Riesgos estáticos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un **riesgo estático en los 1s** (o de 1s) es la posibilidad de que una salida de un circuito, luego de un cambio en una de sus entradas, produzca una interferencia 0 (glitch o pulso espurio) cuando debería permanecer en 1 sin cambio alguno. Es debido a los diferentes retardos de las compuertas.



En forma dual podemos definir un **riesgo estático en los 0s** (o de 0s).



Teoremas:

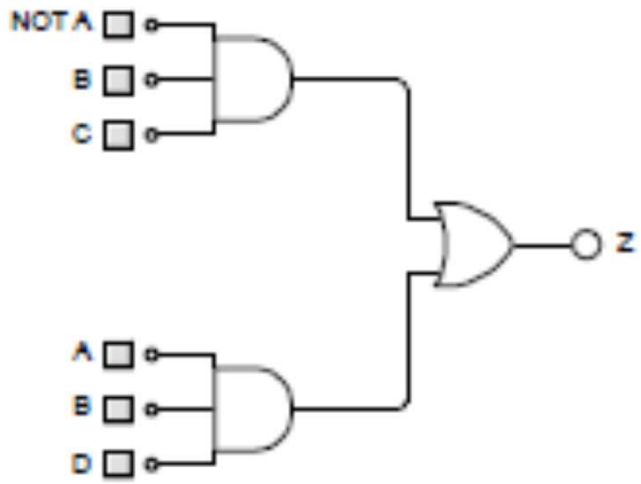
Una función implementada por 1s (**SP**) está **libre de riesgos estáticos en los 0s**.

Una función implementada por 0s (**PS**) está **libre de riesgos estáticos en los 1s**.

Riesgos estáticos

Para el siguiente mapa obtenemos la **función mínima por 1s**.

AB \ CD	00	01	11	10
00				
01			1	1
11		1	1	
10				



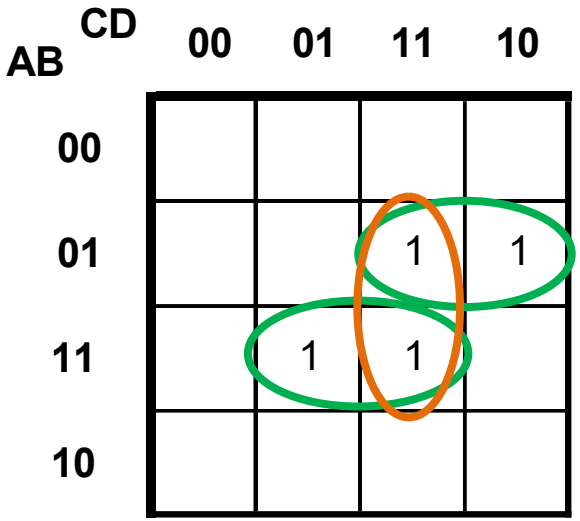
$$Z = \bar{A}BC + ABD$$

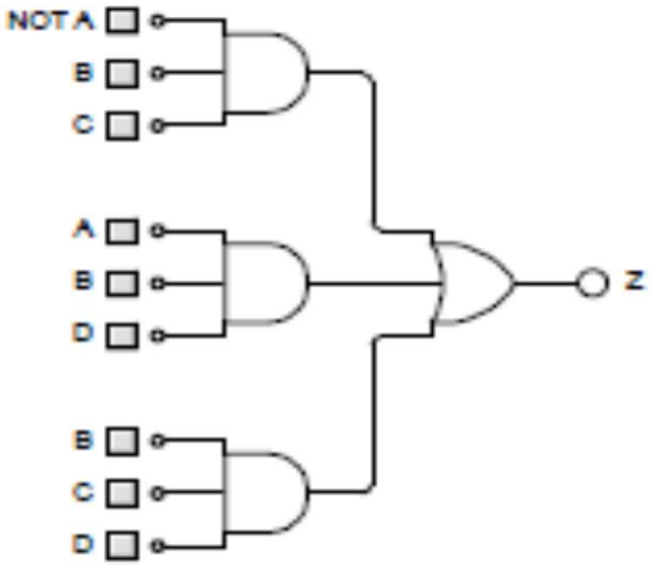
La función implementada **NO es libre de riesgos**, ya que si en la combinación **0**111 (Z = 1), cambia A pasando a **1**111 (Z = 1), existe el riesgo de que Z (debe valer 1 en ambos casos) tenga un 0 espurio o **riesgo estático** en los 1s.

Riesgos estáticos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Para que la implementación sea **libre de riesgos**, debemos garantizar Z = 1 durante la transición.





$$Z = \bar{A}BC + ABD + BCD$$

La función implementada es la **función mínima (SP) libre de riesgos**.

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Familias Lógicas

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Plantear los conceptos básicos de electrónica digital para poder comprender la información contenida en las hojas de datos de los circuitos integrados digitales.

Analizar el concepto de Familia lógica y sus parámetros más relevantes.

Conocer las características principales de las distintas tecnologías utilizadas en los circuitos integrados digitales.

Definición

Denominamos **familia lógica** a aquellos componentes lógicos que tienen características en común, debido a que son de una misma tecnología de fabricación, lo que los hace compatibles entre sí. Valores **min, max, typ**.

Características eléctricas

- Niveles lógicos (tensiones)
- Margen de ruido
- Corrientes de entrada y salida
- Fan Out
- Fan In

Características de conmutación

- Retardo de propagación
- Potencia consumida

Rangos de operación

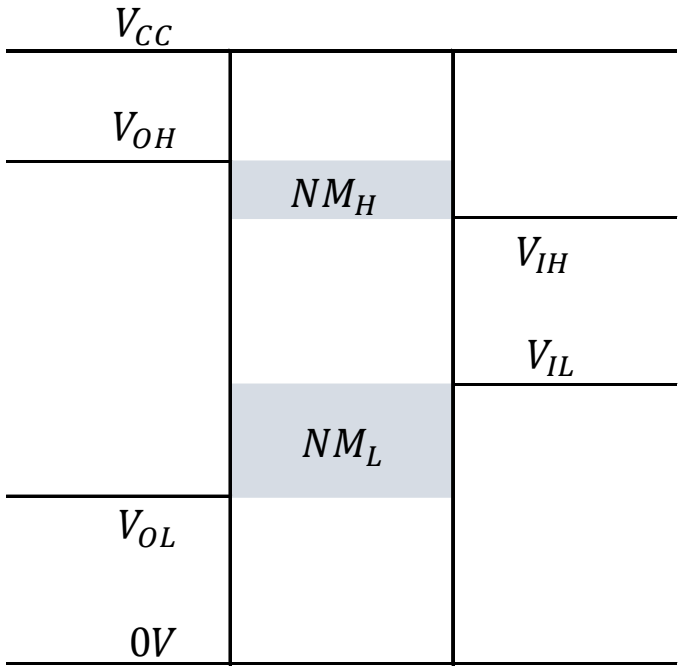
- Tensión de alimentación
- Temperatura

Niveles lógicos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Si bien los **niveles lógicos** (tensiones) ideales son **Vcc** (tensión de alimentación) para el 1 lógico, y **0 V** para el 0 lógico, existe un rango de valores garantizados y tolerados.





Niveles lógicos - Margen de ruido

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Los **niveles lógicos** (tensiones) que se especifican en las hojas de datos son:

V_{OHmin}: Tensión de salida **mínima** en estado **alto** garantizada por el fabricante.

V_{OLmax}: Tensión de salida **máxima** en estado **bajo** garantizada por el fabricante.

V_{IH}: **Mínima** tensión de entrada requerida por el fabricante para ser reconocida como un 1.

V_{IL}: **Máxima** tensión de entrada requerida por el fabricante para ser reconocida como un 0.

Margen de ruido (Noise Margin):

$$\mathbf{NM_H} = V_{OHmin} - V_{IH} \qquad \mathbf{NM_L} = V_{IL} - V_{OLmax}$$

El menor de ambos será el **NM** de la familia.

Ejemplo: inversor

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

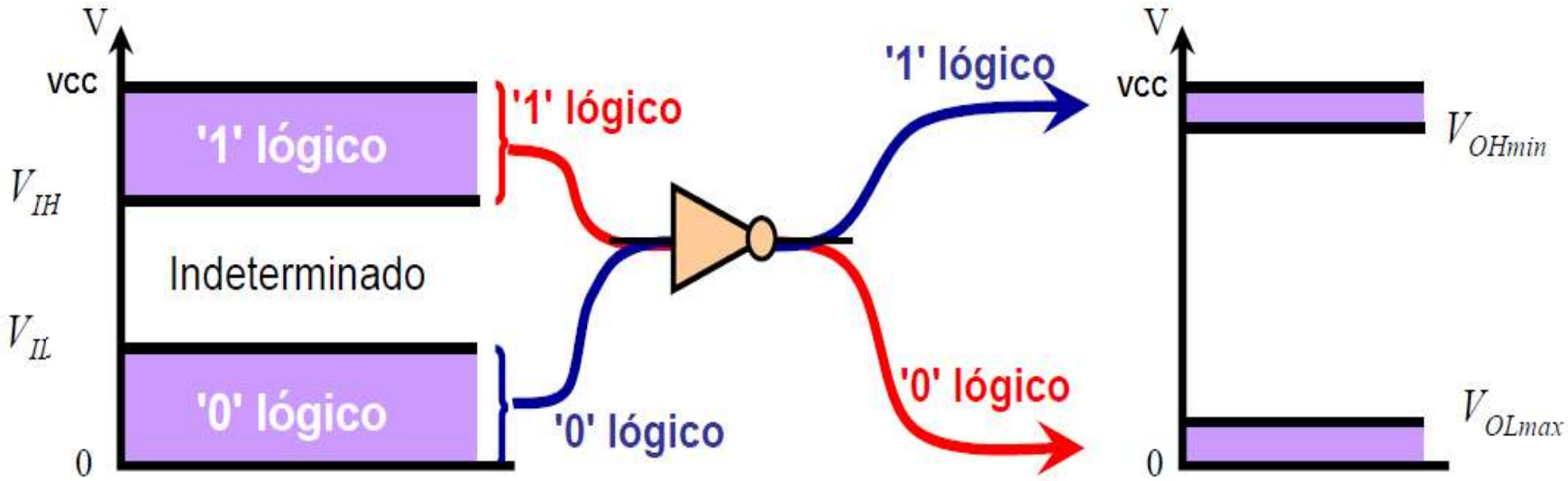
Ejemplo de niveles lógicos para un **inversor**:

Entrada

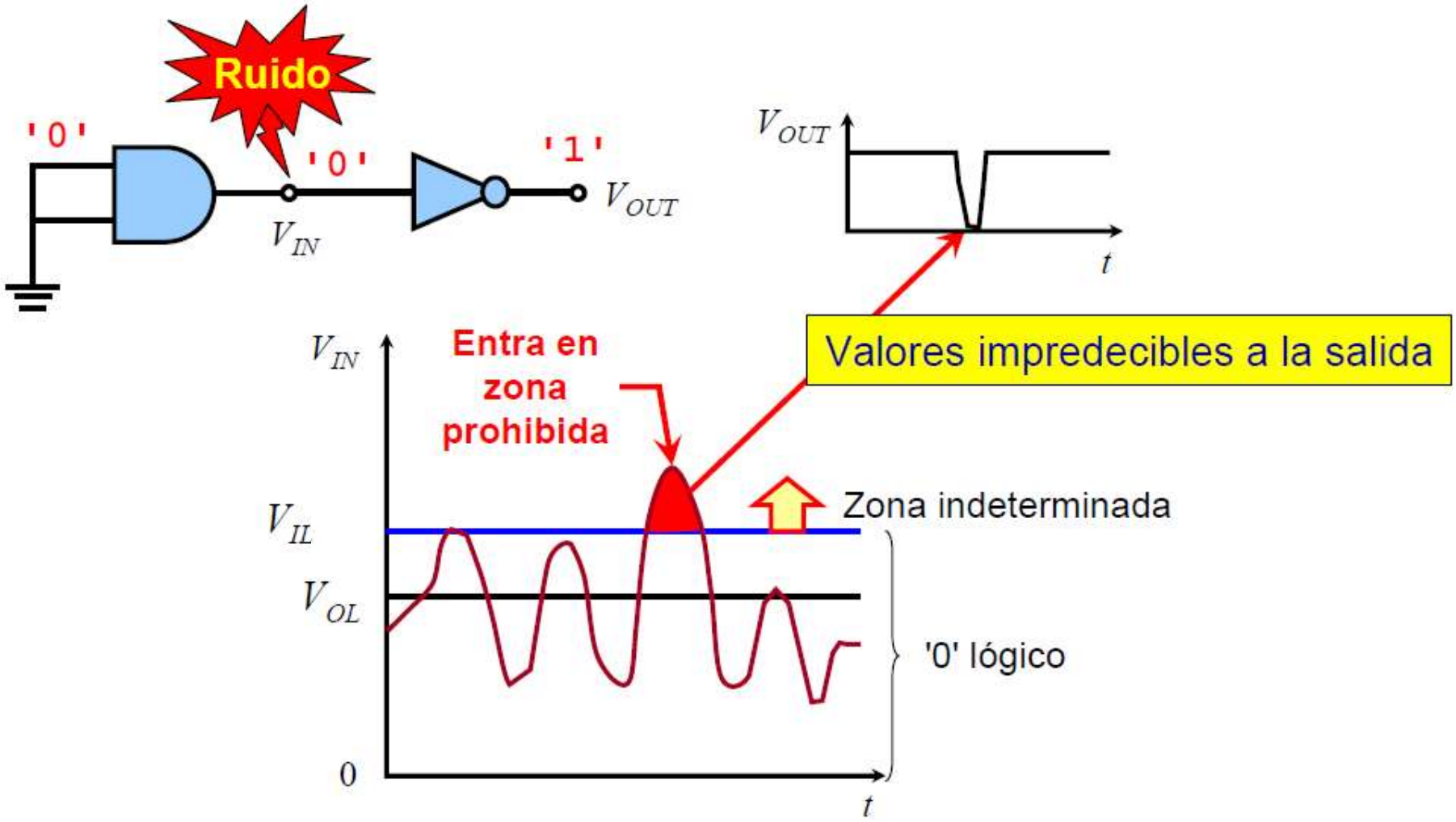
Tensiones que se consideran '0' ó '1'

Salida

Tensiones que se dan como '0' ó '1'



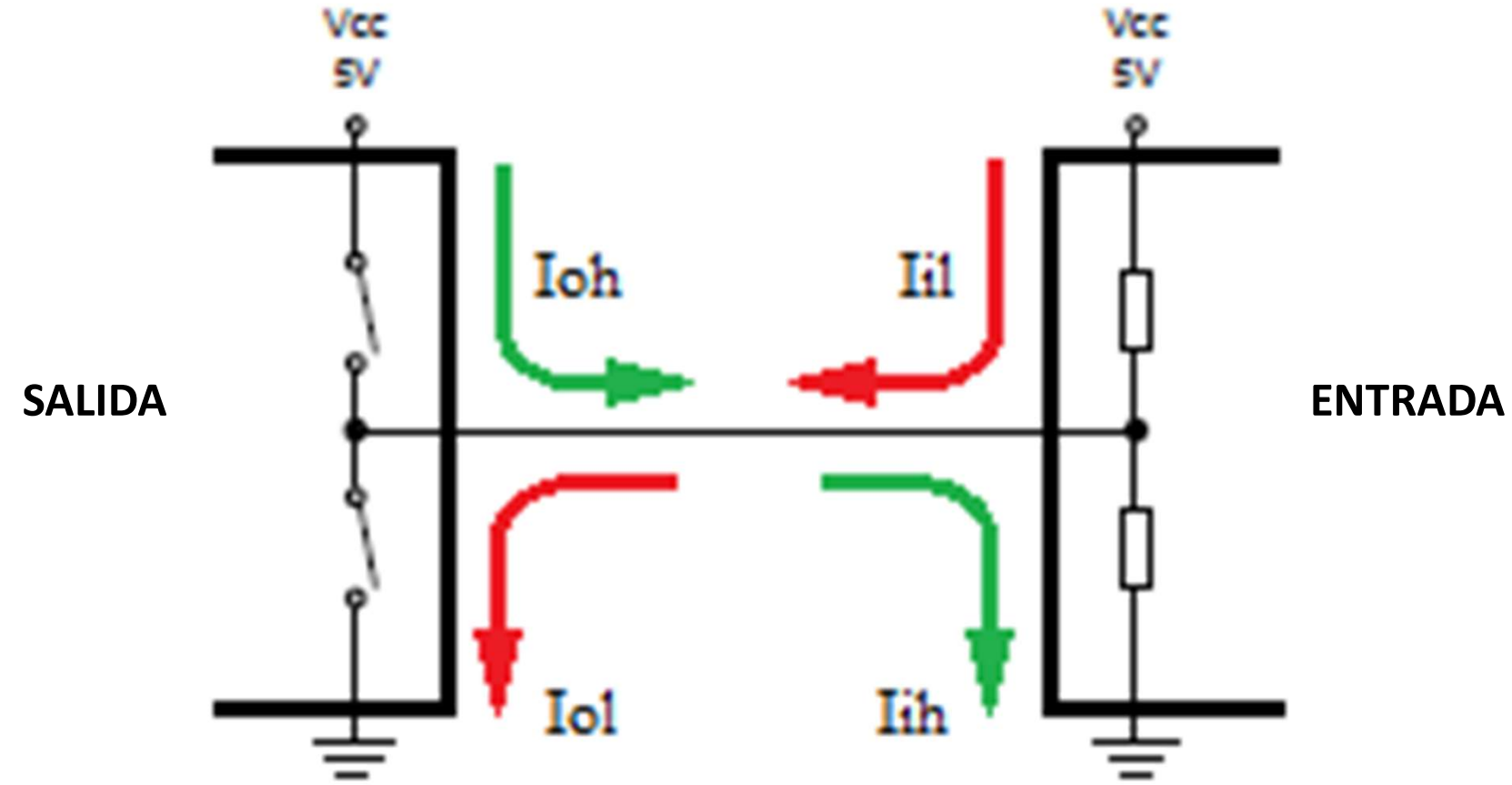
Ejemplo: inversor



Corrientes de entrada y salida

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

También se informan en la hoja de datos las **corrientes** de entrada y salida en ambos niveles lógicos.



Corrientes de entrada y salida

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Las **corrientes** de entrada y salida en ambos niveles lógicos son:

I_{OH}: **Capacidad** de corriente de salida en estado **alto** garantizada por el fabricante.

I_{OL}: **Capacidad** de corriente de salida en estado **bajo** garantizada por el fabricante.

I_{IH}: **Máximo consumo** de corriente de entrada en estado **alto** garantizado por el fabricante.

I_{IL}: **Máximo consumo** de corriente de entrada en estado **bajo** garantizado por el fabricante.

Por convención se toman **positivas** las corrientes **entrantes** y **negativas** las corrientes **salientes**.

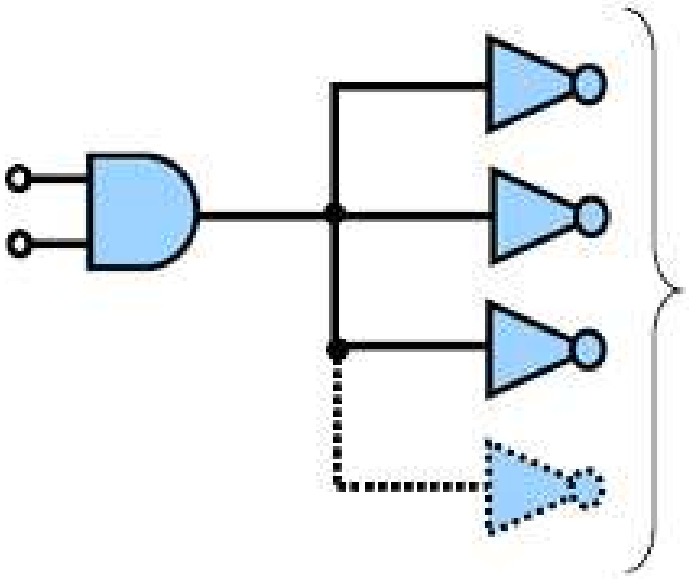
Fan Out

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Definimos el **Fan Out (N o FO)** como la **máxima cantidad de entradas** que puedo alimentar desde una salida sin degradar los niveles lógicos garantizados. Surge de analizar para ambos estados lógicos la relación entre la corriente de salida y la de entrada:

$$\mathbf{FO_H} = |I_{OH} / I_{IH}| \qquad \mathbf{FO_L} = |I_{OL} / I_{IL}|$$

El menor de ambos será el **FO (N)** de la familia.



Máximo N puertas ⇨ fan-out = N

Fan In

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Muchos autores lo definen como la cantidad de entradas que tiene una compuerta lógica, sin embargo esto no aporta mayor información.

Definimos el **Fan In (FI)** como la **cantidad equivalente** de entradas típicas (I_{IH}, I_{IL}) que consume una entrada en particular.

Por ejemplo, en una salida con FO = 10 podré conectarle 10 entradas típicas, o 5 con FI = 2, u otras combinaciones que no superen 10.

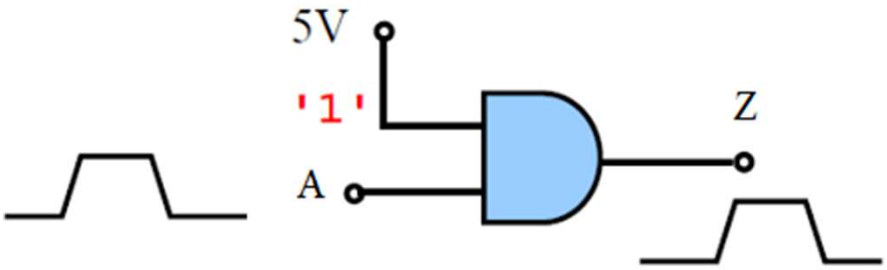
Retardo de propagación

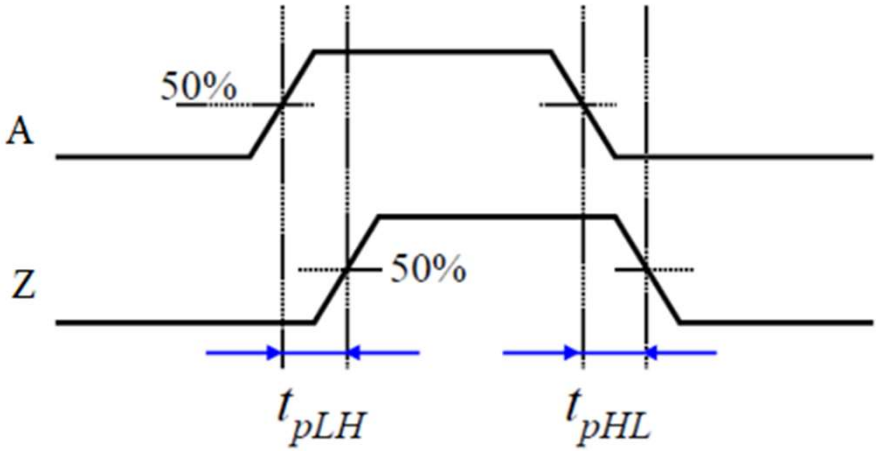
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Retardo de propagación:

Es el tiempo que **demora** una compuerta en tener en su salida el valor correspondiente desde que se produce el cambio en una variable de entrada. Está en el orden de los **nano segundos (ns)**.

El fabricante especifica: $\mathbf{t_p}$ $\mathbf{t_{pLH}}$ $\mathbf{t_{pHL}}$





Potencia y Factor de Mérito

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Potencia consumida o disipada:

Es la energía por unidad de tiempo **suministrada** a una compuerta por la fuente de alimentación. Está en el orden de los **mili Watts (mW)**.

El fabricante la especifica en las hojas de datos.

$$P_d$$

Factor de mérito:

Se define como factor de mérito al producto entre la potencia disipada y el retardo de propagación. Está en el orden de los **pico Joule (pJ = ns . mW)**.

$$FM = P_d \, t_p$$

Como podemos ver, cuanto **más bajo** sea el factor de mérito **mejor** será la familia.

Rangos de operación

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

	Tensión de Alimentación	Temperatura de Operación
Serie militar:		
54XXNN	4,5 - 5,5 V	-55°C +125°C
Serie comercial:		
74XXNN	4,75 - 5,25 V	0°C +70°C

Hoja de datos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Recommended Operating Conditions

Symbol	Parameter	Min	Nom	Max	Units
V_{CC}	Supply Voltage	4.75	5	5.25	V
V_{IH}	HIGH Level Input Voltage	2			V
V_{IL}	LOW Level Input Voltage			0.8	V
I_{OH}	HIGH Level Output Current			−0.4	mA
I_{OL}	LOW Level Output Current			16	mA
T_A	Free Air Operating Temperature	0		70	°C

Electrical Characteristics

over recommended operating free air temperature range (unless otherwise noted)

Symbol	Parameter	Conditions	Min	Typ (Note 2)	Max	Units
V_I	Input Clamp Voltage	V_{CC} = Min, I_I = −12 mA			−1.5	V
V_{OH}	HIGH Level Output Voltage	V_{CC} = Min, I_{OH} = Max V_{IL} = Max	2.4	3.4		V
V_{OL}	LOW Level Output Voltage	V_{CC} = Min, I_{OL} = Max V_{IH} = Min		0.2	0.4	V
I_I	Input Current @ Max Input Voltage	V_{CC} = Max, V_I = 5.5V			1	mA
I_{IH}	HIGH Level Input Current	V_{CC} = Max, V_I = 2.4V			40	µA
I_{IL}	LOW Level Input Current	V_{CC} = Max, V_I = 0.4V			−1.6	mA
I_{OS}	Short Circuit Output Current	V_{CC} = Max (Note 3)	−18		−55	mA
I_{CCH}	Supply Current with Outputs HIGH	V_{CC} = Max		4	8	mA
I_{CCL}	Supply Current with Outputs LOW	V_{CC} = Max		12	22	mA

Note 2: All typicals are at V_{CC} = 5V, T_A = 25°C.

Note 3: Not more than one output should be shorted at a time.

Switching Characteristics

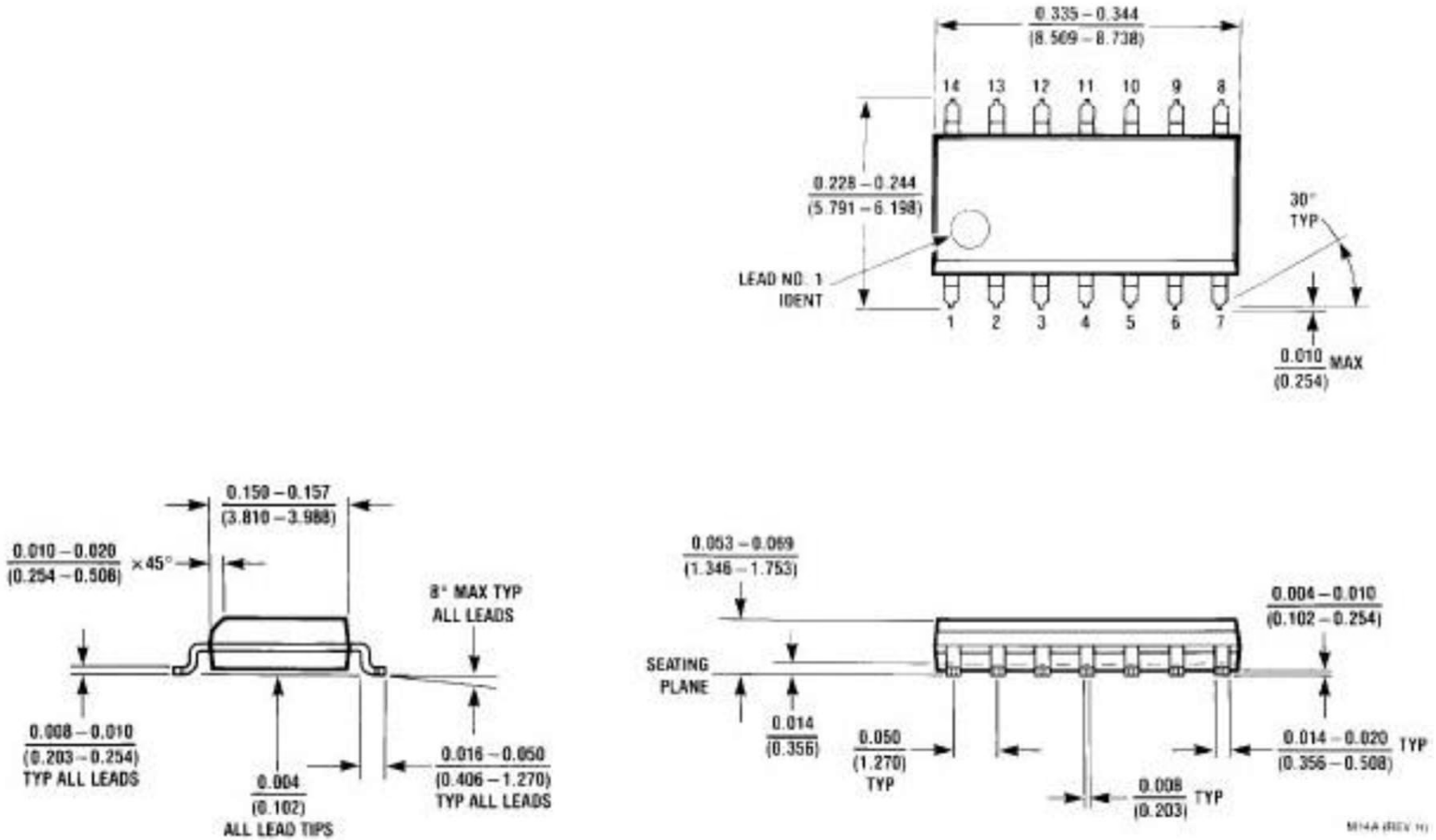
at V_{CC} = 5V and T_A = 25°C

Symbol	Parameter	Conditions	Min	Max	Units
t_{PLH}	Propagation Delay Time LOW-to-HIGH Level Output	C_L = 15 pF R_L = 400Ω		22	ns
t_{PHL}	Propagation Delay Time HIGH-to-LOW Level Output			15	ns

Hoja de datos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Physical Dimensions inches (millimeters) unless otherwise noted



14-Lead Small Outline Integrated Circuit (SOIC), JEDEC MS-012, 0.150 Narrow
Package Number M14A

Familia lógica ideal

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

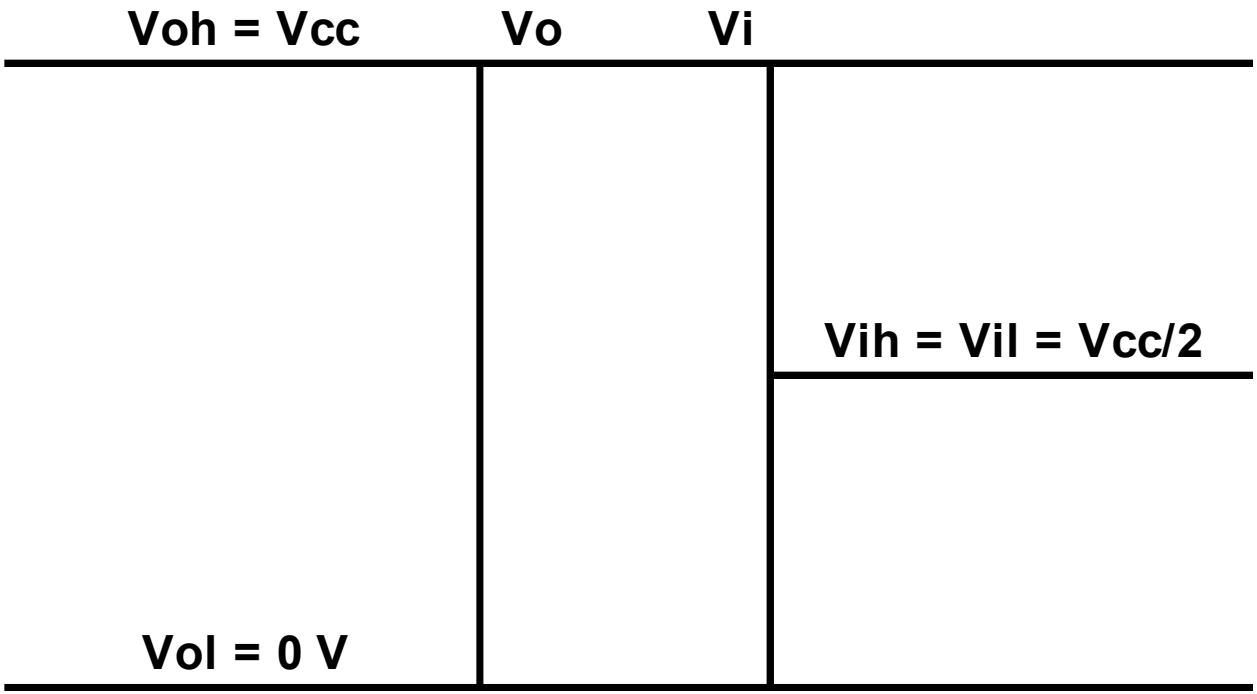
Niveles de tensión:

$V_{OH} = V_{CC}$

$V_{IH} = V_{IL} = \frac{V_{CC}}{2}$

$V_{OL} = 0\ V$

$NM = \frac{V_{CC}}{2}$

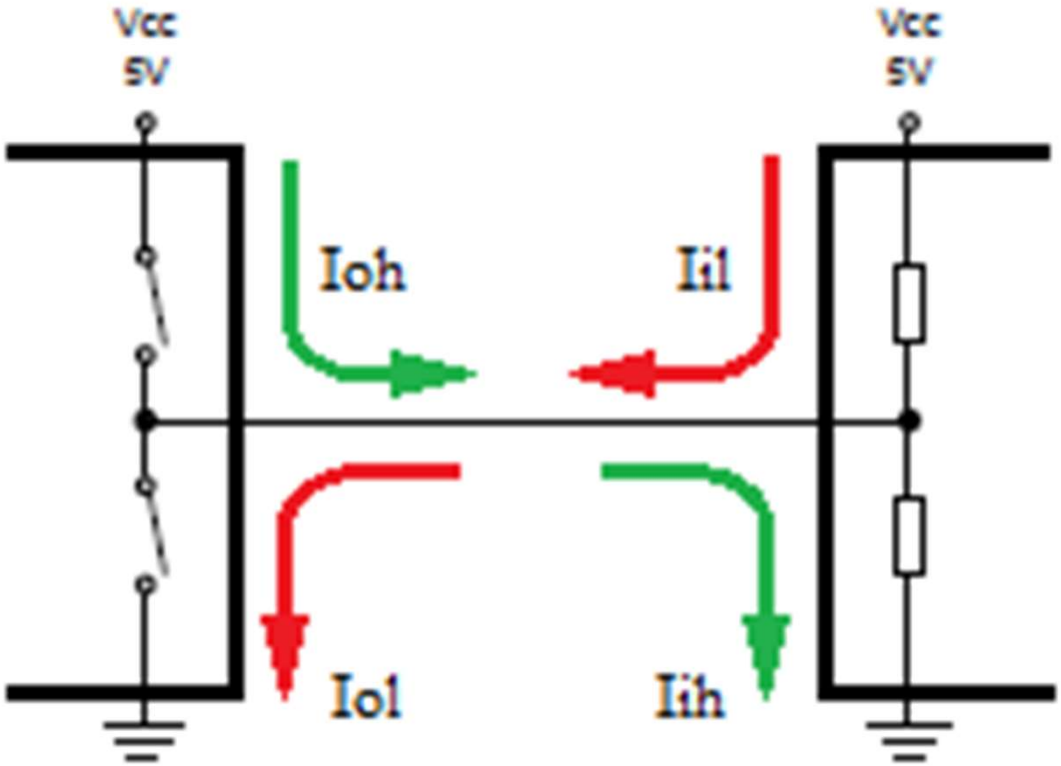


Familia lógica ideal

Corrientes:

$I_{OH} = I_{OL} \rightarrow \infty$ $\qquad$ $I_{IH} = I_{IL} = 0$

$FO \rightarrow \infty$ $\qquad$ $Zi \rightarrow \infty$



Familia lógica ideal

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Conmutación:

$$t_p = 0 \qquad P_d = 0 \qquad FM = 0$$

El valor de FM de una familia nos dará idea de qué tan cerca de la **familia lógica ideal** estará esta familia lógica.

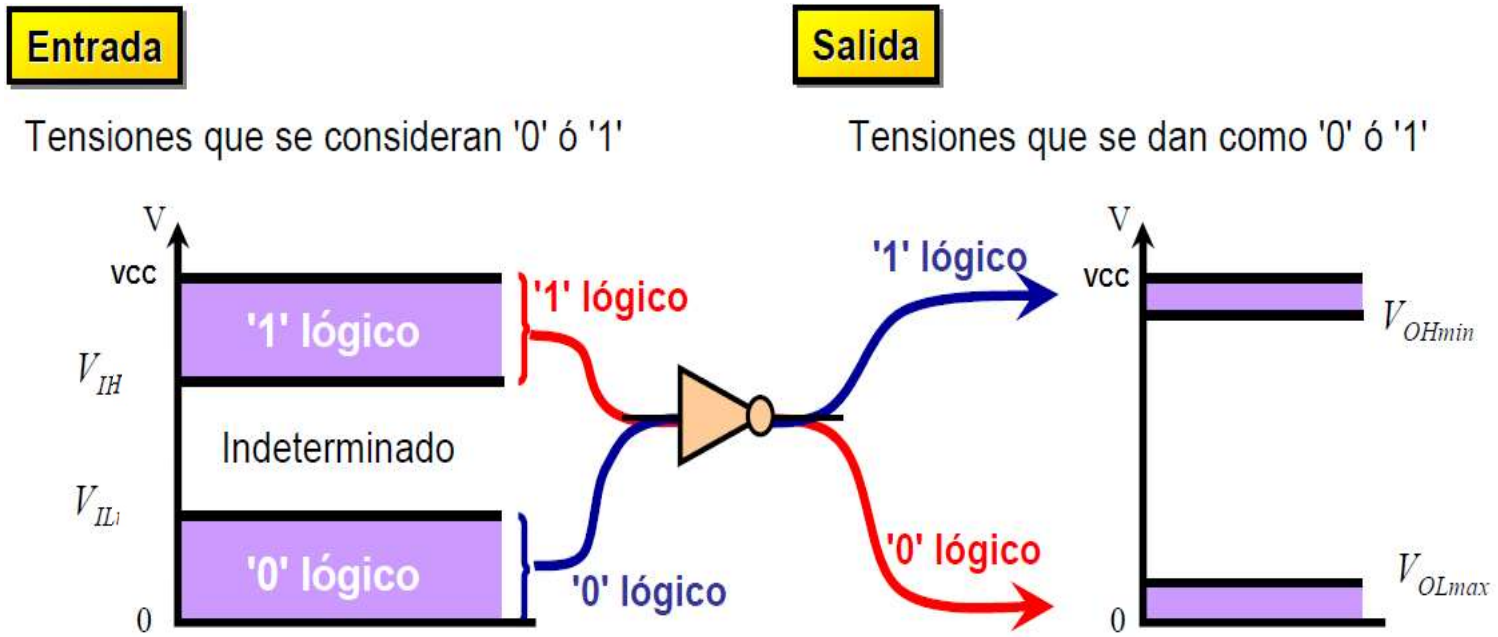
Como esto es físicamente imposible, nos encontramos con el dilema de que si quiero conmutación rápida deberé sacrificar potencia, y si quiero poca potencia disipada tendré por lo general retardos de propagación mayores.

Esto da lugar a implementar familias enfocadas en solo una de estas características, es decir de **bajo consumo** o de **alta velocidad**.

Familias activas y pasivas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

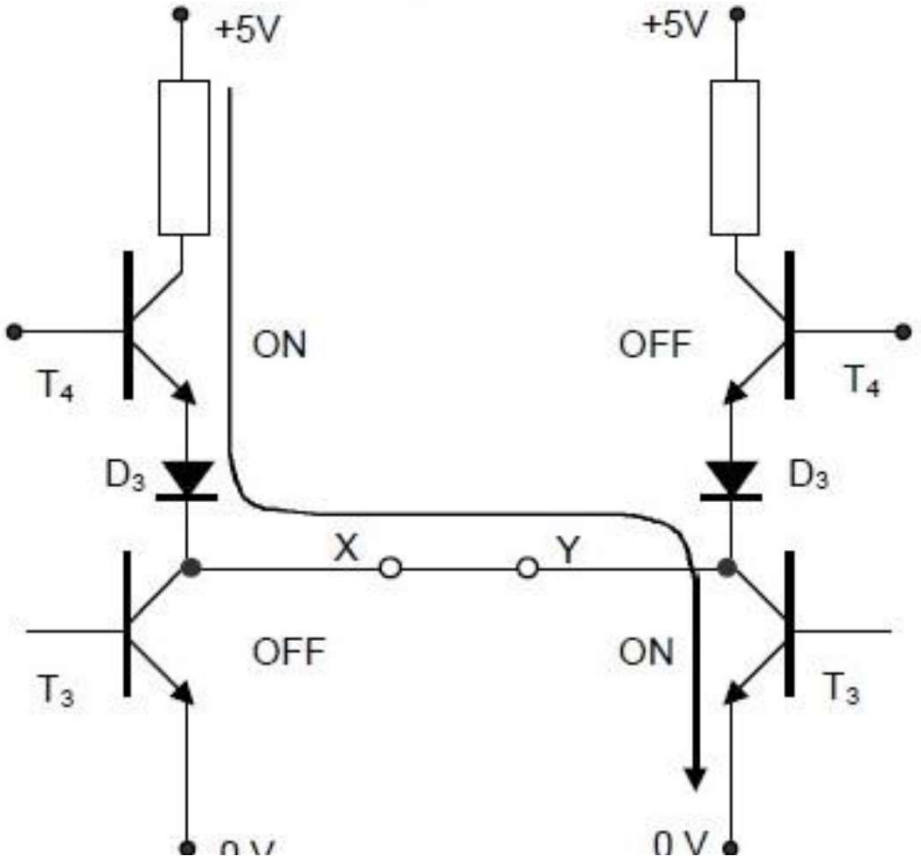
Una **familia activa** no degrada los niveles lógicos, por el contrario, si el nivel de entrada está cercano a los límites de la zona prohibida, a su salida el nivel correspondiente será reestablecido.

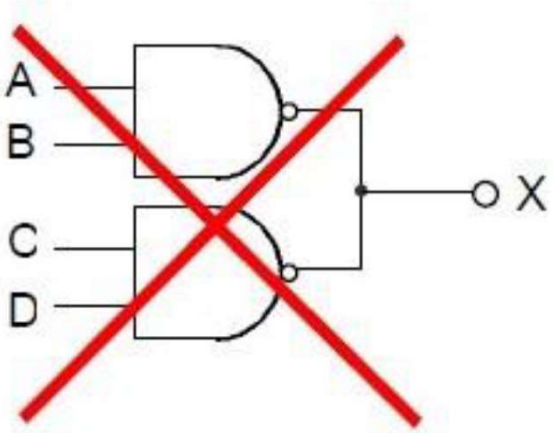


Una **familia pasiva** no reestablece los niveles lógicos, por lo que estos se van degradando por cada nivel atravesado. Por ejemplo: Lógica de diodos.

Topología de Bus

Para conectar salidas a un **bus** no puedo utilizar las Standard.

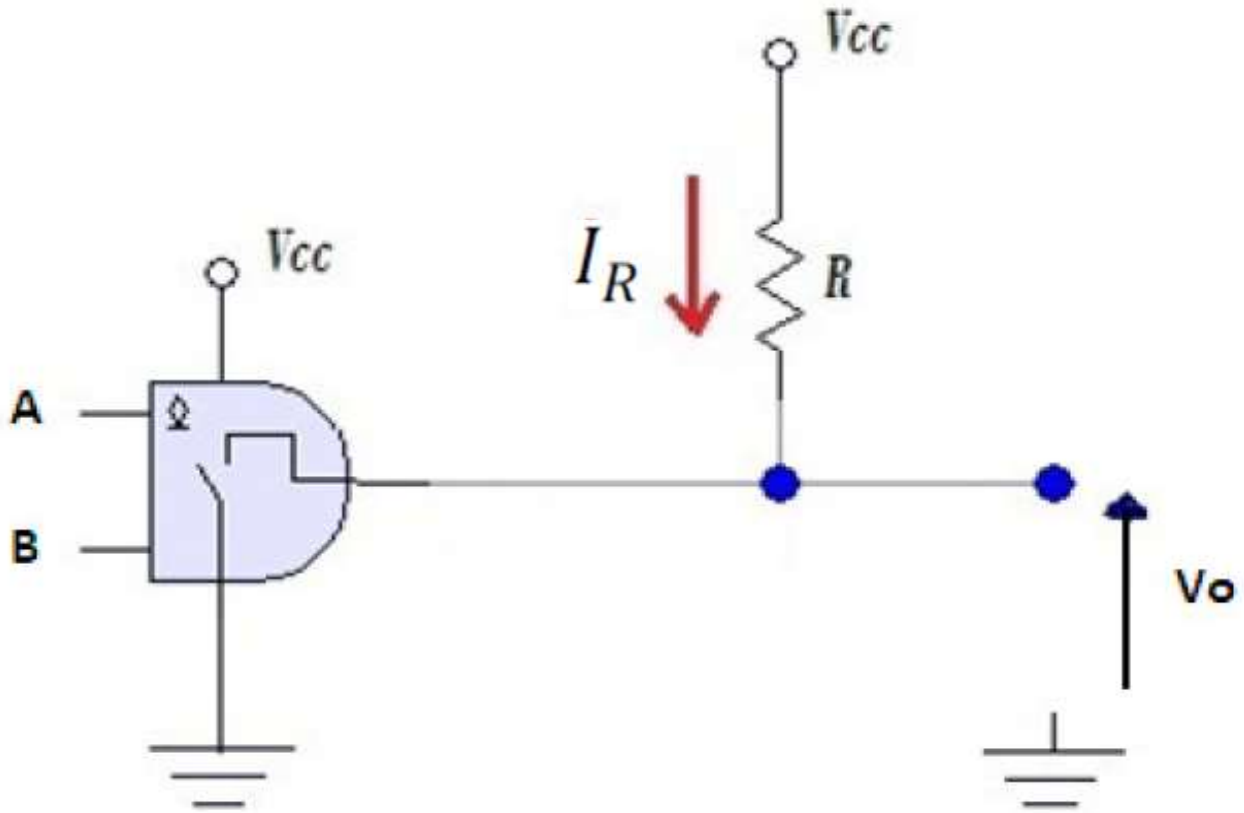




Salidas Open Collector / Open Drain

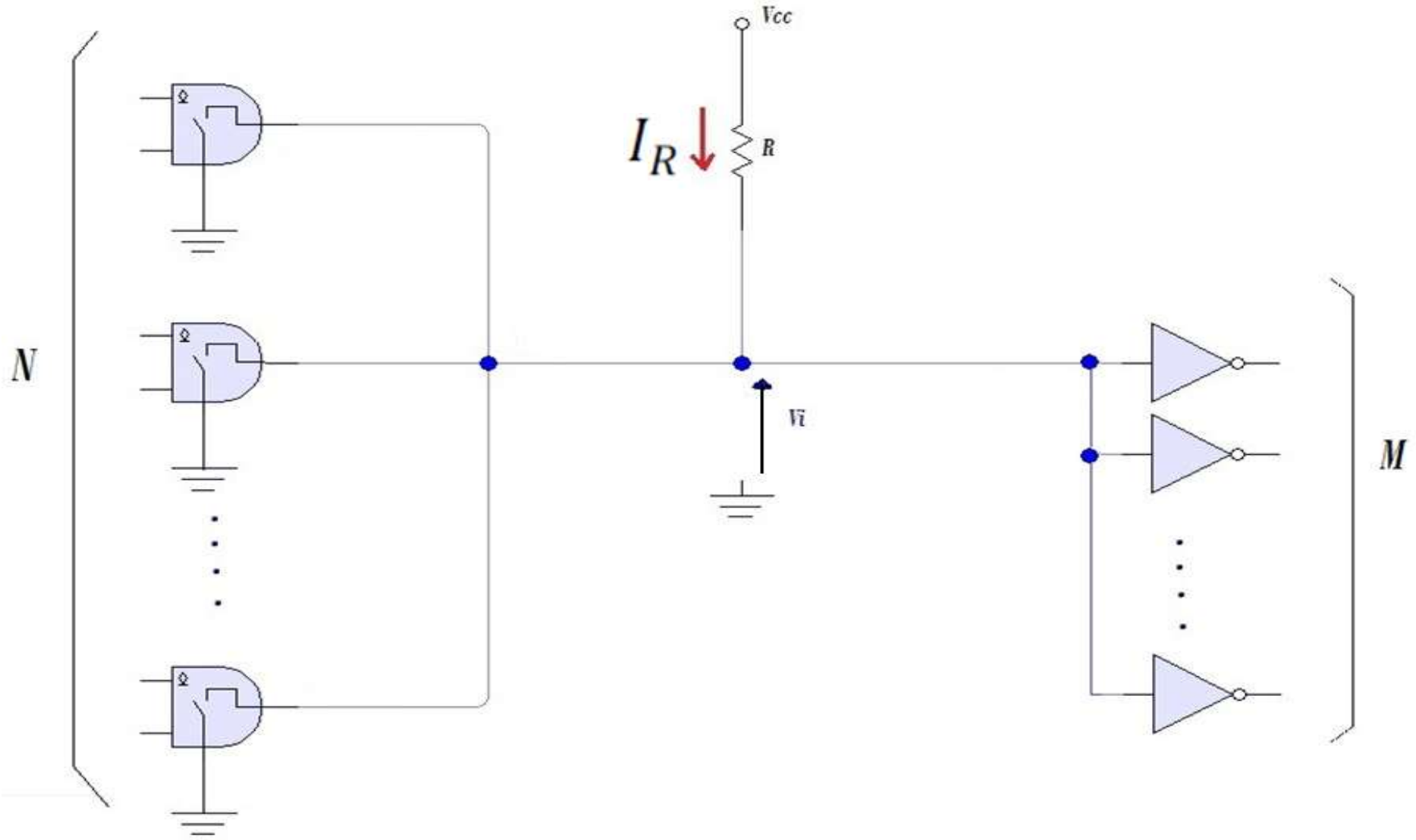
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

La compuerta solo puede poner el 0 lógico. Cuando debería poner el 1 lógico está abierta, pero el **resistor externo R** permite tener un nivel alto.
En las hojas de datos se indica cómo acotar los valores **mínimo** y **máximo** de R.
Para **bajo consumo** elijo R cercana a la máxima (menor corriente, P = V . I) y para **alta velocidad** cercana a la mínima (τ = R . C).

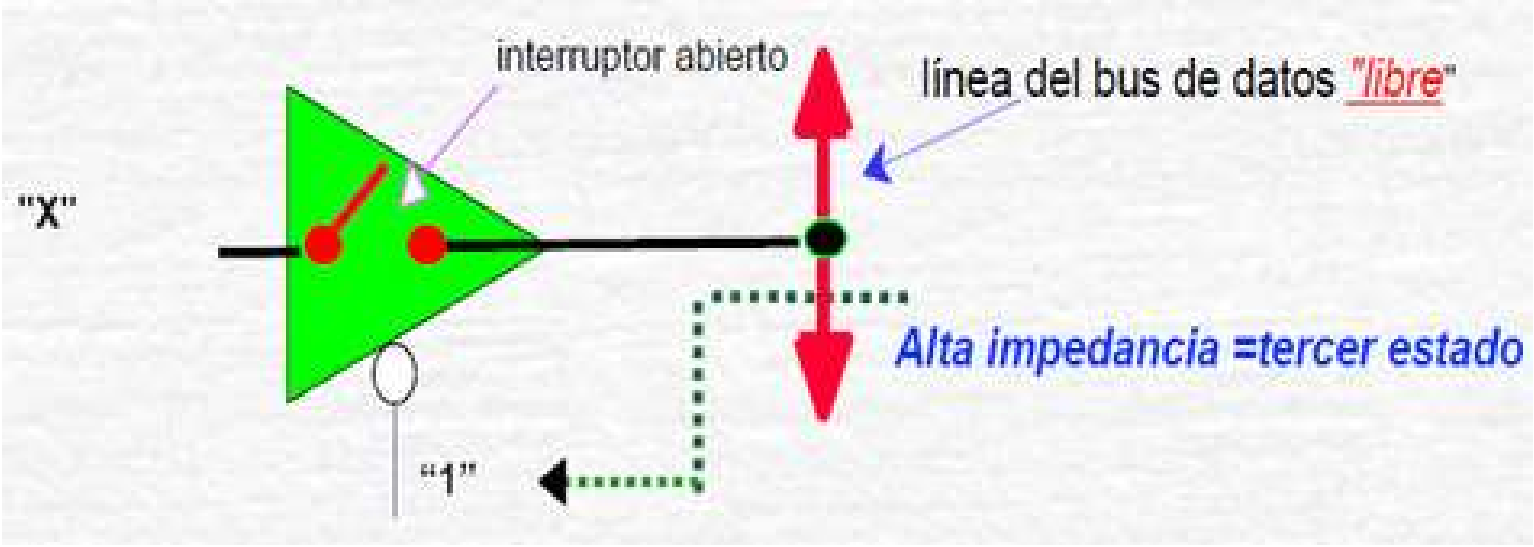


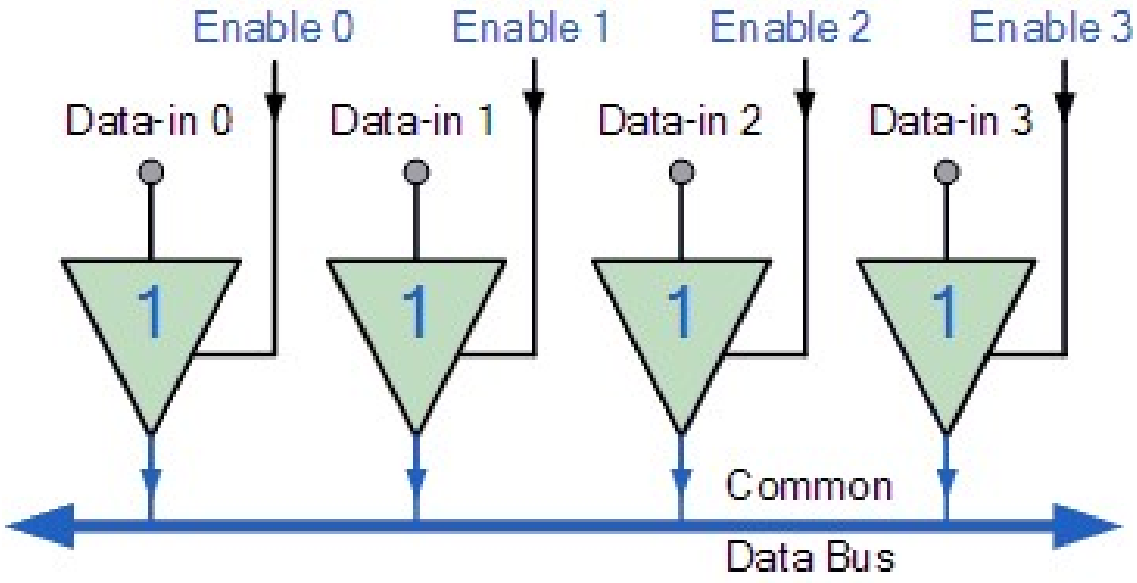
Salidas Open Collector / Open Drain

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



Salida Tri-state





Resumen de tipos de salidas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

TTL

Standard (Totem Pole)
Open Collector
Tri-State

CMOS

Standard
Open Drain
Tri-State

Tipo de salida	Velocidad	Conexión en bus
Totem Pole	SI	NO
Open Collector	NO	SI
Tri- State	SI	SI

Entradas no utilizadas

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Debemos tener en cuenta que si una entrada **no está conectada** a ningún potencial (flotante), podrá tomar valores que afecten al funcionamiento de la compuerta.
Se deben conectar las **entradas** de una compuerta que **no se utilizan** a un potencial que **no afecte la operación lógica**.

A+B = A+B+B = A+B+0



A . B = A . B . B = A . B . 1

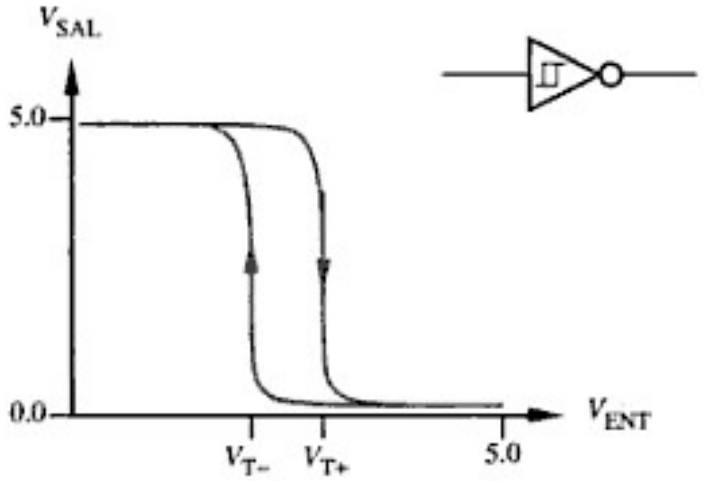


Entrada Schmitt Trigger

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

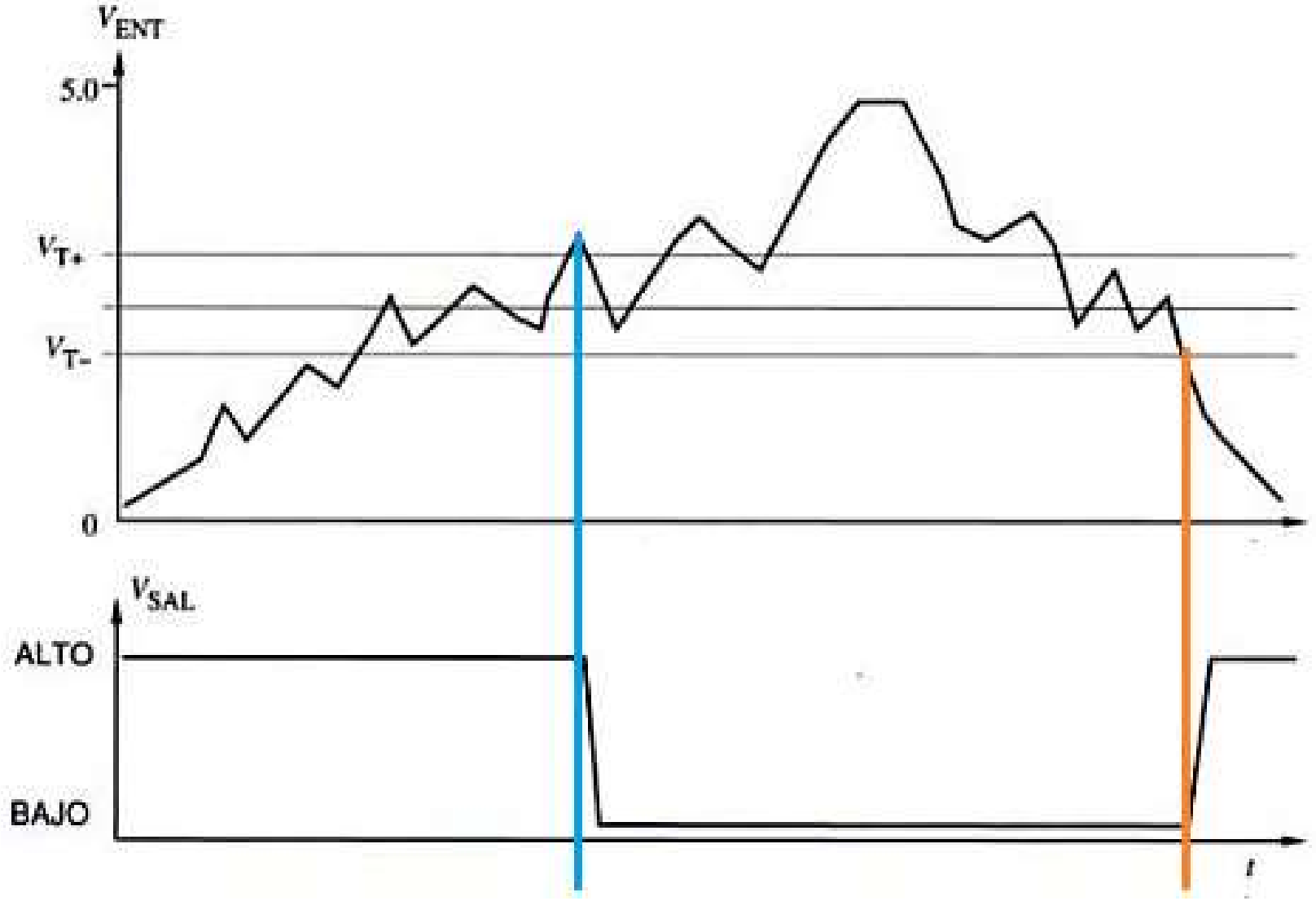
Recommended Operating Conditions

Symbol	Parameter	54LS14			DM74LS14			Units
		Min	Nom	Max	Min	Nom	Max	
V_{CC}	Supply Voltage	4.5	5	5.5	4.75	5	5.25	V
V_{T+}	Positive-Going Input Threshold Voltage (Note 1)	1.5	1.6	2.0	1.4	1.6	1.9	V
V_{T-}	Negative-Going Input Threshold Voltage (Note 1)	0.6	0.8	1.1	0.5	0.8	1	V
HYS	Input Hysteresis (Note 1)	0.4	0.8		0.4	0.8		V
I_{OH}	High Level Output Current			−0.4			−0.4	mA
I_{OL}	Low Level Output Current			4			8	mA
T_A	Free Air Operating Temperature	−55		125	0		70	°C



Entrada Schmitt Trigger

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



Familia TTL

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Subfamilias: 74XXXNN

	Standard
L	Low Power
H	High Speed
S	Schottky
LS	Low Power Schottky
AS	Advanced Schottky
ALS	Advanced Low Power Schottky
F	Fast

Familia CMOS

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Subfamilias: 74XXXNN (llevan una C)

Serie 4000 (antigua)

C		CMOS
AC	ACT	Advanced
HC	HCT	High Speed
AHC	AHCT	Advanced High Speed
VHC	VHCT	Very High Speed

BiCMOS:

Tiene las ventajas de TTL y CMOS

Familia CMOS

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Susceptibilidad a descargas electrostáticas en CMOS:

Son susceptibles a daños como consecuencia de su alta impedancia de entrada. Una pequeña carga electrostática que circule por estas altas impedancias puede dar origen a voltajes peligrosos.

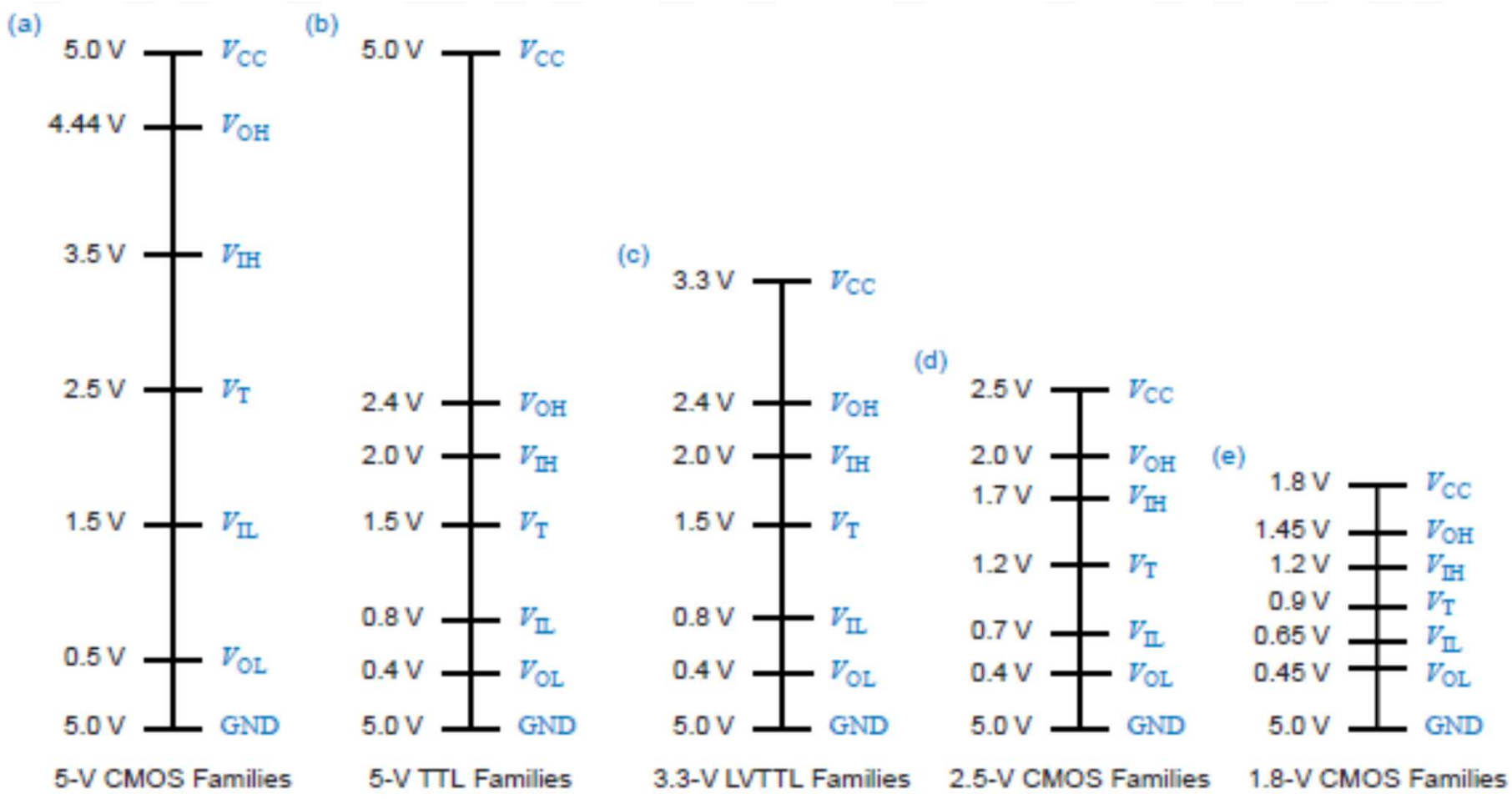
Están protegidos contra este tipo de daño mediante la inclusión en sus entradas de diodos de protección en inversa. Si bien los diodos por lo general cumplen con su finalidad, algunas veces no comienzan a conducir con la rapidez necesaria para evitar que el CI sufra daños.

Por consiguiente, sigue siendo buena idea observar las precauciones de manejo recomendadas por el fabricante para todos los CI CMOS.

Para ello se dispone de una serie de medidas como pisos conductivos, uso de ropa antiestática, referencia del operario a tierra, etc.

LVTTL y LVCMOS

Reduciendo la tensión de alimentación se reduce el consumo de potencia. Por ello aparecen las **familias LV** (Low Voltage): **LVTTL** y **LVCMOS**.



FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Códigos

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar las distintas posibilidades de representar información numérica y alfabética.

Conocer las características principales que puede presentar un código numérico BCD.

Estudiar las características que necesitan los códigos para la seguridad de la información (detectores y correctores de errores).

Evaluar el método de Hamming para detección y corrección de errores.

Definición

Para representar información se utilizan códigos de distintos tipos, primero analizaremos los códigos **numéricos**, particularmente los **BCD** (decimal codificado en binario), que me permiten representar dígitos decimales (**0 a 9**).

Características

- Pesado, posicional o ponderado
- Autocomplementado
- Progresivo
- Cerrado
- Reflejado
- Distancia mínima

Códigos BCD pesados

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Sin peso	Pesado
	8421
0 0000	0 0000
1 0101	1 0001
2 0010	2 0010
3 0100	3 0011
4 1111	4 0100
5 0110	5 0101
6 1011	6 0110
7 1101	7 0111
8 1010	8 1000
9 0011	9 1001

Códigos BCD pesados

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

	6311		5211	
0	0000		0000	
1	0001	(0010)	0001	(0010)
2	0011		0100	(0011)
3	0100		0101	(0110)
4	0101	(0110)	0111	
5	0111		1000	
6	1000		1010	(1001)
7	1001	(1010)	1011	(1100)
8	1011		1110	(1101)
9	1100		1111	

Códigos BCD autocomplementados

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

	84(-2)(-1)	XS-3 (8421)
0	0000	0011
1	0111	0100
2	0110	0101
3	0101	0110
4	0100	0111
5	1011	1000
6	1010	1001
7	1001	1010
8	1000	1011
9	1111	1100

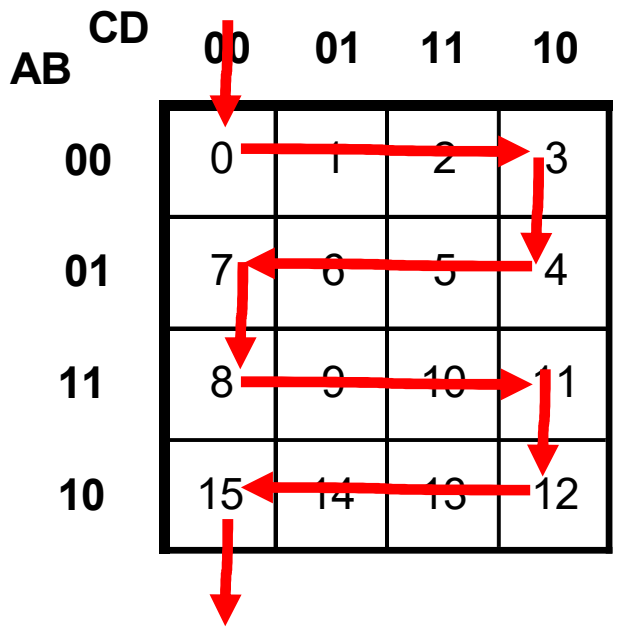
Código reflejado de Gray de 4 bits

0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

Reflejado (por construcción)

Progresivo (solo cambia 1 bit)

Cerrado (solo cambia 1 bit del **15** al **0**)



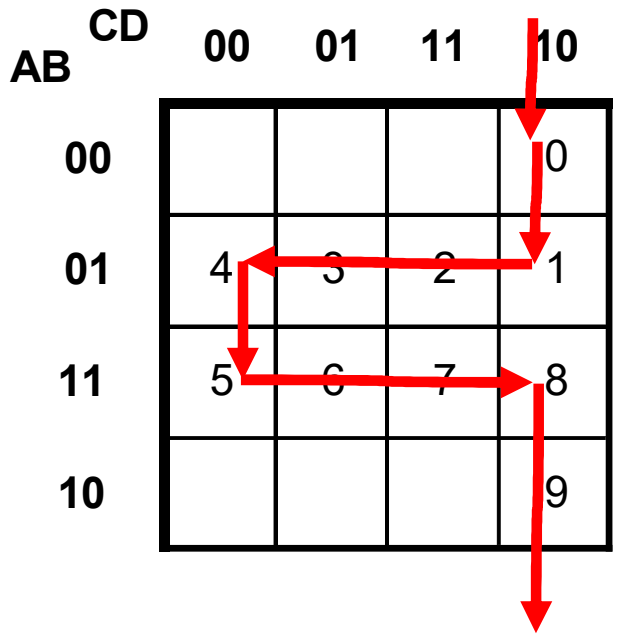
Código de Gray BCD XS-3

	0	0000
	1	0001
	2	0011
0	3	0010
1	4	0110
2	5	0111
3	6	0101
4	7	0100
5	8	1100
6	9	1101
7	10	1111
8	11	1110
9	12	1010
	13	1011
	14	1001
	15	1000

Reflejado (por construcción)

Progresivo (solo cambia 1 bit)

Cerrado (solo cambia 1 bit del **9** al **0**)



Representación de números en BCD

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Puedo representar al número decimal 739 en binario, pero también mediante códigos **BCD** como los vistos:

	7	**3**	**9**	
Binario		$1011100011_{\|2}$		(10 bits)
BCD 8421	0111	0011	1001	(12 bits)
BCD 5211	1011	0101	1111	
BCD XS-3	1010	0110	1100	
BCD Gray XS-3	1111	0101	1010	

Distancia mínima de un código

Interesa estudiar la **distancia mínima** de un código:

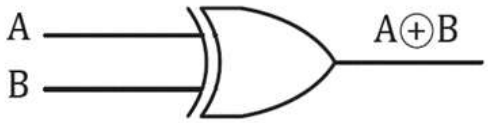
	8421
	ABCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

AB \ CD	00	01	11	10
00	0	1	3	2
01	4	5	7	6
11				
10	8	9		

dmin = 1 >>> no detecta errores de 1 bit

Distancia mínima de un código

Para aumentar **dmin** puedo agregar un **bit de paridad**:



A	B	A⊕B
0	0	0
0	1	1
1	0	1
1	1	0

	8421-P	
	ABCD	P
0	0000	0
1	0001	1
2	0010	1
3	0011	0
4	0100	1
5	0101	0
6	0110	0
7	0111	1
8	1000	1
9	1001	0

P = 0

AB \ CD	00	01	11	10
00	0	x	3	x
01	x	5	x	6
11		x		x
10	x	9	x	

P = 1

AB \ CD	00	01	11	10
00	x	1	x	2
01	4	x	7	x
11	x		x	
10	8	x		x

dmin = 2 >>> detecta errores de 1 bit (no corrige)

Distancia mínima de un código

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Supongamos un código con **dmin = 3**,
analicemos parte del mapa K:

Si asumimos que solo puede haber 1 error:

dmin = 3 >>> detecta 1 error y lo corrige.

Pero si asumimos que puede haber 2 errores:

dmin = 3 >>> detecta 2 errores sin corregirlos.

AB \ CD	00	01	11	10
00	1	1	2	1
01	1	2	2	2
11	x	3	2	x
10	1	x	x	x

AB \ CD	00	01	11	10
00	1	x	x	x
01	x	3	2	x
11	3	3	3	x
10	x	3	x	

Distancia mínima de un código

Ahora supongamos un código con **dmin = 4**, analicemos parte del mapa K:

dmin = 4 >>> detecta 2 errores o corrige 1.

Podemos establecer:

dmin = d + c + 1

1 = 0 + 0 + 1
2 = 1 + 0 + 1
3 = 1 + 1 + 1
3 = 2 + 0 + 1
4 = 2 + 1 + 1
4 = 3 + 0 + 1

AB \ CD	00	01	11	10
00	1	1	1/2	1
01	1	1/2	2	1/2
11	1/2	2	2	2
10	1	1/2	2	1/2

AB \ CD	00	01	11	10
00	1	1/2		1/2
01	1/2		1/2	
11		1/2	2	1/2
10	1/2		1/2	

Códigos redundantes

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

2 entre 5		Biquinario		Johnson	
	7421P		**50 43210**		
0	11000	0	01 00001	0	00000
1	00011	1	01 00010	1	00001
2	00101	2	01 00100	2	00011
3	00110	3	01 01000	3	00111
4	01001	4	01 10000	4	01111
5	01010	5	10 00001	5	11111
6	01100	6	10 00010	6	11110
7	10001	7	10 00100	7	11100
8	10010	8	10 01000	8	11000
9	10100	9	10 10000	9	10000

Método de Hamming

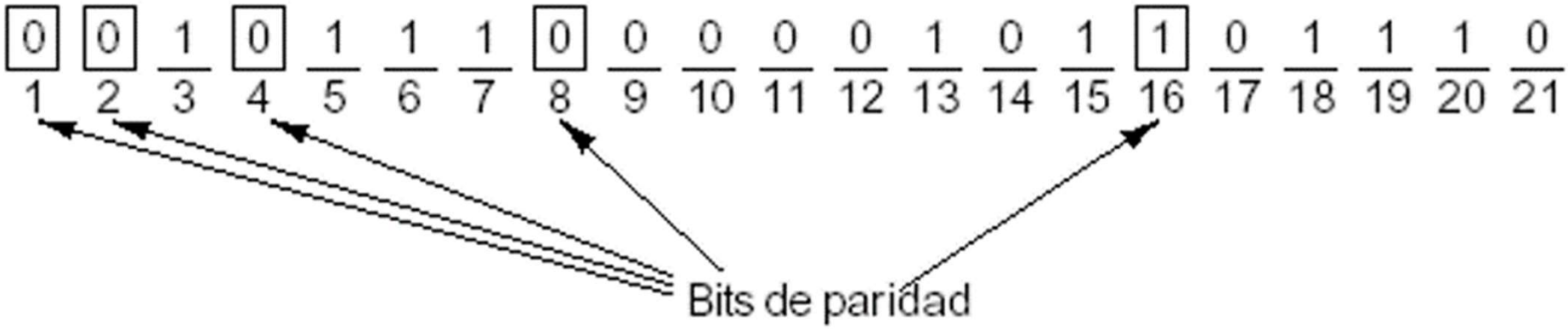
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Hamming **no es un código** en sí mismo. El **método** agrega **bits de paridad** a palabras pertenecientes a cualquier código. Lo hace en posiciones determinadas para poder detectar y corregir errores.

k_i = bit de paridad

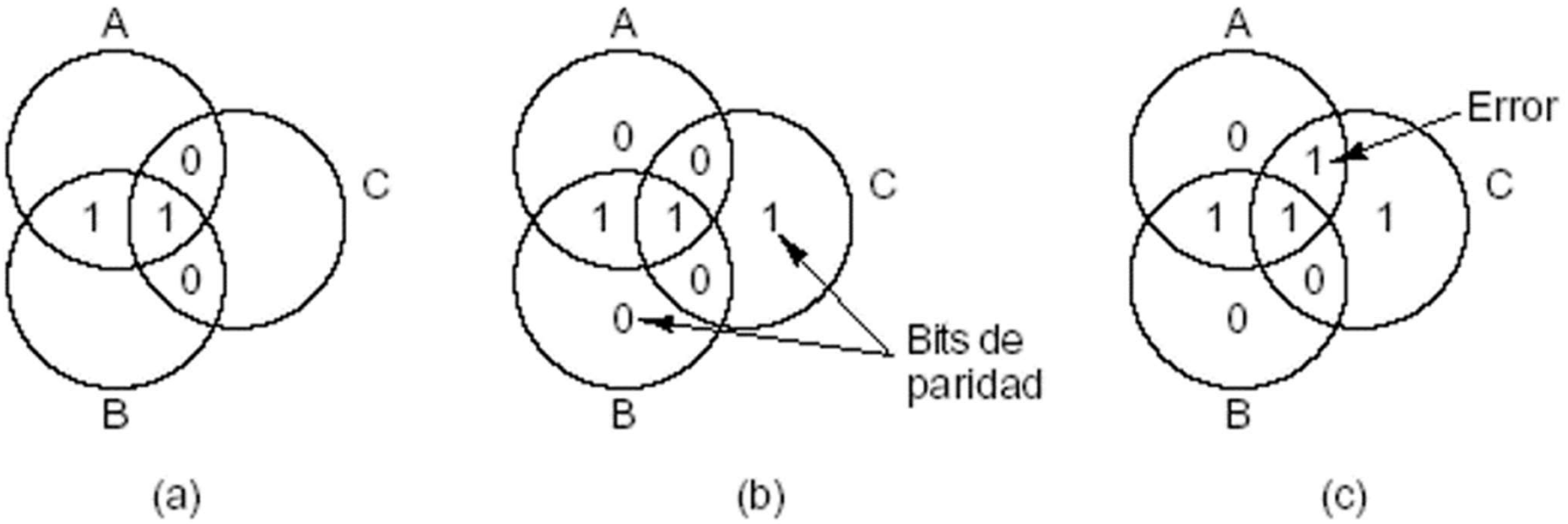
m_i = bit de mensaje

Los bits de paridad ocupan las posiciones que son potencias de 2: k_1, k_2, k_4, k_8, k_{16}, etc.



Método de Hamming

Para 4 bits de mensaje tendremos 3 bits de paridad: $\mathbf{k_1 k_2 m_3 k_4 m_5 m_6 m_7}$



k_i será la paridad par de los bits de mensaje que tengan en su posición en binario el peso i en 1:

k_1 >>> m_3 (011), m_5 (101) y m_7 (111)

k_2 >>> m_3 (011), m_6 (110) y m_7 (111)

k_4 >>> m_5 (101), m_6 (110) y m_7 (111)

Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Calculamos los k_i:

$$\mathbf{k_1} = m_3 \oplus m_5 \oplus m_7$$
$$\mathbf{k_2} = m_3 \oplus m_6 \oplus m_7$$
$$\mathbf{k_4} = m_5 \oplus m_6 \oplus m_7$$

Los calculamos para $\mathbf{m_3 m_5 m_6 m_7 = 1100}$

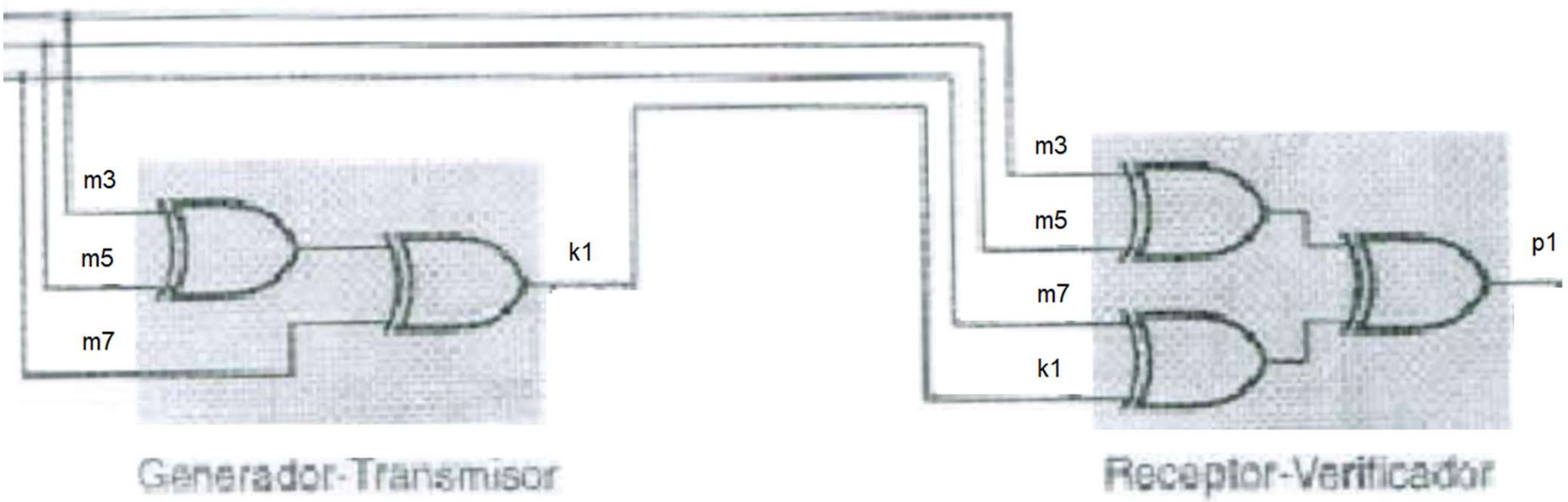
$$\mathbf{k_1} = 1 \oplus 1 \oplus 0 = \mathbf{0}$$
$$\mathbf{k_2} = 1 \oplus 0 \oplus 0 = \mathbf{1}$$
$$\mathbf{k_4} = 1 \oplus 0 \oplus 0 = \mathbf{1}$$

Por lo tanto enviamos: $\mathbf{k_1 k_2 m_3 k_4 m_5 m_6 m_7 = 0111100}$

Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Los circuitos **generador** de paridad y **verificador** quedan:



Debo verificar todas las paridades en el receptor (P_i), resultando **todas 0** si **no hay errores**.

Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En el receptor calculamos los P_i:

$$\mathbf{P_1} = k_1 \oplus m_3 \oplus m_5 \oplus m_7$$
$$\mathbf{P_2} = k_2 \oplus m_3 \oplus m_6 \oplus m_7$$
$$\mathbf{P_4} = k_4 \oplus m_5 \oplus m_6 \oplus m_7$$

Los calculamos para nuestro ejemplo: $\mathbf{k_1 k_2 m_3 k_4 m_5 m_6 m_7 = 0111100}$

$$\mathbf{P_1} = 0 \oplus 1 \oplus 1 \oplus 0 = \mathbf{0}$$
$$\mathbf{P_2} = 1 \oplus 1 \oplus 0 \oplus 0 = \mathbf{0}$$
$$\mathbf{P_4} = 1 \oplus 1 \oplus 0 \oplus 0 = \mathbf{0}$$

Por lo tanto **no hubo error**.

Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Si recibimos un **error** en m_6 : $k_1k_2m_3k_4m_5m_6m_7$ = **0111110**

$$P_1 = 0 \oplus 1 \oplus 1 \oplus 0 = \mathbf{0}$$
$$P_2 = 1 \oplus 1 \oplus 1 \oplus 0 = \mathbf{1}$$
$$P_4 = 1 \oplus 1 \oplus 1 \oplus 0 = \mathbf{1}$$

Por obtener paridades distintas de 0 (impares), estoy en presencia de un **error**.

Para determinar en qué posición analizo:

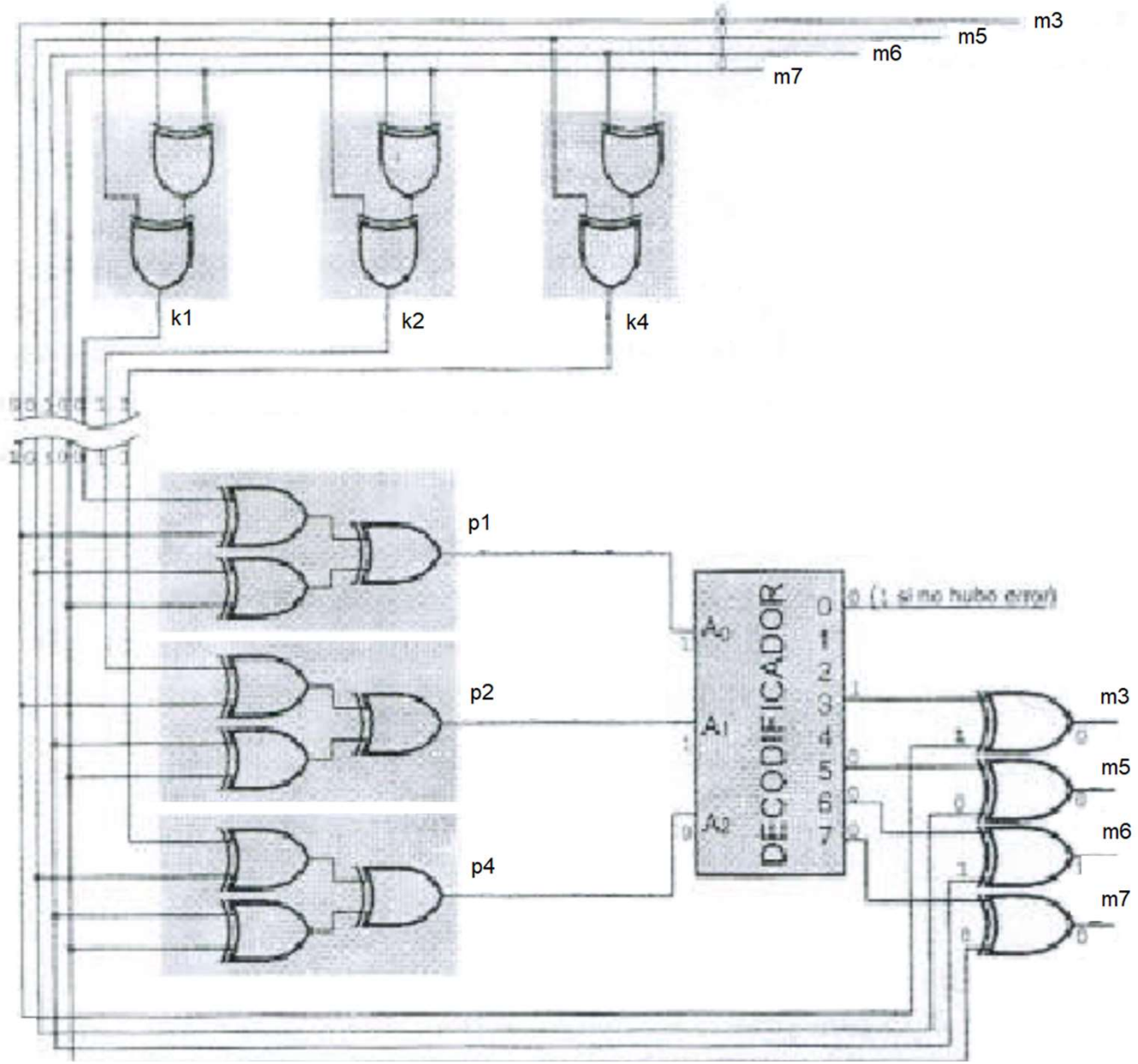
$P_4P_2P_1$ = **110** >>> el **error** está en el **bit 6**

Por lo tanto lo corrijo invirtiéndolo para obtener el mensaje original, y desecho los bits de paridad:

0111110 >>> **0111100** >>> $m_3m_5m_6m_7$ = **1100**

Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



Método de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Para 4 bits de mensaje agregué 3 bits de paridad, 75 % de gasto adicional, sin embargo, por **cada bit** de paridad agregado **duplico** la cantidad de bits de mensaje, resultando apropiado para palabras largas:

Word size	Check bits	Total size	% de gasto extra
8	4	12	50
16	5	21	31
32	6	38	19
64	7	71	11
128	8	136	6
256	9	265	4
512	10	522	2

Códigos alfanuméricos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Los códigos **alfanuméricos**, tal como su nombre lo indica, permite representar letras, números, caracteres especiales y comandos.
Actualmente uno de los más utilizados es el ASCII (American Standard Code for Information Interchange). El estándar **ASCII** utiliza **7 bits** para representar los caracteres del alfabeto latino (inglés).
Los primeros 32 caracteres del código ASCII son conocidos como "**comandos** de control", estos sirven para configurar algunas opciones del texto sin imprimir ningún caracter en pantalla. Originalmente fue usado para enviar información a dispositivos de texto, como teleimpresoras, por lo que actualmente solo se utilizan unos pocos comandos.
El resto son caracteres imprimibles que incluyen números, letras mayúsculas y minúsculas (inglés), símbolos y caracteres especiales, etc.
Existe también el **ASCII extendido** o de **8 bits** (1 byte) que permite representar otros caracteres adicionales, como por ejemplo letras pertenecientes a otros idiomas, letras con tildes, etc.

Códigos alfanuméricos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Comandos ASCII

Comando ASCII	Código hexadecimal	Función
NUL	0x00	Carácter nulo
SOH	0x01	Inicio de encabezado
STX	0x02	Inicio de texto
ETX	0x03	Fin de texto
EOT	0x04	Fin de transmisión
ENQ	0x05	Consulta
ACK	0x06	Acuse de recibo
BEL	0x07	Timbre
BS	0x08	Retroceso *
HT	0x09	Tabulación horizontal
LF	0x0A	Salto de línea *
VT	0x0B	Tabulación vertical
FF	0x0C	De avance
CR	0x0D	Retorno de carro *
SO	0x0E	Mayúsculas fuera
SI	0x0F	En mayúsculas
DEL	0x10	Enlace de datos/Escape
DC1	0x11	Dispositivo de control 1
DC2	0x12	Dispositivo de control 2
DC3	0x13	Dispositivo de control 3
DC4	0x14	Dispositivo de control 4
NAK	0x15	Confirmación negativa
SYN	0x16	Síncrono en espera
ETB	0x17	Fin de transmisión de bloque
CAN	0x18	Cancelar

Códigos alfanuméricos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Caracteres imprimibles ASCII 7 bits

Código hexadecimal	Símbolo ASCII	Código hexadecimal	Símbolo ASCII	Código hexadecimal	Símbolo ASCII
0x20	Espacio ' '	0x40	@	0x60	`
0x21	!	0x41	A	0x61	a
0x22	"	0x42	B	0x62	b
0x23	#	0x43	C	0x63	c
0x24	$	0x44	D	0x64	d
0x25	%	0x45	E	0x65	e
0x26	&	0x46	F	0x66	f
0x27	'	0x47	G	0x67	g
0x28	(	0x48	H	0x68	h
0x29	)	0x49	I	0x69	i
0x2A	*	0x4A	J	0x6A	j
0x2B	+	0x4B	K	0x6B	k
0x2C	,	0x4C	L	0x6C	l
0x2D	-	0x4D	M	0x6D	m
0x2E	.	0x4E	N	0x6E	n
0x2F	/	0x4F	O	0x6F	o
0x30	0	0x50	P	0x70	p
0x31	1	0x51	Q	0x71	q
0x32	2	0x52	R	0x72	r
0x33	3	0x53	S	0x73	s
0x34	4	0x54	T	0x74	t

Códigos alfanuméricos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Caracteres imprimibles ASCII 7 bits (continuación)

0x35	5	0x55	U	0x75	u
0x36	6	0x56	V	0x76	v
0x37	7	0x57	W	0x77	w
0x38	8	0x58	X	0x78	x
0x39	9	0x59	Y	0x79	y
0x3A	:	0x5A	Z	0x7A	z
0x3B	;	0x5B	[	0x7B	{
0x3C	<	0x5C	\	0x7C	|
0x3D	=	0x5D	]	0x7D	}
0x3E	>	0x5E	^	0x7E	~
0x3F	?	0x5F	_		

FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Decodificadores y Multiplexores

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar el concepto de Módulo Universal o bloques lógicos combinacionales básicos.

Estudiar los conceptos de codificación y decodificación como también los circuitos que los llevan a cabo (Encoder y Decoder).

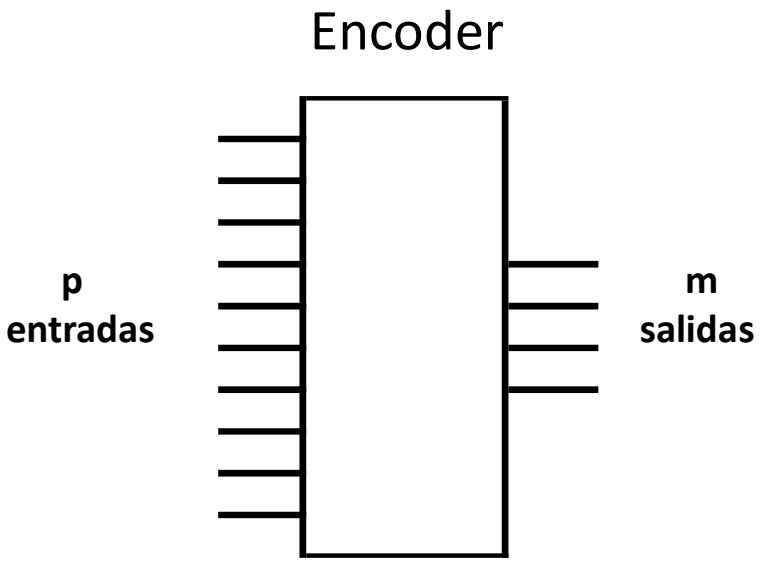
Ver la necesidad de la conversión entre distintos tipos de códigos.

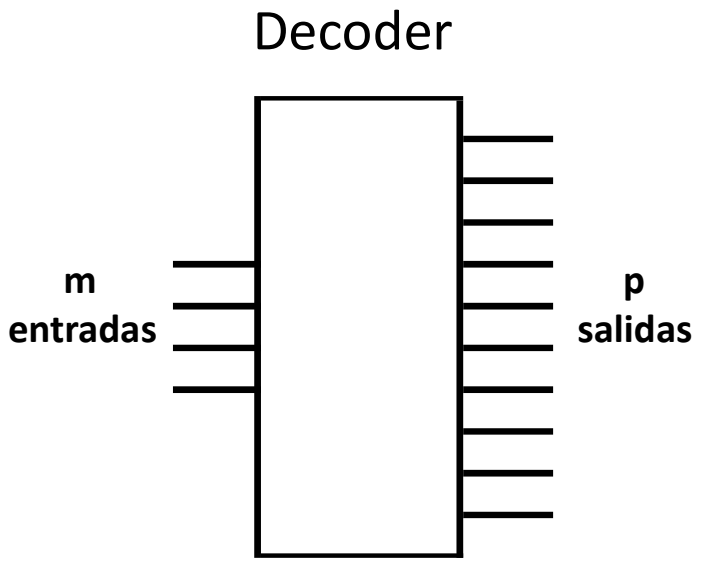
Analizar el concepto de multiplexación y demultiplexación analógica y digital como también las características de los circuitos Mux y Demux digitales.

Utilizar los bloques universales (CIs) anteriores para la implementación alternativa de funciones lógicas.

Codificación y decodificación

Es un proceso en el cual se “compacta” la información para luego de transmitirla y / o almacenarla, volverla a su estado original.
En el área digital tendremos circuitos **combinacionales** que realizarán esas tareas: Codificadores o Encoders y Decodificadores o Decoders.
En general los Codificadores tienen más entradas que salidas mientras que los Decodificadores tienen más salidas que entradas.

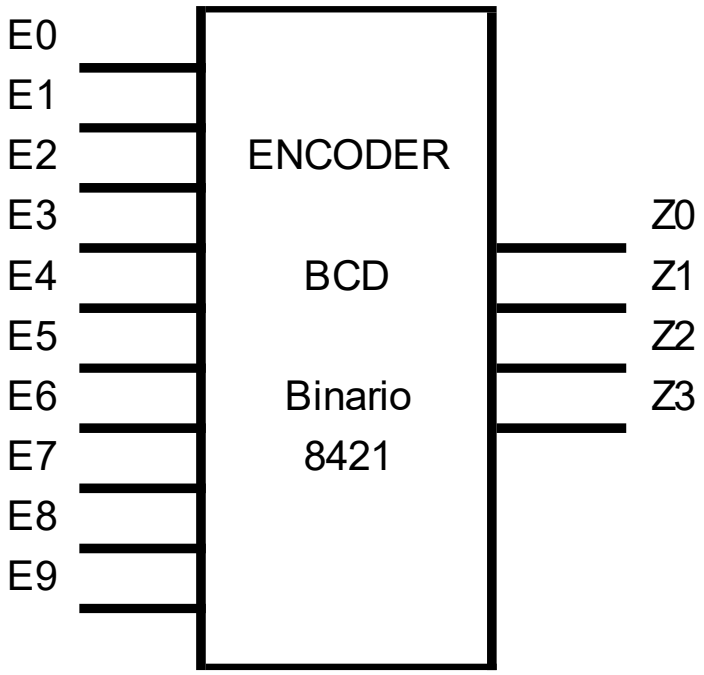




Codificador (Encoder)

Es un circuito **combinacional** que asigna un **código** binario de salida único a cada señal de entrada aplicada al dispositivo. Si un codificador tiene **n** entradas, el número de salidas **s** es tal que se debe cumplir la siguiente condición: $\mathbf{2^S \geq n}$. Para los comerciales, salvo para el BCD, se cumple $\mathbf{2^S = n}$ (ej: 4/2, 8/3, 16/4, etc.)

Encoder **BCD** a binario 8421:

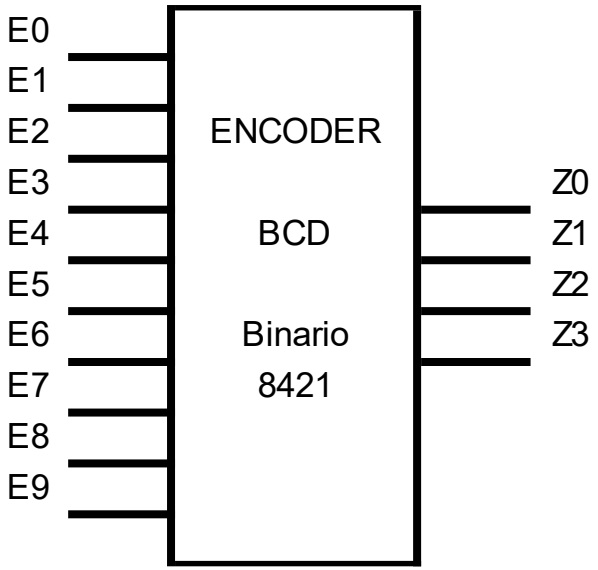


Dígito decimal	Código BCD A_3	A_2	A_1	A_0
0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1
8	1	0	0	0
9	1	0	0	1

Codificador (Encoder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

E0	E1	E2	E3	E4	E5	E6	E7	E8	E9	Z3	Z2	Z1	Z0
1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	**1**	0	0	0	0	0	0	0	0	0	0	0	**1**
0	0	**1**	0	0	0	0	0	0	0	0	0	**1**	0
0	0	0	**1**	0	0	0	0	0	0	0	0	**1**	**1**
0	0	0	0	**1**	0	0	0	0	0	0	**1**	0	0
0	0	0	0	0	**1**	0	0	0	0	0	**1**	0	**1**
0	0	0	0	0	0	**1**	0	0	0	0	**1**	**1**	0
0	0	0	0	0	0	0	**1**	0	0	0	**1**	**1**	**1**
0	0	0	0	0	0	0	0	**1**	0	**1**	0	0	0
0	0	0	0	0	0	0	0	0	**1**	**1**	0	0	**1**



$$Z_0 = E_1 + E_3 + E_5 + E_7 + E_9$$

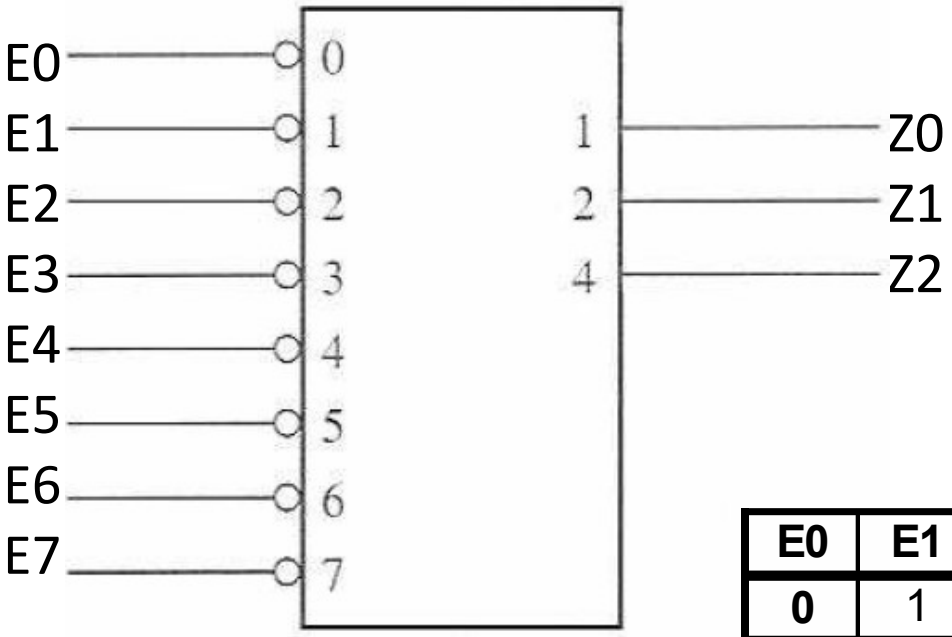
$$Z_1 = E_2 + E_3 + E_6 + E_7$$

$$Z_2 = E_4 + E_5 + E_6 + E_7$$

$$Z_3 = E_8 + E_9$$

Codificador (Encoder)

Veamos ahora el caso de un Encoder de "**8 a 3**" con entradas **negadas**:



$$Z_0 = \overline{E_1} + \overline{E_3} + \overline{E_5} + \overline{E_7}$$

$$Z_1 = \overline{E_2} + \overline{E_3} + \overline{E_6} + \overline{E_7}$$

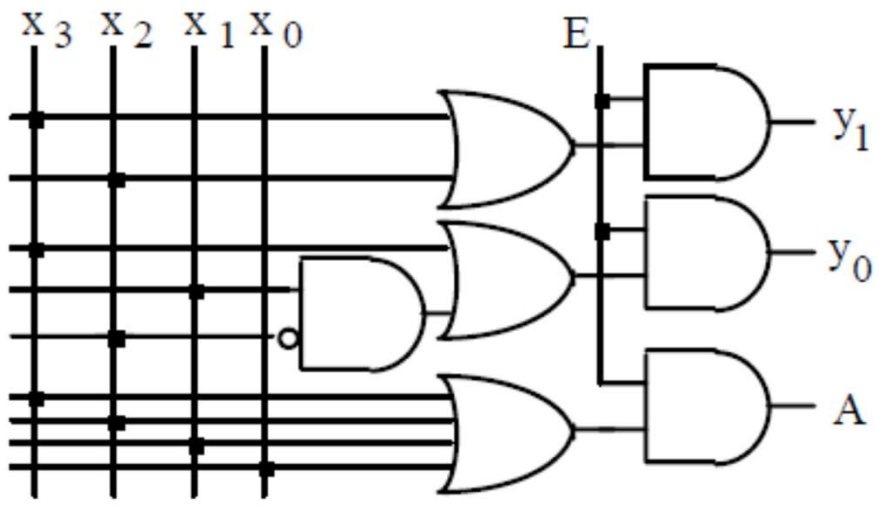
$$Z_2 = \overline{E_4} + \overline{E_5} + \overline{E_6} + \overline{E_7}$$

E0	E1	E2	E3	E4	E5	E6	E7	Z2	Z1	Z0
0	1	1	1	1	1	1	1	0	0	0
1	**0**	1	1	1	1	1	1	0	0	**1**
1	1	**0**	1	1	1	1	1	0	**1**	0
1	1	1	**0**	1	1	1	1	0	**1**	**1**
1	1	1	1	**0**	1	1	1	**1**	0	0
1	1	1	1	1	**0**	1	1	**1**	0	**1**
1	1	1	1	1	1	**0**	1	**1**	**1**	0
1	1	1	1	1	1	1	**0**	**1**	**1**	**1**

Codificador (Encoder)

En previsión de que haya **más de una entrada activa a la vez**, se puede definir una **prioridad** de entradas, actuando siempre de acuerdo con la entrada activa de prioridad más alta.
Ejemplo de codificador 4 a 2 con prioridad. La entrada E es de habilitación. La salida A en 1 indica que hay alguna entrada activa.

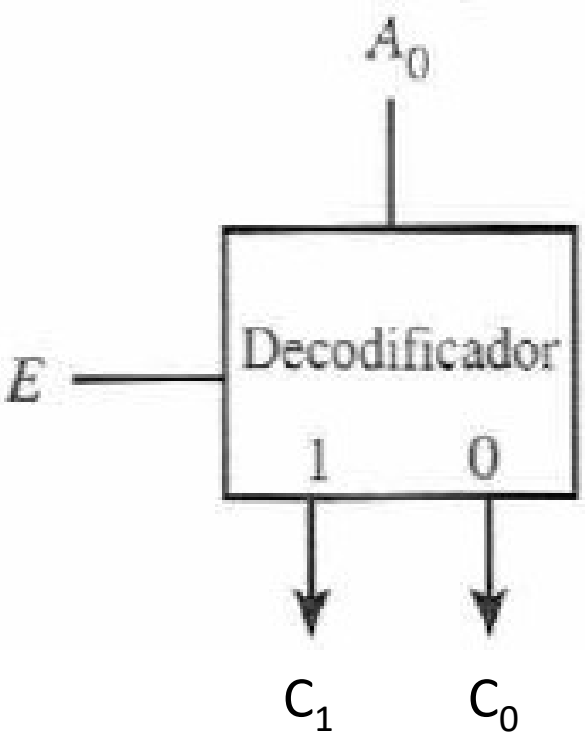
E	x_3	x_2	x_1	x_0	A	y_1	y_0
0	X	X	X	X	0	0	0
1	0	0	0	0	0	0	0
1	0	0	0	1	1	0	0
1	0	0	1	X	1	0	1
1	0	1	X	X	1	1	0
1	1	X	X	X	1	1	1



Decodificador (Decoder)

Es un circuito **combinacional** que para una combinación de valores en sus entradas activa **solo una** de sus **salidas**. Si tiene **s** entradas, el número de salidas **n** es tal que se debe cumplir la siguiente condición: $2^S \geq n$. Generalmente tienen una entrada de **habilitación E**. Para los comerciales, salvo para el BCD, se cumple $2^S = n$ (ej: 2/4, 3/8, 4/16, etc.)

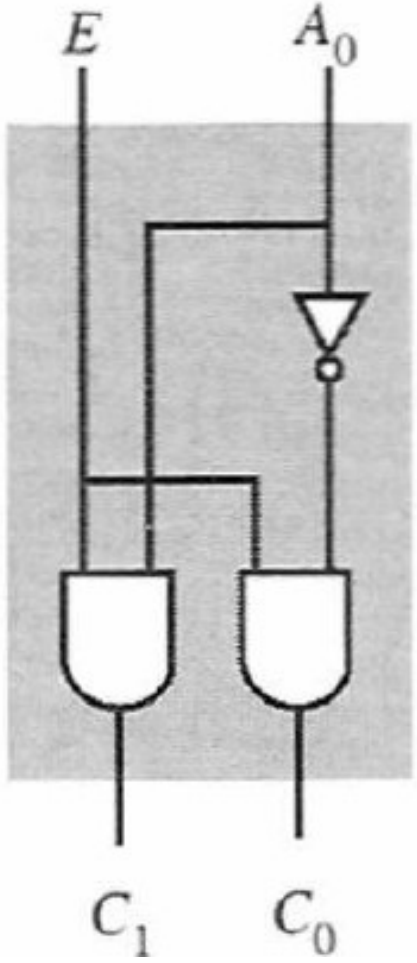
Diseño de un Decoder de **1 entrada** y **2 salidas**:



E	A_0	C_1	C_0
1	0	0	1
1	1	1	0
0	X	0	0

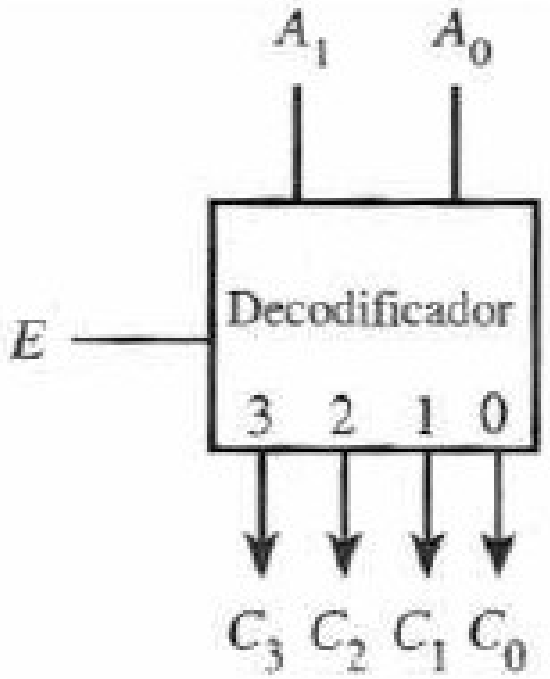
$$C_0 = E \cdot \overline{A}_0$$

$$C_1 = E \cdot A_0$$



Decodificador (Decoder)

Diseño de un Decoder de
2 entradas y **4 salidas**:



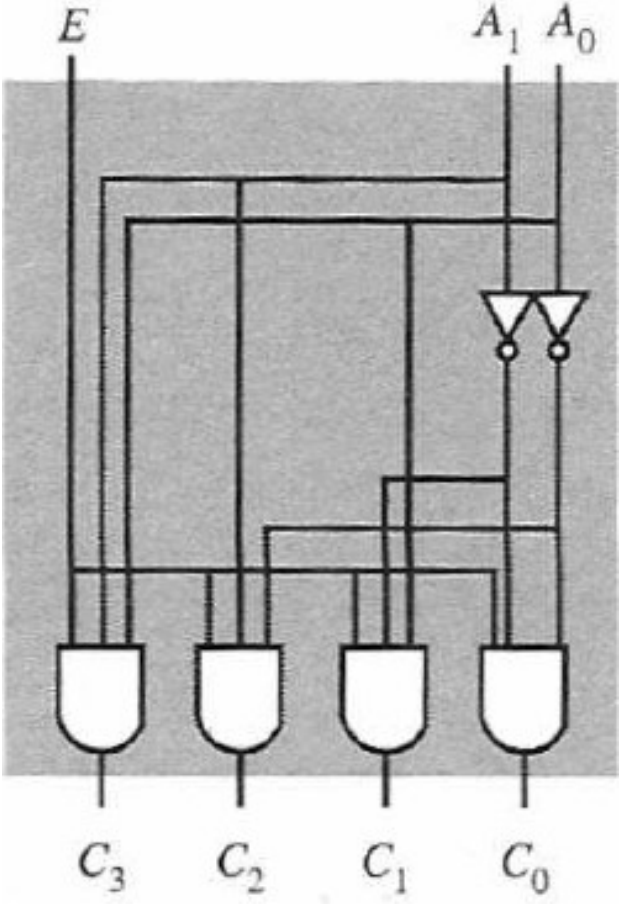
E	A_1	A_0	C_3	C_2	C_1	C_0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0
0	X	X	0	0	0	0

$$C_0 = E \cdot \overline{A}_1 \cdot \overline{A}_0$$

$$C_1 = E \cdot \overline{A}_1 \cdot A_0$$

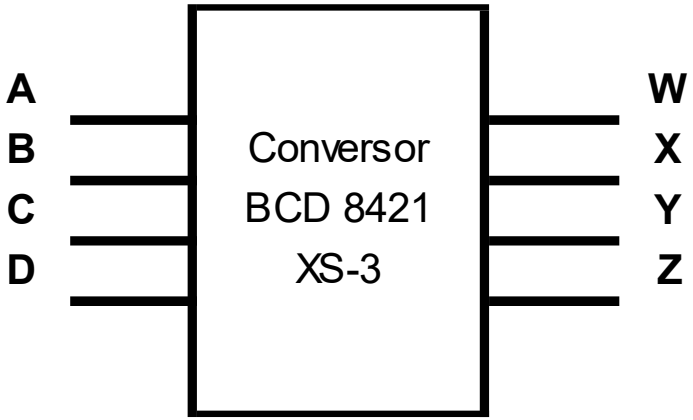
$$C_2 = E \cdot A_1 \cdot \overline{A}_0$$

$$C_3 = E \cdot A_1 \cdot A_0$$



Conversor de código

Es un circuito **combinacional** que convierte información de un código a otro.
Por ejemplo del código **BCD 8421** al código **XS-3 8421**.

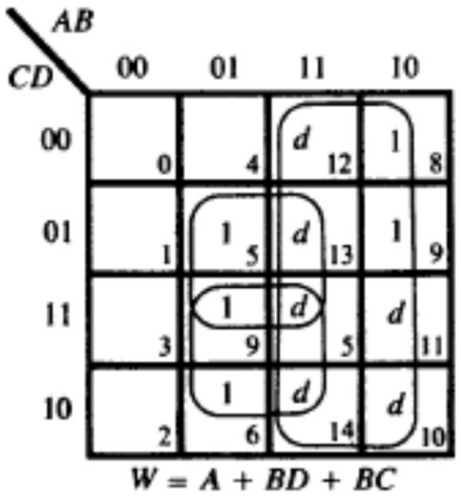


La tabla de verdad será:

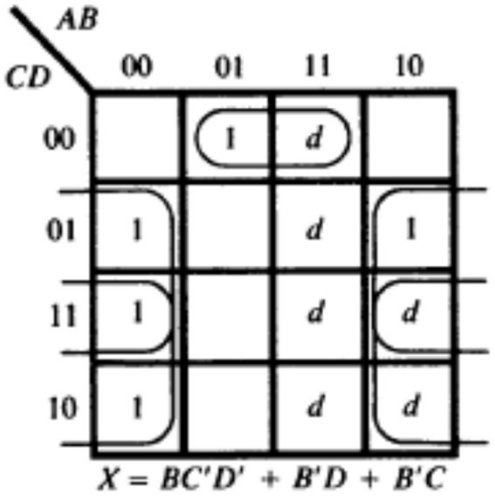
Inputs (BCD) A	B	C	D	Outputs (Excess-3) W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	*d*	*d*	*d*	*d*
1	0	1	1	*d*	*d*	*d*	*d*
1	1	*0*	*0*	*d*	*d*	*d*	*d*
1	1	0	1	*d*	*d*	*d*	*d*
1	1	1	0	*d*	*d*	*d*	*d*
1	1	1	1	*d*	*d*	*d*	*d*

Conversor de código

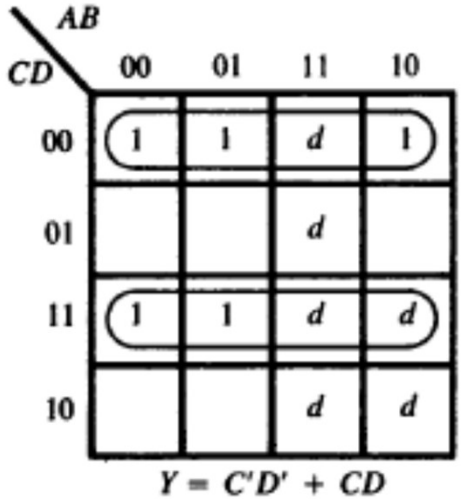
Resolviendo obtenemos el circuito interno del conversor:



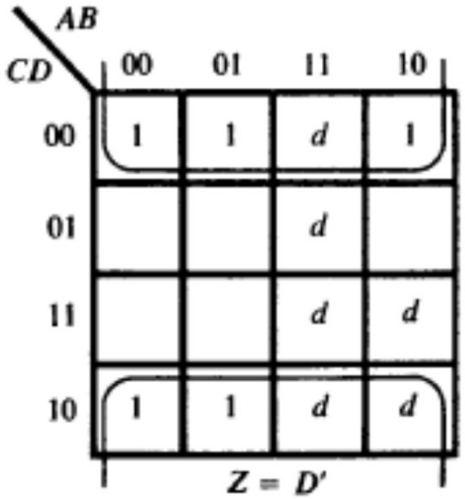
$W = A + BD + BC$



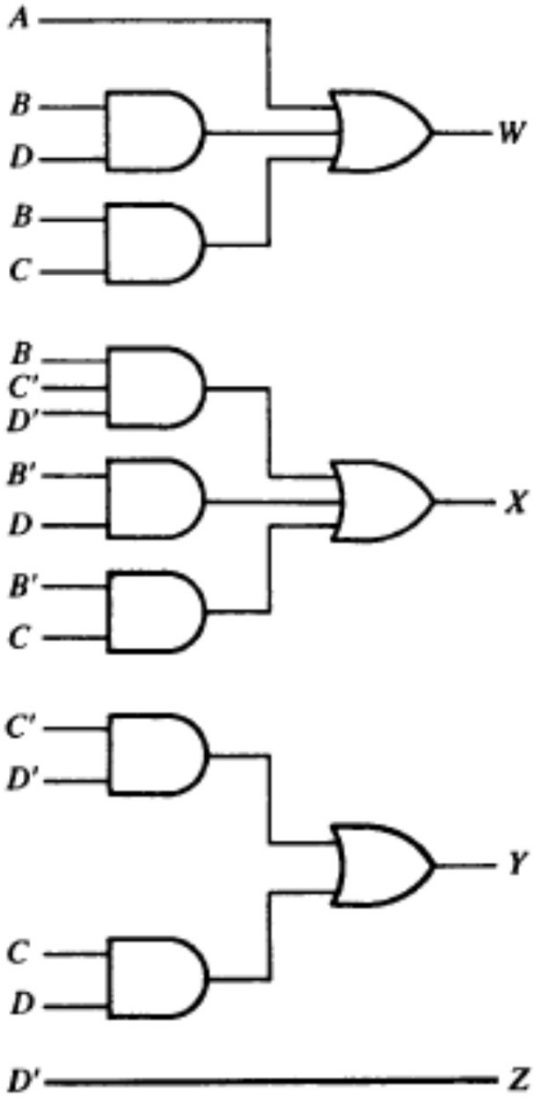
$X = BC'D' + B'D + B'C$



$Y = C'D' + CD$

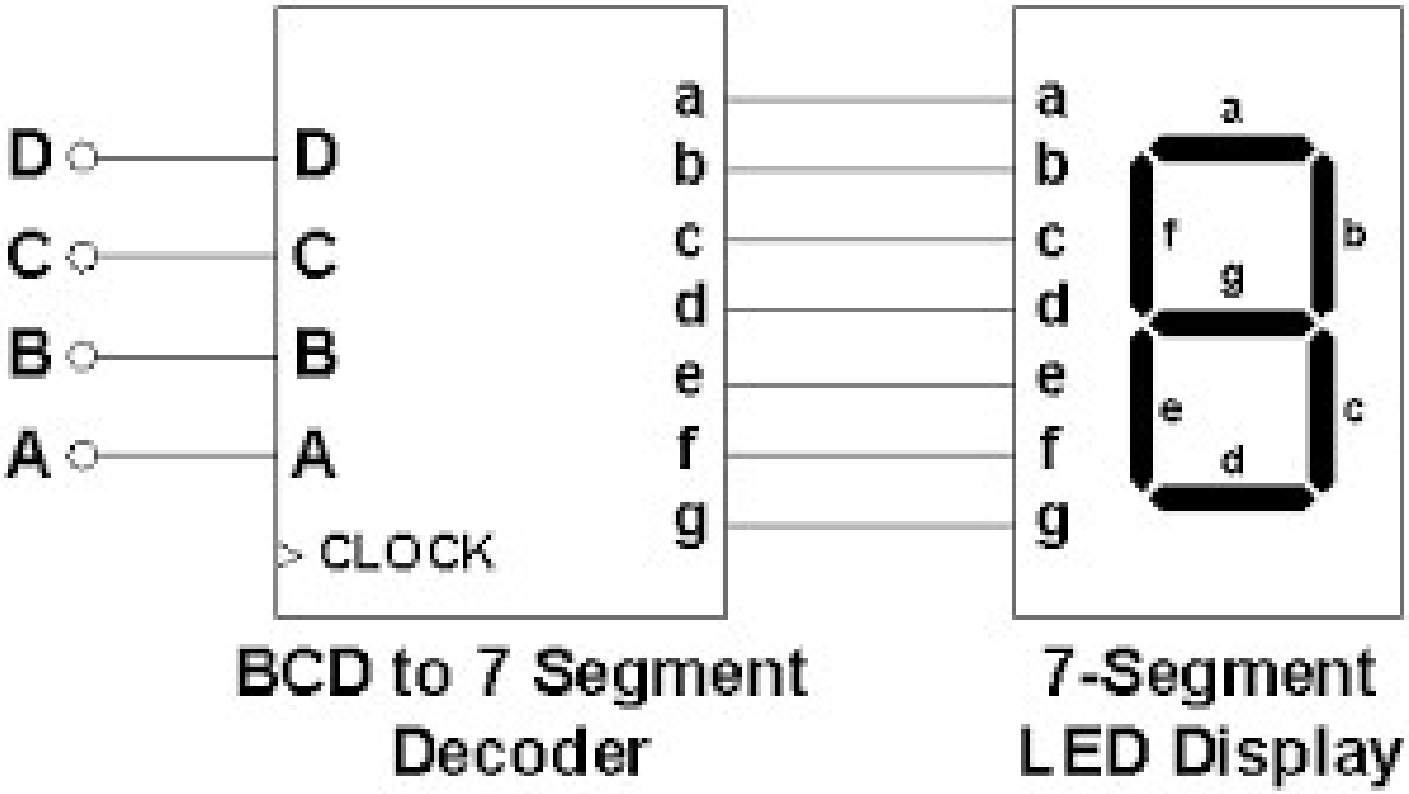


$Z = D'$



Conversor de código

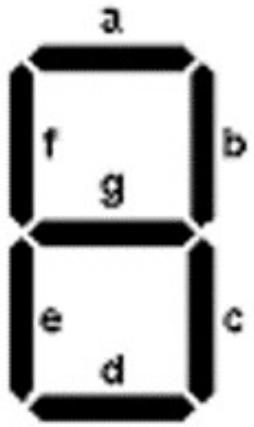
Conversor del código **BCD 8421** a **7 segmentos**:



Conversor de código

La TV quedaría:

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	x	x	x	x	x	x	x
1	0	1	1	x	x	x	x	x	x	x
1	1	0	0	x	x	x	x	x	x	x
1	1	0	1	x	x	x	x	x	x	x
1	1	1	0	x	x	x	x	x	x	x
1	1	1	1	x	x	x	x	x	x	x



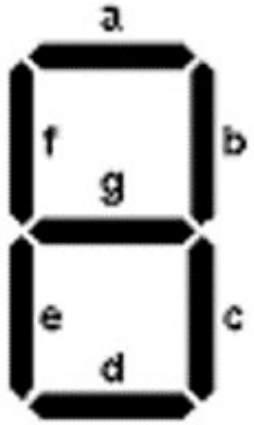
La solución la obtenemos resolviendo los 7 K de 4 variables c/u teniendo en cuenta las x según sea más conveniente para la simplificación.

Conversor de código

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Si queremos que al aparecer una combinación no esperada indique **E** (error):

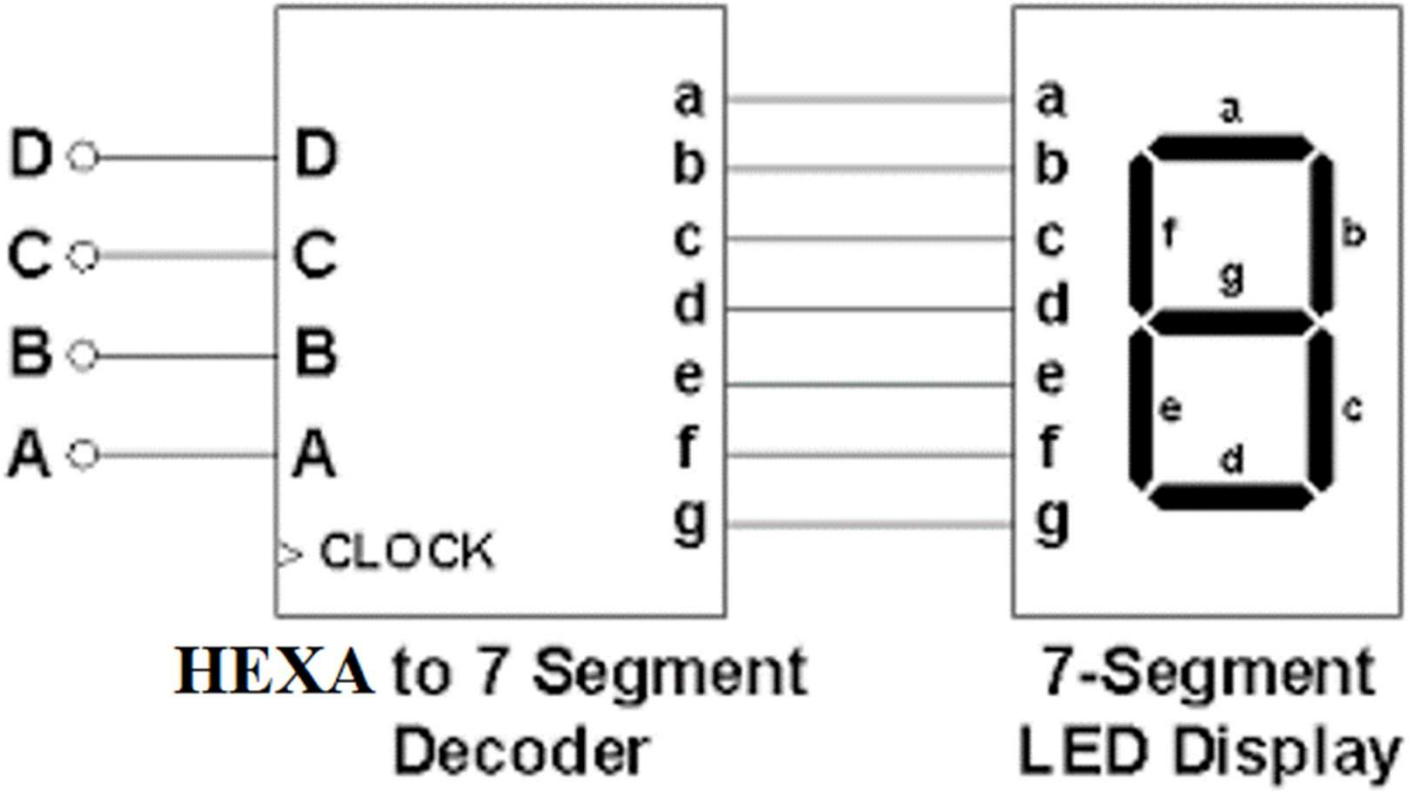
A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	0	0	1	1	1	1
1	0	1	1	1	0	0	1	1	1	1
1	1	0	0	1	0	0	1	1	1	1
1	1	0	1	1	0	0	1	1	1	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	1	1	1	1



Nuevamente la solución la obtenemos resolviendo los 7 K de 4 variables c/u.

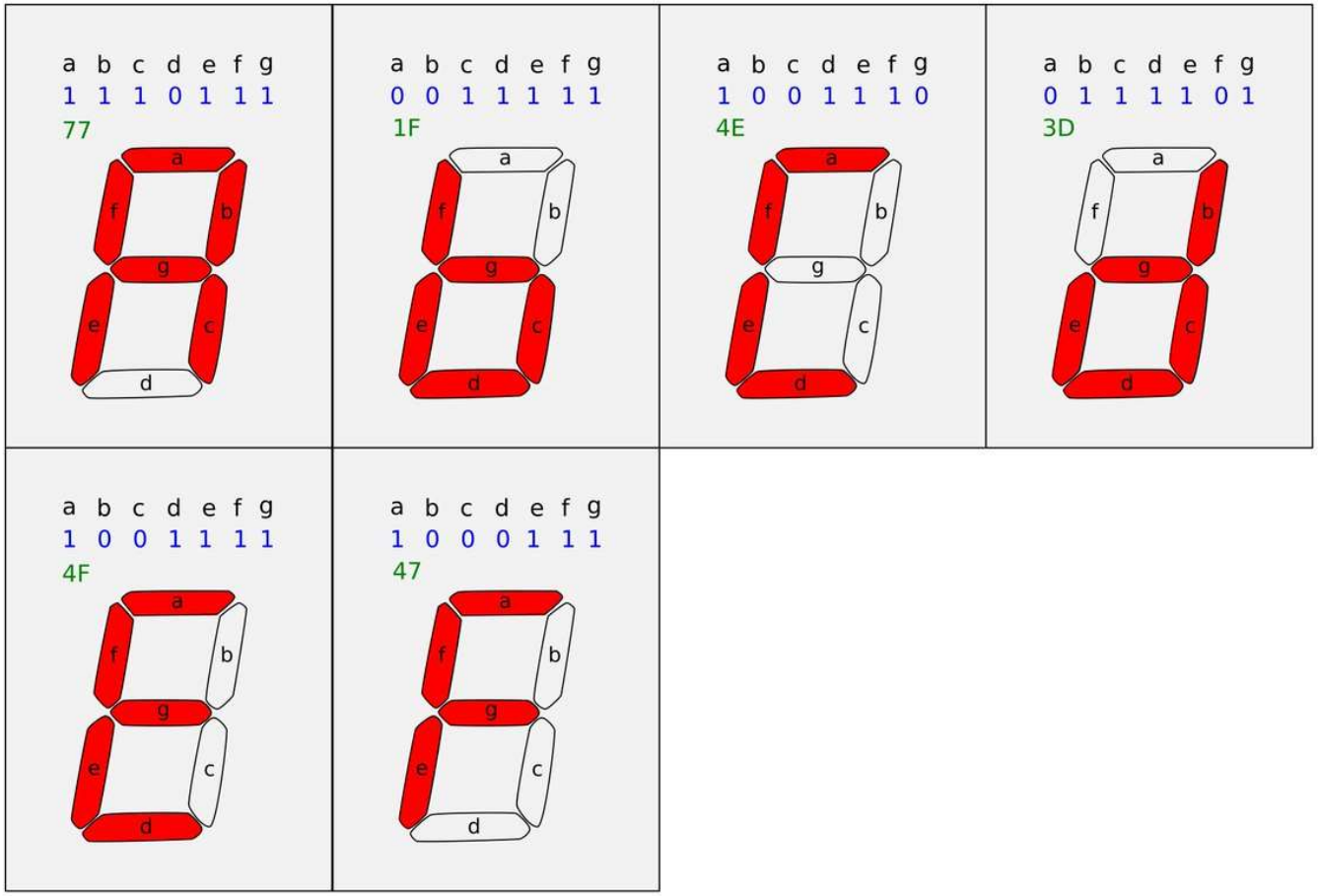
Conversor de código

Conversor del código **Hexadecimal** a **7 segmentos**:



Conversor de código

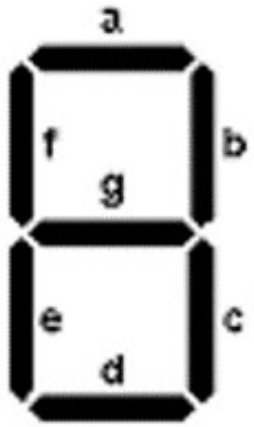
Ahora debemos también representar los 6 casos del **10** al **15** (**A** a **F**):



Conversor de código

Entonces tenemos las 16 combinaciones posibles (sin x):

A	B	C	D	a	b	c	d	e	f	g
0	0	0	0	1	1	1	1	1	1	0
0	0	0	1	0	1	1	0	0	0	0
0	0	1	0	1	1	0	1	1	0	1
0	0	1	1	1	1	1	1	0	0	1
0	1	0	0	0	1	1	0	0	1	1
0	1	0	1	1	0	1	1	0	1	1
0	1	1	0	1	0	1	1	1	1	1
0	1	1	1	1	1	1	0	0	0	0
1	0	0	0	1	1	1	1	1	1	1
1	0	0	1	1	1	1	1	0	1	1
1	0	1	0	1	1	1	0	1	1	1
1	0	1	1	0	0	1	1	1	1	1
1	1	0	0	1	0	0	1	1	1	0
1	1	0	1	0	1	1	1	1	0	1
1	1	1	0	1	0	0	1	1	1	1
1	1	1	1	1	0	0	0	1	1	1



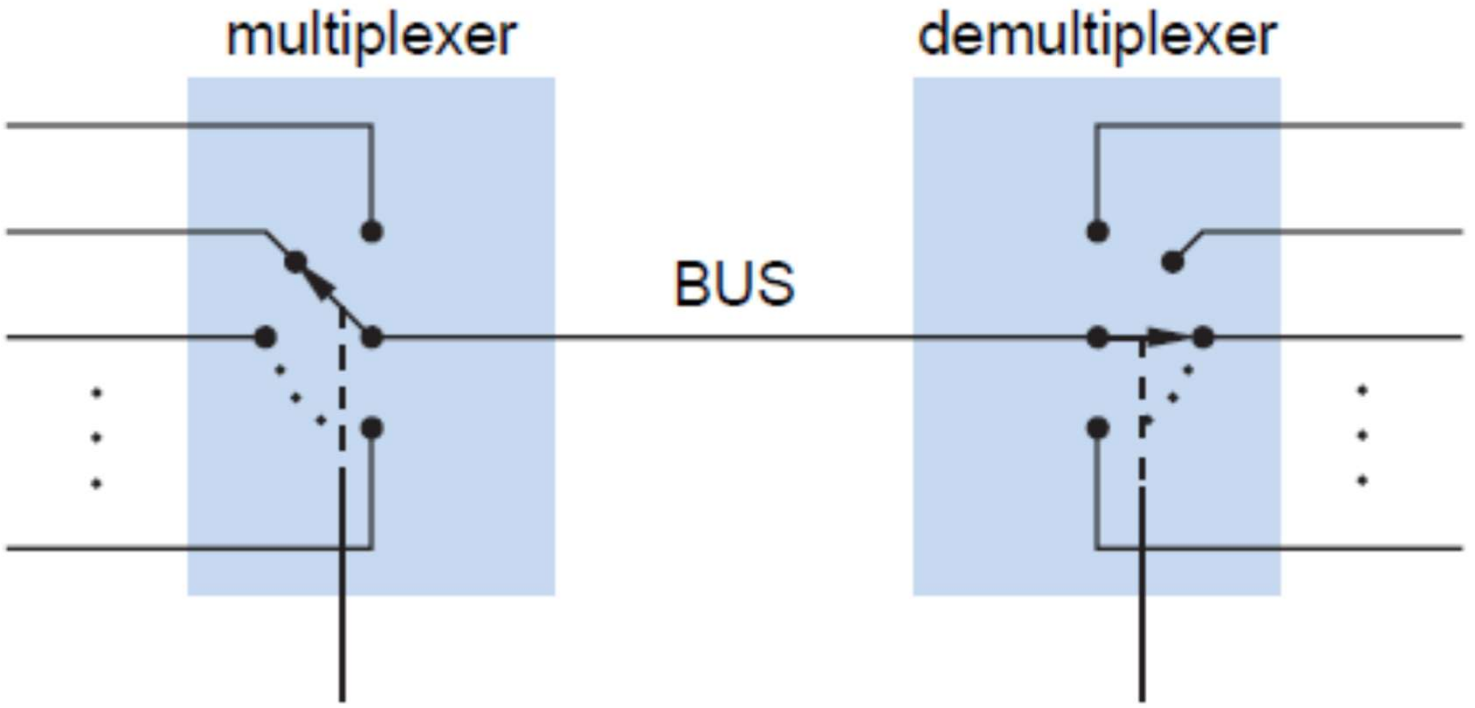
Nuevamente la solución la obtenemos resolviendo los 7 K de 4 variables c/u.

Multiplexación y Demultiplexación

Un **multiplexor (MUX) analógico** es un circuito que permite seleccionar cual de sus canales de entrada enviará la información que presenta hacia su única salida.

Un **demultiplexor (DEMUX) analógico** es un circuito que permite seleccionar cual de sus salidas recibirá la información que se presenta en su única entrada.

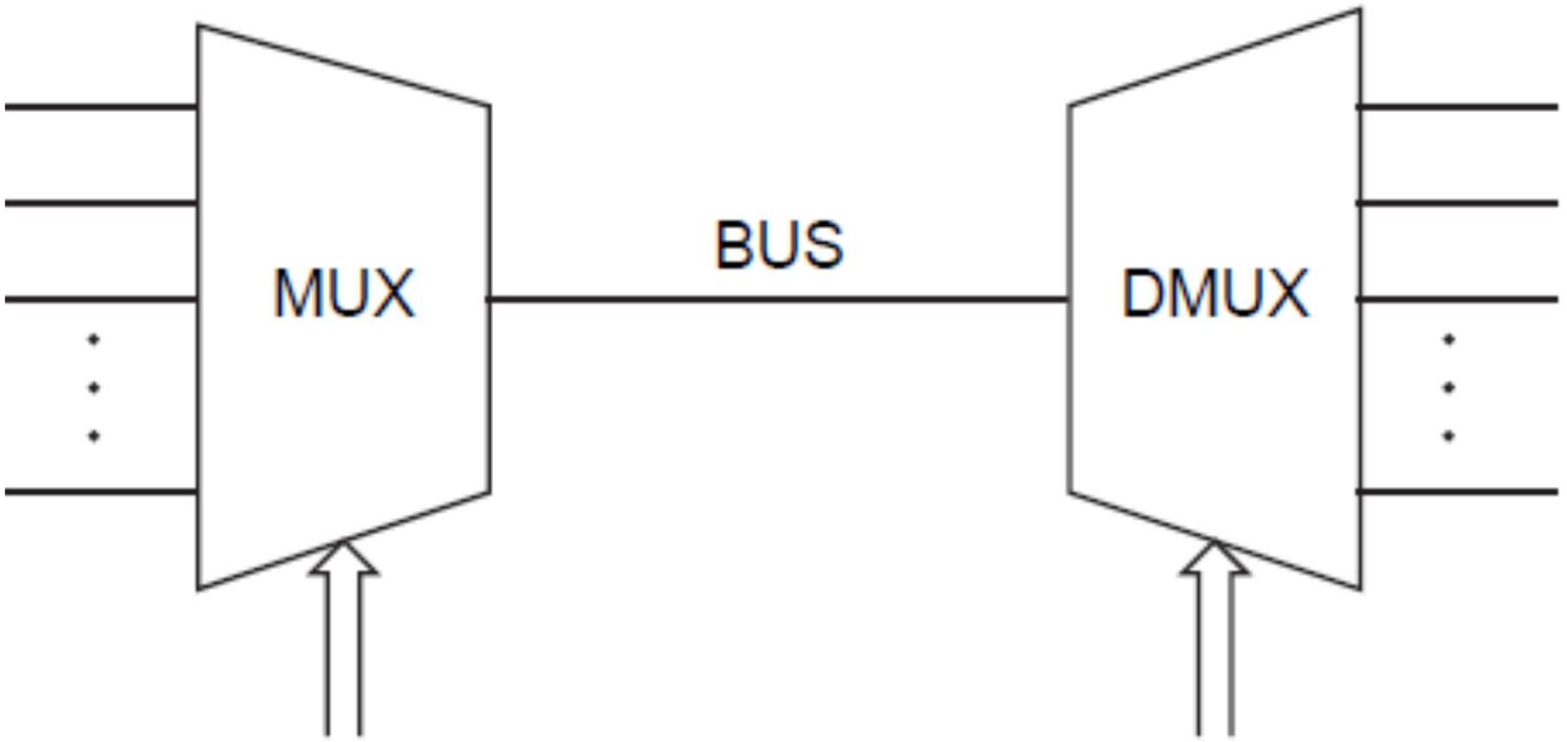
Ambos pueden ser bidireccionales.



Multiplexación y Demultiplexación

El **multiplexor** y el **demultiplexor digitales** son circuitos lógicos combinacionales que permiten realizar las mismas funciones que los analógicos pero con información binaria.

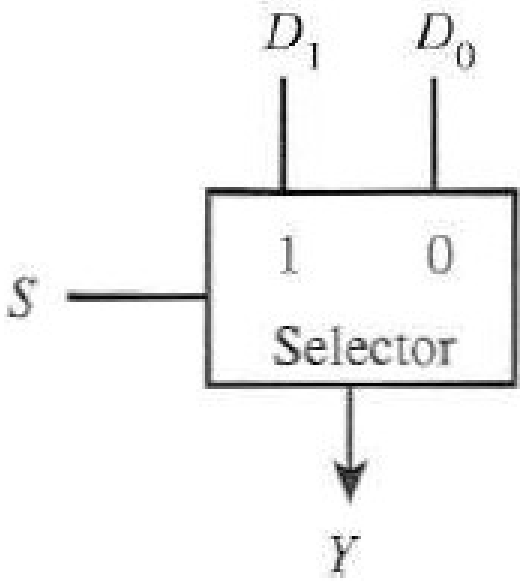
Tienen definidas sus entradas y salidas, por lo tanto son **unidireccionales**.



Multiplexor (MUX)

Un **multiplexor** es un circuito combinacional que permite transmitir la información digital presente en sus entradas (de datos) hacia una única salida.
Para poder transmitir la información es necesario entradas de selección o control, mediante las cuales se selecciona qué entrada se conecta a la salida.
Un multiplexor de **2^n** entradas de datos tendrá **n** entradas de control.

Diseño de un MUX de **2 entradas de datos** y **1 entrada de control**:



S	Y
0	D_0
1	D_1

S	D1	D0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

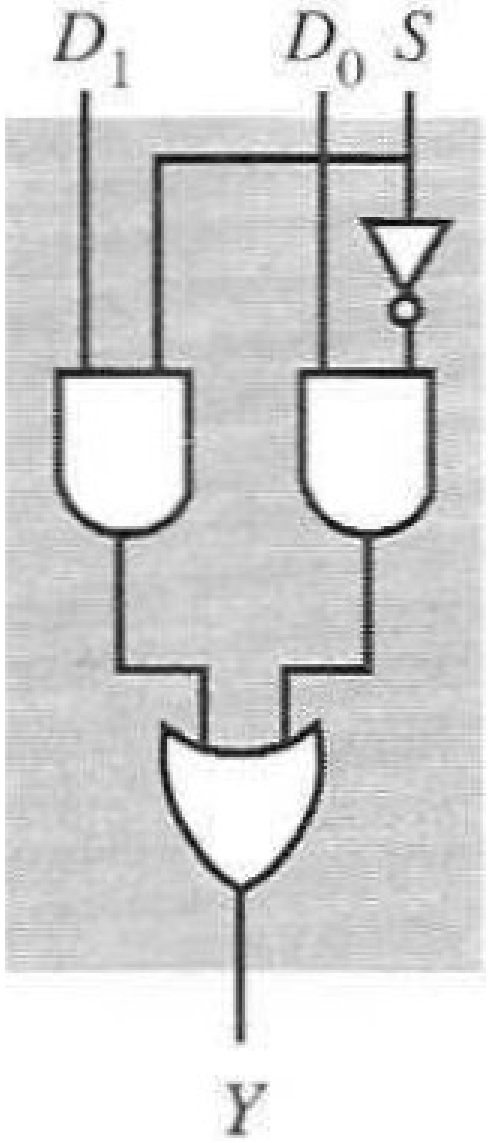
Multiplexor (MUX)

Resolviendo:

S	D1	D0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

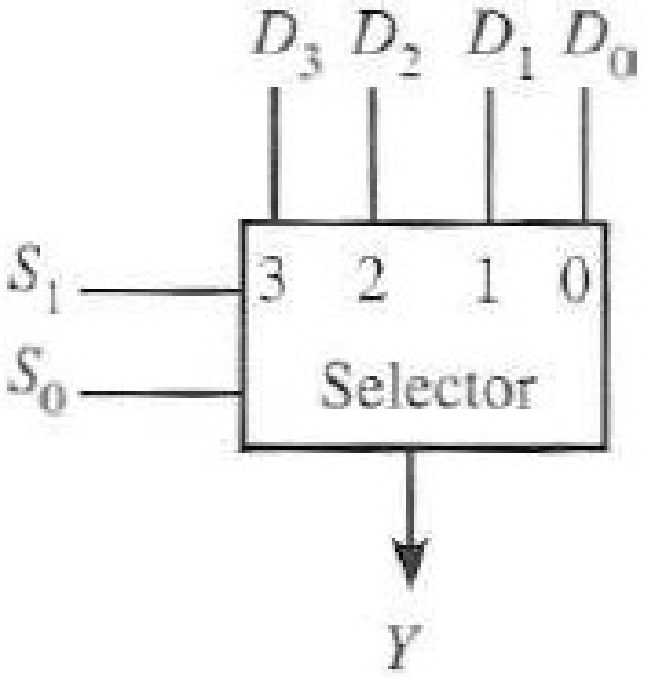
S \ D1D0	00	01	11	10
0	0	1	1	0
1	0	0	1	1

$$Y = \overline{S} \cdot D_0 + S \cdot D_1$$



Multiplexor (MUX)

Diseño de un MUX de **4 entradas de datos** y **2 entradas de control**:



S_1	S_0	Y
0	0	D_0
0	1	D_1
1	0	D_2
1	1	D_3

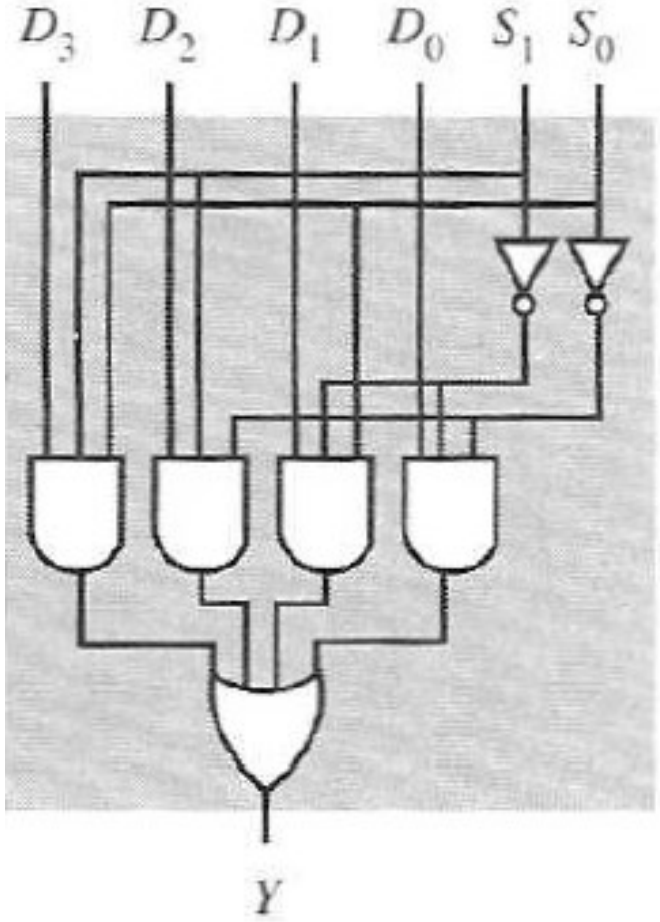
S1	S0	D3	D2	D1	D0	Y
0	0	x	x	x	0	0
0	0	x	x	x	1	1
0	1	x	x	0	x	0
0	1	x	x	1	x	1
1	0	x	0	x	x	0
1	0	x	1	x	x	1
1	1	0	x	x	x	0
1	1	1	x	x	x	1

Multiplexor (MUX)

Resolviendo:

S1	S0	D3	D2	D1	D0	Y
0	0	x	x	x	0	0
0	0	x	x	x	1	1
0	1	x	x	0	x	0
0	1	x	x	1	x	1
1	0	x	0	x	x	0
1	0	x	1	x	x	1
1	1	0	x	x	x	0
1	1	1	x	x	x	1

$$Y=\overline{S_0}\,\overline{S_1}D_0+ S_0\overline{S_1}D_1+ \overline{S_0}S_1D_2+ S_0S_1D_3$$

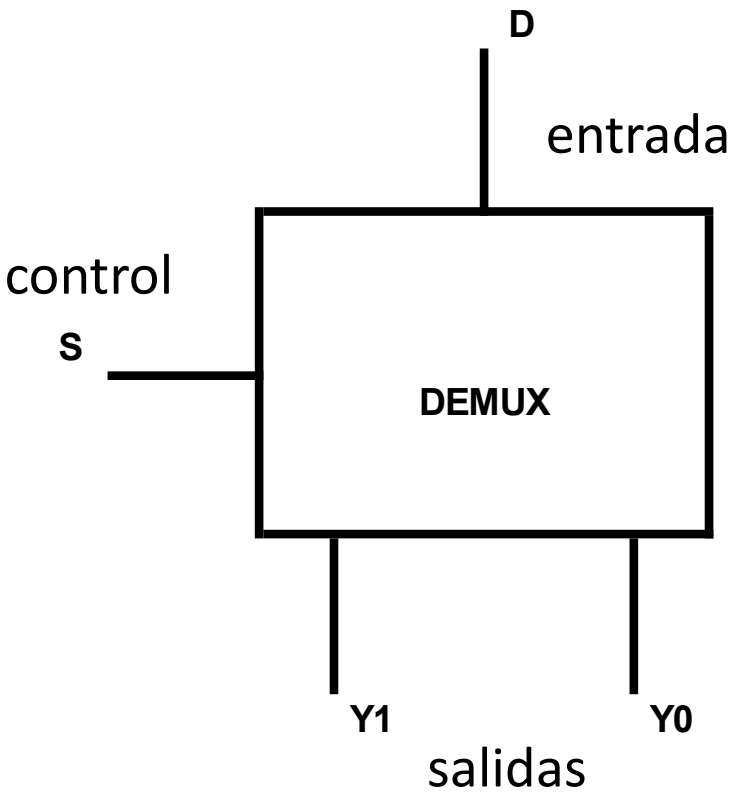


Demultiplexor (DEMUX)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un **demultiplexor** realiza la operación inversa a la que realiza un multiplexor. Distribuye los datos provenientes de una entrada entre varias salidas.
Un demultiplexor de 2^n salidas tendrá **n** entradas de control.

Diseño de un DEMUX de **2 salidas** y **1 entrada de control**:



S	Y1	Y0
0	0	D
1	D	0

S	D	Y1	Y0
0	0	0	0
0	1	0	1
1	0	0	0
1	1	1	0

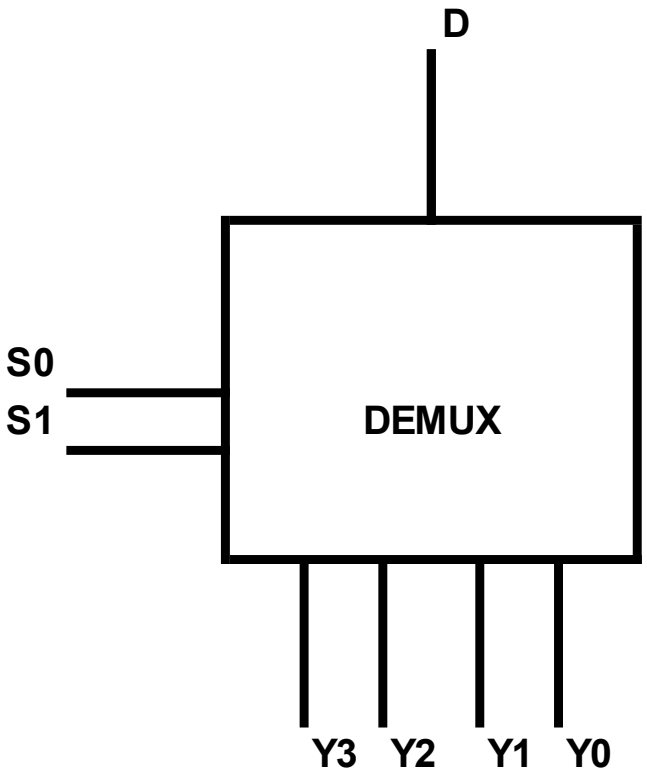
$$Y_0 = \bar{S}\, D$$

$$Y_1 = S\, D$$

Demultiplexor (DEMUX)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Diseño de un DEMUX de **4 salidas** y **2 entradas de control**:



S1	S0	Y3	Y2	Y1	Y0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

S1	S0	D	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0
0	0	1	0	0	0	1
0	1	0	0	0	0	0
0	1	1	0	0	1	0
1	0	0	0	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	0	0
1	1	1	1	0	0	0

Demultiplexor (DEMUX)

Resolviendo:

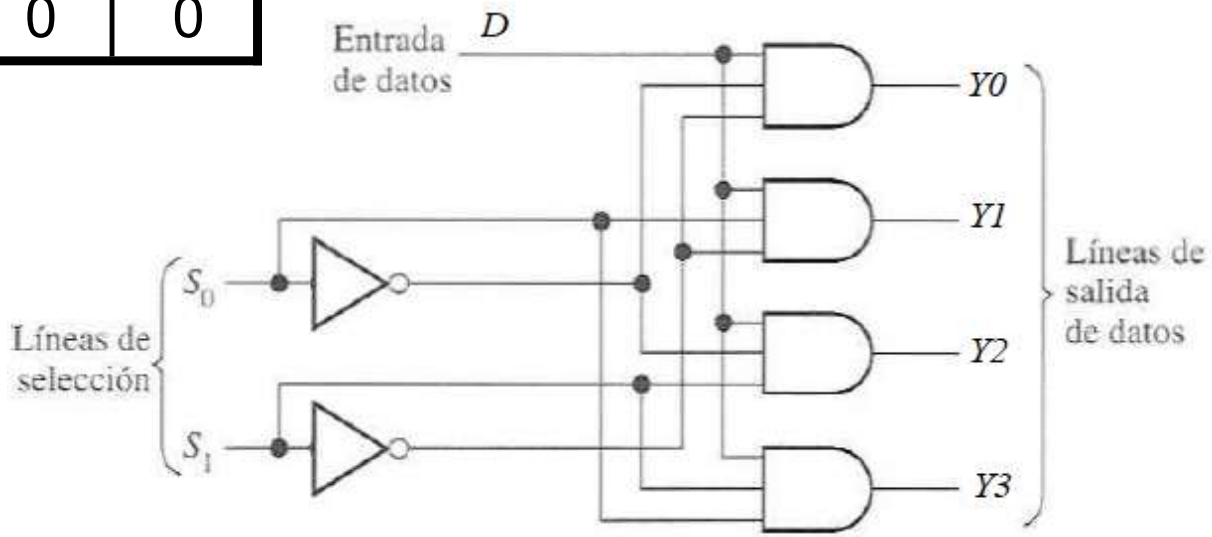
S1	S0	D	Y3	Y2	Y1	Y0
0	0	0	0	0	0	0
0	0	1	0	0	0	1
0	1	0	0	0	0	0
0	1	1	0	0	1	0
1	0	0	0	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	0	0
1	1	1	1	0	0	0

$$Y_0 = \bar{S}_1 \bar{S}_0 D$$

$$Y_1 = \bar{S}_1 S_0 D$$

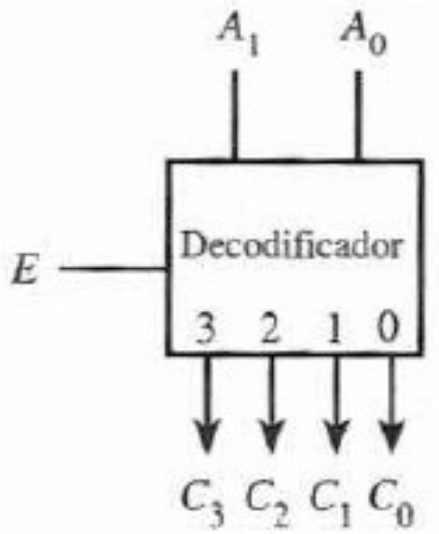
$$Y_2 = S_1 \bar{S}_0 D$$

$$Y_3 = S_1 S_0 D$$



Decoder / Demultiplexer

Analizando vemos que si bien son conceptos distintos, las TV son similares y las funciones y circuitos son iguales:

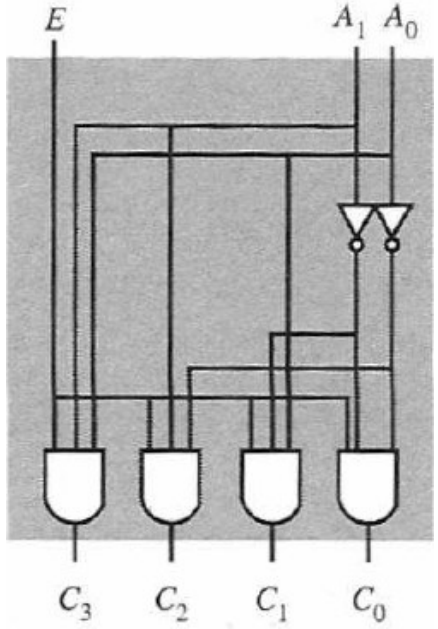


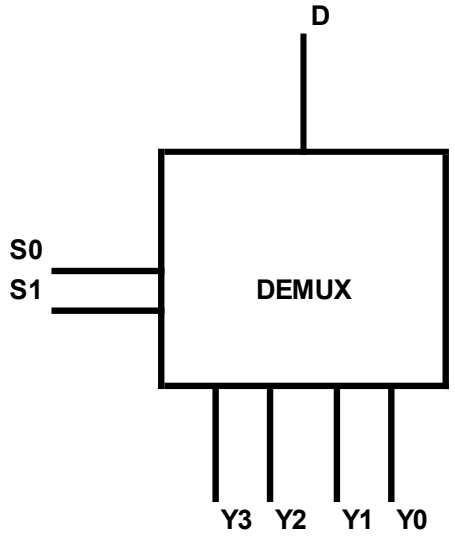
$$C_0 = E \cdot \overline{A}_1 \cdot \overline{A}_0$$

$$C_1 = E \cdot \overline{A}_1 \cdot A_0$$

$$C_2 = E \cdot A_1 \cdot \overline{A}_0$$

$$C_3 = E \cdot A_1 \cdot A_0$$



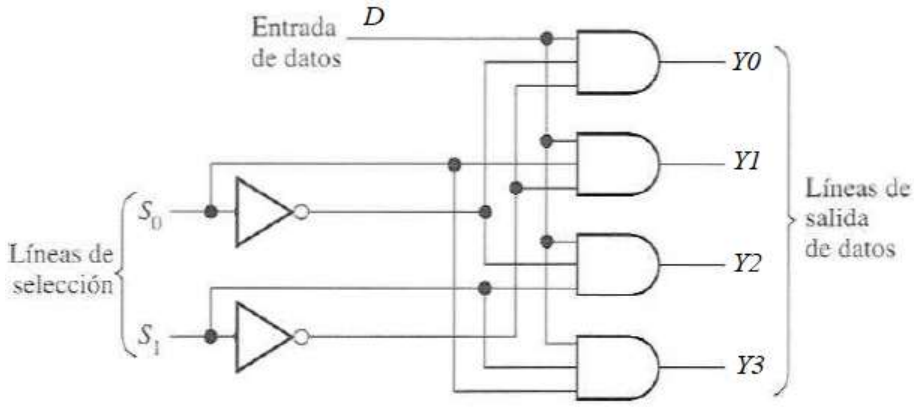


$$Y_0 = \overline{S}_1 \, \overline{S}_0 \, D$$

$$Y_1 = \overline{S}_1 \, S_0 \, D$$

$$Y_2 = S_1 \, \overline{S}_0 \, D$$

$$Y_3 = S_1 \, S_0 \, D$$



Decoder / Demultiplexer

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo comercial:

74HC/HCT138

3-to-8 line decoder/demultiplexer; inverting

INPUTS						OUTPUTS							
$\overline{E}_1$	$\overline{E}_2$	E_3	A_0	A_1	A_2	$\overline{Y}_0$	$\overline{Y}_1$	$\overline{Y}_2$	$\overline{Y}_3$	$\overline{Y}_4$	$\overline{Y}_5$	$\overline{Y}_6$	$\overline{Y}_7$
H	X	X	X	X	X	H	H	H	H	H	H	H	H
X	H	X	X	X	X	H	H	H	H	H	H	H	H
X	X	L	X	X	X	H	H	H	H	H	H	H	H
L	L	H	L	L	L	L	H	H	H	H	H	H	H
L	L	H	H	L	L	H	L	H	H	H	H	H	H
L	L	H	L	H	L	H	H	L	H	H	H	H	H
L	L	H	H	H	L	H	H	H	L	H	H	H	H
L	L	H	L	L	H	H	H	H	H	L	H	H	H
L	L	H	H	L	H	H	H	H	H	H	L	H	H
L	L	H	L	H	H	H	H	H	H	H	H	L	H
L	L	H	H	H	H	H	H	H	H	H	H	H	L

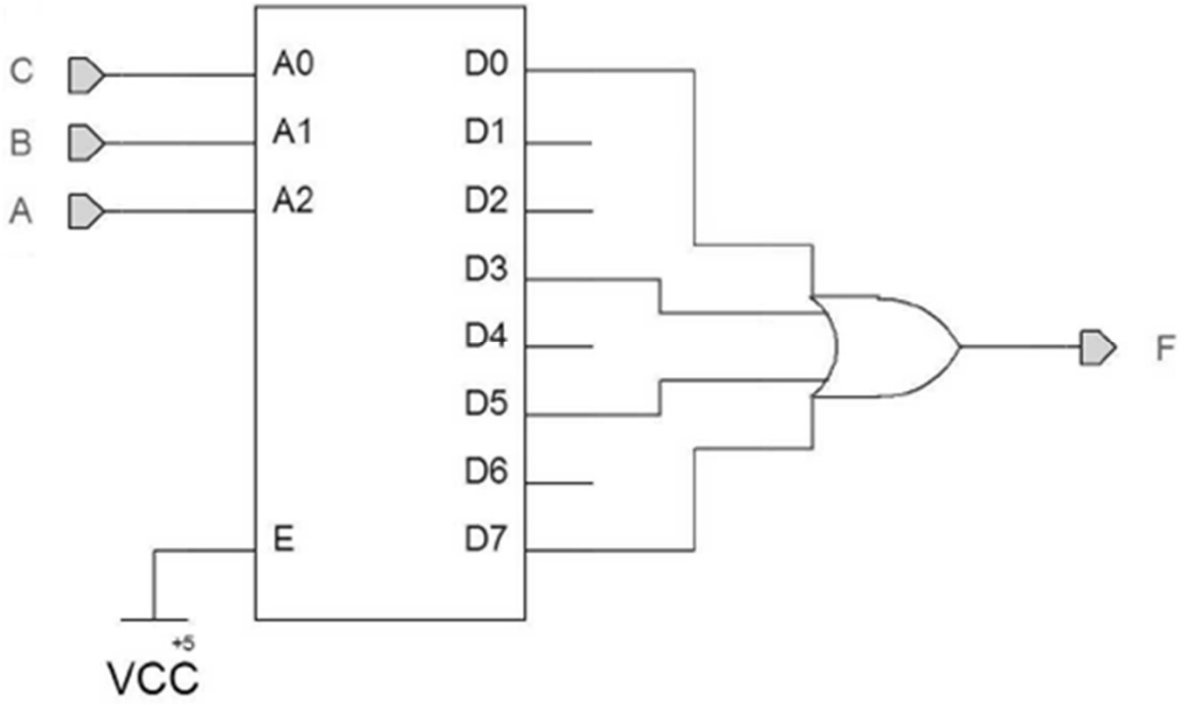
Implementación con Decoder / Demux

Podemos **implementar funciones** a partir de su TV con **Decoder / Demux** ya que estos generan los minitérminos de una función de **n** variables. Luego los sumamos mediante una compuerta **OR**.

Ejemplo: Implementar con un Decoder / Demux la función dada.

$$F(A,B,C) = \sum(0,3,5,7)$$

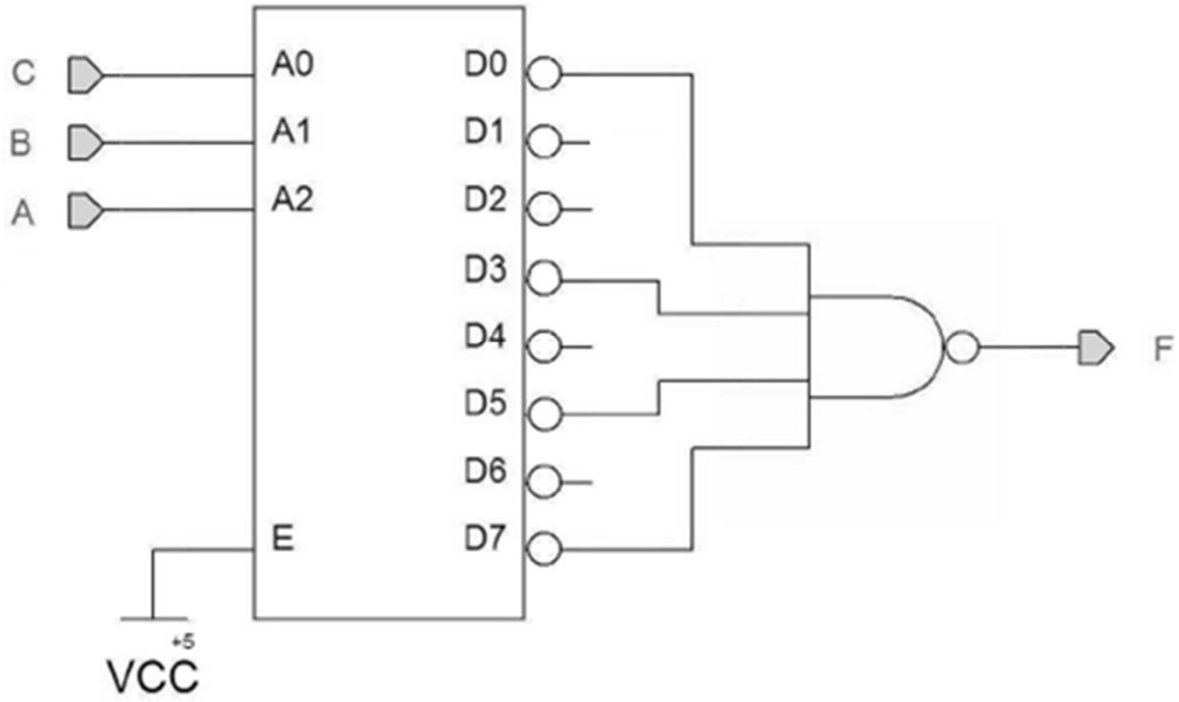
	A	B	C	F
0)	0	0	0	1
1)	0	0	1	0
2)	0	1	0	0
3)	0	1	1	1
4)	1	0	0	0
5)	1	0	1	1
6)	1	1	0	0
7)	1	1	1	1



Implementación con Decoder / Demux

Si el **Decoder / Demux** tiene salidas negadas, aplicando De Morgan tenemos:

	A	B	C	F
0)	0	0	0	1
1)	0	0	1	0
2)	0	1	0	0
3)	0	1	1	1
4)	1	0	0	0
5)	1	0	1	1
6)	1	1	0	0
7)	1	1	1	1



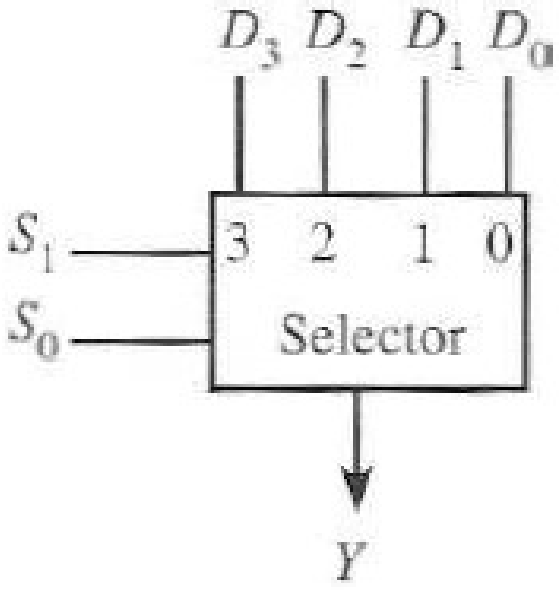
Implementación con MUX único

También podemos implementar funciones de **n** variables con **MUX** de **n-1** entradas de control, colocando **n-1** variables de la función en las entradas de control, quedando las entradas de datos como funciones de la variable no usada.
Ejemplo: Implementar la función de 3 variables con **un solo MUX**.
La función es de 3 variables, entonces el MUX debe tener 2 entradas de control.

$F(A,B,C) = \sum(1,2,4,5)$

$F(A,B,C) = \prod(0,3,6,7)$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



AB \ C	0	1	
00	0	1	D0
01	1	0	D1
11	0	0	D3
10	1	1	D2

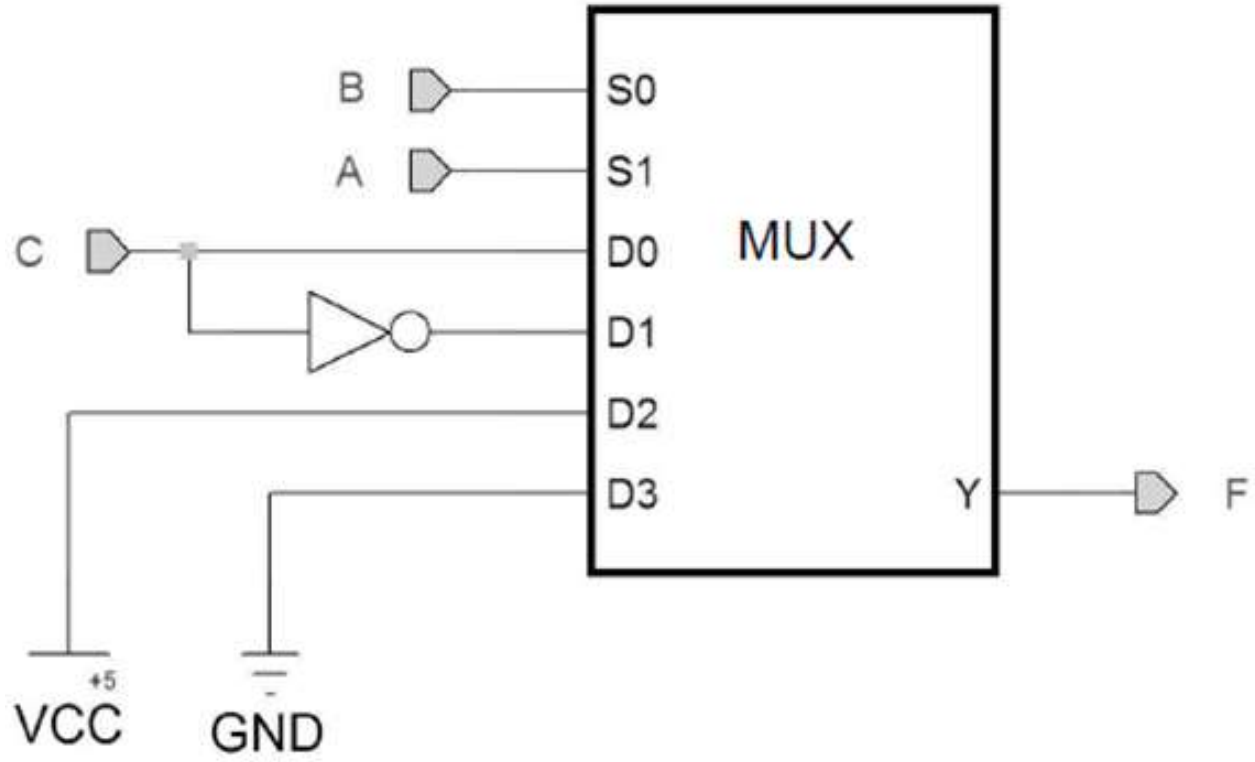
Implementación con MUX único

Analizando el mapa K nos queda:

AB \ C	0	1	
00	0	1	D0
01	1	0	D1
11	0	0	D3
10	1	1	D2

$F(A,B,C) = \sum(1,2,4,5)$

$F(A,B,C) = \prod(0,3,6,7)$



Implementación con MUX único

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: Implementar la función de 4 variables con **un solo MUX**.

$F(A,B,C,D) = \sum(1,2,4,5,9,10,11)$
$F(A,B,C,D) = \prod(0,3,6,7,8,12,13,14,15)$

A	B	C	D	F
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

AB \ CD	00	01	11	10
00	0	1	0	1
01	1	1	0	0
11	0	0	0	0
10	0	1	1	1

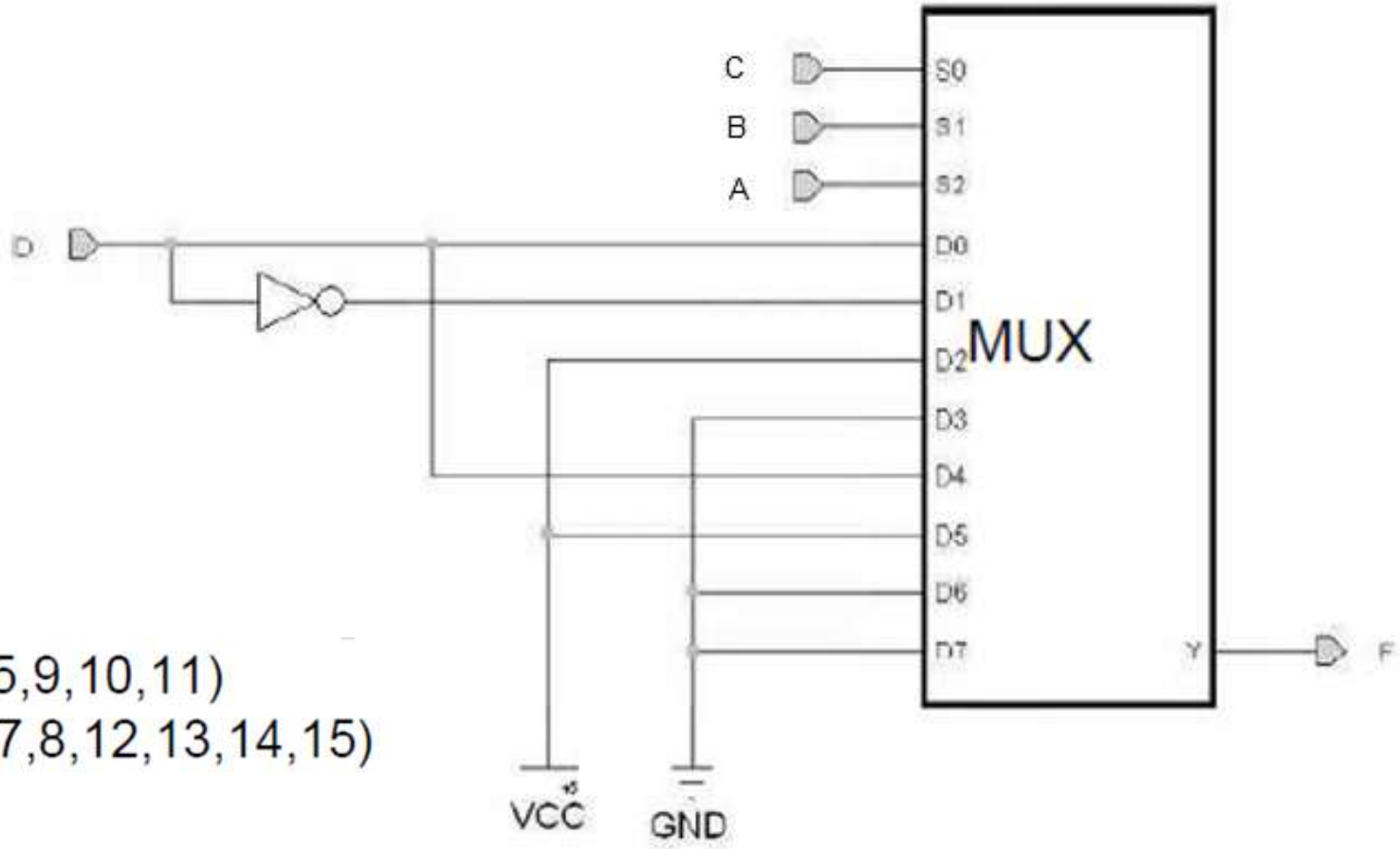
DO	D1
D2	D3
D6	D7
D4	D5

D	Not D
1	0
0	0
D	1

Implementación con MUX único

Otra forma de visualizar el mapa K y el circuito final:

ABC \ D	0	1	
000	0	1	D0
001	1	0	D1
011	0	0	D3
010	1	1	D2
110	0	0	D6
111	0	0	D7
101	1	1	D5
100	0	1	D4



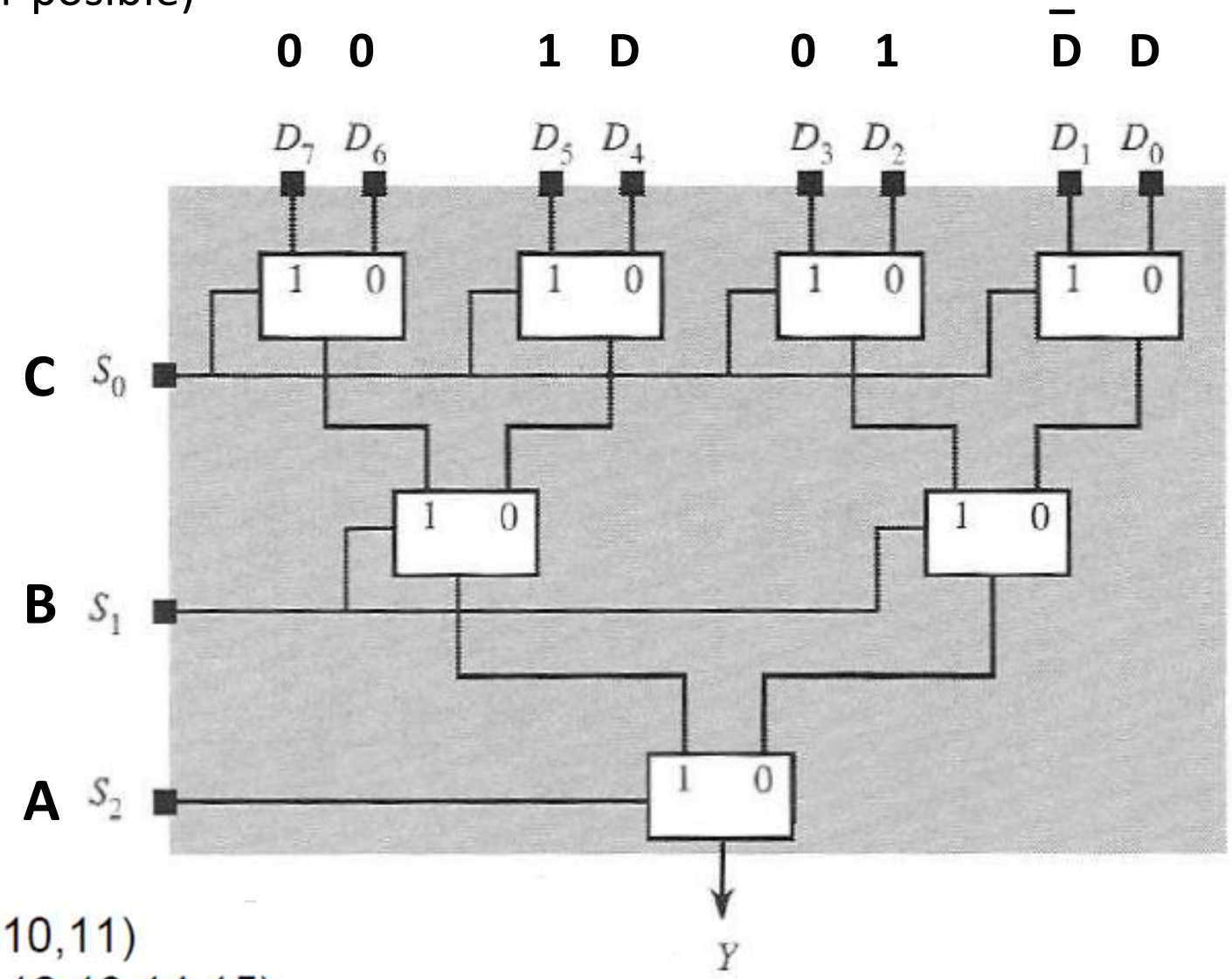
$F(A,B,C,D) = \sum(1,2,4,5,9,10,11)$

$F(A,B,C,D) = \prod(0,3,6,7,8,12,13,14,15)$

Implementación c/ MUX de 1 var. (árbol)

Ejemplo: Implementar la misma función pero solo con MUX de **1 variable de control**. (Simplificar de ser posible)

ABC \ D	0	1	
000	0	1	D0
001	1	0	D1
011	0	0	D3
010	1	1	D2
110	0	0	D6
111	0	0	D7
101	1	1	D5
100	0	1	D4

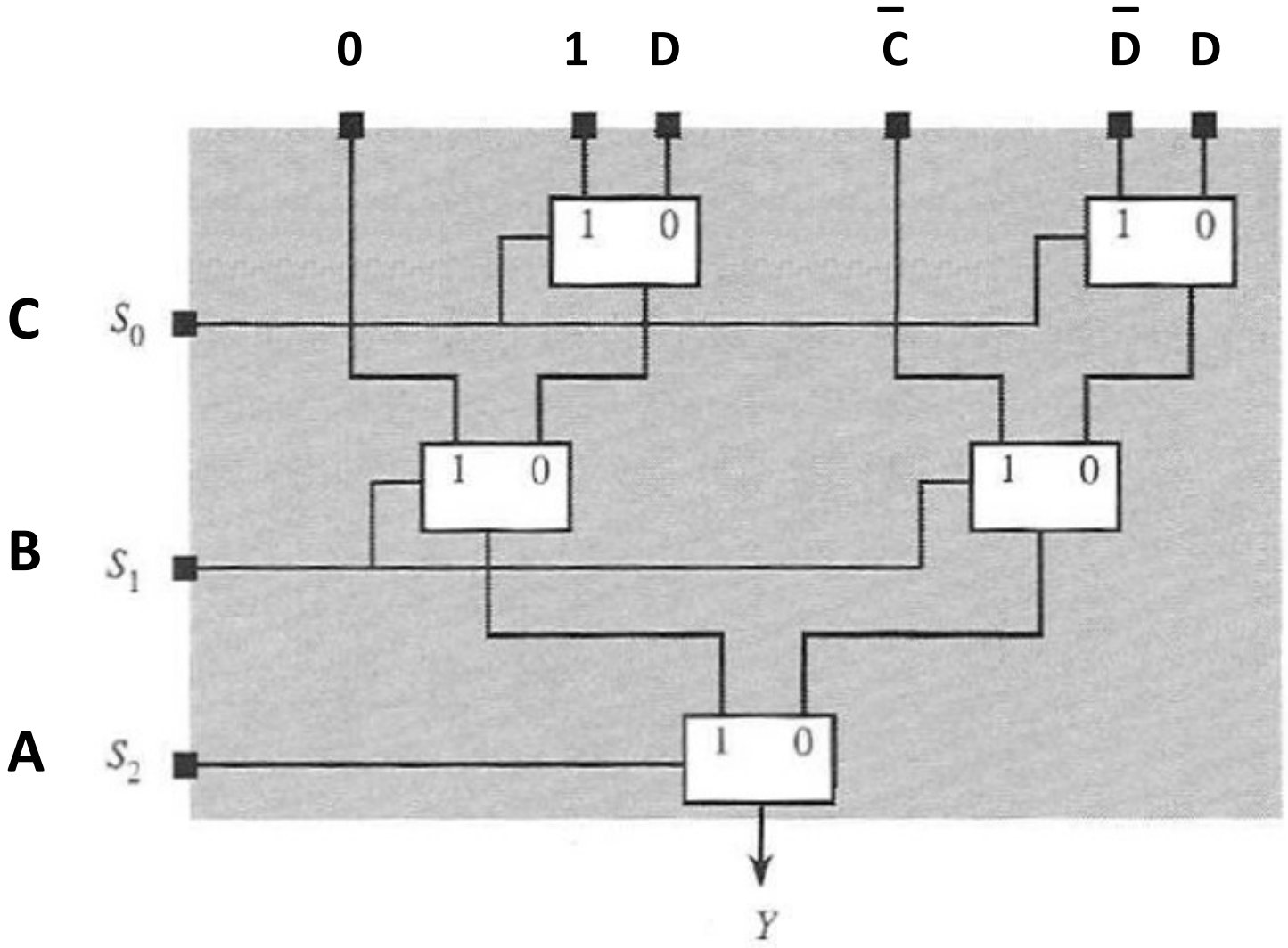


$F(A,B,C,D) = \sum(1,2,4,5,9,10,11)$

$F(A,B,C,D) = \prod(0,3,6,7,8,12,13,14,15)$

Implementación c/ MUX de 1 var. (árbol)

El circuito simplificado quedará:



Conclusiones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Análisis y síntesis de:

- Encoder y Decoder
- Conversor de código
- Mux y Demux

Síntesis de circuitos combinacionales utilizando:

- Decoder / Demux (+ compuerta)
- Mux
  - Único Mux
  - En árbol

Conclusiones

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Estamos en condiciones de sintetizar funciones:

- Como SP (Mapa K por 1s)
- Como PS (Mapa K por 0s)
- Solamente con NAND
- Solamente con NOR
- Con Decoder / Demux (+ compuerta)
- Con Mux (único / en árbol)

Más adelante:

- Con PLDs (Dispositivos Lógicos Programables)



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Unidades Aritméticas

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Analizar las celdas aritméticas básicas, sumadores, restadores, comparadores de magnitud.

Utilizar los bloques universales anteriores para la implementación de unidades aritméticas de más de un bit.

Estudiar los conceptos de sumador serie y sumador paralelo con acarreo serie y sus tiempos de respuesta.

Conocer las ventajas del sumador con acarreo anticipado (CLA).

Analizar el concepto de unidad aritmético lógica (ALU).

Analizar los circuitos comparadores de magnitud y su implementación.

Sumador de 2 Nros de 1 bit (Half Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Una de las operaciones más frecuentes en una computadora es **sumar** dos números.

Esta operación la realiza una unidad aritmético lógica (ALU), mediante un circuito **sumador.**

1 0 1 1

+ 1 0 0 1

1 0 1 0 0

Asignaremos por convención en estos circuitos al 1 lógico el 1 aritmético binario, y al 0 lógico el 0 aritmético binario.

Sumador de 2 Nros de 1 bit (Half Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Para desarrollar un circuito binario que realice la suma aritmética de 2 bits (**Half Adder**), debemos basarnos en la tabla de la **suma binaria**:

1

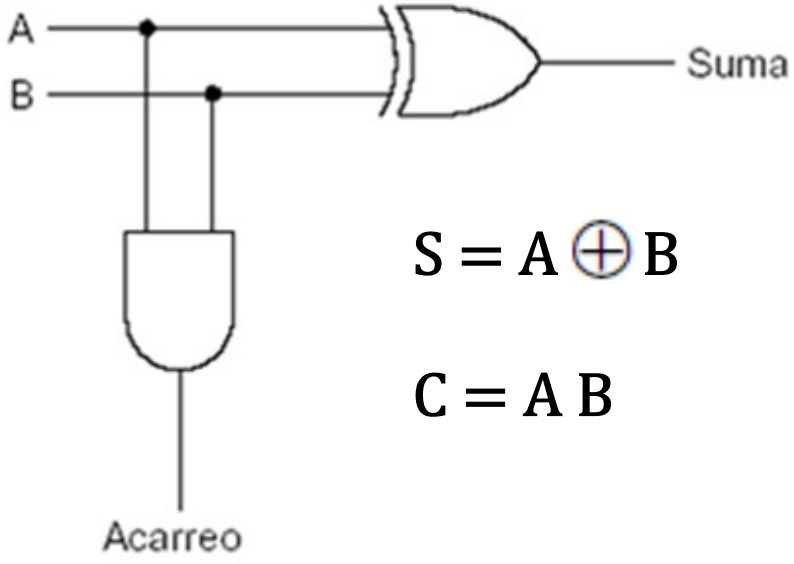
+ 1

1 0

+	0	1
0	0	1
1	1	10

Vemos que el circuito tendrá **2 entradas** (A y B) y **2 salidas** (S y C).

A	B	Suma	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



$$S = A \oplus B$$

$$C = A\,B$$

Sumador de 3 Nros de 1 bit (Full Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En una suma de **2 números de n bits**, el circuito anterior queda restringido solo a la primer columna.

Para sumar la segunda columna y subsiguientes se deberá tener en cuenta el **acarreo** (me llevo) producido por la columna **anterior**.

0 1 1

1 0 1 1

+ 1 0 0 1

1 0 1 0 0

Por lo tanto necesitaré un sumador de **3 números de 1 bit** cada uno (**Full Adder**).

Sumador de 3 Nros de 1 bit (Full Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En este caso tendremos un circuito de **3 entradas** (A_i, B_i y C_{i-1}) y **2 salidas** (S_i y C_i).

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S_i = A_i \oplus B_i \oplus C_{i-1}$$

Ai \ BiCi-1	00	01	11	10
0	0	0	1	0
1	0	1	1	1

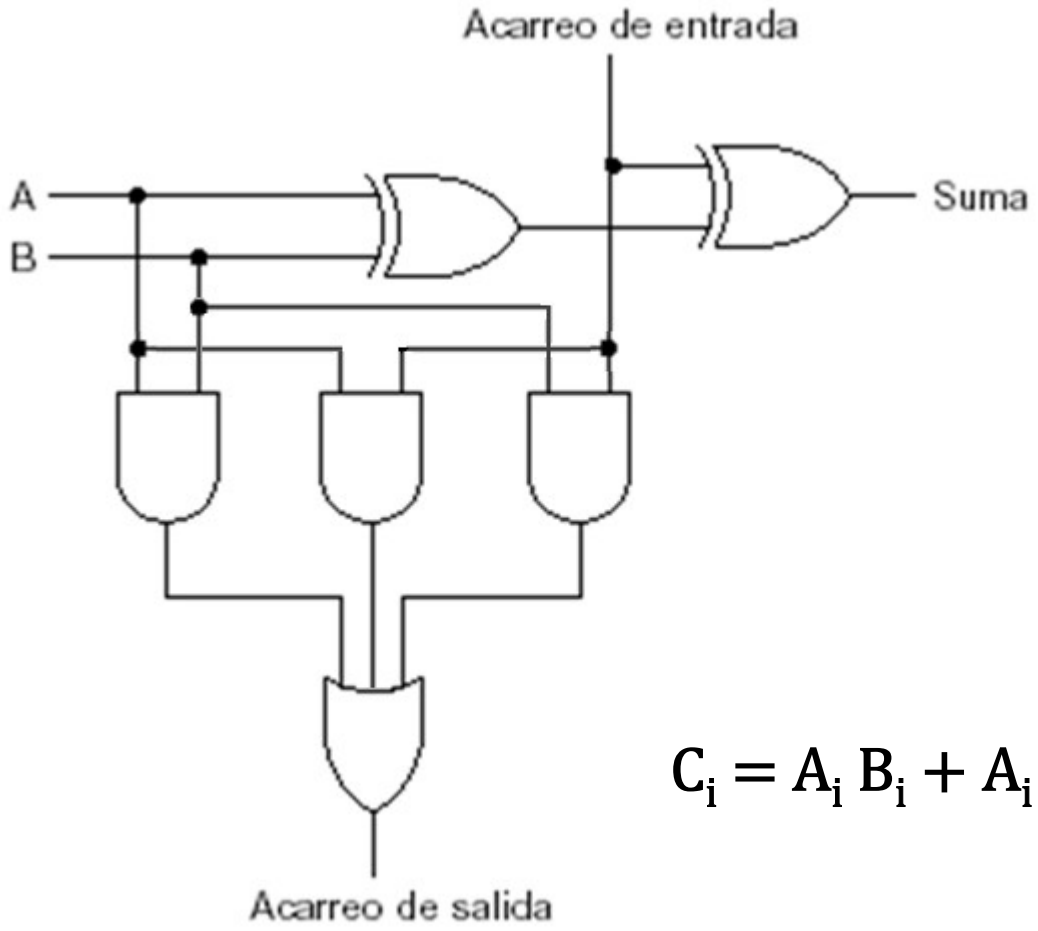
$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

Suponiendo implementaciones en 2 niveles, tanto S_i como C_i presentarán un retardo de 2 compuertas (**2** δ, donde δ es el retardo típico de una compuerta).

Sumador de 3 Nros de 1 bit (Full Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

El circuito del **FA** quedará:



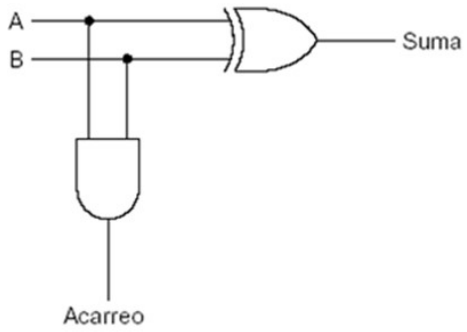
$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

Sumador de 3 Nros de 1 bit (Full Adder)

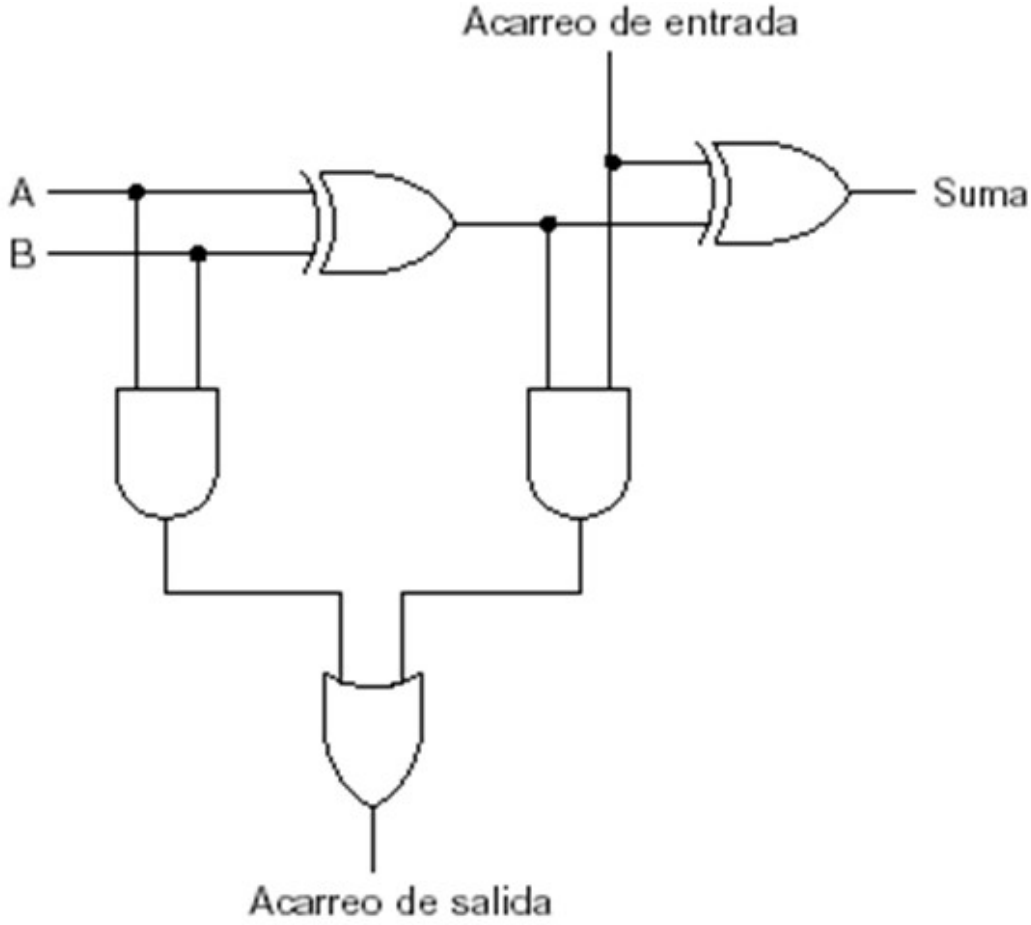
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

También podremos implementar un **FA** como la combinación de 2 **HA** y una compuerta **OR**. Atención con los retardos, 3 niveles.

HA



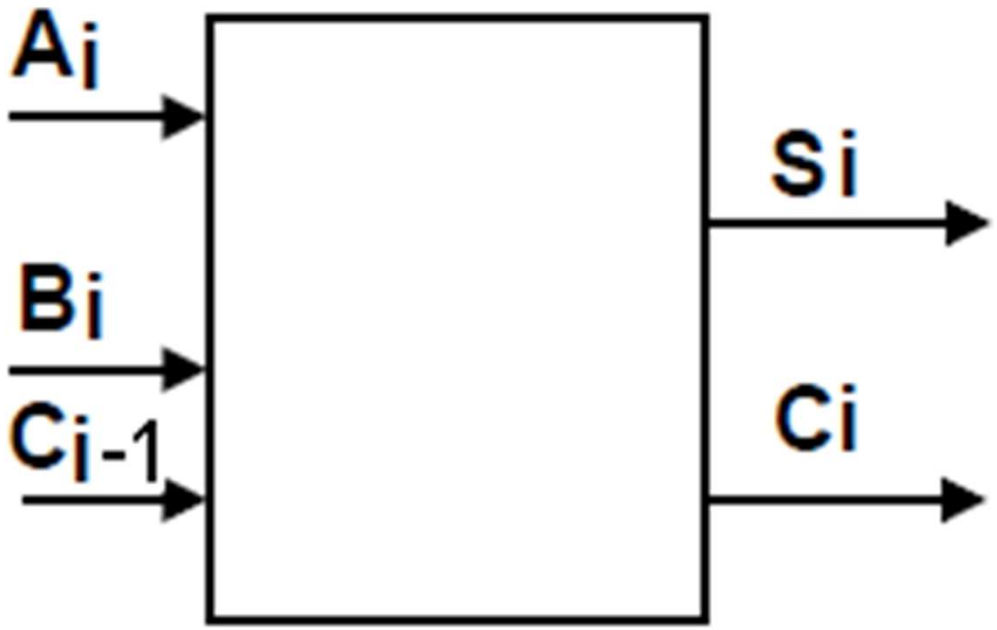
FA



Sumador de 3 Nros de 1 bit (Full Adder)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Tenemos entonces un **bloque funcional FA**.



Asumimos que tanto $\mathbf{S_i}$ como $\mathbf{C_i}$ presentarán un retardo de 2 compuertas (**2** δ).

Restador de 2 números de 1 bit

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ahora desarrollaremos un circuito binario que realice la **resta** de **2 bits**, debemos basarnos en la siguiente TV:

A	B	Resta	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$R = A \oplus B$$

$$Bw = \overline{A}\ B$$

Al igual que con los sumadores, este restador solo sirve para la primer columna.

Para restar la segunda columna y subsiguientes se deberá tener en cuenta el **borrow** (pedir prestado) producido por la columna **anterior**.

Restador de 3 números de 1 bit

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

En este caso tendremos un circuito de **3 entradas** (A_i, B_i y Bw_{i-1}) y **2 salidas** (R_i y Bw_i).

A_i	B_i	Bw_{i-1}	R_i	Bw_i
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$R_i = A_i \oplus B_i \oplus Bw_{i-1}$$

Ai \ BiBwi-1	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$Bw_i = \overline{A}_i B_i + \overline{A}_i Bw_{i-1} + B_i Bw_{i-1}$$

Equivalencias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Vemos que los sumadores y restadores son **circuitalmente similares**, por lo que podemos implementar unos con otros y viceversa agregando alguna compuerta o inversor:

Sumador de 3 bits a partir de 2 sumadores de 2 bits

Sumador de 3 bits a partir de 1 sumador y 1 restador de 2 bits

Sumador de 3 bits a partir de 1 restador de 3 bits

Sumador de 2 bits a partir de 1 restador de 2 bits

Restador de 2 bits a partir de 1 sumador de 2 bits

Restador de 3 bits a partir de 1 sumador de 3 bits

Restador de 3 bits a partir de 2 sumadores de 2 bits

Tipos de sumadores

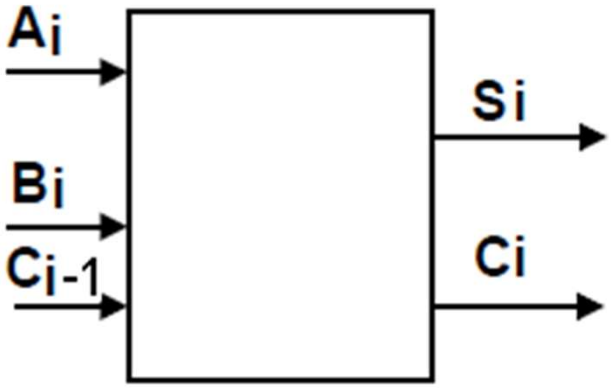
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Basándonos en el **bloque funcional FA** ya visto, ahora analizaremos varios conceptos distintos de **sumadores de más de 1 bit** y sus tiempos de cálculo:

Sumador Serie

Sumador Paralelo con Acarreo Serie (Ripple Adder)

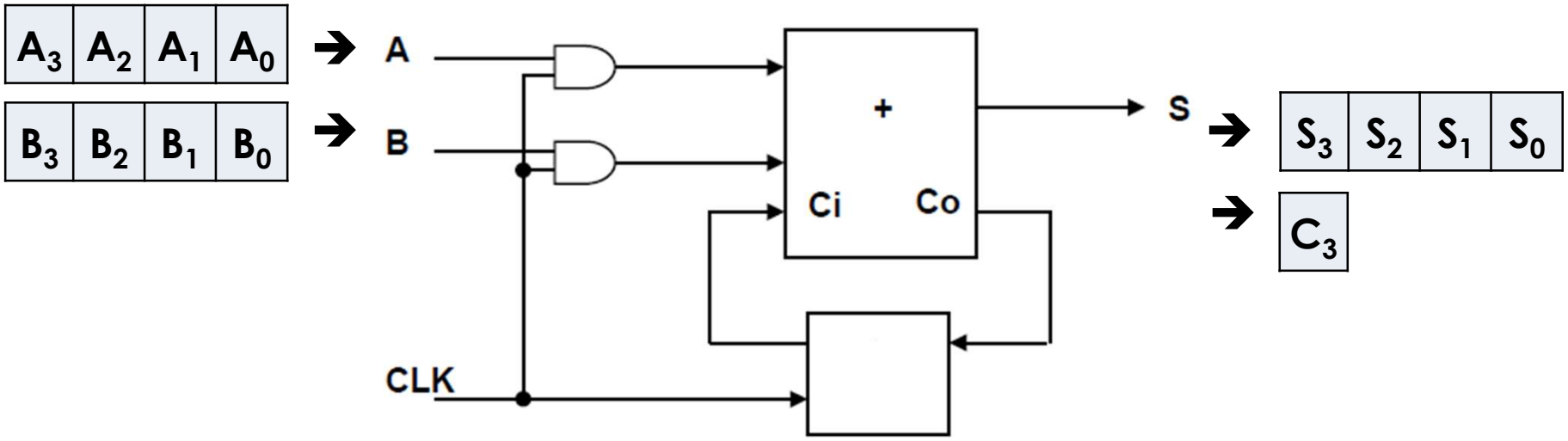
Sumador Paralelo con Acarreo Anticipado (Carry Look Ahead)



Sumador serie

Este concepto requiere poco hardware (**1 único FA**) pero necesita almacenar los sumandos y resultados (memoria) y también secuenciar las sumas (software).

$$\begin{array}{r} C_2\ C_1\ C_0 \\ A_3\ A_2\ A_1\ A_0 \\ +\ B_3\ B_2\ B_1\ B_0 \\ \hline C_3\ S_3\ S_2\ S_1\ S_0 \end{array}$$



Sumador serie

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Debemos considerar que por cada bit (columna) tendremos los retardos propios del **FA**, a esto deberemos adicionar los tiempos de transporte de los sumandos, almacenamiento de los resultados en la memoria, etc.

$$\begin{array}{r} C_2\ C_1\ C_0 \\ A_3\ A_2\ A_1\ A_0 \\ +\ B_3\ B_2\ B_1\ B_0 \\ \hline C_3\ S_3\ S_2\ S_1\ S_0 \end{array}$$

El tiempo total depende **linealmente** de la cantidad de bits del sumador más los tiempos de almacenamiento. Considerando el retardo del FA como 2δ tenemos:

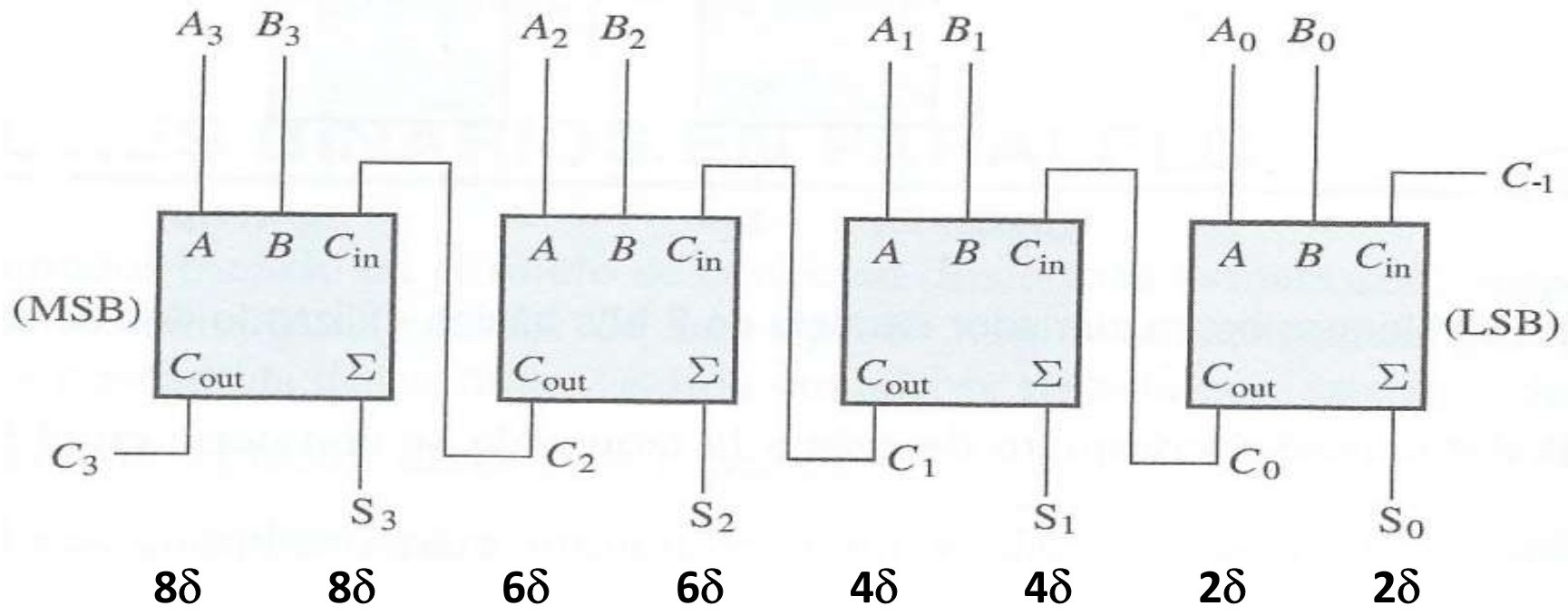
Para **n bits**: $\mathbf{Tt = 2\ n\ \delta + ts}$

Sumador paralelo con acarreo serie

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Este concepto es puro hardware (**1 FA por cada bit**).

$$\begin{array}{r} C_2\ C_1\ C_0 \\ A_3\ A_2\ A_1\ A_0 \\ +\ \underline{B_3\ B_2\ B_1\ B_0} \\ \hline C_3\ S_3\ S_2\ S_1\ S_0 \end{array}$$



Sumador paralelo con acarreo serie

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Debemos considerar que cada FA debe esperar los resultados válidos del anterior, y a partir de ese momento debe transcurrir su propio tiempo de cálculo.

$$\begin{array}{r} C_2\ C_1\ C_0 \\ A_3\ A_2\ A_1\ A_0 \\ +\ B_3\ B_2\ B_1\ B_0 \\ \hline C_3\ S_3\ S_2\ S_1\ S_0 \end{array}$$

El tiempo total depende **linealmente** de la cantidad de bits del sumador. Nuevamente considerando el retardo del FA como 2δ tenemos:

Para **n bits**: **Tt = 2 n** δ

Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Este sumador (**CLA: Carry Look Ahead**) precalcula todos los acarreos para así tener una velocidad independiente de la cantidad de bits a sumar:

$$\begin{array}{r} C_2\ C_1\ C_0 \\ A_3\ A_2\ A_1\ A_0 \\ +\ B_3\ B_2\ B_1\ B_0 \\ \hline C_3\ S_3\ S_2\ S_1\ S_0 \end{array}$$

Debemos precalcular los C_i para cada columna, de la solución del **FA** tenemos:

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1} \quad \rightarrow \quad C_i = A_i B_i + C_{i-1} (A_i + B_i)$$

si: $g_i = A_i B_i$ y $p_i = A_i + B_i$

$$C_i = g_i + C_{i-1} p_i$$

Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Debemos precalcular los C_i para las 4 columnas.

$$C_i = g_i + C_{i-1}\, p_i$$

$$C_0 = g_0 + C_{-1}\, p_0$$

$$C_1 = g_1 + g_0\, p_1 + C_{-1}\, p_0\, p_1$$

$$C_2 = g_2 + g_1\, p_2 + g_0\, p_1\, p_2 + C_{-1}\, p_0\, p_1\, p_2$$

$$C_3 = g_3 + g_2\, p_3 + g_1\, p_2\, p_3 + g_0\, p_1\, p_2\, p_3 + C_{-1}\, p_0\, p_1\, p_2\, p_3$$

Son todas funciones de 2 niveles, que se hacen 3 considerando g_i y p_i.

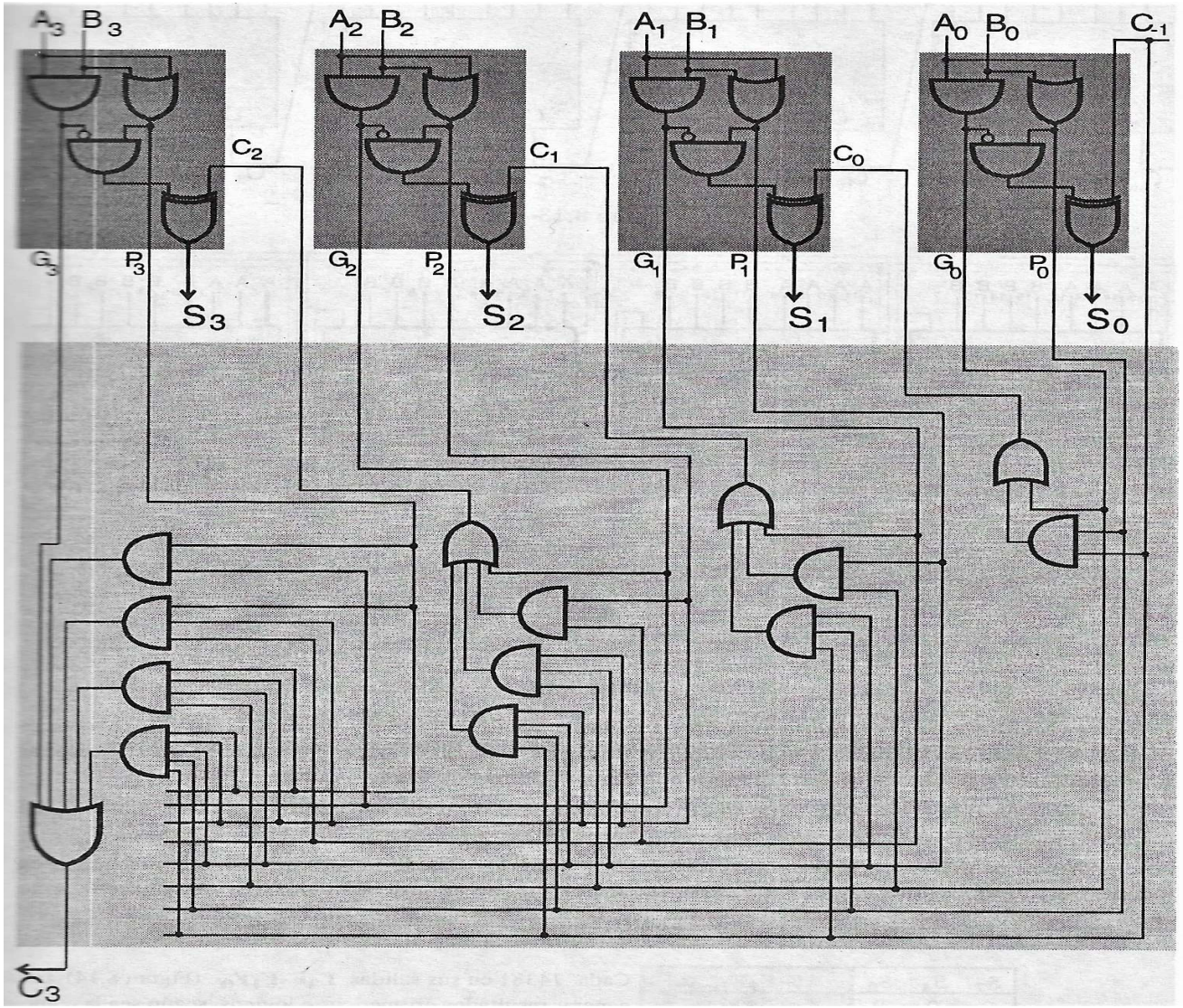
Por lo tanto, en **3** δ tendremos precalculados todos los $\mathbf{C_i}$.

Adicionando **2** δ de los FA tendremos: **Tt = 5** δ (**independiente de n**)

Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

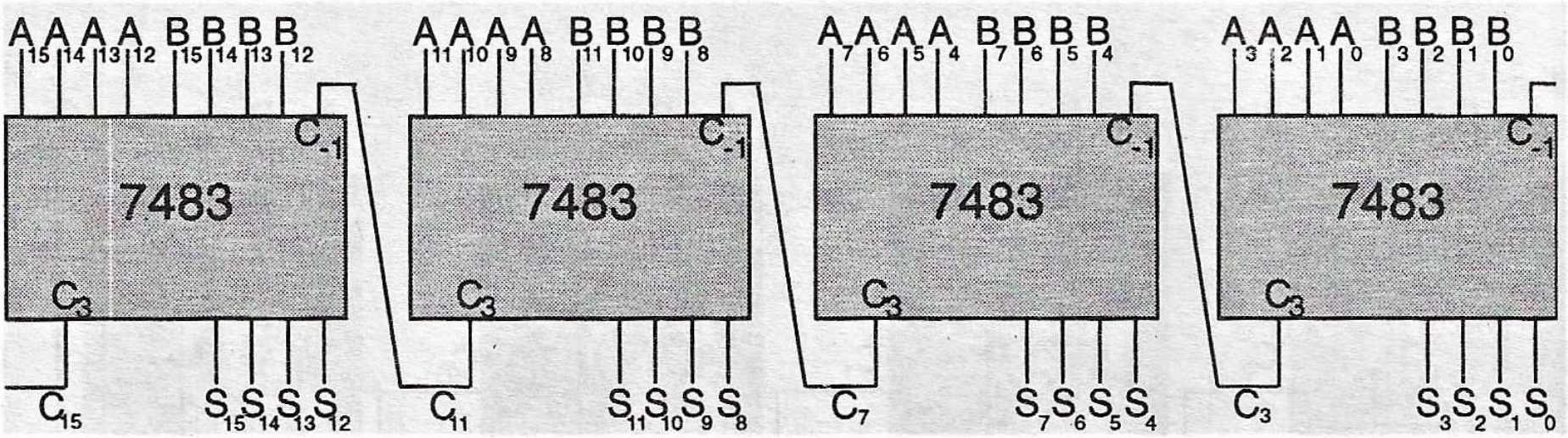
El circuito completo para 4 bits queda (CI 7483 Sumador de 4 bits CLA):



Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

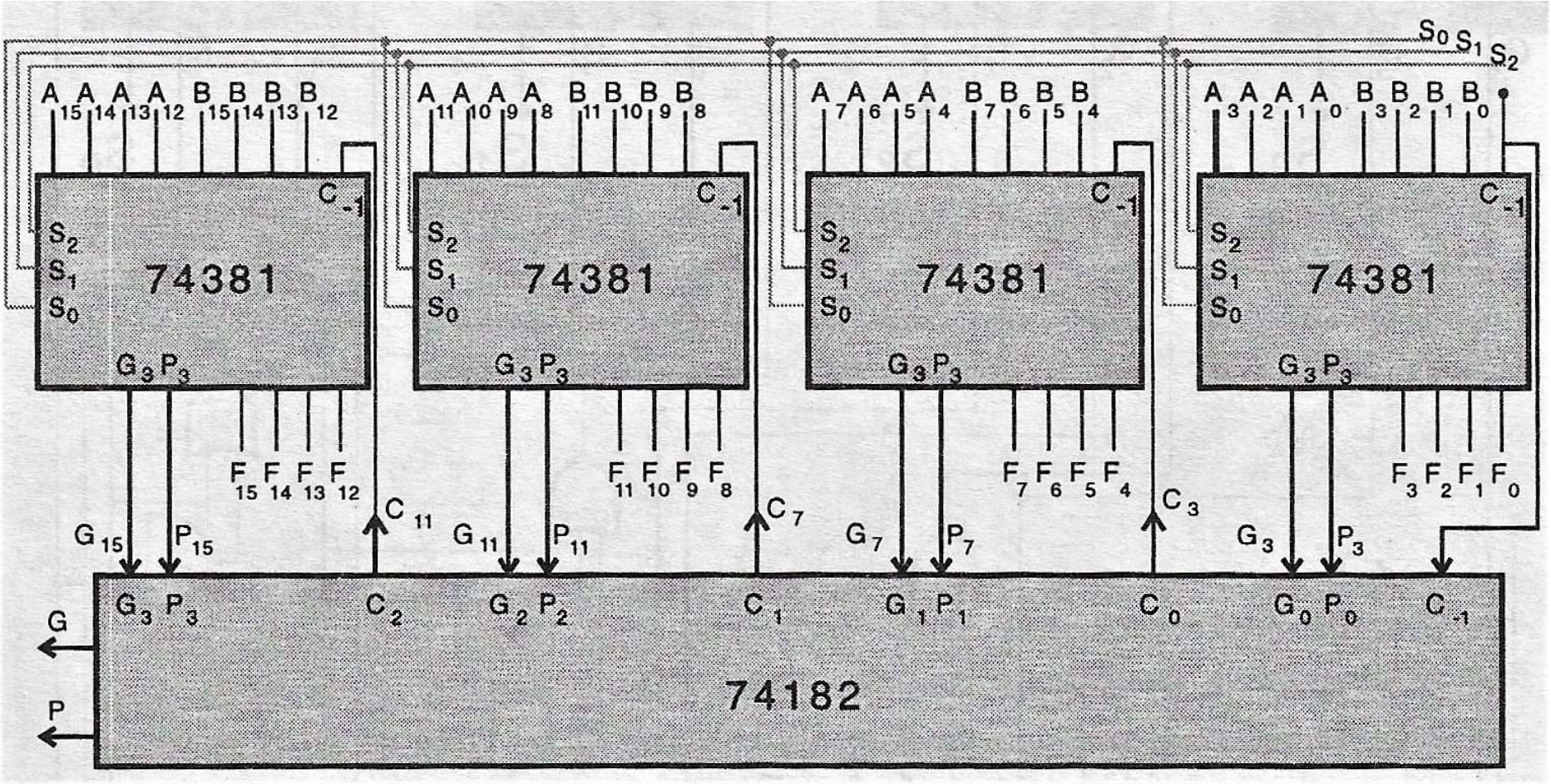
Sumador paralelo de 16 bits con acarreo anticipado (4 CI 7483 Sumador de 4 bits CLA) y acarreo serie entre ellos:



Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

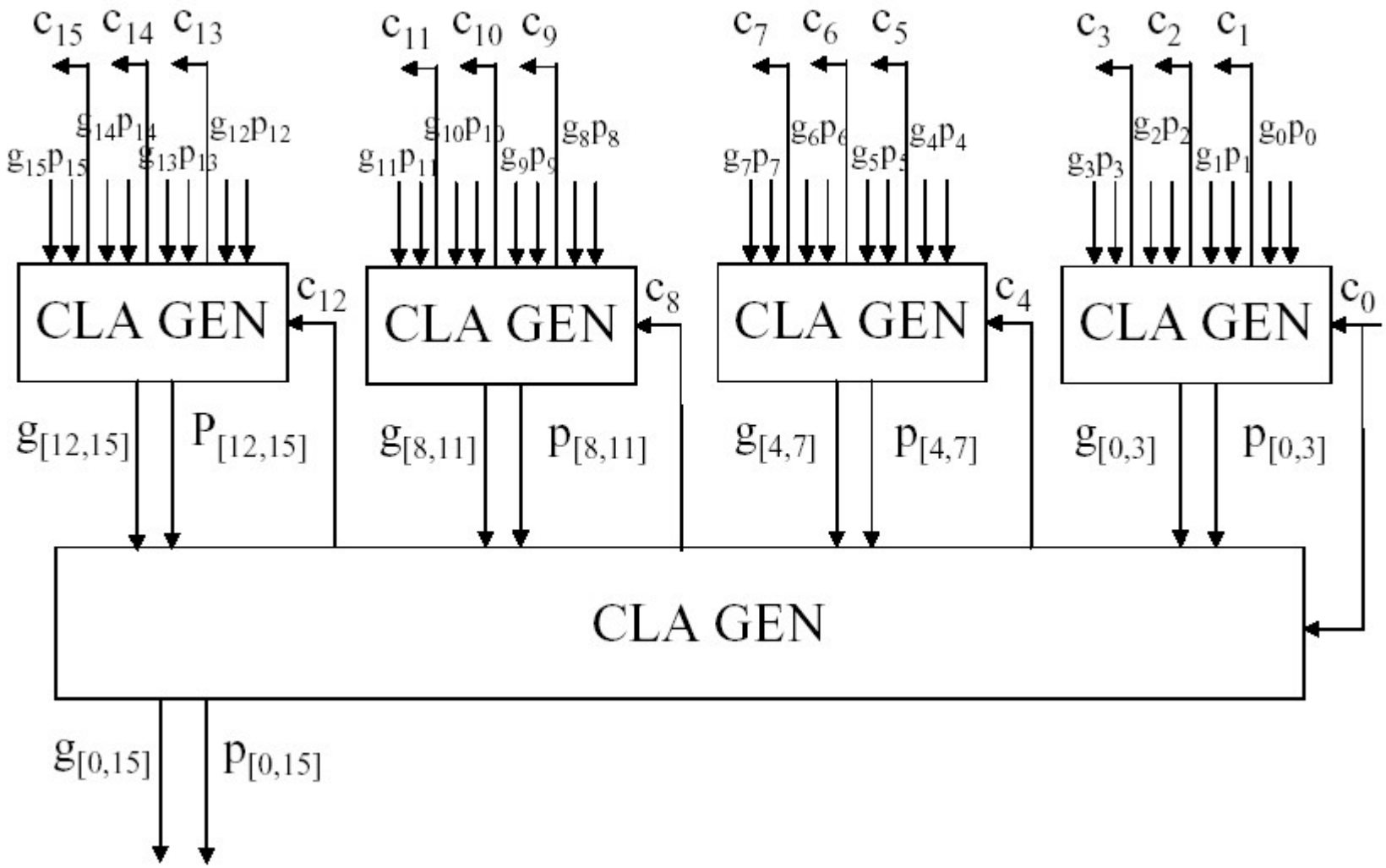
Sumador paralelo de 16 bits con acarreo anticipado (74381/74182):



Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

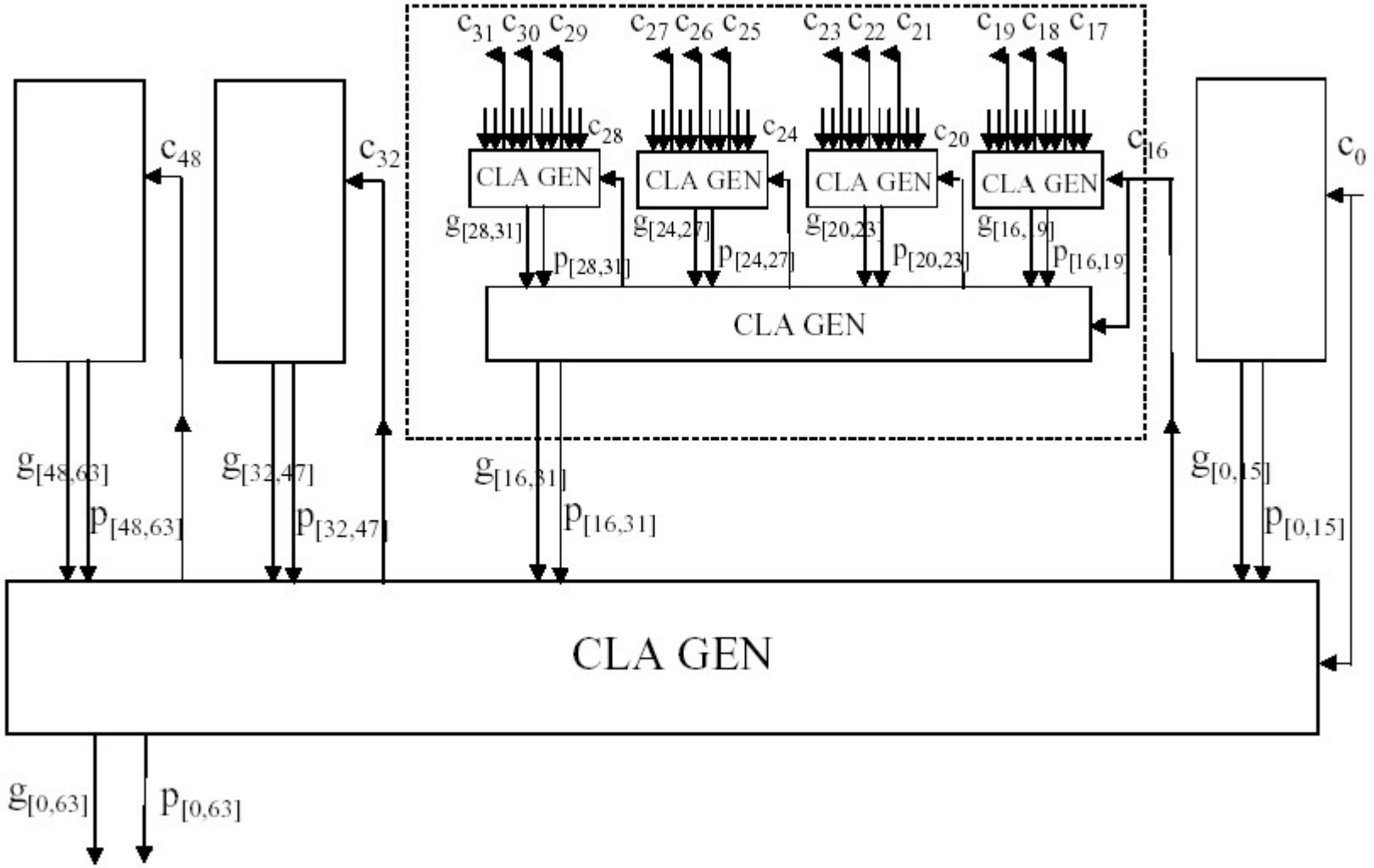
Generador de acarreo de 16 bits implementado en 2 niveles (sin los FA):



Sumador paralelo con acarreo anticipado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Generador de acarreo de 64 bits implementado en 3 niveles (sin los FA):



Tipos de sumadores (resumen)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Sumador Serie

Ventaja: 1 solo FA

Desventaja: Necesita Software y memoria

Tiempo de cálculo: $Tt = 2\ n\ \delta + ts$

P. ej.: para 4 bits: $> 8\ \delta$ para 64 bits: $> 128\ \delta$

Sumador Paralelo con Acarreo Serie (Ripple Adder)

Ventaja: No necesita Software ni memoria

Desventaja: 1 FA por cada bit

Tiempo de cálculo: $Tt = 2\ n\ \delta$

P. ej.: para 4 bits: $8\ \delta$ para 64 bits: $128\ \delta$

Sumador Paralelo con Acarreo Anticipado (Carry Look Ahead)

Ventaja: Velocidad

Desventaja: 1 FA por cada bit

Tiempo de cálculo: $Tt = 5\ \delta$

Independiente de la cantidad de bits

Sumador (implementación alternativa)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

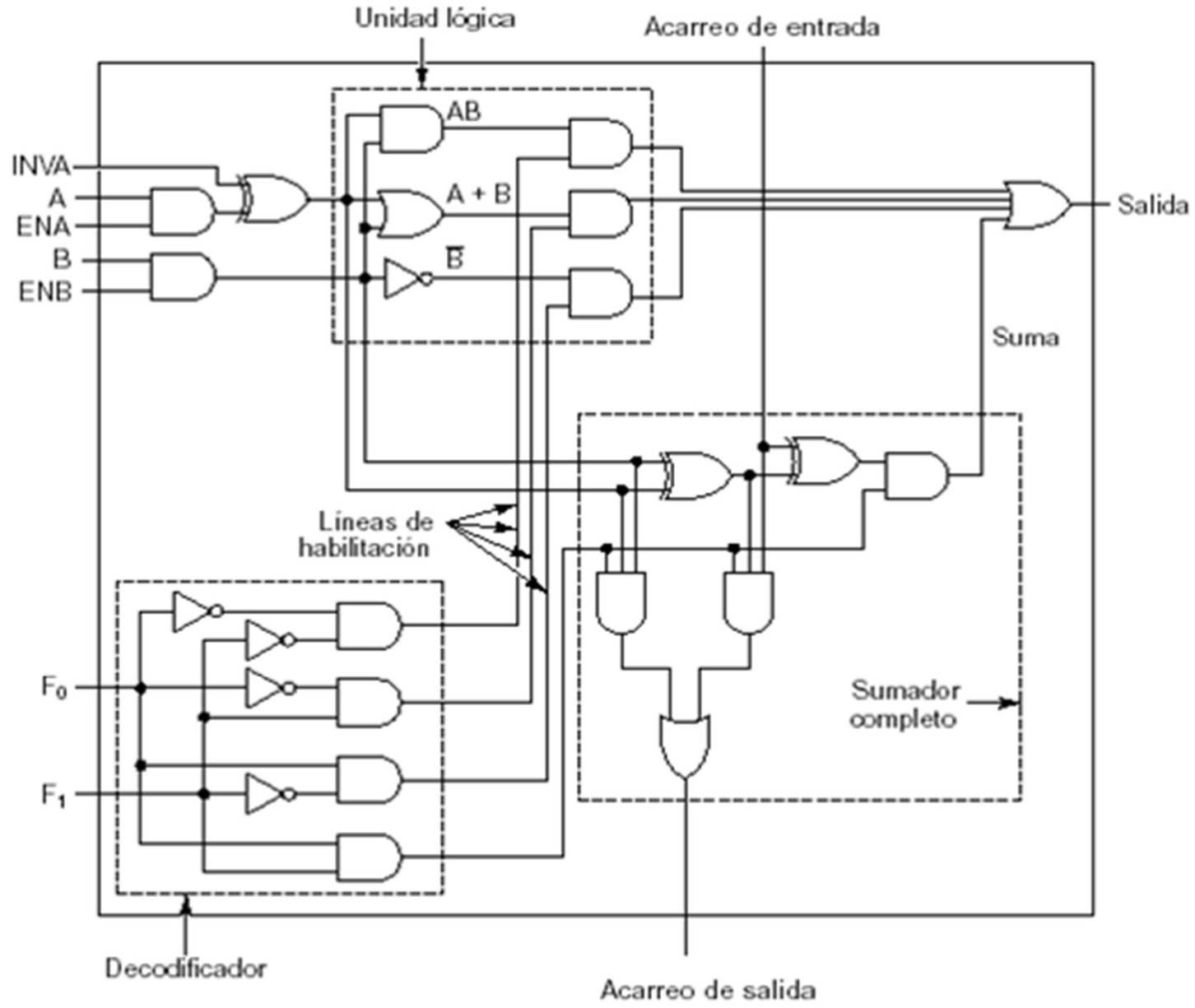
Considerando que al sumar 2 números de n bits el resultado podrá tener hasta n + 1 bits, otro abordaje podría plantear un circuito de 2 niveles con 2 n variables de entrada y n + 1 salidas. Ventajas y desventajas. Retardos.

En nuestro ejemplo tendríamos 8 entradas y 5 salidas.

$A_3\ A_2\ A_1\ A_0$	4 entradas
$+\ B_3\ B_2\ B_1\ B_0$	4 entradas
$C_3\ S_3\ S_2\ S_1\ S_0$	5 salidas

Unidad aritmético lógica (ALU)

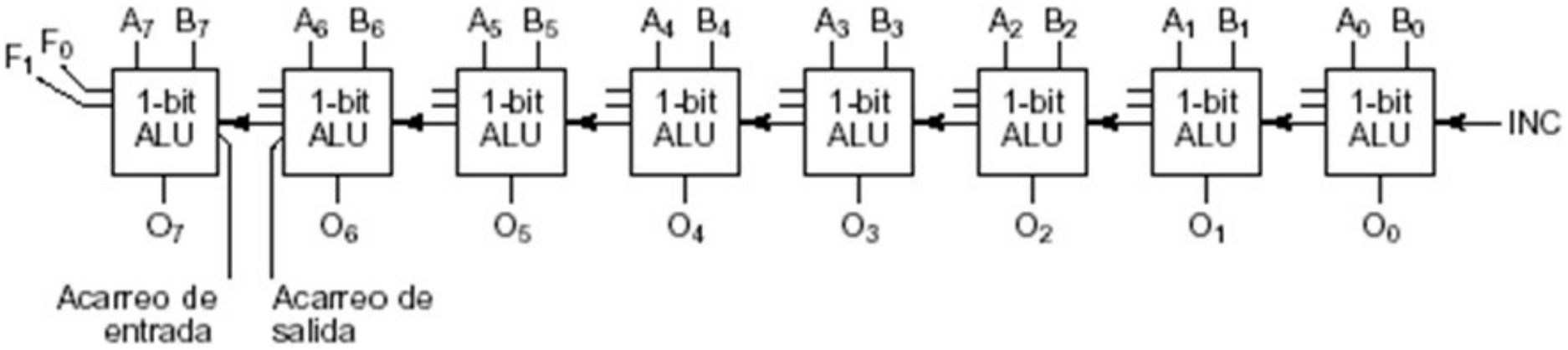
FACULTAD DE INGENIERIA
Universidad de Buenos Aires



Unidad aritmético lógica (ALU)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

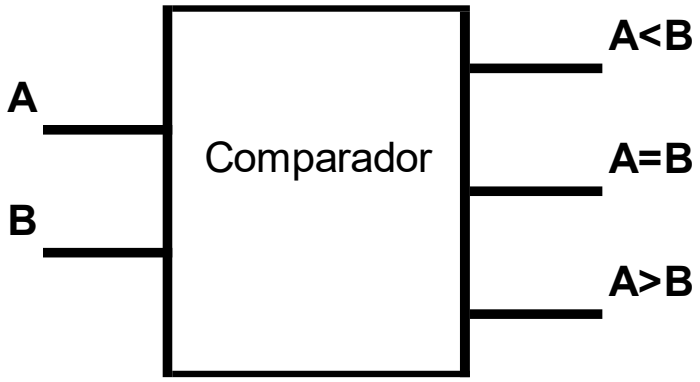
Conjunto de 8 ALUs de 1 bit formando una ALU de 8 bits:



Comparador de magnitud de 1 bit

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Consideremos un circuito que compare 2 números de **1 bit** cada uno:



A	B	A<B	A≤B	A=B	A≥B	A>B	A≠B
0	0	0	1	1	1	0	0
0	1	1	1	0	0	0	1
1	0	0	0	0	1	1	1
1	1	0	1	1	1	0	0

$$S_{A\leq B} = S_{A<B} + S_{A=B}$$

$$S_{A\geq B} = S_{A>B} + S_{A=B}$$

$$S_{A\leq B} = \overline{S_{A>B}}$$

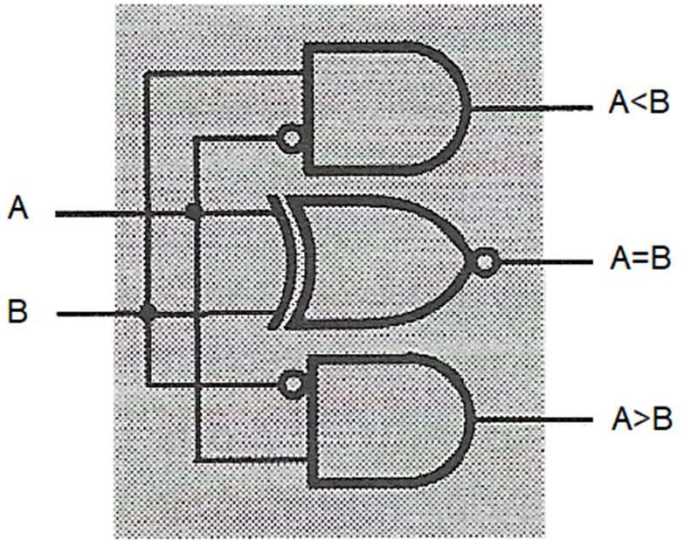
$$S_{A\geq B} = \overline{S_{A<B}}$$

$$S_{A\neq B} = \overline{S_{A=B}}$$

$$S_{A<B} = \overline{A}\, B$$

$$S_{A=B} = \overline{A \oplus B}$$

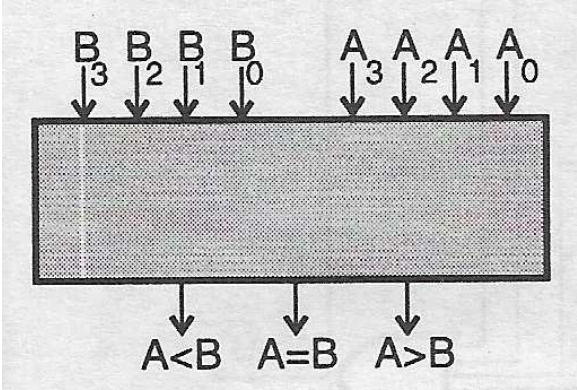
$$S_{A>B} = A\, \overline{B}$$



Comparador de magnitud de 4 bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Veamos ahora un circuito que compare 2 números de **4 bits** cada uno:



Tendríamos 3 funciones de 8 variables cada una. Ventajas y desventajas.

A3	A2	A1	A0	B3	B2	B1	B0	A<B	A=B	A>B
0	0	0	0	0	0	0	0	0	1	0
		256 combinaciones								
1	1	1	1	1	1	1	1	0	1	0

Comparador de magnitud de 4 bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Utilizando comparadores de **1 bit**, implementando solo **A>B** y **A=B** tendríamos:

A=B cuando $A_3=B_3$ y $A_2=B_2$ y $A_1=B_1$ y $A_0=B_0$

A>B cuando $A_3>B_3$ o $A_3=B_3$ y $A_2>B_2$ o $A_3=B_3$ y $A_2=B_2$ y $A_1>B_1$ o

$A_3=B_3$ y $A_2=B_2$ y $A_1=B_1$ y $A_0>B_0$

$$S_{A=B} = \overline{A_3 \oplus B_3}\ \overline{A_2 \oplus B_2}\ \overline{A_1 \oplus B_1}\ \overline{A_0 \oplus B_0}$$

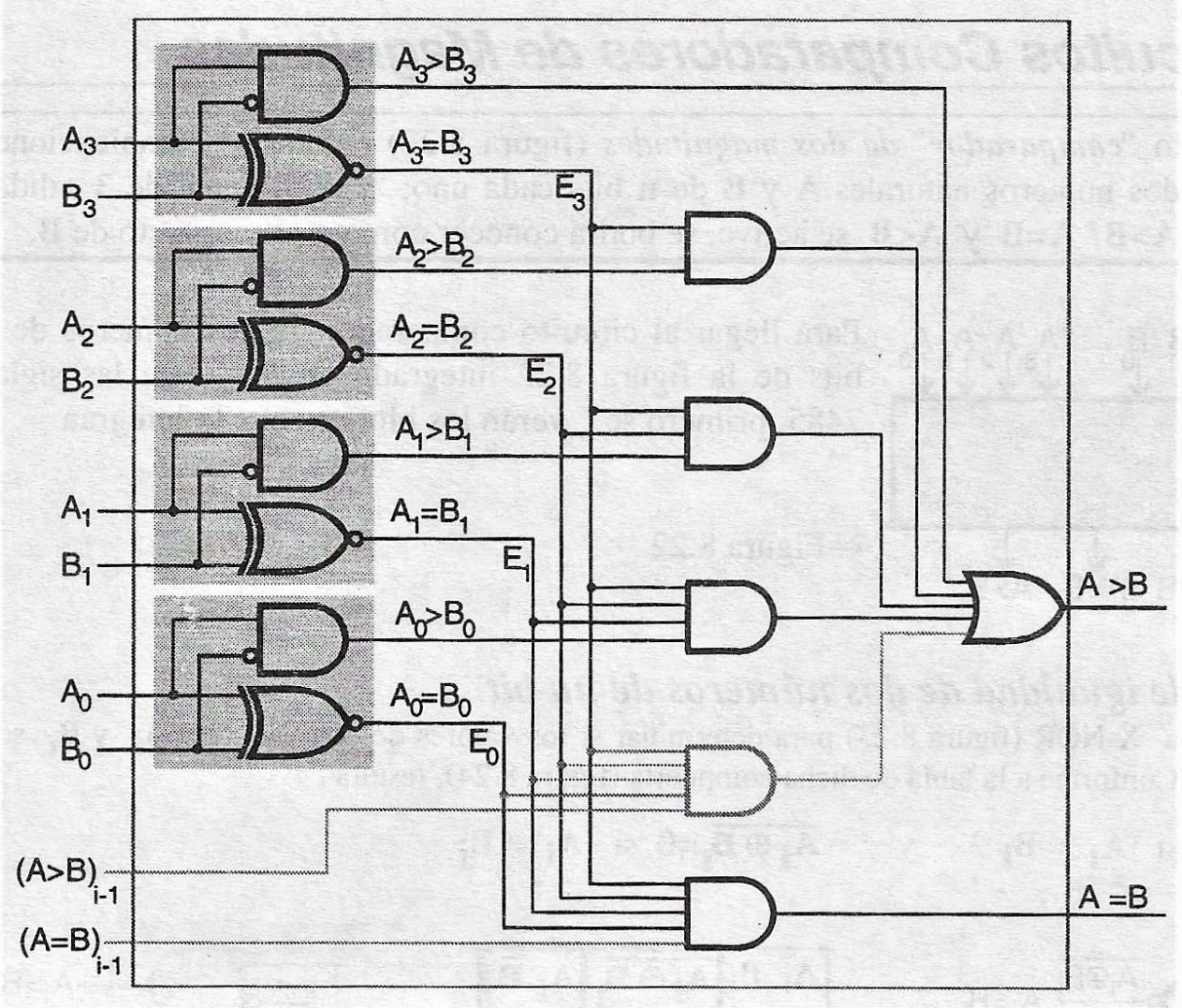
$$S_{A>B} = A_3\,\overline{B}_3 + \overline{A_3 \oplus B_3}\ A_2\,\overline{B}_2 + \overline{A_3 \oplus B_3}\ \overline{A_2 \oplus B_2}\ A_1\,\overline{B}_1 +$$

$$+ \overline{A_3 \oplus B_3}\ \overline{A_2 \oplus B_2}\ \overline{A_1 \oplus B_1}\ A_0\,\overline{B}_0$$

Comparador de magnitud de 4 bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

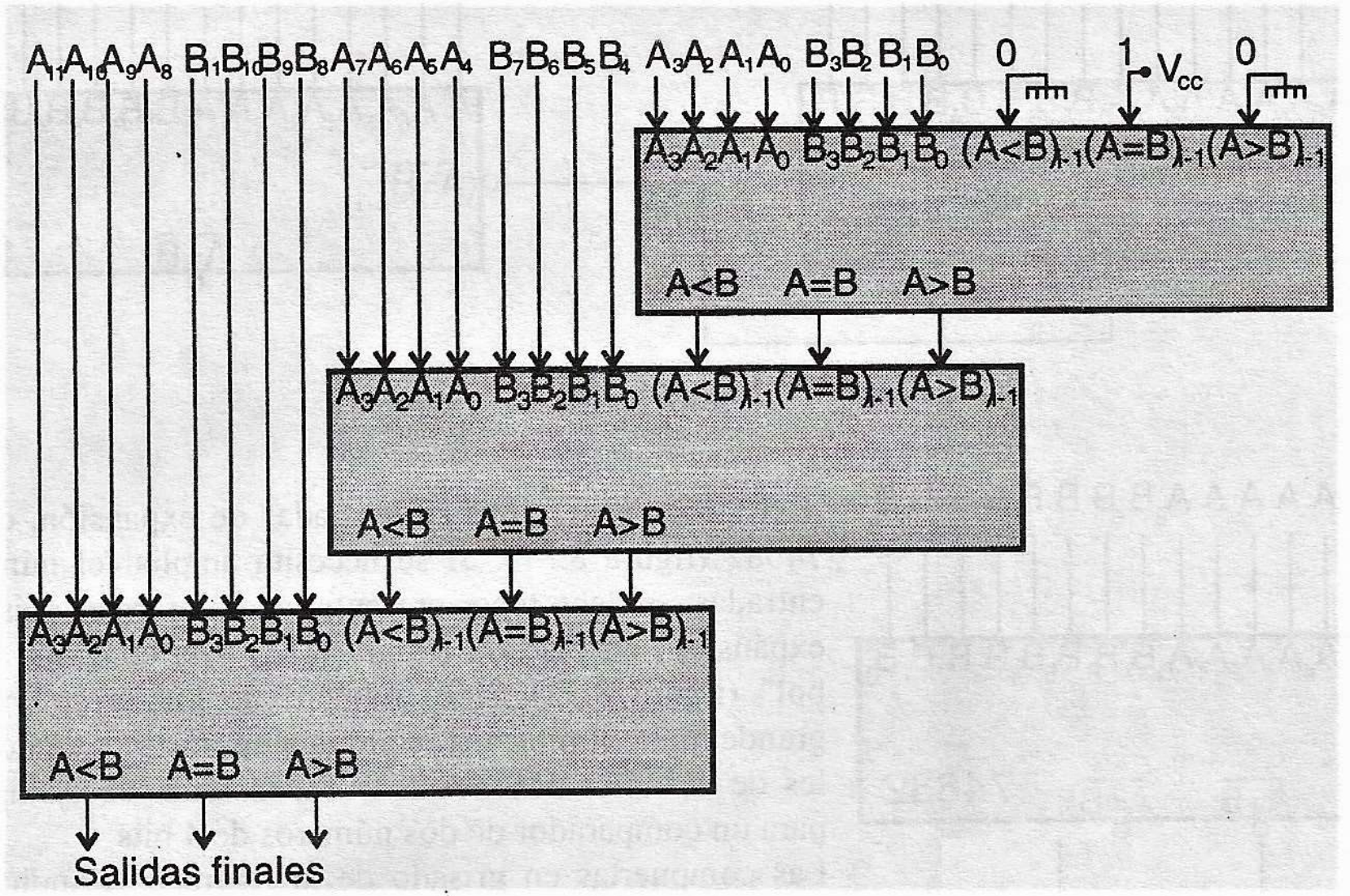
Agregando entradas adicionales para otro comparador anterior (en gris) queda:



Comparador de magnitud de 12 bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

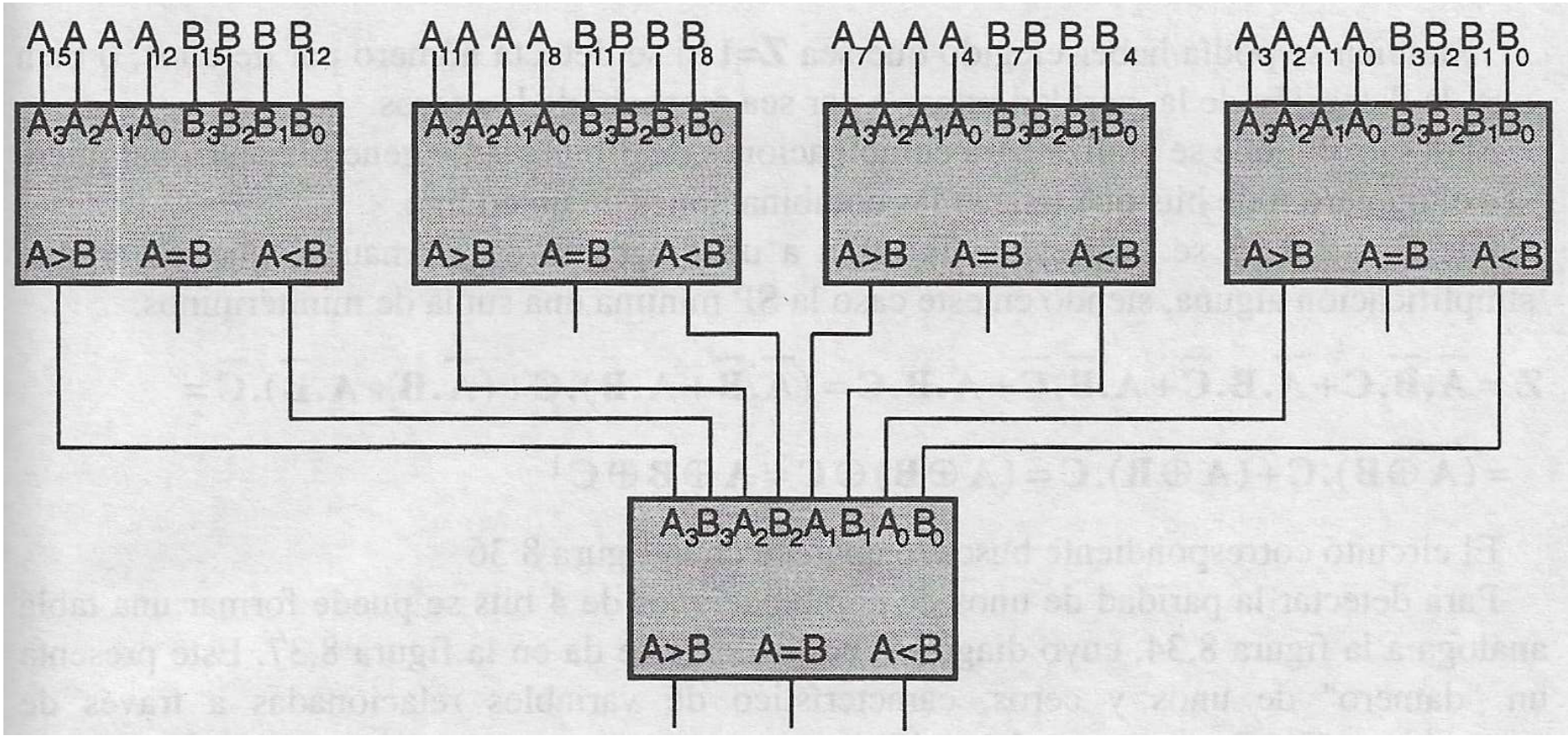
Conectando 3 comparadores de 4 bits en cascada (CI 7485):



Comparador de magnitud de 16 bits

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Conectando 5 comparadores de 4 bits en árbol:



Generador / detector de paridad

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

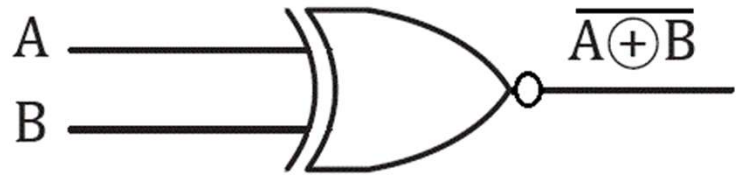
Las compuertas exclusivas permiten conocer la **paridad** de sus entradas:

A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0



Generador de paridad **PAR**

A	B	$\overline{A \oplus B}$
0	0	1
0	1	0
1	0	0
1	1	1

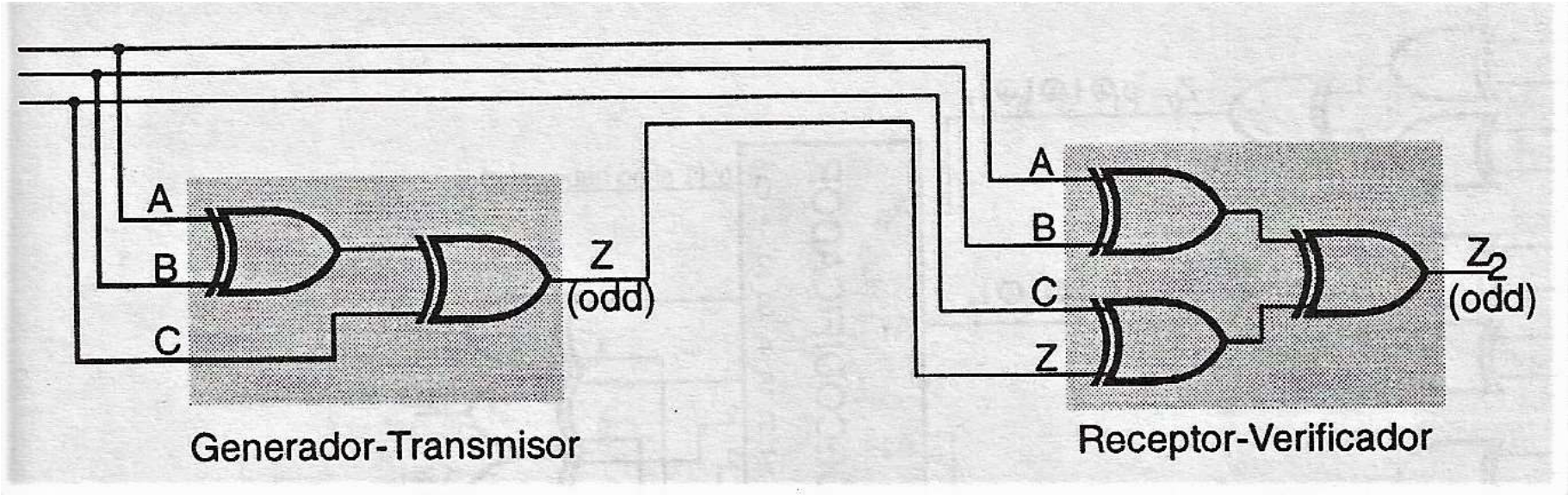


Generador de paridad **IMPAR**

Generador / detector de paridad

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: Generación y detección de las paridades en el método de Hamming. (atención con la cantidad de niveles en el receptor).



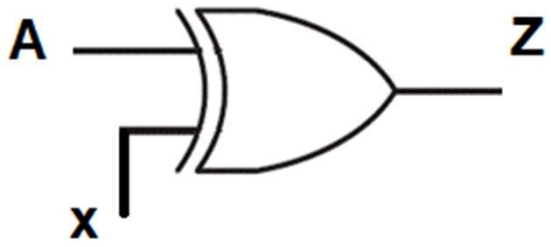
Inversor controlado

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Utilizando una de las entradas de una XOR como entrada de control (x) puedo **controlar la inversión** o no del dato presente en la otra entrada (A):

x	Z
0	A
1	Not A

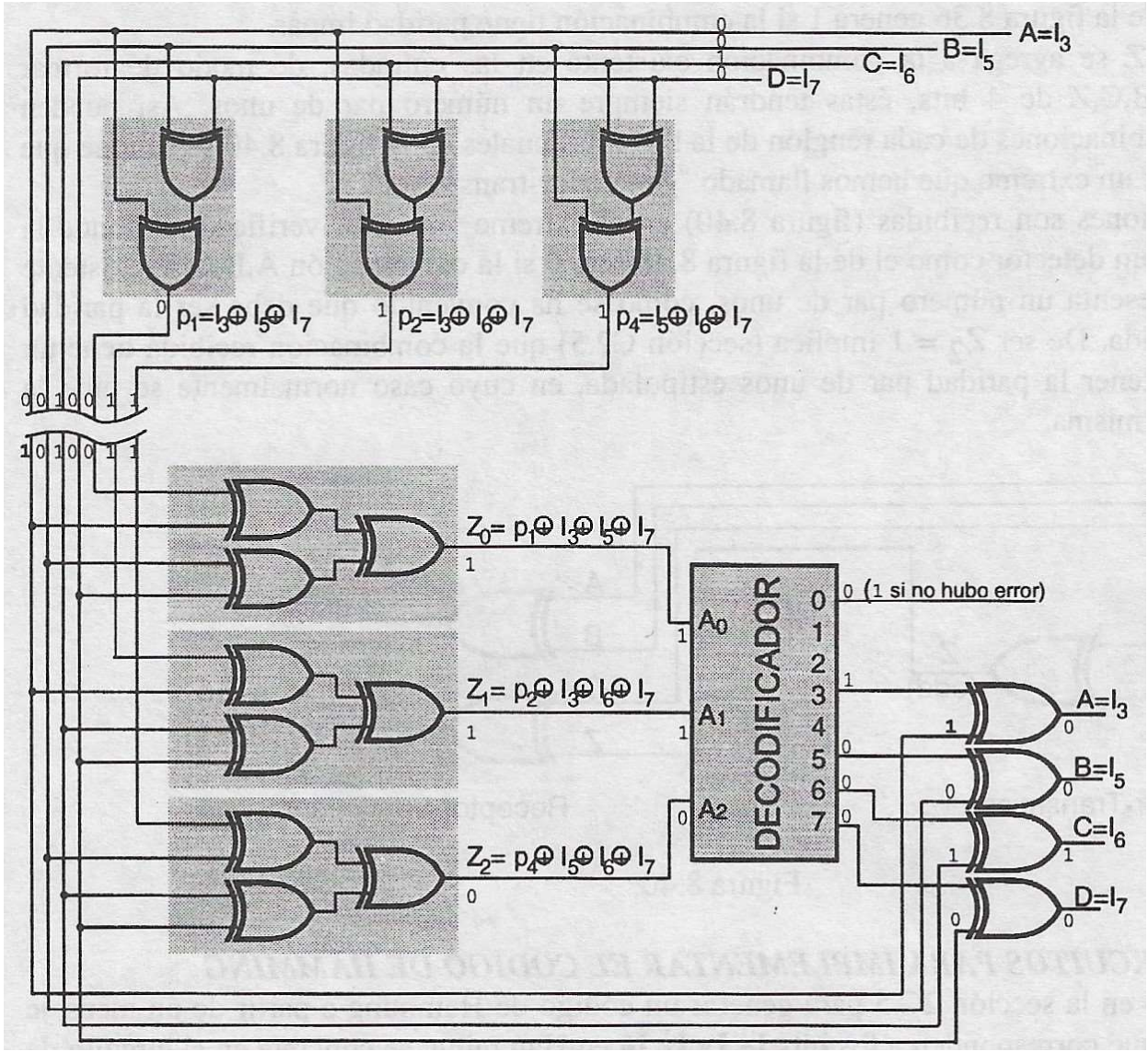
x	A	Z
0	0	0
0	1	1
1	0	1
1	1	0



Circuito de Hamming

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Aquí vemos varias de las unidades funcionales vistas:



FACULTAD DE INGENIERIA
Universidad de Buenos Aires



FACULTAD DE INGENIERIA
Universidad de Buenos Aires

86.01 Técnica Digital

Memorias y Dispositivos Lógicos Programables

Ing. Jorge H. Fuchs

Introducción

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Objetivos de la clase:

Conocer los distintos tipos de memorias como también su organización interna.

Analizar el concepto de módulo universal en el sentido de un circuito capaz de permitir la síntesis de cualquier tipo de función lógica dada su facilidad de programación.

Conocer los distintos tipos de Dispositivos Lógicos Programables (PLD) y sus características.

Estudiar su utilización en la implementación de funciones lógicas de circuitos combinacionales.

Memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Clasificación de las memorias según su acceso:

Memorias de acceso secuencial

El tiempo de acceso a los datos depende de su ubicación. Ejemplos: Buffer, Stack, etc. Las veremos más adelante.

Memorias de acceso aleatorio (Random Access Memory)

El tiempo de acceso a los datos es independiente de su ubicación. Es una característica de cada una. Ejemplos: RAM, ROM, HDD, SSD, Pen Drive, etc.

Memorias de acceso aleatorio

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Clasificación de las memorias de acceso aleatorio (RAM):

Memorias de Lectura / Escritura (conocidas como RAM)

Memorias de sólo lectura (ROM)

Memorias de mayormente lectura

Memorias de acceso aleatorio

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Memorias de Lectura/Escritura (conocidas como RAM):

DRAM (Dynamic Random Access Memory):

Los datos se almacenan como en la carga de un capacitor. Tiende a descargarse y, por lo tanto, es necesario un proceso de refresco periódico. Son de mayor capacidad, más simples y baratas que las SRAM. Son volátiles.

SRAM (Static Random Access Memory):

Los datos se almacenan formando biestables (las veremos más adelante), por lo que no requiere refresco. Igual que la DRAM son volátiles. Son más rápidas que las DRAM y más caras.

Memorias de acceso aleatorio

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Memorias de sólo lectura (ROM):

ROM (Read Only Memory):

Se usan principalmente en microprogramación de sistemas. Los fabricantes las suelen emplear cuando producen componentes de forma masiva.

PROM (Programmable Read Only Memory):

El proceso de escritura es eléctrico. Se puede grabar (por el usuario) posteriormente a la fabricación del chip, a diferencia de las anteriores que se graba durante la fabricación. Permite una única grabación y es más cara que la ROM.

Memorias de acceso aleatorio

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Memorias de sobre todo lectura:

EPROM (Erasable Programmable Read Only Memory):

Se puede escribir varias veces de forma eléctrica, sin embargo, el borrado de los contenidos es completo y a través de la exposición a rayos ultravioletas (suelen tener una pequeña “ventanita” en el chip).

EEPROM o E^2PROM (Electrically Erasable PROM):

Se puede borrar selectivamente byte a byte con corriente eléctrica. Es más cara que la EPROM. El proceso de borrado es lento.

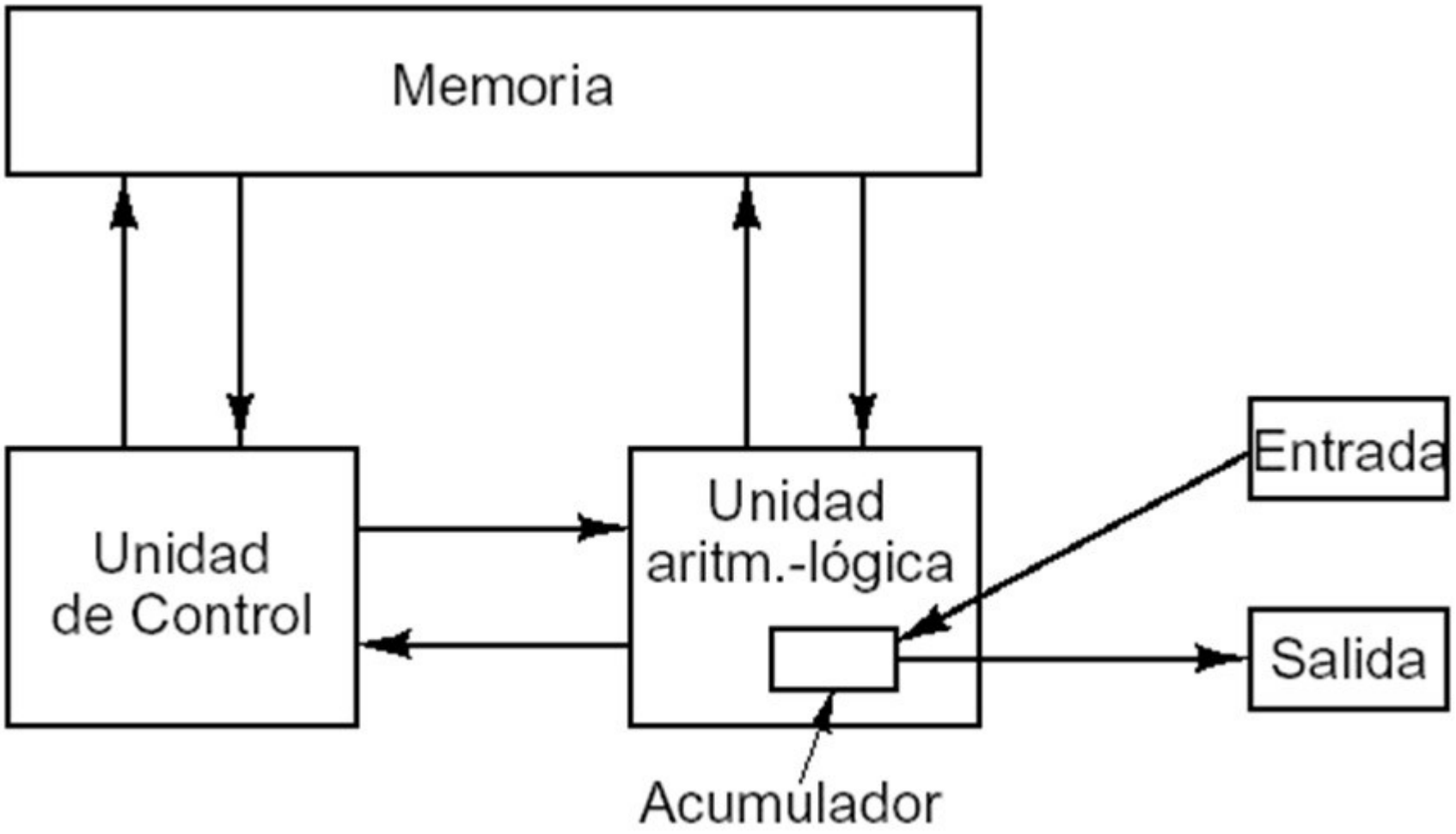
FLASH:

Está basada en las memorias EEPROM pero permite el borrado por bloques, por lo que este proceso es mucho más veloz. Es más barata y densa.

Memorias de acceso aleatorio

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Esquema básico de una computadora:



Buses

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Es un camino eléctrico común entre varios dispositivos y la CPU (Unidad Central de Procesamiento).

Ejemplos:

Bus de Direcciones (Address Bus)

Bus de Datos (Data Bus)

Bus de Control

Buses

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Bus de Direcciones (Address Bus)

n líneas de Address ➔ 2^n locaciones

n: 16, 20, 32, 64, etc.

Unidireccional

Tamaño de la memoria

Bus de Datos (Data Bus)

d líneas de Data ➔ d bit / palabra

d: 8, 16, 32, 64.

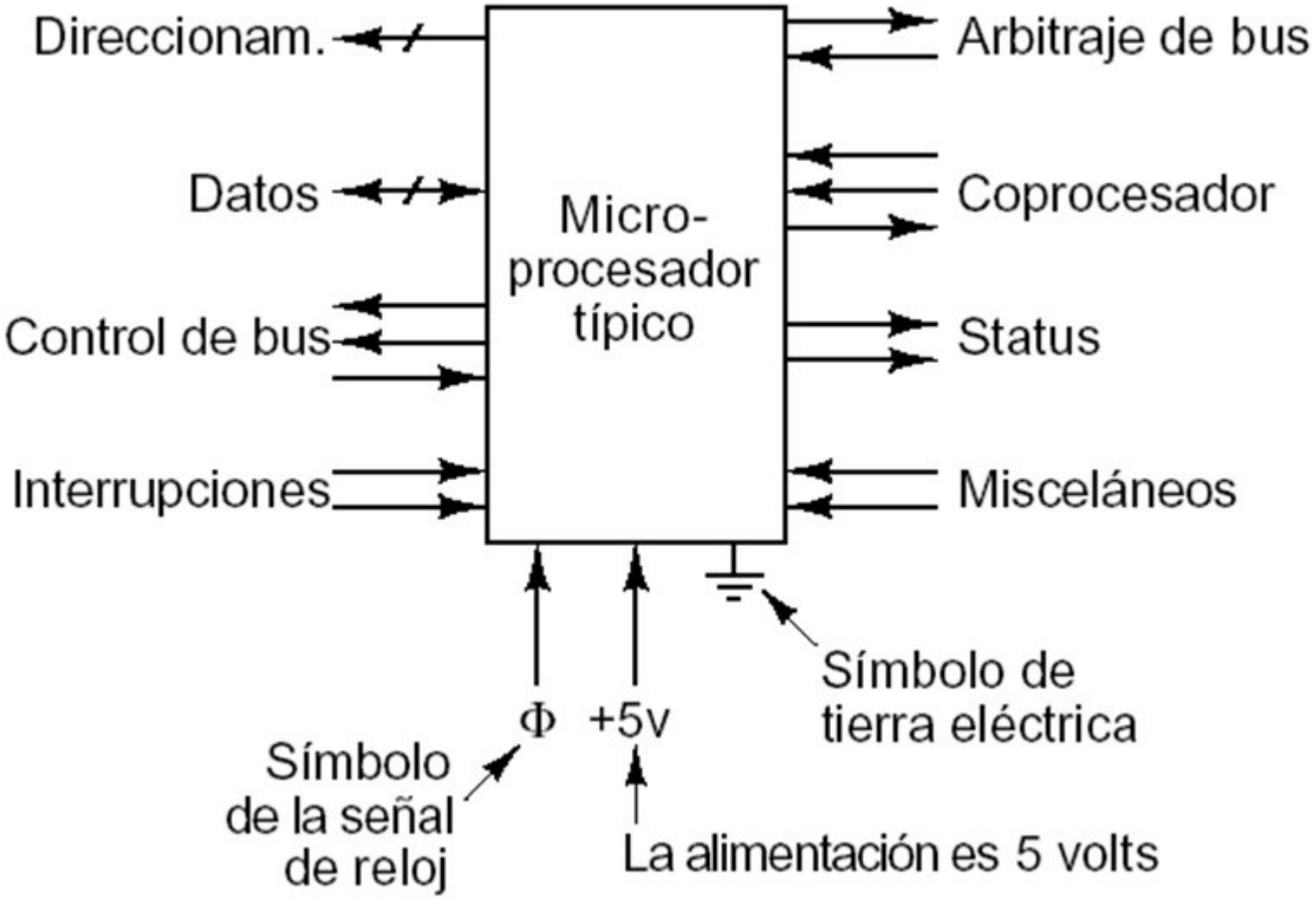
Bidireccional

Ancho de banda

Buses

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

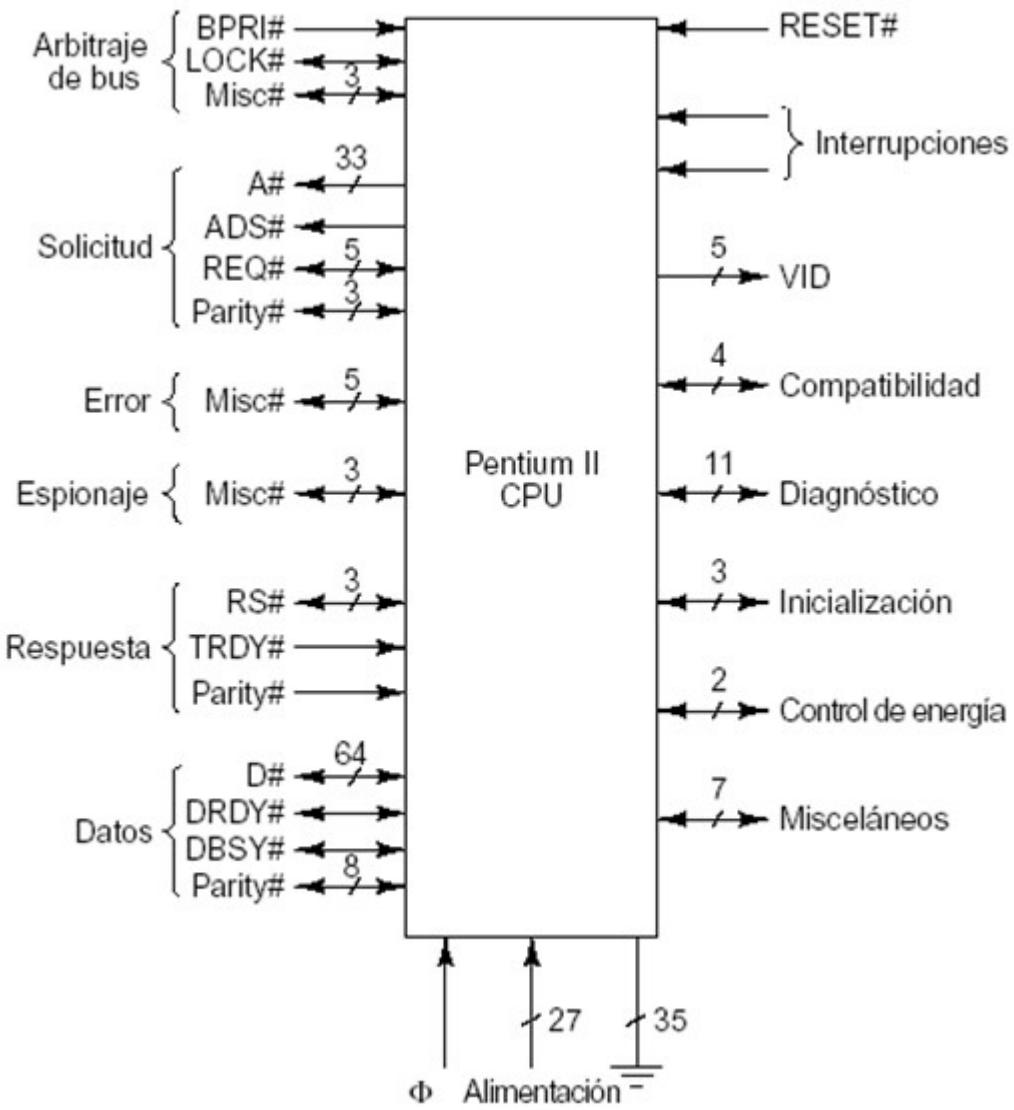
Buses externos de una CPU:



Buses

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

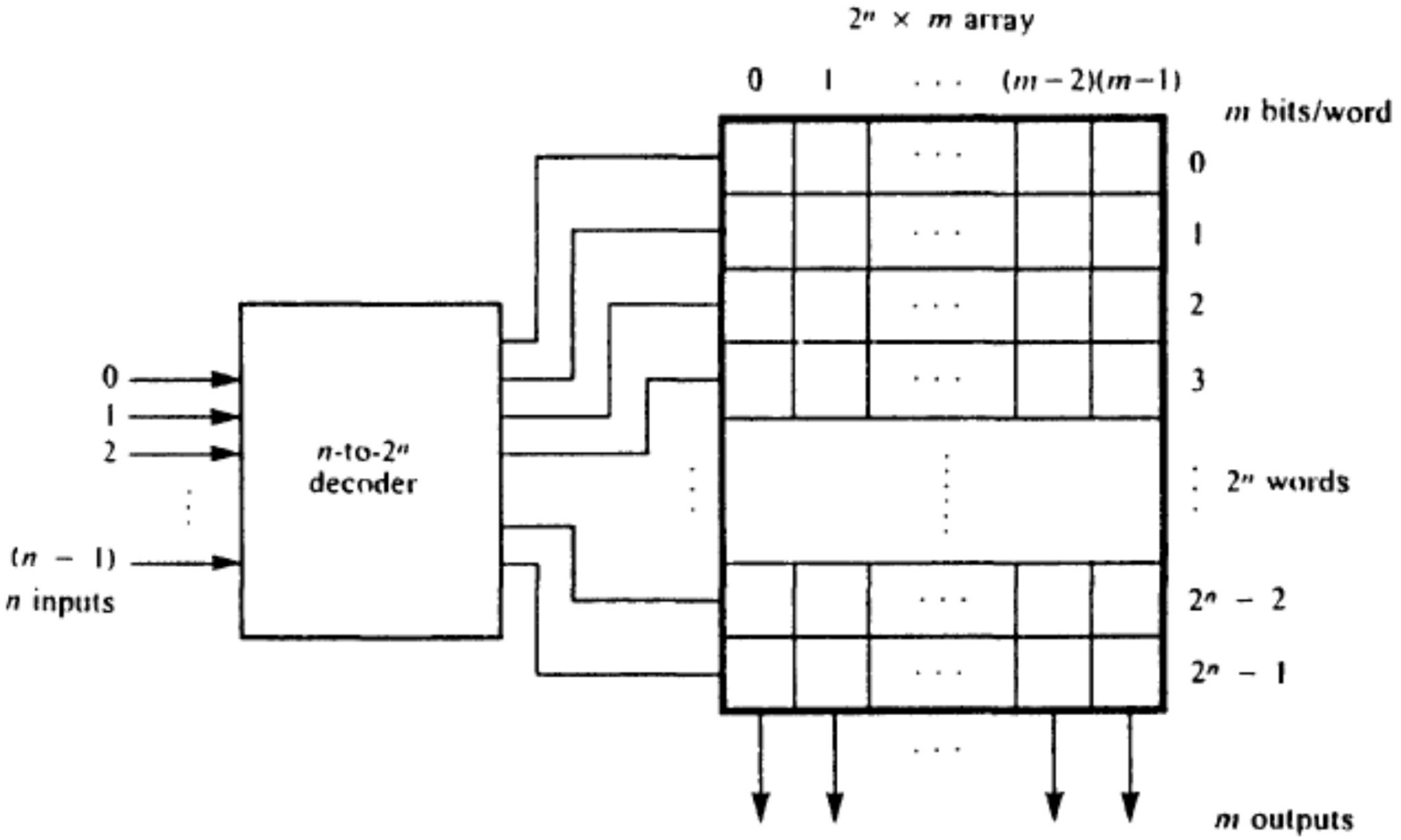
Ejemplo de buses externos de una CPU:



Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Organización de una memoria de acceso aleatorio:

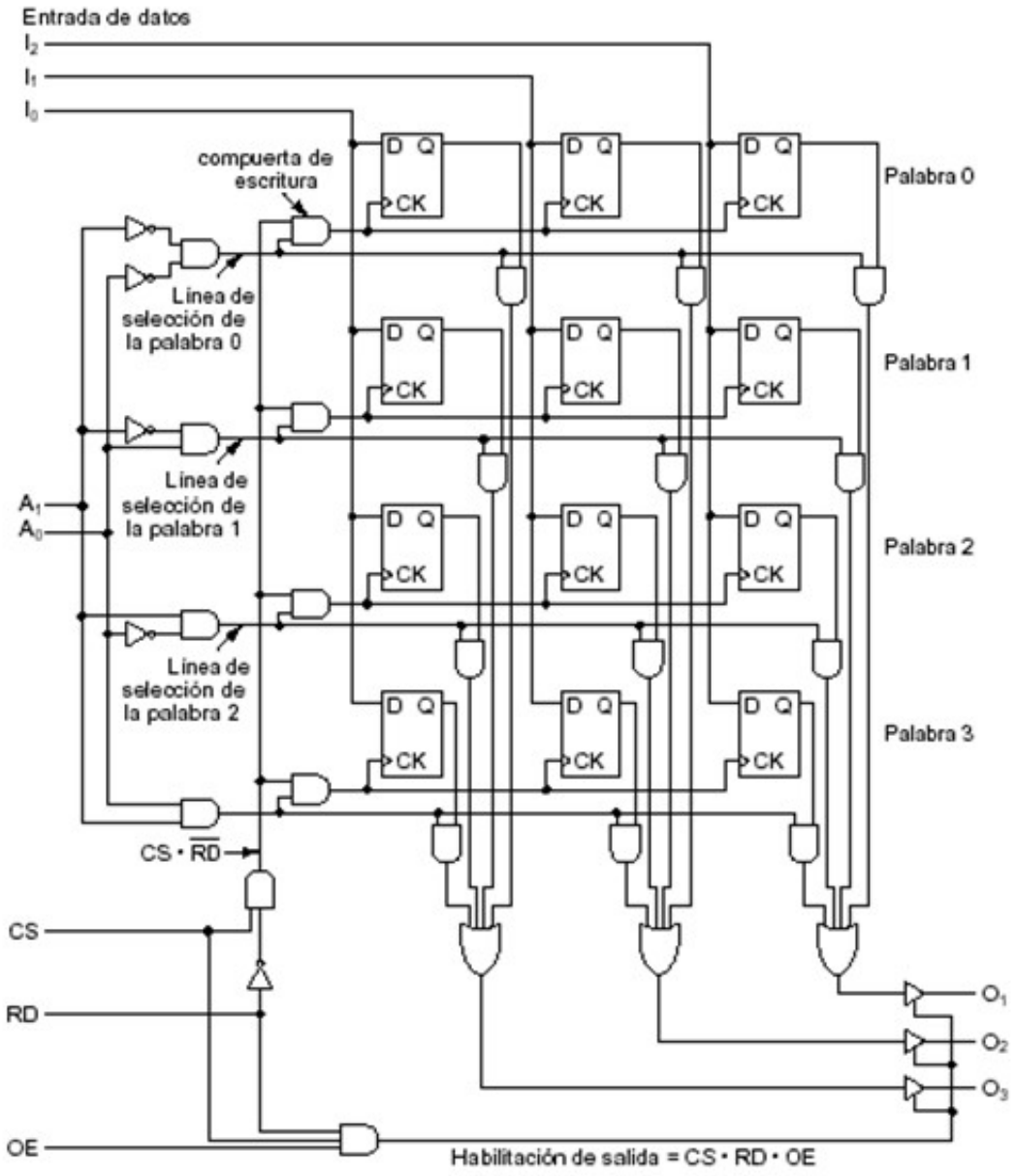


Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Organización de una memoria de 4 palabras de 3 bits cada una.

Cada renglón tiene 3 bits, una operación de lectura o escritura siempre lee o escribe una palabra completa (3 bits).



Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Selección de las m palabras (Address Bus: n líneas)

- Cantidad de palabras: $\mathbf{m = 2^n}$

Organización en palabras de L bits (Data Bus: d líneas)

- Longitud de palabra: $\mathbf{L = d}$

Capacidad de la memoria

- Capacidad (cant. total de celdas): $\mathbf{c = m\ L}$

Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: Memoria de 8 palabras de 4 bits c/u.

• # palabras	$m = 2^n$	$8 = 2^3$ >>> n = 3 (líneas de Address)
• # líneas de datos	d = L	4
• # total celdas	c = m x L	4 x 8 = 32 bits

Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

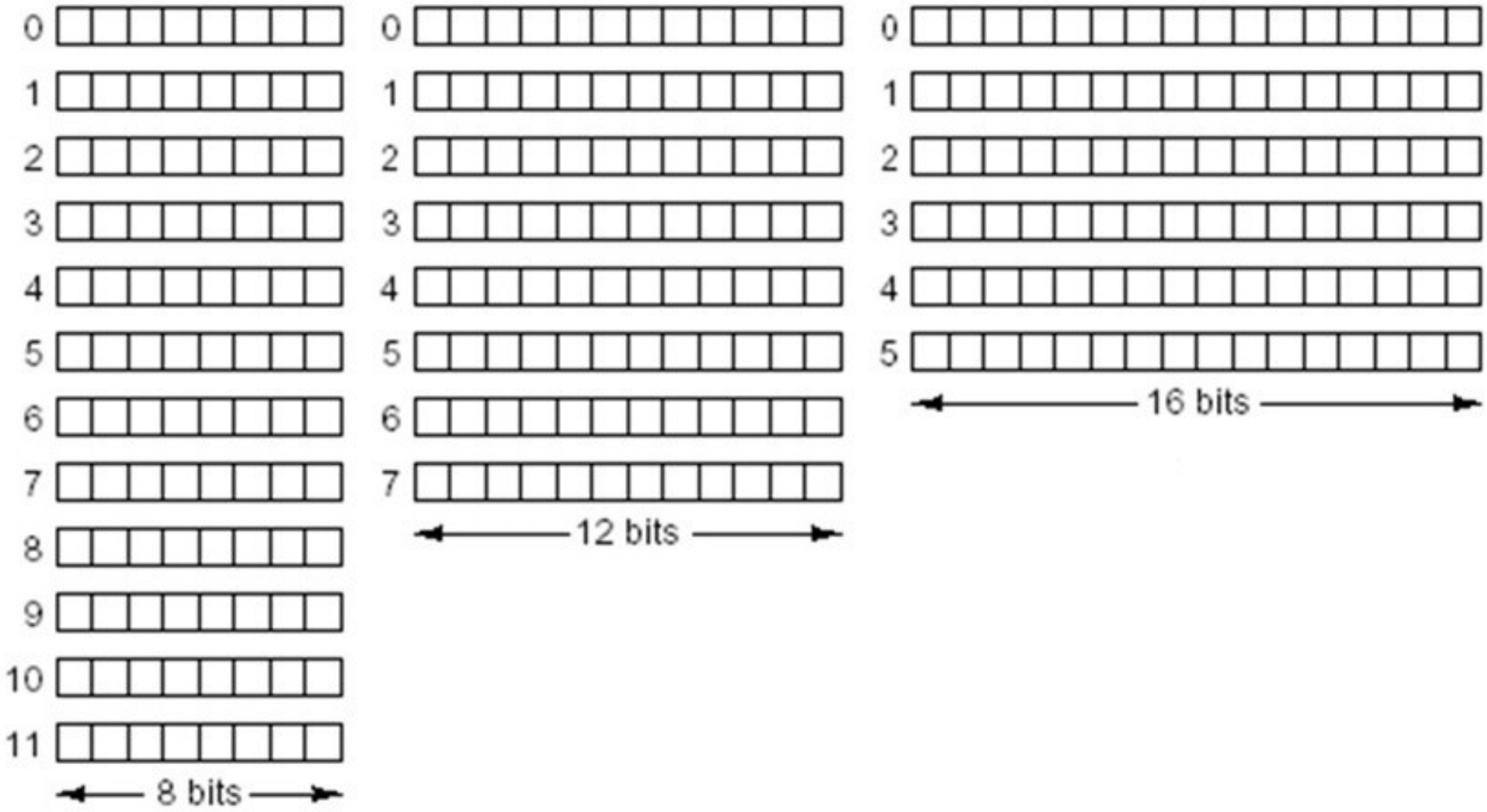
Más ejemplos:

	Bus de datos	Bus de direcciones	Capacidad en bits
1024 x 1	1	10	1024
256 x 4	4	8	1024
2048 x 8	8	11	16384
4096 x 1	1	12	4096
8K x 8	8	13	65568
1M x 1	1	20	1048576

Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

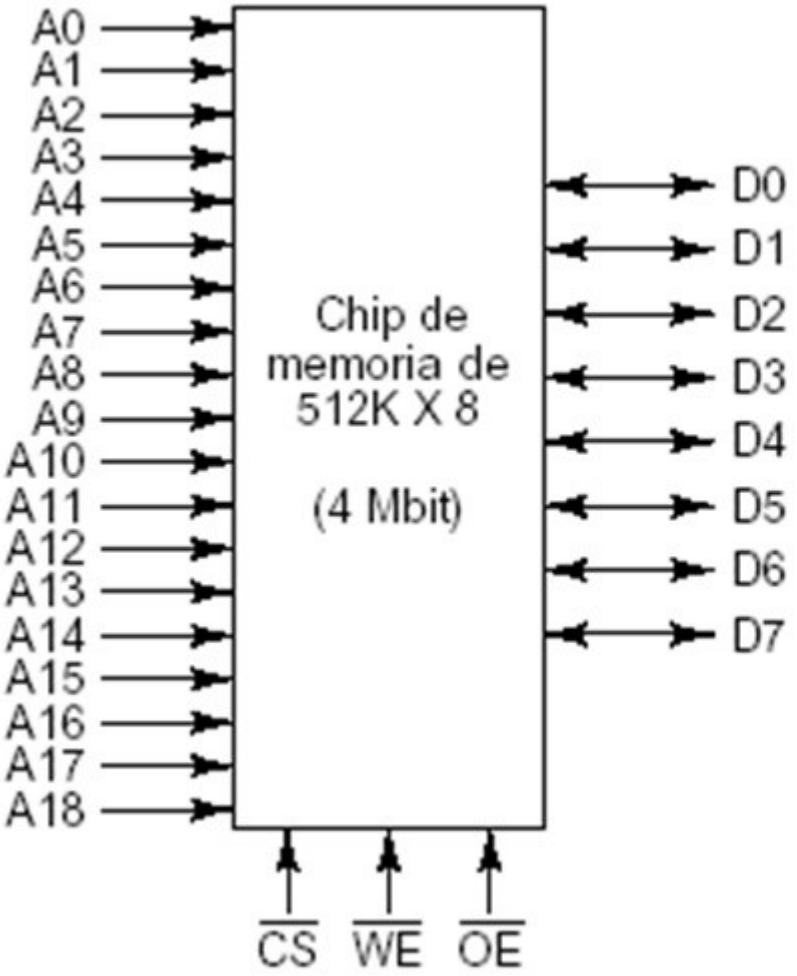
Varias formas de organización de una memoria de 96 bits:

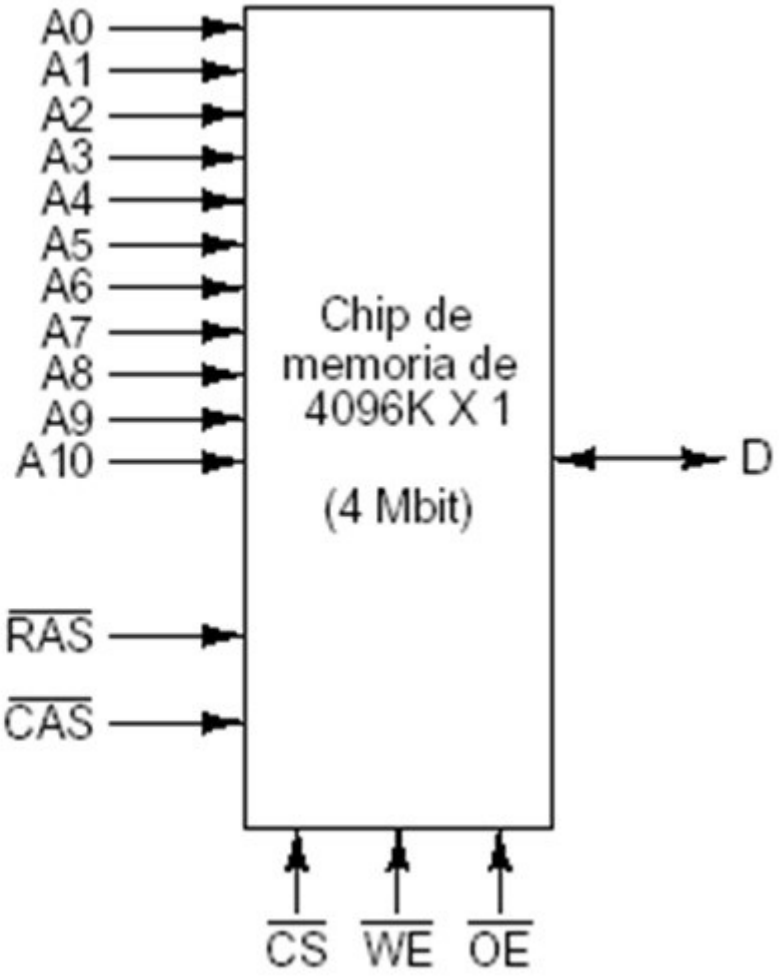


Organización de memorias

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Dos formas de organizar un chip de memoria de 4 Mbit:





Tipos de memorias y usos

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Comparación de distintos tipos de memorias:

Tipo	Categoría	Borrado	Alterable por byte	Volátil	Uso típico
SRAM	Lect./escrit.	Eléctrico	Sí	Sí	Caché nivel 2
DRAM	Lect./escrit.	Eléctrico	Sí	Sí	Memoria principal
ROM	Sólo lectura	Imposible	No	No	Aparatos en vol. grandes
PROM	Sólo lectura	Imposible	No	No	Equipos en vol. pequeños
EPROM	Princ. lectura	Luz UV	No	No	Prototipos de dispositivos
EEPROM	Princ. lectura	Eléctrico	No	No	Prototipos de dispositivos
Flash	Lect./escrit.	Eléctrico	No	No	Película para cámara digital

Dispositivos Lógicos Programables (PLD)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Tipos de PLD:

Read Only Memory (ROM)

Programmable ROM (PROM, EPROM, E^2PROM)

Programmable Logic Arrays (PLA)

Programmable Array Logic (PAL)

Erasable PLD (EPLD)

Field Programmable Gate Arrays (FPGA)

Dispositivos Lógicos Programables (PLD)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Arreglos lógicos sencillos

Basados en circuitos lógicos de 2 niveles (AND/OR).

Tienen una estructura de arreglo regular.

Distintos tipos:

Memorias de solo lectura (ROM, PROM, E^2PROM)

Programmable Logic Array (PLA)

Programmable Array Logic (PAL)

Arreglos programables en campo Field Programmable Gate Arrays (FPGA)

Bloque básico común replicado muchas veces.

Cada bloque puede configurarse para producir distintas funciones lógicas y en general incluyen biestables (flip-flops).

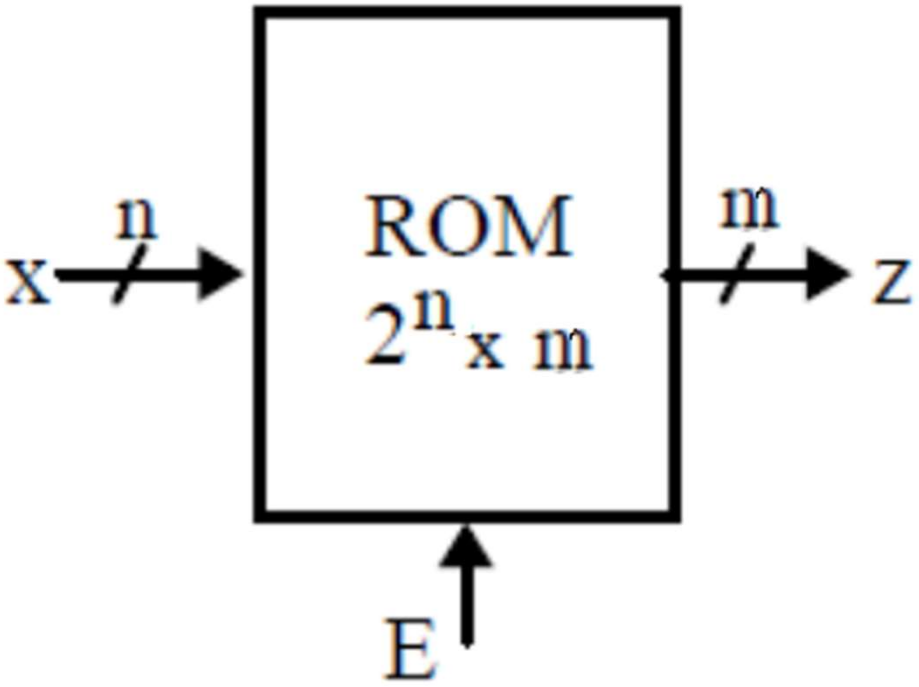
Interconexiones programables.

Las FPGAs más grandes tienen alrededor de 500K compuertas además de 500 Kb de SRAM.

Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Una **ROM** de **2^n x m** bits es un circuito lógico combinacional con **n** entradas y **m** salidas que implementa **m funciones lógicas de n variables**. El circuito puede disponer también de entradas adicionales de habilitación.

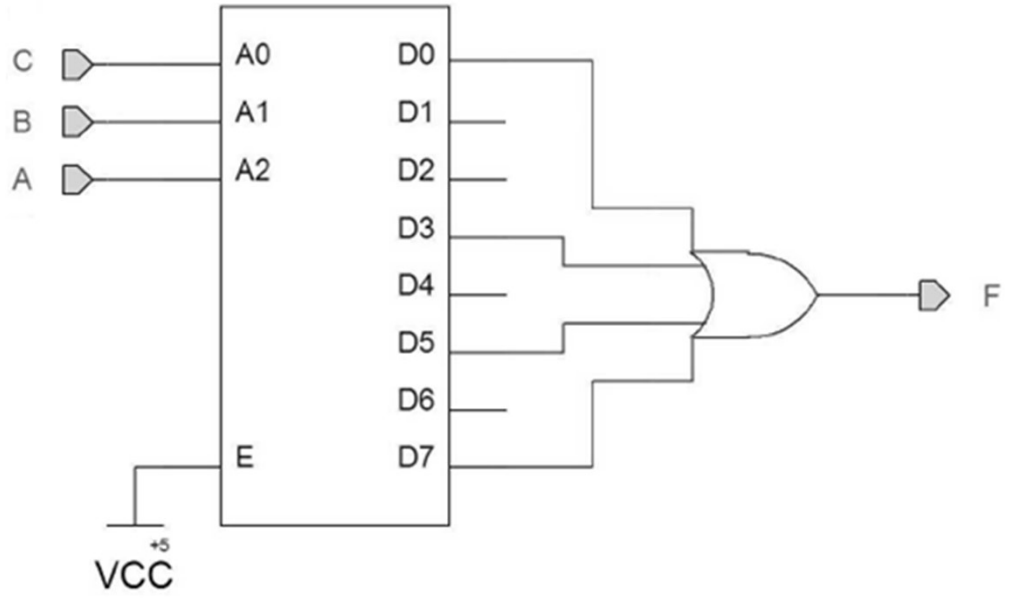


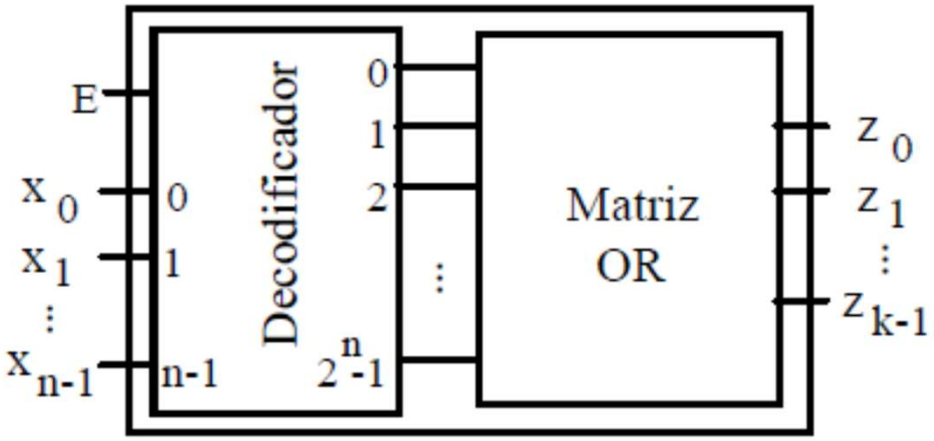
Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Recordando la implementación de funciones con Decodificadores y OR:

	A	B	C	F
0)	0	0	0	1
1)	0	0	1	0
2)	0	1	0	0
3)	0	1	1	1
4)	1	0	0	0
5)	1	0	1	1
6)	1	1	0	0
7)	1	1	1	1

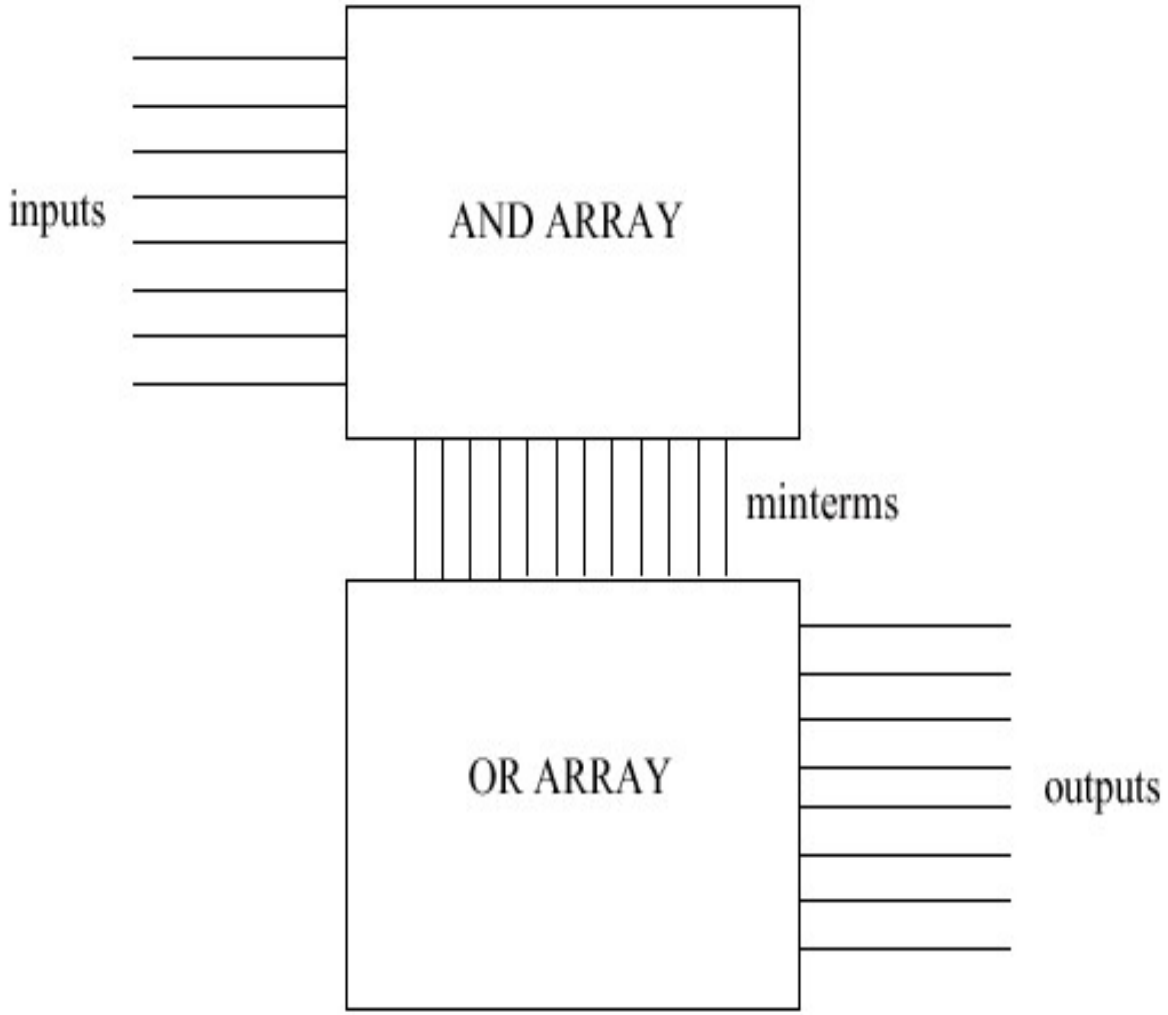




Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

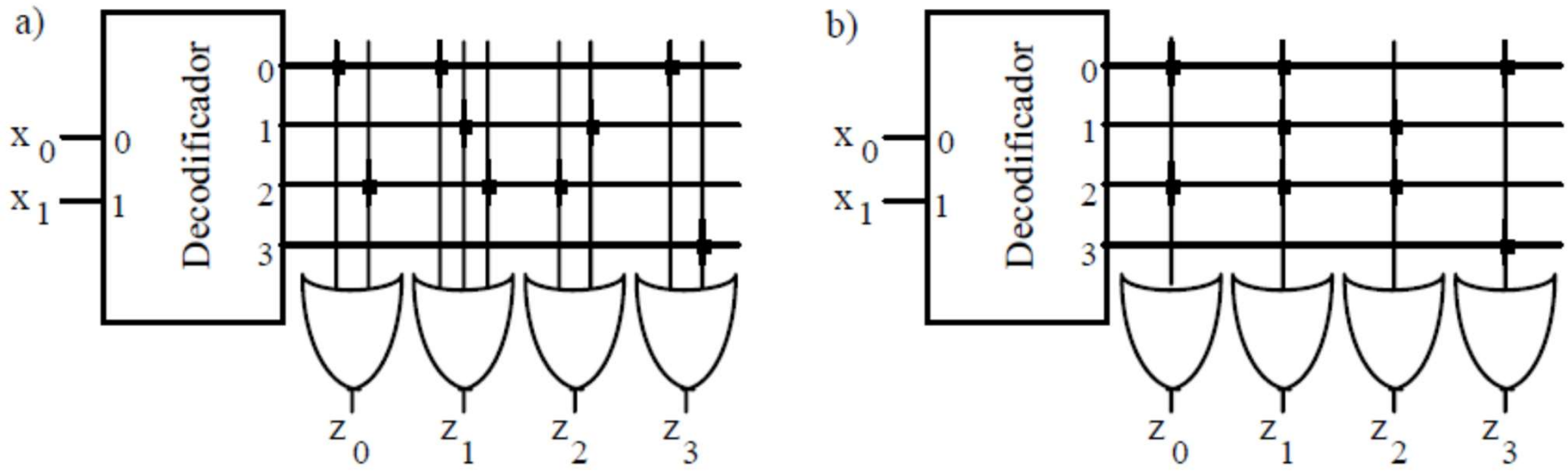
Esquema genérico:



Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

a) ROM de 4 x 4 bits. b) Representación abreviada de la matriz OR.



Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un **decodificador** genera para las **n** variables de entradas **2^n minitérminos**.

Utilizando compuertas **OR** para unir los minitérminos de las funciones Booleanas, se puede generar cualquier circuito combinatorio.

Incluyen el decodificador (compuertas **AND**) y las compuertas **OR** en un solo circuito integrado.

Es un depósito de información binaria fija (combinacional).

Los CI ROM y PROM tienen uniones internas que pueden quemarse o romperse programando.

Consisten en **n** líneas de entrada y **m** de salida.

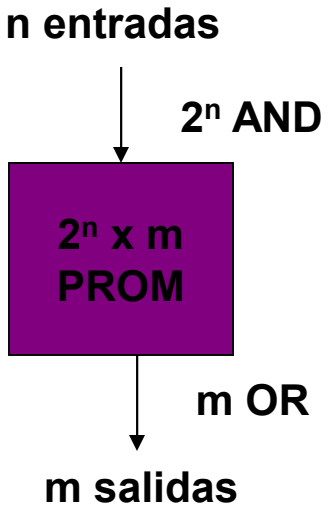
Cada combinación de entrada se llama **dirección**.

Cada combinación de salida es una **palabra**.

La cantidad de bits por palabra es **m**.

La cantidad de direcciones posibles es **2^n**.

Una palabra de salida es seleccionada por una dirección única.



Read Only Memory (ROM)

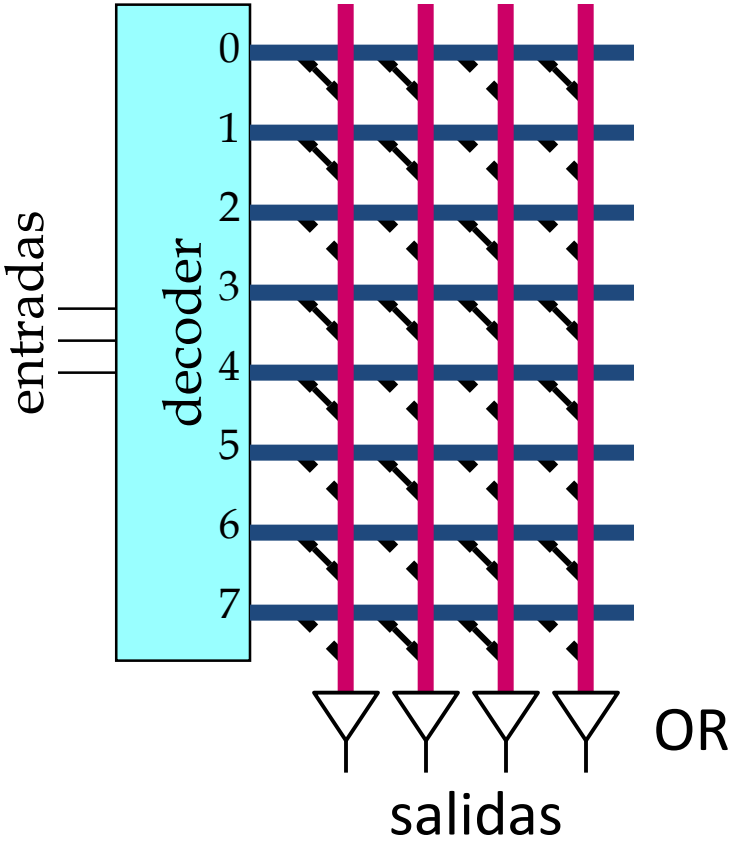
Las ROM y PROM pueden construirse utilizando arreglos ortogonales de cables.

- Conexión opcional en cada intersección.
- El decodificador coloca un 1 lógico en solo uno de los cables horizontales, que aparecerá a la salida de una de las OR si hay una conexión realizada.

Algunas PROM se pueden configurar quebrando conexiones.

- Alta tensión ubicada entre una entrada y una salida por vez.
- La gran corriente causa que el fusible en la interconexión se queme.

Otras “ROM” pueden borrarse y reprogramarse (EPROM, EEPROM, Flash).



- **Funciones almacenadas**

$\sum(0,1,3,4,6)$, $\sum(0,1,3,5,7)$, $\sum(2,3,6,7)$, $\sum(0,3,4,6)$

Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: ROM 4 x 4 con funciones implementadas.

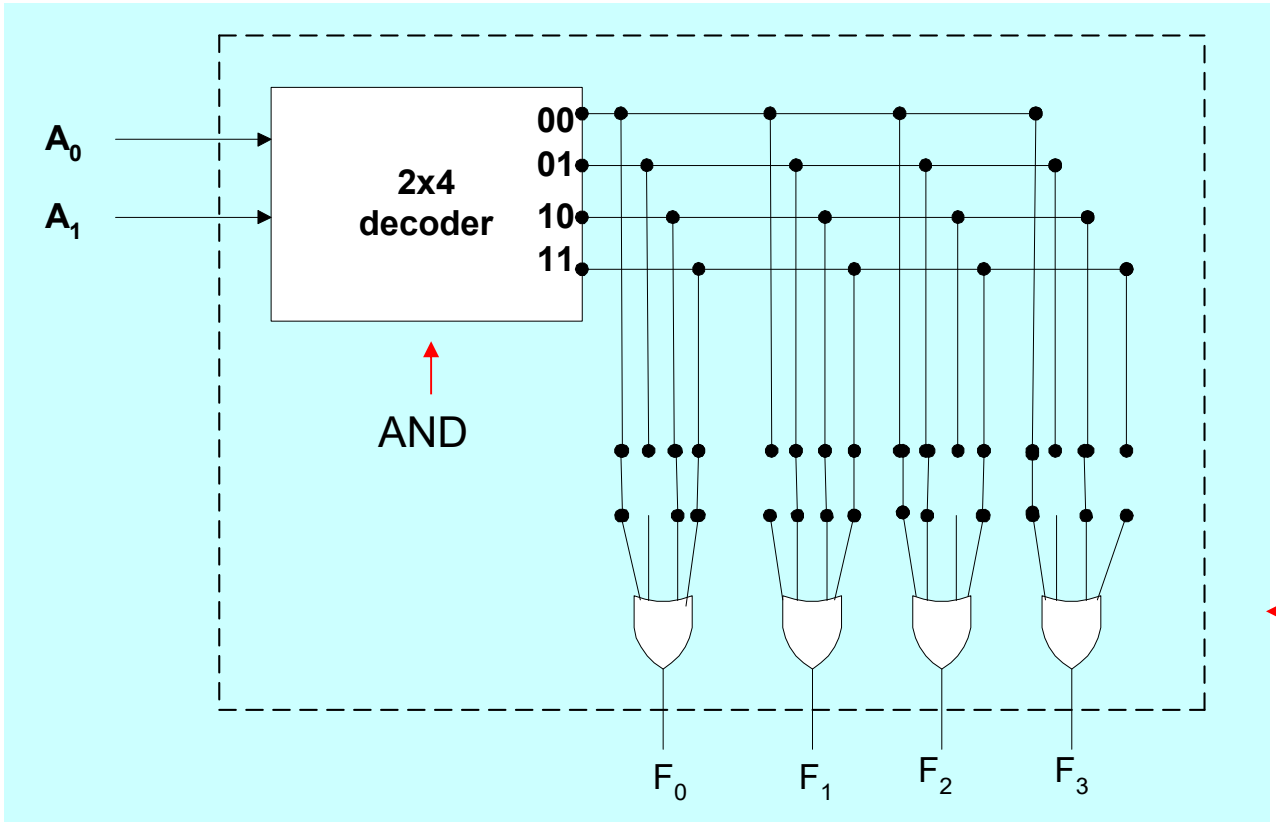


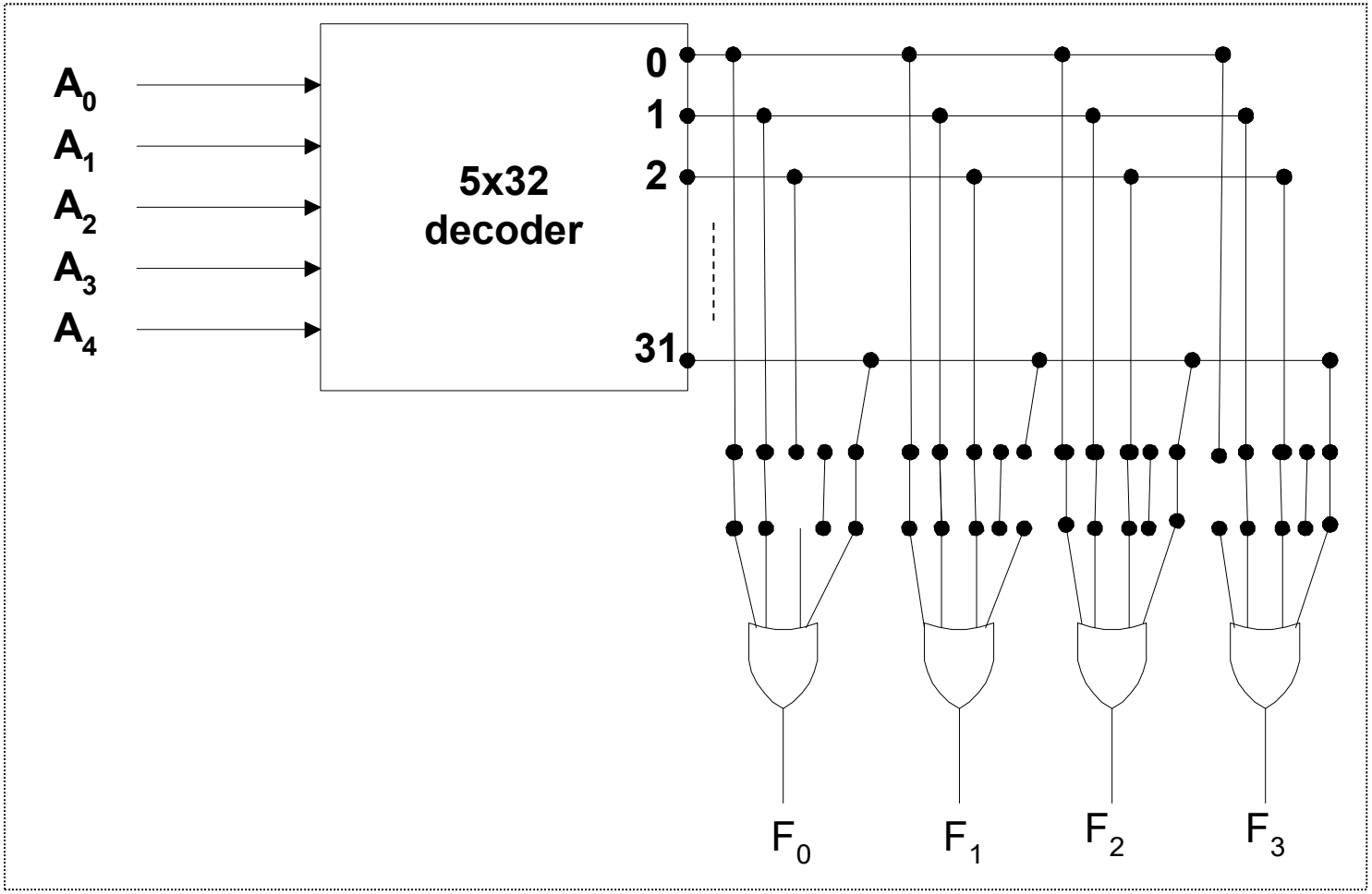
Tabla de verdad

A_0	A_1	F_0	F_1	F_2	F_3
0	0	1	0	1	1
0	1	0	1	1	0
1	0	1	1	0	1
1	1	1	1	1	0

Read Only Memory (ROM)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

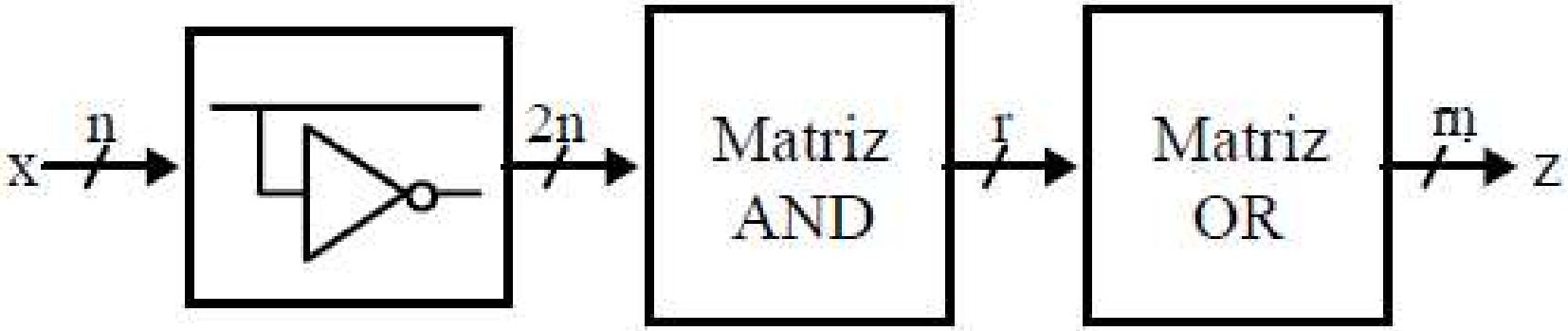
Ejemplo: ROM 32 x 4



Programmable Logic Array (PLA)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Un arreglo lógico programable (**PLA**) de **n x r x m** es un circuito lógico combinacional con **n** entradas y **m** salidas que implementa **m funciones lógicas de n variables** utilizando **r** productos, siendo **r < 2^n**. Adicionalmente el circuito puede disponer de entradas de habilitación.

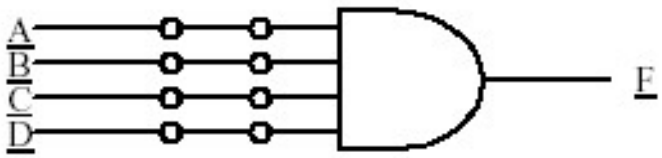


Programmable Logic Array (PLA)

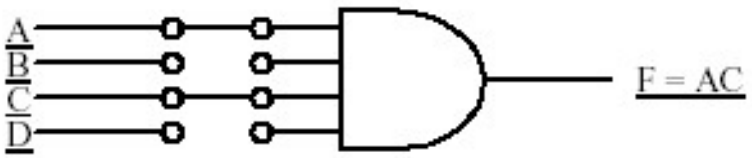
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Las **PLA** son estructuras más generales que las ROM. También tienen un arreglo de **AND** y uno de **OR ambos programables**. El tamaño del arreglo AND depende de la cantidad de entradas pero **no** tiene la cantidad de AND como una ROM (no genera 2^n minitérminos); mientras que el tamaño del arreglo OR depende de la cantidad de salidas.

Notación de programación:



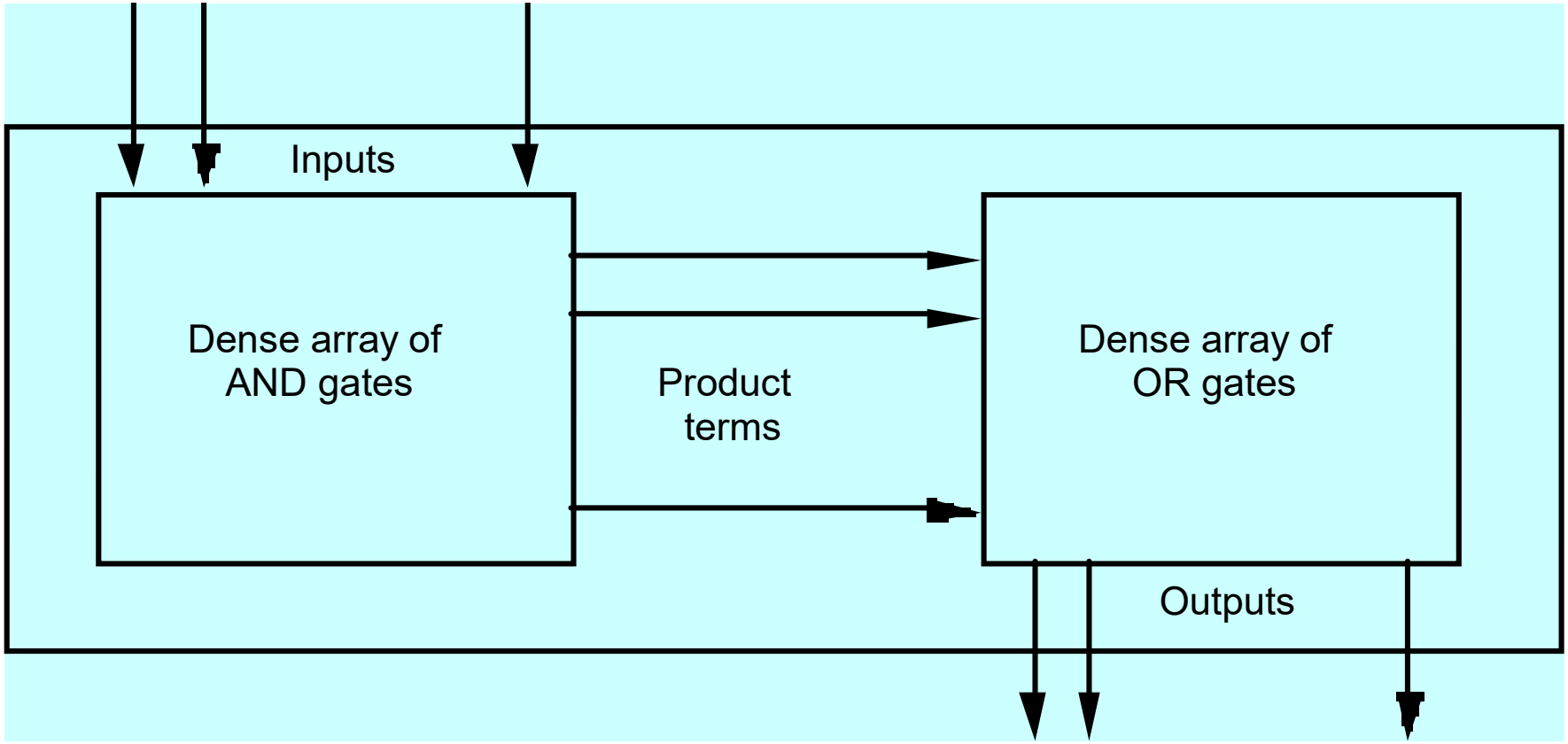
before programming: all fuses intact



after programming: fuses blown to remove unneeded connections

Programmable Logic Array (PLA)

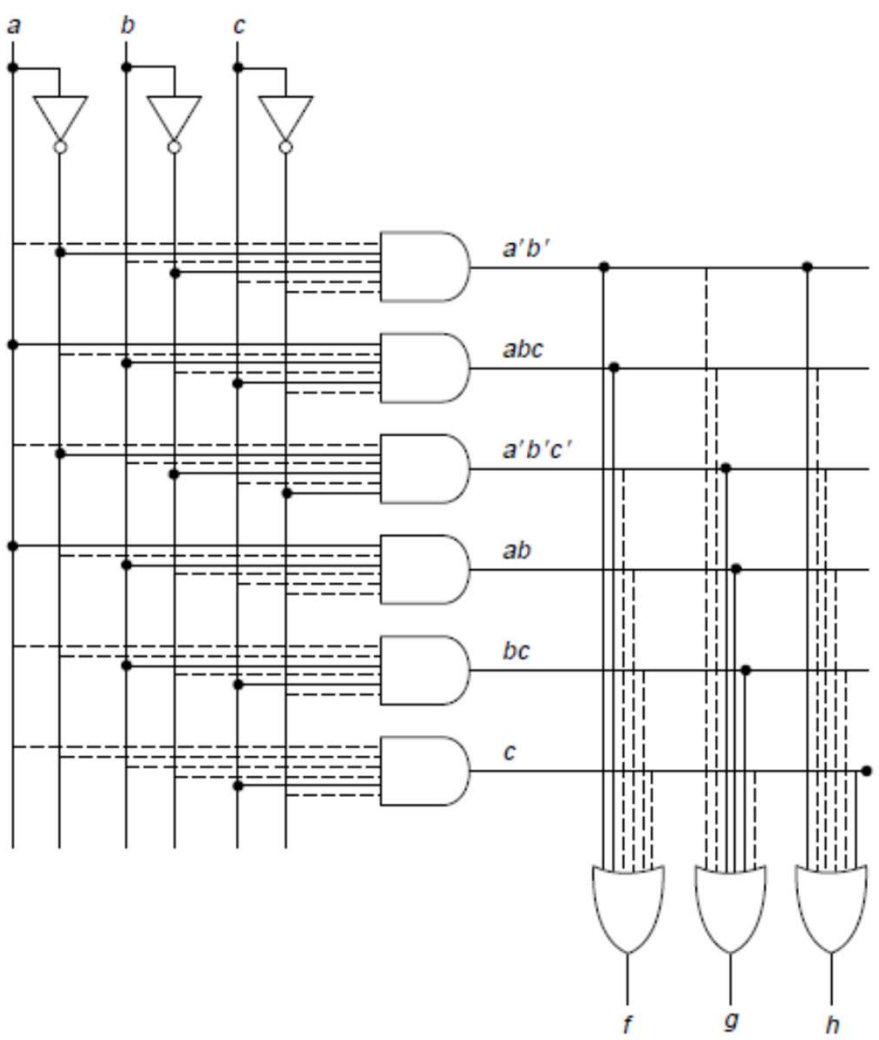
Las PLA tienen configurable la matriz AND y también la matriz OR. Permiten implementar suma de productos (no necesariamente minitérminos) de dos niveles.

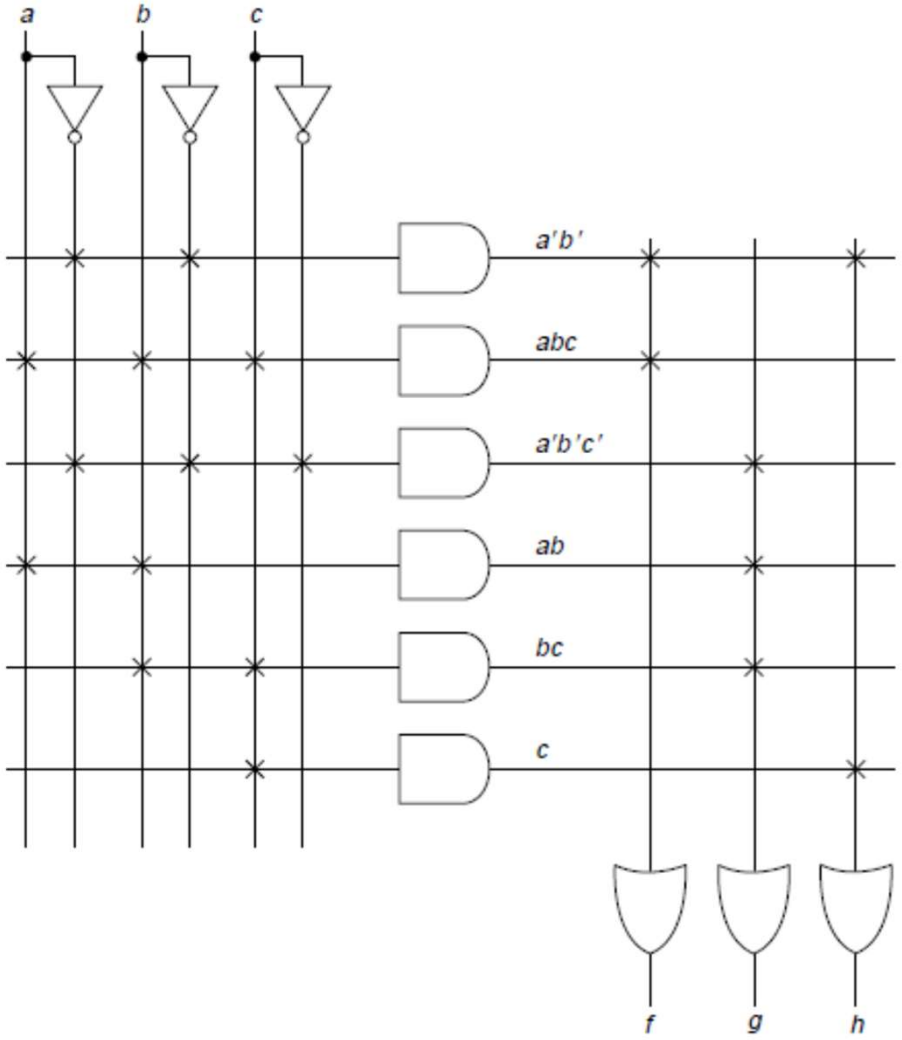


Programmable Logic Array (PLA)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo de programación y notación de una PLA (Suma de Productos):

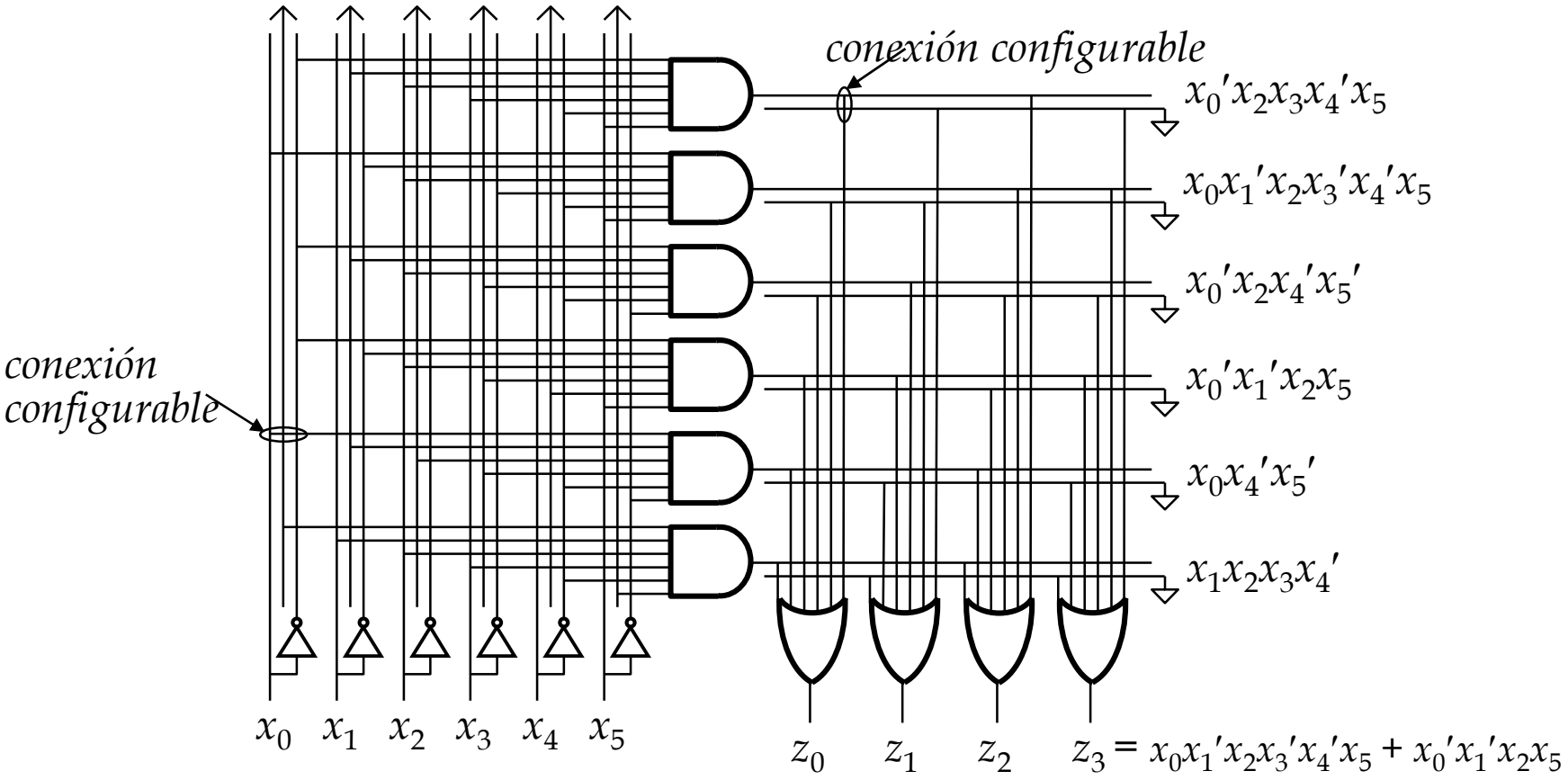




Programmable Logic Array (PLA)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo:



Programmable Logic Array (PLA)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo: Implementación de 6 funciones de 3 variables:

F1 = A B C

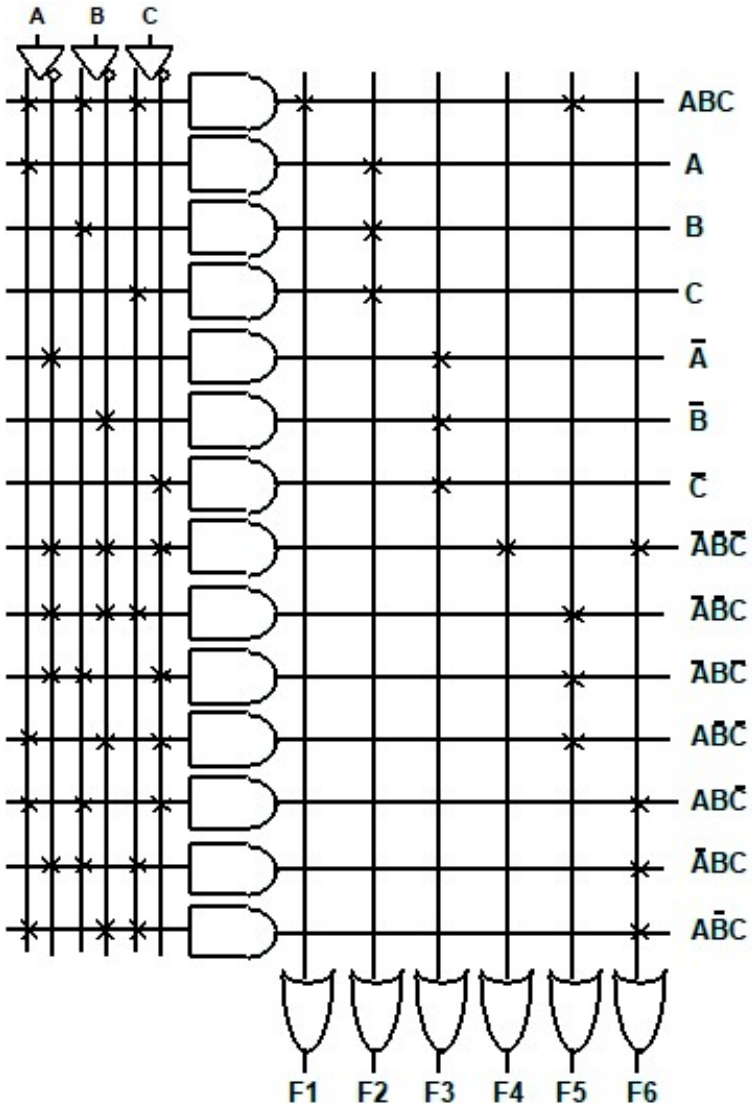
F2 = A + B + C

F3 = A B C

F4 = A + B + C

F5 = A xor B xor C

F6 = A xnor B xnor C

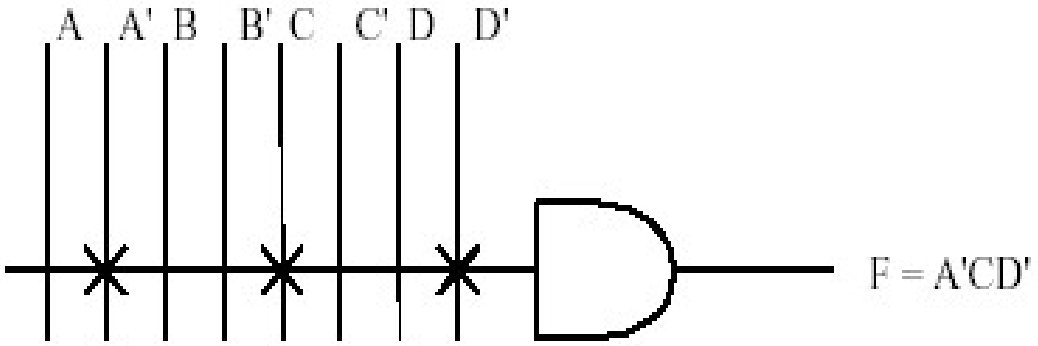


Programmable Array Logic (PAL)

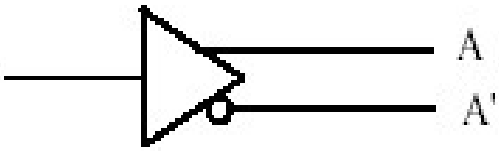
FACULTAD DE INGENIERIA
Universidad de Buenos Aires

PAL es similar a la PLA pero tiene fijo el arreglo OR.
Más simple de programar y de construcción más barata.
Para cada salida está determinada la cantidad de términos de la SP.
Generalmente se dispone también de las salidas complementadas.

Notación de programación:



an X is used to indicate a programmed

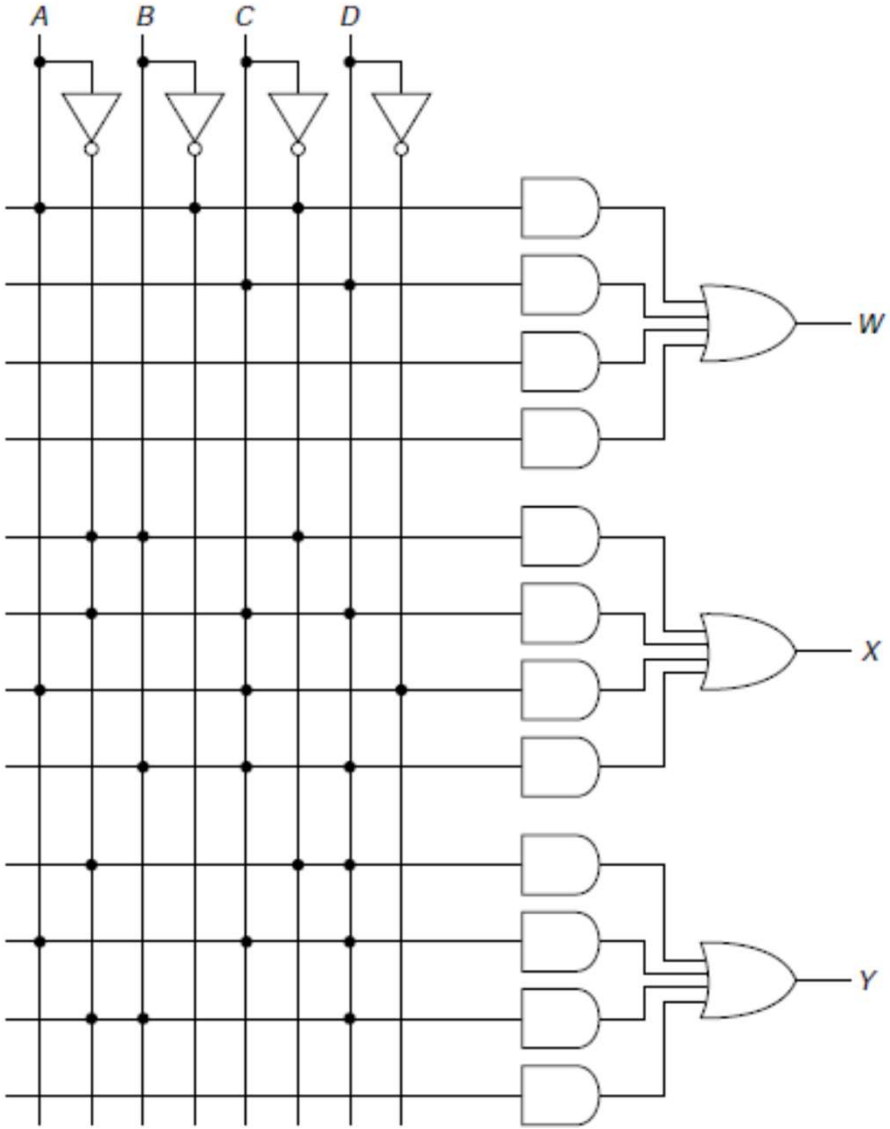


complemented and uncomplemented outputs are available

Programmable Array Logic (PAL)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

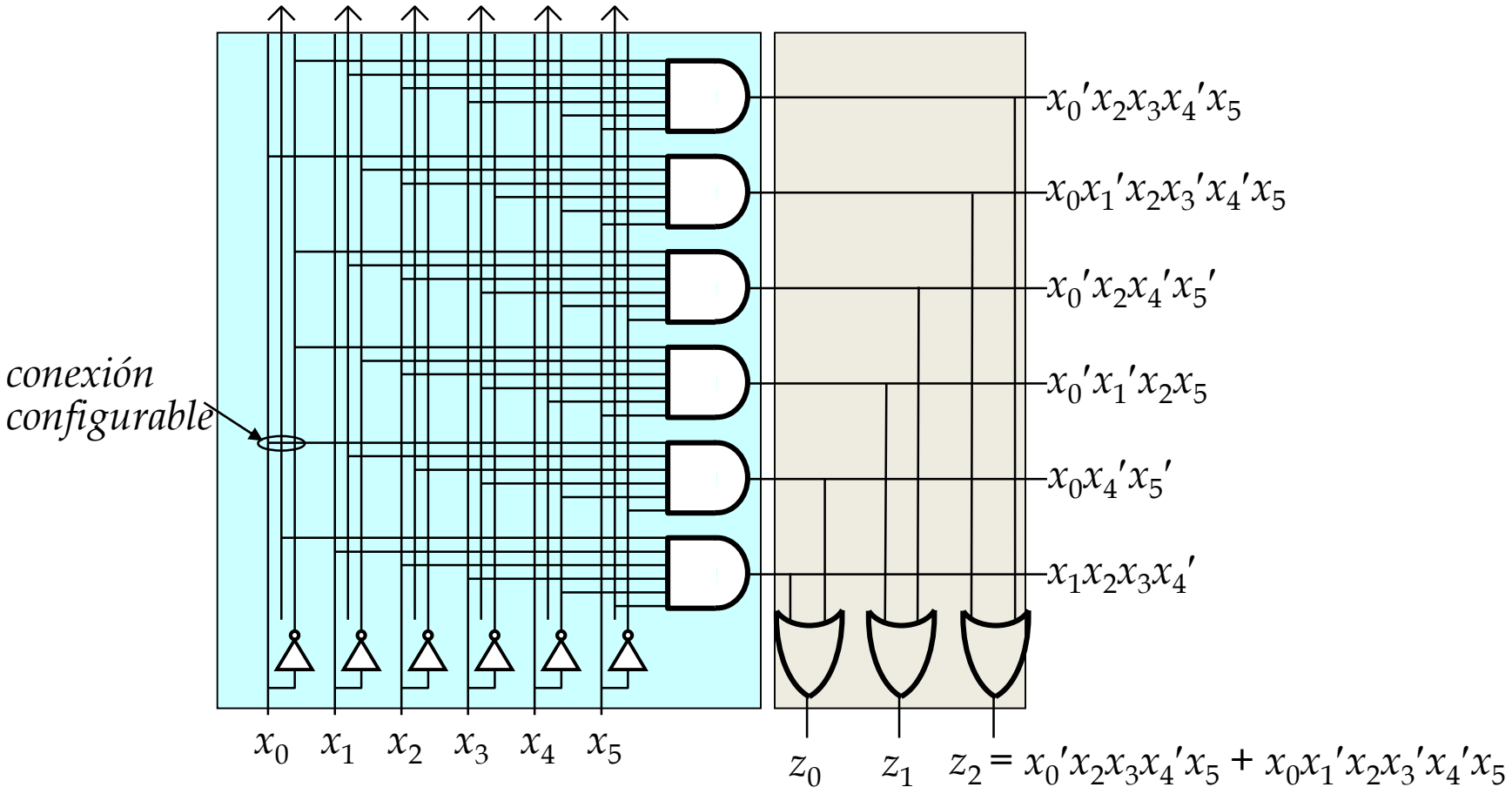
Ejemplo:



Programmable Array Logic (PAL)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

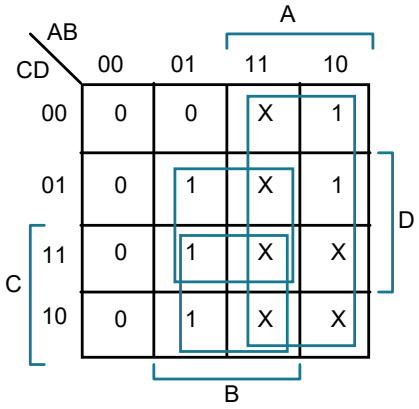
Ejemplo:



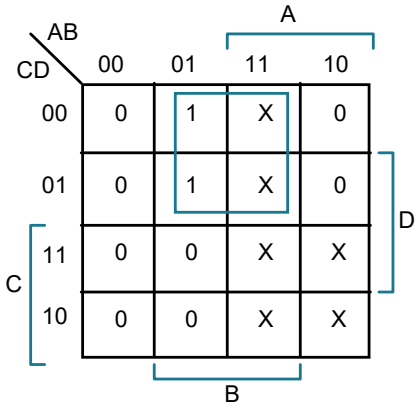
Programmable Array Logic (PAL)

Ejemplo de diseño: Conversor de código BCD a Gray

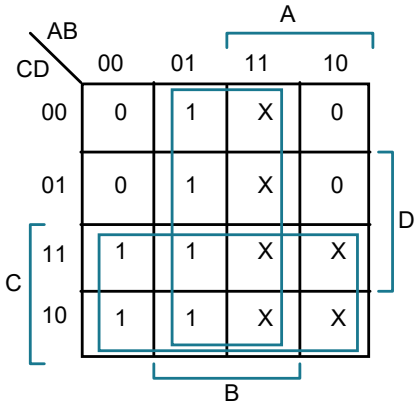
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	1	1	1	0
0	1	1	0	1	0	1	0
0	1	1	1	1	0	1	1
1	0	0	0	1	0	0	1
1	0	0	1	1	0	0	0
1	0	1	0	X	X	X	X
1	0	1	1	X	X	X	X
1	1	0	0	X	X	X	X
1	1	0	1	X	X	X	X
1	1	1	0	X	X	X	X
1	1	1	1	X	X	X	X



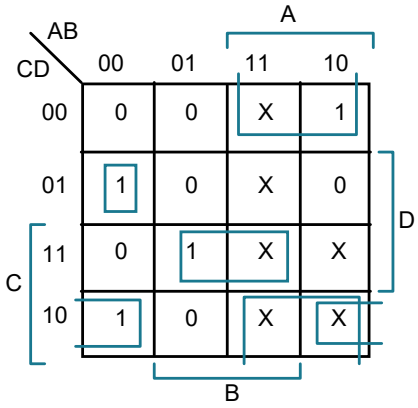
K-map for W



K-map for X



K-map for Y



K-map for Z

Funciones mínimas:

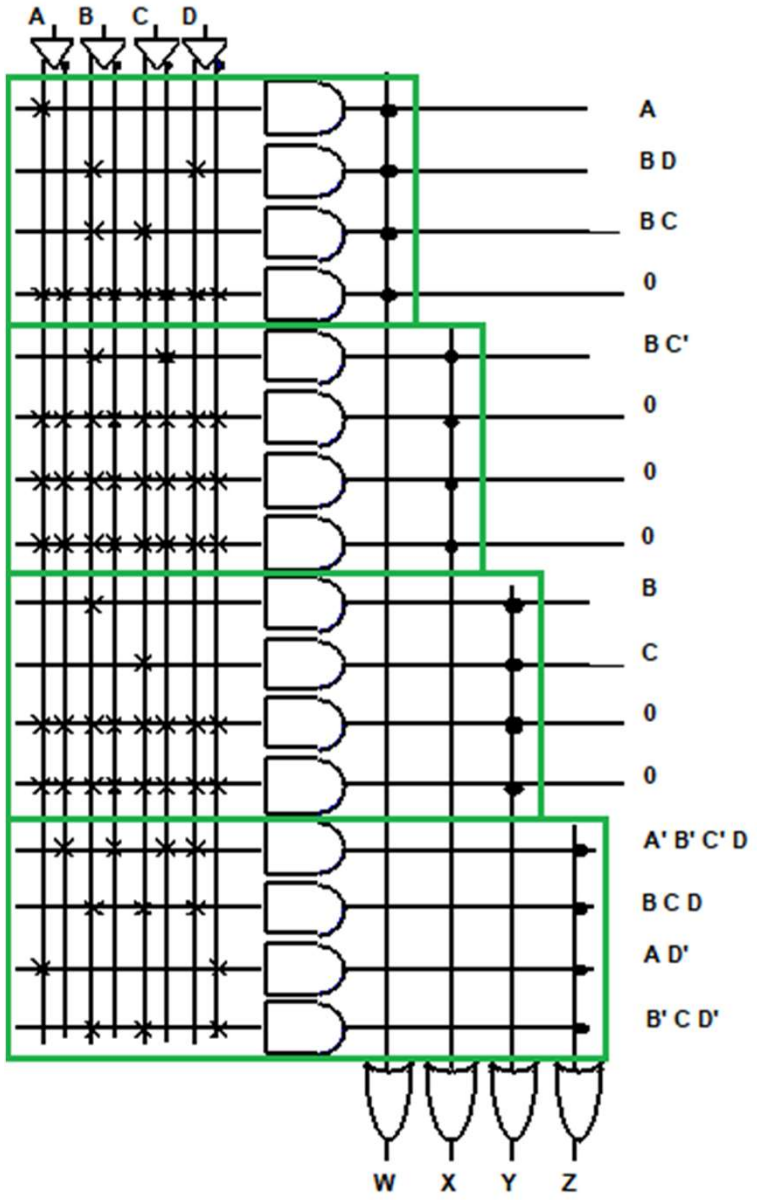
W = A + B D + B C
X = B C'
Y = B + C
Z = A'B'C'D + B C D + A D' + B' C D'

Programmable Array Logic (PAL)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Ejemplo de diseño: Conversor de código BCD a Gray

PAL programada



Funciones mínimas:

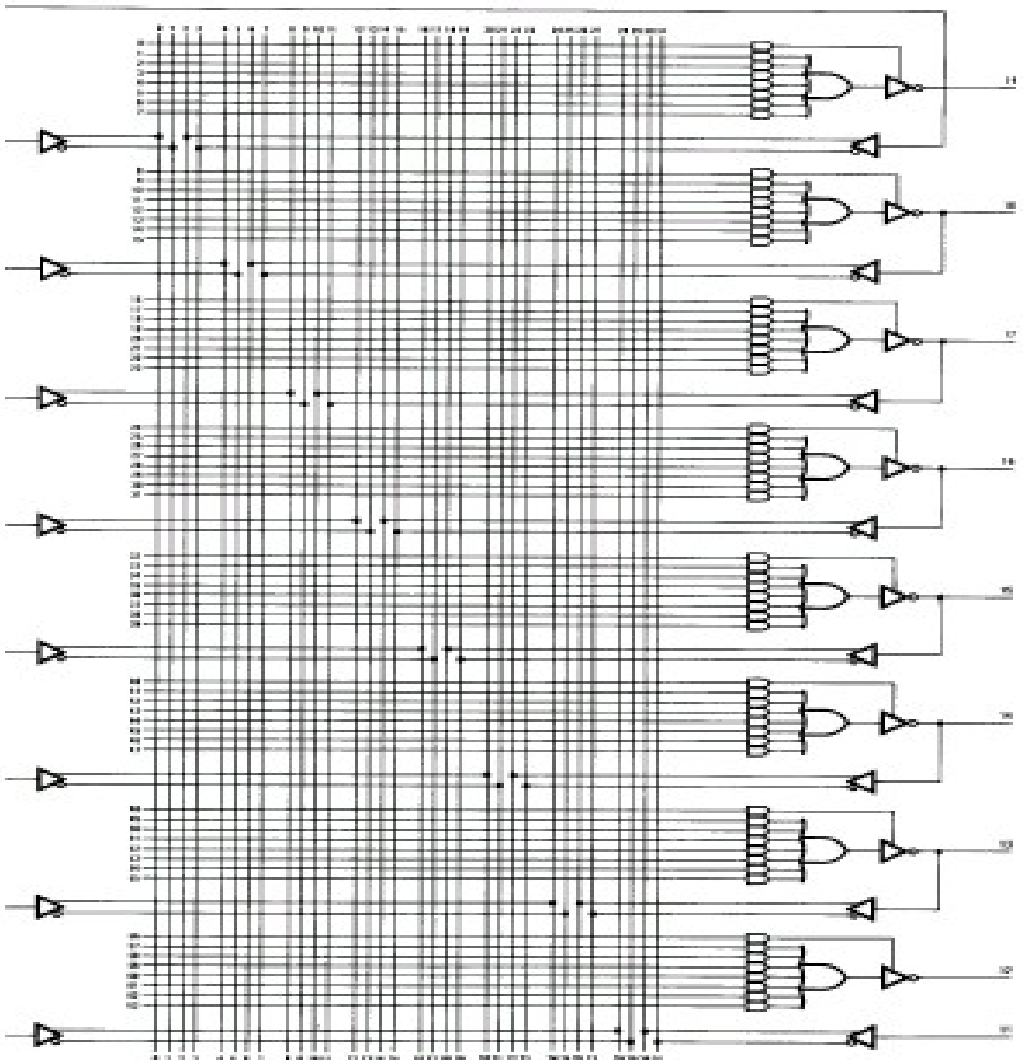
W = A + B D + B C
X = B C'
Y = B + C
Z = A'B'C'D + B C D + A D' + B' C D'

4 términos de producto para cada compuerta **OR**

Programmable Array Logic (PAL)

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

La PAL de la figura consiste de un número fijo de compuertas AND de entradas múltiples combinadas con compuertas OR. Las conexiones a cada AND son programables, hay flexibilidad en las funciones lógicas realizables. La única limitación es la cantidad fija de términos. Mayormente tienen la posibilidad de realimentar las salidas.



Comparación

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

PROM se programa a partir de los minitérminos de las funciones

A partir de la TV directamente, ya que el decodificador genera todos los minitérminos y sumo solo los que forman parte de la función.

PLA se programa a partir de la simplificación de las funciones

Se simplifican las funciones a implementar y a partir de allí se generan los términos producto.

PAL se programa a partir de la simplificación de las funciones

Se simplifican las funciones a implementar y a partir de allí se generan los términos producto considerando que debo tener como mínimo tantas AND como sumandos en cada función a implementar.

	Matriz AND	Matriz OR
PROM	Fija	Programable
PLA	Programable	Programable
PAL	Programable	Fija

Comparación

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Se pueden considerar a PROM y PAL como versiones restringidas de la PLA

La PROM es una PLA con arreglo AND fijo que genera todos los minitérminos y un arreglo OR configurable.

La PAL es una PLA con un arreglo OR fijo, y cada salida es un subconjunto de las AND.

Selección de acuerdo a las necesidades

Se observará las ecuaciones a implementar.

Tener en cuenta la cantidad de entradas y salidas.

Comparar costos.

La PROM es más fácil, pero utilizar PAL y PLA donde resulte posible

Las que incluyen biestables permiten realizar circuitos secuenciales sincrónicos.

Herramientas de diseño

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

Las herramientas CAD realizan diseño en base a especificaciones en lenguajes descriptores de hardware.

PALASM y ABEL elementales.

VHDL y Verilog avanzados (FPGA).

FACULTAD DE INGENIERIA
Universidad de Buenos Aires

